

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2016

Huấn luyện viên: Yu Linyun, Chen Xumin

China Computer Federation, 2016

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2016

IOI China candidate team papers / National training team 2016 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2016. Phần bìa và mục lục có lỗi văn bản đọc được, nhưng ngay từ bài đầu tiên phần thân đã bị lỗi ánh xạ phong chữ và trích xuất thành mojibake. Bản dịch này chuyển ngữ các thông tin đầu sách, mục lục, bài đầu tiên của Ren Zhizhou về một số phương pháp tính tổng hàm nhân tính, bài thứ hai của Jiang Zhihao về một số phương pháp mô hình hóa bằng luồng mạng, bài thứ ba của Dong Kefan về quy hoạch tuyến tính cùng bài toán đối ngẫu, bài thứ tư của Wang Wentao về một số thuật toán cùng ứng dụng cho bài toán cắt nhỏ nhất trên đồ thị vô hướng, bài thứ năm của Zou Xiaoyao về ứng dụng của quy hoạch tuyến tính trong thi tin học, bài thứ sáu của Ji Ruiyi về phép toán cực trị trên đoạn cùng bài toán cực trị lịch sử, bài thứ bảy của Mao Xiao về việc xét lại biến đổi Fourier nhanh, bài thứ tám của Luo Zhezhen về một lớp phương pháp duy trì thông tin đoạn có hỗ trợ chèn và xóa ở cuối, bài thứ chín của Hong Huadun về báo cáo ra đề mảng hậu tố của Tiểu C, bài thứ mười của Zhang Haowei về báo cáo ra đề *Xiaoxiaokan*, và bài thứ mười một của Li Zihao về báo cáo ra đề *strakf*, cùng bài thứ mười hai của Wang Wenxiao về báo cáo ra đề *Tập hợp của quá khứ*, bài thứ mười ba của Wu Zuofan về báo cáo ra đề *Lái tàu rời Qin Chuan*, bài thứ mười bốn của Jin Ce về báo cáo ra đề *Bài luyện tập thuật toán sắp xếp cơ bản*, và bài cuối của Yuan Yutao về báo cáo ra đề *move*. Phần thân các bài được dựng từ OCR và đối chiếu công thức với lớp văn bản PDF.

1 Trạng thái và ghi chú trích xuất

Đã dịch cả mười lăm bài trong tập luận văn. Bài 1 dùng trang vật lý 5–20 của PDF nguồn, tương ứng trang in 1–16. Bài 2 dùng trang vật lý 21–35, tương ứng trang in 17–31; trang vật lý 36, tương ứng trang in 32, là trang trắng ngăn cách trước bài kế tiếp. Bài 3 bắt đầu ở trang vật lý 37, tương ứng trang in 33, và kết thúc ở trang vật lý 66, tương ứng trang in 62. Bài 4 dùng trang vật lý 67–84, tương ứng trang in 63–80. Bài 5 dùng trang vật lý 85–102, tương ứng trang in 81–98. Bài 6 dùng trang vật lý 103–126, tương ứng trang in 99–122; trang vật lý 126, tương ứng trang in 122, là trang trắng ngăn cách trước bài kế tiếp. Bài 7 dùng trang vật lý 127–144, tương ứng trang in 123–140. Bài 8 dùng trang vật lý 145–166, tương ứng trang in 141–162; phần nội dung kết thúc ở trang in 161, trang in 162 là trang trắng ngăn cách, và bài kế tiếp bắt đầu ở trang vật lý 167, tương ứng trang in 163. Bài 9 dùng trang vật lý 167–174, tương ứng trang in 163–170; bài 10 bắt đầu ở trang vật lý 175, tương ứng trang in 171, và dùng trang vật lý 175–182, tương ứng trang in 171–178. Bài 11 dùng trang vật lý 183–192, tương ứng trang in 179–188; phần nội dung kết thúc ở trang in 187, trang in 188 là trang trắng ngăn cách, và bài kế tiếp bắt đầu ở trang vật lý 193, tương ứng trang in 189. Bài 12 dùng trang vật lý 193–198, tương ứng trang in 189–194; phần nội dung kết thúc ở trang in 193, trang in 194 là trang trắng ngăn cách, và bài 13 bắt đầu ở trang vật lý 199, tương ứng trang in 195. Bài 13 dùng trang vật lý 199–212, tương ứng trang in 195–208; phần nội

dung kết thúc ở trang in 207, trang in 208 là trang trắng ngăn cách, và bài kế tiếp bắt đầu ở trang vật lý 213, tương ứng trang in 209. Bài 14 dùng trang vật lý 213–223, tương ứng trang in 209–219; trang vật lý 224, tương ứng trang in 220, là trang trắng ngăn cách trước bài kế tiếp ở trang in 221. Bài 15 dùng trang vật lý 225–234, tương ứng trang in 221–230, và kết thúc ở trang cuối của PDF nguồn. Phần thân các bài 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13 và 14 có lớp văn bản bị mojibake, nên được OCR bằng pdftoppm và tesseract -l chi_sim+eng, rồi lọc nhiễu thủ công khi dựng bản dịch; bài 15 cũng dùng cùng quy trình này.

2 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2016*. Huấn luyện viên ghi trên bìa là Yu Linyun và Chen Xumin. Đơn vị xuất bản là China Computer Federation.

3 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Một số phương pháp tính tổng hàm nhân tính	Ren Zhizhou, Trường Trung học số 1 Thiệu Hưng
17	Một số phương pháp mô hình hóa bằng luồng mạng	Jiang Zhihao, Trường Trung học số 1 Shengli, Đồng Doanh
33	Bàn sơ lược về quy hoạch tuyến tính và bài toán đối ngẫu	Dong Kefan, Trường Trung học số 1 Phúc Châu, Phúc Kiến
63	Bàn sơ lược về một số thuật toán và ứng dụng cho bài toán cắt nhỏ nhất trên đồ thị vô hướng	Wang Wentao, Trường Trung học số 1 Thiệu Hưng
81	Bàn sơ lược về ứng dụng của quy hoạch tuyến tính trong thi tin học	Zou Xiaoyao, Trường Trung học Trần Hải, Ninh Ba
99	Phép toán cực trị trên đoạn và bài toán cực trị lịch sử	Ji Ruyi, Trường Học Quân Hàng Châu
123	Xét lại biến đổi Fourier nhanh	Mao Xiao, Trường Yali
141	Từ <i>Unknown</i> bàn về một lớp phương pháp duy trì thông tin đoạn có hỗ trợ chèn và xóa ở cuối	Luo Zhezheng, Trường THPT trực thuộc Đại học Sư phạm An Huy
163	Báo cáo ra đề mảng hậu tố của Tiểu C	Hong Huadun, Trường Trung học số 1 Thiệu Hưng
171	Báo cáo ra đề <i>Xiaoxiaokan</i>	Zhang Haowei, Trường Trung học Dư Diêu, Chiết Giang
179	Báo cáo ra đề <i>strakf</i>	Li Zihao, Trường Trung học Shimen, Phúc Sơn
189	Báo cáo ra đề <i>Tập hợp của quá khứ</i>	Wang Wenxiao, Trường Trung học Song Thập, Hạ Môn
195	Báo cáo ra đề <i>Lái tàu rời Qin Chuan</i>	Wu Zuofan, Trường THPT trực thuộc Đại học Sư phạm An Huy
209	Bài luyện tập thuật toán sắp xếp cơ bản	Jin Ce, Trường Học Quân Hàng Châu, Chiết Giang
221	Báo cáo ra đề <i>move</i>	Yuan Yutao, Trường Yali, Trường Sa

Bài 1. Một số phương pháp tính tổng hàm nhân tính

Ren Zhizhou, Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Hàm nhân tính là một lớp hàm số học đặc biệt. Bài viết này tổng hợp một số kỹ thuật tính toán thường gặp trong các bài toán liên quan tới hàm nhân tính, và từ cảm hứng của các thuật toán đó đưa ra một phương pháp tính tổng hàm nhân tính có phạm vi áp dụng tương đối rộng.

1. Dẫn nhập

Trong thi tin học, đôi khi ta gặp các hàm được định nghĩa trên miền số nguyên dương, tức các hàm số học. Hàm nhân tính, với tư cách một lớp hàm số học đặc biệt, thường là đối tượng được thảo luận. Trong loại bài toán này, thí sinh thường phải trước hết thực hiện một số suy luận, rồi chọn phương pháp thích hợp để tính biểu thức sau khi đã biến đổi. Khi đó ta cần những thuật toán hiệu quả để hoàn tất bước tính toán cuối cùng.

Trong bài viết này, tác giả trước hết chỉnh lý một số phương pháp tính toán thường dùng trong thi tin học.

Mục 2 giới thiệu định nghĩa hàm nhân tính và liệt kê một số ví dụ thường gặp.

Mục 3 giới thiệu cách dùng sàng tuyến tính để nhanh chóng tính các giá trị đầu của một hàm nhân tính.

Mục 4 giới thiệu định nghĩa tích chập Dirichlet và đảo Mobius, làm nền cho mục sau.

Mục 5 kết hợp nội dung hai mục trước để đưa ra một phương pháp tính tổng hàm số học hiệu quả. Trong thi tin học Trung Quốc, thuật toán này thường được gọi là sàng Du Jiao; nó xây dựng công thức bằng tích chập Dirichlet. Tuy không có liên hệ quá lớn với tính nhân tính, và đòi hỏi chính hàm cần tính có tính chất đặc biệt nên điều kiện sử dụng hơi nghiêm ngặt, nó vẫn đem lại nhiều gợi ý.

Trong Mục 6, từ cảm hứng của các thuật toán trước đó, tác giả đưa ra một phương pháp tính tổng hàm nhân tính mới. Hiệu suất của phương pháp này tuy hơi kém thuật toán ở Mục 5, nhưng phạm vi áp dụng rộng hơn. Ở mục này, tác giả bắt đầu từ động cơ thiết kế thuật toán, từng bước xây dựng toàn bộ quy trình, rồi giới thiệu ứng dụng của thuật toán này ngoài bài toán tính tổng hàm nhân tính.

2. Định nghĩa

Định nghĩa 2.1. Hàm có miền xác định là tập số nguyên dương và miền giá trị là tập số phức được gọi là **hàm số học**.

Định nghĩa 2.2. Giả sử $f(x)$ là một hàm số học. Nếu với mọi cặp số nguyên dương nguyên tố cùng nhau a, b đều có

$$f(ab) = f(a)f(b),$$

thì $f(x)$ là một **hàm nhân tính**.

Định nghĩa 2.3. Giả sử $f(x)$ là một hàm số học. Nếu với mọi cặp số nguyên dương a, b đều có

$$f(ab) = f(a)f(b),$$

thì $f(x)$ là một **hàm hoàn toàn nhân tính**.

Nếu không nói riêng, mọi hàm xuất hiện trong bài viết này đều là hàm số học.

2.1. Một số hàm nhân tính thường gặp

Sau đây là một số hàm nhân tính thường gặp:

- Hàm ước $\sigma_k(n)$, biểu thị tổng lũy thừa bậc k của mọi ước của n .
- Hàm Euler $\varphi(n)$, biểu thị số lượng số nguyên dương không vượt quá n và nguyên tố cùng nhau với n .
- Hàm Mobius

$$\mu(n) = \begin{cases} 1, & n = 1, \\ (-1)^k, & n \text{ là tích của } k \text{ số nguyên tố khác nhau,} \\ 0, & \text{các trường hợp khác.} \end{cases}$$

2.2. Một số hàm hoàn toàn nhân tính thường gặp

Sau đây là một số hàm hoàn toàn nhân tính thường gặp:

- Hàm lũy thừa $Id_k(n) = n^k$.
- Hàm đơn vị

$$\epsilon(n) = \begin{cases} 1, & n = 1, \\ 0, & n > 1. \end{cases}$$

3. Sàng tuyến tính

Sàng tuyến tính là thuật toán rất thường dùng khi tính hàm nhân tính. Nó có thể tính trong $O(n)$ thừa số nguyên tố nhỏ nhất của mọi số từ 2 tới n ; dùng các thông tin này, ta có thể truy hồi xong các giá trị hàm của n hạng đầu.

3.1. Tính thừa số nguyên tố nhỏ nhất

Xét phân tích thừa số nguyên tố của một số nguyên dương n , và sắp các thừa số nguyên tố theo thứ tự giảm dần:

$$n = \prod_{i=1}^k p_i,$$

trong đó p_i là số nguyên tố và với $1 \leq i < k$ có $p_i \geq p_{i+1}$. Cách phân tích này là duy nhất; có thể dùng tính duy nhất đó để thu được thuật toán sau:

- Gọi pr_i là thừa số nguyên tố nhỏ nhất của i , và liệt kê i từ 2 để tính.
- Nếu pr_i của số hiện tại chưa được tính, thì i là số nguyên tố và $pr_i = i$.
- Liệt kê mọi số nguyên tố p không vượt quá pr_i , rồi gán $pr_{ip} = p$.

Mỗi số chỉ bị sàng bởi thừa số nguyên tố nhỏ nhất của nó, nên độ phức tạp thời gian của thuật toán này là $O(n)$.

3.2. Tính n hạng đầu của một hàm nhân tính

Giả sử $f(x)$ là một hàm nhân tính, nay cần tính các giá trị $f(x)$ của n hạng đầu.

Nếu có thể phân tích x thành một cặp a, b nguyên tố cùng nhau với $a, b > 1$, thì giá trị $f(x)$ có thể truy hồi theo Định nghĩa 2.2. Vì vậy chỉ còn cần tính trường hợp x chỉ có một loại thừa số nguyên tố.

Ta có thể dùng sàng tuyến tính $O(n)$ để tính thừa số nguyên tố nhỏ nhất của mỗi số, rồi truy hồi số mũ của thừa số nguyên tố nhỏ nhất đó trong mỗi số. Giả sử thừa số nguyên tố nhỏ nhất của x là p , số mũ là c , và $p^c \mid x$. Khi đó

$$f(x) = f(p^c)f\left(\frac{x}{p^c}\right).$$

3.3. Ví dụ 1

Ví dụ 1. Tính n giá trị đầu của hàm Euler $\varphi(n)$.

Với $n > 1$, giả sử

$$n = \prod_{i=1}^k p_i^{c_i},$$

trong đó các p_i đều là số nguyên tố và $p_i \neq p_j$ khi $i \neq j$. Hàm Euler có công thức kinh điển

$$\varphi(n) = n \prod_{i=1}^k \frac{p_i - 1}{p_i} = \prod_{i=1}^k (p_i - 1)p_i^{c_i-1}.$$

Với trường hợp n chỉ có một loại thừa số nguyên tố, ta có thể tính trực tiếp; phần còn lại dùng cách ở Mục 3.2 để truy hồi bằng sàng tuyến tính. Độ phức tạp thời gian là $O(n)$.

4. Đảo Mobius

4.1. Tích chập Dirichlet

Định nghĩa 4.1. Định nghĩa tích chập Dirichlet của hai hàm số học $f(x), g(x)$, ký hiệu $(f * g)(x)$, là

$$(f * g)(n) = \sum_{d \mid n} f(d)g\left(\frac{n}{d}\right).$$

Tích chập Dirichlet có các tính chất phép toán sau:

- Giao hoán: $f * g = g * f$.
- Kết hợp: $(f * g) * h = f * (g * h)$.
- Phân phối: $f * (g + h) = f * g + f * h$.
- Đơn vị: $f * \epsilon = f$.
- Nếu f, g đều là hàm nhân tính thì $f * g$ cũng là hàm nhân tính.

Các tính chất trên chứng minh khá đơn giản, xin để độc giả tự suy nghĩ.

4.2. Đảo Mobius

Bổ đề 4.1. $\mu * 1 = \epsilon$, tức

$$\sum_{d \mid n} \mu(d) = \epsilon(n),$$

trong đó hàm 1 là hàm hằng luôn trả về 1.

Chứng minh. Giả sử n có k loại thừa số nguyên tố khác nhau. Khi đó

$$\sum_{d|n} \mu(d) = \sum_{i=0}^k (-1)^i \binom{k}{i} = (1-1)^k = [k=0] = \epsilon(n).$$

ϵ là hàm đơn vị đã nhắc tới ở Mục 2.2.

Định lý 4.1 (đảo Mobius). Nếu có hai hàm số học f, g thỏa

$$f(n) = \sum_{d|n} g(d),$$

thì chúng cũng thỏa

$$g(n) = \sum_{d|n} \mu(d) f\left(\frac{n}{d}\right).$$

Điều ngược lại cũng đúng, tức

$$f = g * 1 \iff g = \mu * f.$$

Chứng minh. Giả sử

$$f = g * 1.$$

Chập hai vế với hàm μ , ta được

$$\mu * f = \mu * g * 1.$$

Theo Bổ đề 4.1, suy ra

$$\mu * f = \epsilon * g = g.$$

Ngược lại, chập hai vế với hàm 1, ta được

$$g * 1 = \mu * f * 1 = \epsilon * f = f.$$

4.3. Mở rộng của đảo Mobius

Đảo Mobius còn có nhiều dạng mở rộng. Tiếp theo ta giới thiệu một mở rộng về hàm số học.

Bổ đề 4.2. Giả sử x, y, a, b là các số nguyên dương, trong đó $a, b \leq x$, và

$$y = \left\lfloor \frac{x}{a} \right\rfloor.$$

Khi đó

$$\left\lfloor \frac{y}{b} \right\rfloor = \left\lfloor \frac{x}{ab} \right\rfloor.$$

Chứng minh. Đặt $x = kab + c$, trong đó k, c là các số nguyên không âm và $c < ab$. Khi đó $k = \lfloor x/(ab) \rfloor$. Ta có

$$y = \left\lfloor \frac{x}{a} \right\rfloor = kb + \left\lfloor \frac{c}{a} \right\rfloor,$$

và $\lfloor c/a \rfloor < b$, nên $\lfloor y/b \rfloor = k$.

Định lý 4.2. Giả sử f, g là hai hàm số học, t là một hàm hoàn toàn nhân tính và $t(1) = 1$. Khi đó

$$f(n) = \sum_{k=1}^n t(k) g\left(\left\lfloor \frac{n}{k} \right\rfloor\right)$$

khi và chỉ khi

$$g(n) = \sum_{k=1}^n \mu(k) t(k) f\left(\left\lfloor \frac{n}{k} \right\rfloor\right).$$

Chứng minh. Xét thay công thức đầu vào công thức sau. Theo Bổ đề 4.2, ta có

$$\begin{aligned}\sum_{k=1}^n \mu(k)t(k)f\left(\left\lfloor \frac{n}{k} \right\rfloor\right) &= \sum_{k=1}^n \mu(k)t(k) \sum_{i=1}^{\lfloor n/k \rfloor} t(i)g\left(\left\lfloor \frac{n}{ki} \right\rfloor\right) \\ &= \sum_{j=1}^n g\left(\left\lfloor \frac{n}{j} \right\rfloor\right) \sum_{i|j} \mu(i)t(i)t\left(\frac{j}{i}\right) \\ &= \sum_{j=1}^n g\left(\left\lfloor \frac{n}{j} \right\rfloor\right) t(j) \sum_{i|j} \mu(i).\end{aligned}$$

Theo Bổ đề 4.1,

$$\sum_{k=1}^n \mu(k)t(k)f\left(\left\lfloor \frac{n}{k} \right\rfloor\right) = \sum_{j=1}^n g\left(\left\lfloor \frac{n}{j} \right\rfloor\right) t(j)\epsilon(j) = g(n)t(1) = g(n).$$

Chiều ngược lại cũng tương tự.

5. Xây dựng bằng tích chập Dirichlet

Trong thi tin học Trung Quốc, thuật toán này thường được gọi là sàng Du Jiao. Nó có thể hoàn thành hiệu quả phép tính cho một số hàm số học thường gặp.

5.1. Dạng chính

Giả sử $f(n)$ là một hàm số học, cần tính

$$S(n) = \sum_{i=1}^n f(i).$$

Theo tính chất của hàm $f(n)$, ta xây dựng một công thức truy hồi của $S(n)$ theo các giá trị

$$S\left(\left\lfloor \frac{n}{i} \right\rfloor\right),$$

như dưới đây.

Tìm một hàm số học thích hợp $g(n)$:

$$\sum_{i=1}^n \sum_{d|i} f(d)g\left(\frac{i}{d}\right) = \sum_{i=1}^n g(i)S\left(\left\lfloor \frac{n}{i} \right\rfloor\right).$$

Khi đó thu được công thức truy hồi

$$g(1)S(n) = \sum_{i=1}^n (f * g)(i) - \sum_{i=2}^n g(i)S\left(\left\lfloor \frac{n}{i} \right\rfloor\right).$$

Bổ đề 5.1. Với một số nguyên dương n , xét mọi số nguyên dương $i \in [1, n]$. Các giá trị $\lfloor n/i \rfloor$ chỉ có $O(\sqrt{n})$ loại.

Chứng minh. Với mọi $i \leq \sqrt{n}$, ta có $\lfloor n/i \rfloor \geq \sqrt{n}$, nên phần này có $O(\sqrt{n})$ loại giá trị. Với mọi $i > \sqrt{n}$, ta có $\lfloor n/i \rfloor < \sqrt{n}$, nên phần này cũng có $O(\sqrt{n})$ loại giá trị.

Theo Bổ đề 4.2 và Bổ đề 5.1, trong toàn bộ quá trình truy hồi tính $S(n)$, chỉ có $O(\sqrt{n})$ giá trị

$$S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

cần được tính.

Nếu có thể nhanh chóng lấy tổng tiền tố của $(f * g)(i)$ và $g(i)$, ta có thể chia đoạn theo giá trị của $\lfloor n/i \rfloor$. Khi đó tính một $S(n)$ có độ phức tạp $O(\sqrt{n})$, còn toàn bộ quá trình có độ phức tạp

$$\sum_{i=1}^{\lfloor \sqrt{n} \rfloor} O(\sqrt{i}) + \sum_{i=1}^{\lfloor \sqrt{n} \rfloor} O\left(\sqrt{\frac{n}{i}}\right).$$

Phần sau chắc chắn lớn hơn phần trước, nên chỉ cần xét

$$\sum_{i=1}^{\lfloor \sqrt{n} \rfloor} O\left(\sqrt{\frac{n}{i}}\right) \approx O\left(\int_0^{\sqrt{n}} \sqrt{\frac{n}{x}} dx\right) = O(n^{3/4}).$$

Cách xây dựng công thức truy hồi cần phân tích theo hàm cụ thể. Có thể tham khảo các ví dụ sau.

5.2. Ví dụ 2

Ví dụ 2. Tính

$$S(n) = \sum_{i=1}^n \mu(i).$$

Theo Bổ đề 4.1, dùng phương pháp ở Mục 5.1 ta thu được công thức truy hồi

$$S(n) = \sum_{i=1}^n \epsilon(i) - \sum_{i=2}^n S\left(\left\lfloor \frac{n}{i} \right\rfloor\right) = 1 - \sum_{i=2}^n S\left(\left\lfloor \frac{n}{i} \right\rfloor\right).$$

Độ phức tạp tính trực tiếp là $O(n^{3/4})$. Xét việc dùng sàng tuyến tính xử lý trước $n^{1/3}$ hạng đầu, khi đó độ phức tạp của phần truy hồi là

$$O\left(\int_0^{n^{1/3}} \sqrt{\frac{n}{x}} dx\right) = O(n^{2/3}).$$

Kết hợp với phần sàng tuyến tính, tổng độ phức tạp là $O(n^{2/3})$.

5.3. Ví dụ 3

Ví dụ 3 (SPOJ DIVCNT2). Tính

$$S(n) = \sum_{i=1}^n \sigma_0(i^2).$$

Tương tự, xét xây dựng quan hệ tích chập Dirichlet. Đặt

$$f(n) = \sigma_0(n^2), \quad g(n) = 2^{\omega(n)},$$

trong đó $\omega(n)$ biểu thị số lượng thừa số nguyên tố khác nhau của n . Ta có $f = g * 1$, tức

$$\sigma_0(n^2) = \sum_{d|n} 2^{\omega(d)}.$$

Chứng minh. Xét từng ước d của n . Trong d^2 , ta có thể bỏ đi một tập thừa số nguyên tố của d , tức có $2^{\omega(d)}$ cách bỏ; bằng cách này có thể liệt kê mọi ước của n^2 .

Dễ thấy $g = \mu^2 * 1$, do đó ta có các quan hệ truy hồi sau:

$$\begin{aligned}\sum_{i=1}^n \sigma_0(i^2) &= \sum_{i=1}^n 2^{\omega(i)} \left\lfloor \frac{n}{i} \right\rfloor, \\ \sum_{i=1}^n 2^{\omega(i)} &= \sum_{i=1}^n \mu^2(i) \left\lfloor \frac{n}{i} \right\rfloor, \\ \sum_{i=1}^n \mu^2(i) &= \sum_{i=1}^{\lfloor \sqrt{n} \rfloor} \mu(i) \left\lfloor \frac{n}{i^2} \right\rfloor.\end{aligned}$$

Tương tự Ví dụ 2, có thể dùng sàng tuyến tính để hoàn thành tính toán trong $O(n^{2/3})$.¹

5.4. Tiểu kết

Theo Bổ đề 4.2 và Bổ đề 5.1, khi truy hồi tính hạng thứ n , loại công thức này chỉ liên quan tới $O(\sqrt{n})$ trạng thái. Bằng tích phân, có thể tính độ phức tạp của chuyển trạng thái truy hồi là $O(n^{3/4})$; dùng sàng tuyến tính xử lý trước một đoạn đầu có thể tiếp tục hạ độ phức tạp.

Lấy tư tưởng này làm nền tảng, ta có thể xây dựng thuật toán trong mục sau. Về nội dung của mục này, có thể xem tài liệu tham khảo [2] để biết thêm chi tiết; bài viết này không triển khai sâu hơn.

6. Một phương pháp tính tổng dựa trên phân tích thừa số nguyên tố

Trong nhiều trường hợp, ta không thể dễ dàng tìm được tính chất của hàm cần tính tổng. Khi đó ta hy vọng có một phương pháp tính tổng hiệu quả với phạm vi áp dụng rộng hơn. Tiếp theo bài viết đưa ra một thuật toán tính tổng cho các hàm nhân tính có dạng tương đối tổng quát.

6.1. Nhắc lại

Giả sử $F(n)$ là một hàm nhân tính, và phân tích thừa số nguyên tố của n là

$$n = \prod_{i=1}^k p_i^{c_i}.$$

Xét cách mô tả một hàm nhân tính như sau:

- Khi n là số nguyên tố, tức $n = p$, $F(p) = G(p)$.
- Khi n là lũy thừa của một số nguyên tố, tức $n = p^c$ với $c > 1$, $F(p^c) = T(p^c)$.
- Các trường hợp còn lại tính theo tính nhân tính:

$$F(n) = \prod_{i=1}^k F(p_i^{c_i}).$$

Ví dụ, với hai hàm nhân tính thường gặp sau:

- Với hàm Euler $\varphi(n)$, $G(p) = p - 1$ và $T(p^c) = (p - 1)p^{c-1}$.
- Với hàm Mobius $\mu(n)$, $G(p) = -1$ và $T(p^c) = 0$.

¹Công thức tính tổng của μ^2 có thể nhận được từ bao hàm-loại trừ.

Tổng quát hơn, $G(p)$ có thể là một đa thức theo p , còn $T(p^c)$ có thể là một đa thức theo p và c . Với các trường hợp này, hoặc với những biểu thức phức tạp hơn, thuật toán ở mục trước có thể không còn phù hợp. Ta xét không dựa vào tính chất riêng của hàm nữa, mà trực tiếp hoàn thành tính toán bằng tính nhân tính.

6.2. Đưa vào bài toán

Lấy một dạng đơn giản của loại bài toán này làm ví dụ.

Ví dụ 4. Định nghĩa một biến thể $\phi(n, d)$ của hàm Euler. Phân tích thừa số nguyên tố của n là

$$n = \prod_{i=1}^k p_i^{c_i},$$

trong đó p_i là các thừa số nguyên tố đôi một khác nhau và $c_i > 0$. Với trường hợp $n = 1$, ta hiểu tích sau là tích rỗng. Đặt

$$\phi(n, d) = \prod_{i=1}^k (p_i^{c_i} + d).$$

Đặc biệt, định nghĩa $\phi(1, d) = 1$. Với n, d cho trước, hãy tính

$$\left(\sum_{i=1}^n \phi(i, d) \right) \bmod (10^9 + 7).$$

Đặt $F(n) = \phi(n, d)$. Theo cách biểu diễn ở Mục 6.1, ta có

$$G(p) = p + d, \quad T(p^c) = p^c + d.$$

6.3. Biến đổi bước đầu

Bổ đề 6.1. Với một số nguyên dương $x \leq n$, x có nhiều nhất một thừa số nguyên tố lớn hơn $\sqrt{n}$.

Chứng minh. Giả sử x có hai thừa số nguyên tố p_1, p_2 lớn hơn $\sqrt{n}$. Khi đó $p_1 p_2 > n$, mâu thuẫn với $x \leq n$.

Theo Bổ đề 6.1, có thể chia $F(x)$ thành hai loại để xét:

- x không có thừa số nguyên tố lớn hơn $\sqrt{n}$;
- x có một thừa số nguyên tố lớn hơn $\sqrt{n}$.

Theo tính nhân tính của hàm, ta có

$$\sum_{i=1}^n F(i) = \sum_{\substack{x \leq n \\ x \text{ không có thừa số nguyên tố} > \sqrt{n}}} F(x) \left(1 + \sum_{\substack{\sqrt{n} < p \leq \lfloor n/x \rfloor \\ p \text{ nguyên tố}}} G(p) \right).$$

Hệ số nhân với $F(x)$ chỉ liên quan tới giá trị $\lfloor n/x \rfloor$. Vì vậy chỉ có $O(\sqrt{n})$ loại hệ số khác nhau, và ta có thể phân $F(x)$ thành $O(\sqrt{n})$ nhóm theo giá trị $\lfloor n/x \rfloor$, mỗi nhóm ứng với một loại hệ số.

Đặt y là một giá trị cố định của $\lfloor n/x \rfloor$. Khi đó cần tính hai phần

$$\sum_{\substack{\sqrt{n} < p \leq y \\ p \text{ nguyên tố}}} G(p), \quad \sum_{\substack{\lfloor n/x \rfloor = y \\ x \text{ không có thừa số nguyên tố} > \sqrt{n}}} F(x).$$

6.4. Tính $G(p)$

6.4.1. Phân tích bước đầu. Trong công thức của Ví dụ 4, $G(p) = p + d$. Không mất tính tổng quát, giả sử $G(p)$ là một đa thức bậc thấp theo p , và xét tính riêng từng hạng của đa thức.

Giả sử các số nguyên tố không vượt quá $\sqrt{n}$ có tổng cộng m số, sắp theo thứ tự tăng dần là $p_1, \dots, p_m$. Đặt $g_k[i][j]$ là tổng lũy thừa bậc k của mọi số trong đoạn $[1, j]$ nguyên tố cùng nhau với i số nguyên tố đầu. Phần $i = 0$ có thể tính bằng nội suy trong $O(k)$.

Với $i \geq 1$, xét trong đoạn $[1, j]$ có bao nhiêu số chứa thừa số nguyên tố p_i và nguyên tố cùng nhau với $i - 1$ số nguyên tố trước. Đặt

$$l = \left\lfloor \frac{j}{p_i} \right\rfloor.$$

Ta có truy hồi

$$g_k[i][j] = g_k[i-1][j] - p_i^k g_k[i-1][l].$$

Giá trị cuối cùng $g_k[m][j] - 1$ chính là tổng lũy thừa bậc k của các số nguyên tố lớn hơn $\sqrt{n}$ trong đoạn $[1, j]$. Theo Bổ đề 4.2 và Bổ đề 5.1, trong quá trình tính $g_k[m][n]$, số trạng thái chiều thứ hai cần dùng là $O(\sqrt{n})$, và các trạng thái này đúng là từng loại tổng $G(p)$ cần tính.

6.4.2. Cài đặt thô. Cài đặt thô công thức truy hồi này cần lần lượt liệt kê số nguyên tố p_i và tính trên từng trạng thái.

Xét với mỗi giá trị $\lfloor n/x \rfloor$ có bao nhiêu số nguyên tố cần chuyển. Phần $\lfloor n/x \rfloor > \sqrt{n}$ cần chuyển mọi số nguyên tố trong phạm vi $\sqrt{n}$, nên

$$\sum_{i=1}^{\lfloor \sqrt{n} \rfloor} O\left(\frac{i}{\log i}\right) + \sum_{i=1}^{\lfloor \sqrt{n} \rfloor} O\left(\frac{\sqrt{n}}{\log \sqrt{n}}\right) \approx O\left(\frac{n}{\log n}\right).$$

6.4.3. Tối ưu. Khi $p_{i+1} > j$, chắc chắn $g_k[i][j] = 1$. Vì vậy nếu $p_i^2 > j \geq p_i$, tức

$$l = \left\lfloor \frac{j}{p_i} \right\rfloor < p_i,$$

thì

$$g_k[i-1][l] = 1, \quad g_k[i][j] = g_k[i-1][j] - p_i^k.$$

Do đó thực ra có thể bỏ qua tính toán khi $p_i^2 > j$. Với mỗi j , ghi lại giá trị i ở lần chuyển cuối cùng; khi gọi vị trí $g_k[i][j]$, cộng bù đóng góp của đoạn số nguyên tố chưa được tính.

Với mỗi giá trị $\lfloor n/x \rfloor$, chỉ cần chuyển các số nguyên tố không vượt quá $\sqrt{\lfloor n/x \rfloor}$. Độ phức tạp có thể ước lượng là

$$\sum_{i=1}^{\lfloor \sqrt{n} \rfloor} O\left(\frac{\sqrt{i}}{\log i}\right) + \sum_{i=1}^{\lfloor \sqrt{n} \rfloor} O\left(\frac{\sqrt{n/i}}{\log \sqrt{n/i}}\right).$$

Đóng góp của phần sau chắc chắn lớn hơn phần trước, và $\sqrt{n/i} \geq n^{1/4}$, nên có thể ước lượng độ phức tạp bởi

$$\sum_{i=1}^{\lfloor \sqrt{n} \rfloor} O\left(\frac{\sqrt{n/i}}{\log \sqrt{n/i}}\right) \approx O\left(\frac{\int_0^{\sqrt{n}} \sqrt{n/x} dx}{\log n}\right) = O\left(\frac{n^{3/4}}{\log n}\right).$$

6.5. Tính $F(x)$

6.5.1. Phân tích bước đầu. Vì các $F(x)$ có cùng giá trị $\lfloor n/x \rfloor$ cần nhân cùng một hệ số, có thể trực tiếp dùng $\lfloor n/x \rfloor$ để biểu diễn trạng thái.

Tương tự, giả sử các số nguyên tố không vượt quá $\sqrt{n}$ có tổng cộng m số, sắp theo thứ tự tăng dần là $p_1, \dots, p_m$. Đặt $f[i][j]$ là tổng các $F(x)$ sao cho x chỉ chứa i loại thừa số nguyên tố đầu và $\lfloor n/x \rfloor = j$. Trong đó $f[0][n] = 1$.

Theo Bổ đề 4.2 và Bổ đề 5.1, số trạng thái chiều thứ hai vẫn có thể thu gọn xuống $O(\sqrt{n})$; mỗi loại ứng với một nhóm hệ số.

6.5.2. Cài đặt thô. Xét chuyển từ $f[i-1][j]$ bằng số nguyên tố p_i . Liệt kê số mũ chuyển c , đặt

$$l = \left\lfloor \frac{j}{p_i^c} \right\rfloor.$$

Khi đó cộng vào $f[i][l]$ giá trị

$$T(p_i^c) f[i-1][j],$$

hoặc $G(p_i) f[i-1][j]$ trong trường hợp $c = 1$.

Tương tự như trước, tính xem mỗi giá trị $\lfloor n/x \rfloor$ có bao nhiêu số nguyên tố cần chuyển. Xét lượng tính toán khi liệt kê từng loại c , đặt

$$h(n) = \sum_{k=2}^{\lfloor \log_2 n \rfloor} O(n^{1/k}) \approx O(\sqrt{n}).$$

Ước lượng độ phức tạp tương tự Mục 6.4:

$$\sum_{i=1}^{\lfloor \sqrt{n} \rfloor} O\left(\frac{i + h(i)}{\log i}\right) + \sum_{i=1}^{\lfloor \sqrt{n} \rfloor} O\left(\frac{\sqrt{n} + h(n/i)}{\log n}\right) \approx O\left(\frac{n}{\log n}\right).$$

6.5.3. Tối ưu. Ở tiểu mục trước, bằng cách bỏ qua tính toán khi $p_i^2 > j$, ta hạ độ phức tạp xuống $O(n^{3/4} / \log n)$. Xét áp dụng tối ưu tương tự.

Chú ý rằng khi $\lfloor n/x \rfloor$ không vượt quá $\sqrt{n}$,

$$1 + \sum_{\substack{\sqrt{n} < p \leq \lfloor n/x \rfloor \\ p \text{ nguyên tố}}} G(p) = 1.$$

Mà công thức tính đáp án cuối cùng là

$$\sum_{i=1}^n F(i) = \sum_{\substack{x \leq n \\ x \text{ không có thừa số nguyên tố} > \sqrt{n}}} F(x) \left(1 + \sum_{\substack{\sqrt{n} < p \leq \lfloor n/x \rfloor \\ p \text{ nguyên tố}}} G(p) \right).$$

Hệ số đóng góp của phần $F(x)$ này vào đáp án là giống nhau, nên có thể dùng đó làm điểm đột phá để tối ưu.

Khi $p_i^2 > y = \lfloor n/x \rfloor$, ta có $\lfloor y/p_i \rfloor < p_i \leq \sqrt{n}$, nên trạng thái y nhiều nhất chỉ có thể chuyển bằng một số nguyên tố vượt quá p_i , và giá trị sau khi chuyển chắc chắn có hệ số đóng góp vào đáp án bằng 1.

Chiến lược chuyển sau khi tối ưu như sau:

- Với một số nguyên tố p_i , chỉ chuyển các trạng thái $y \geq p_i^2$.

- Đặt $l = \lfloor y/p_i^c \rfloor$. Nếu $l < p_i^2$, thì trạng thái này sẽ bị bỏ qua trong các lần chuyển sau; cần tính ngay đóng góp của nó vào đáp án, tức thống kê tổng $G(p)$ trong đoạn $[p_{i+1}, l]$.
- Duy trì với mỗi trạng thái số nguyên tố p_i ở lần chuyển cuối cùng, rồi thống kê tổng $G(p)$ của đoạn bị bỏ qua.

Độ phức tạp được ước lượng tương tự Mục 6.4:

$$\sum_{i=1}^{\lfloor \sqrt{n} \rfloor} O\left(\frac{h(i)}{\log i}\right) + \sum_{i=1}^{\lfloor \sqrt{n} \rfloor} O\left(\frac{h(n/i)}{\log n}\right) \approx O\left(\frac{n^{3/4}}{\log n}\right).$$

6.5.4. Một cách cài đặt khác. Chú ý rằng chỉ khi $x < \sqrt{n}$, ta mới có thể tiếp tục nhân thêm một số nguyên tố lớn hơn $\sqrt{n}$; phần x này cũng không thể chứa bất kỳ thừa số nguyên tố nào lớn hơn $\sqrt{n}$.

Có thể trước hết dùng sàng tuyến tính để xử lý giá trị $F(x)$ của mọi $x < \sqrt{n}$, nhân hệ số tương ứng rồi cộng vào đáp án. Phần còn lại là tính

$$\sum_{\substack{\sqrt{n} \leq x \leq n \\ x \text{ không có thừa số nguyên tố} > \sqrt{n}}} F(x).$$

Khác với mục trước, sắp các số nguyên tố không vượt quá $\sqrt{n}$ theo thứ tự giảm dần là $p_1, \dots, p_m$. Đặt $f'[i][j]$ là tổng các $F(x)$ sao cho chỉ chứa i loại thừa số nguyên tố đầu và $x \leq j$. Để có $f'[0][j] = 1$; giá trị $f'[m][n]$ sau khi trừ tổng các $F(x)$ với $x < \sqrt{n}$ chính là kết quả cần tìm.

Để thu được một chuyển truy hồi: liệt kê số mũ c của thừa số nguyên tố p_i , đặt $l_c = \lfloor j/p_i^c \rfloor$, ta có

$$f'[i][j] = f'[i-1][j] + G(p_i)f'[i-1][l_1] + \sum_{c \geq 2} T(p_i^c)f'[i-1][l_c].$$

Tham khảo mục trước, chiều thứ hai của truy hồi này cũng chỉ có $O(\sqrt{n})$ trạng thái. Cài đặt thô có độ phức tạp $O(n/\log n)$; xét áp dụng tối ưu tương tự.

Khi $p_i^2 > j$, vì p_i được sắp theo thứ tự giảm dần, chỉ cần thống kê tổng giá trị hàm của các số nguyên tố trong đoạn $[p_i, j]$, nên vẫn có thể bỏ qua tính toán của phần này và hạ độ phức tạp xuống

$$O\left(\frac{n^{3/4}}{\log n}\right).$$

So sánh hai cách cài đặt, khác biệt chính nằm ở thứ tự liệt kê thừa số nguyên tố. Có thể chọn theo tình huống cụ thể.²

6.6. Mở rộng

Nhắc lại vài bước biến đổi chính trong thuật toán này:

- Theo Bổ đề 6.1, dùng tính nhân tính để chia thừa số nguyên tố của mỗi số thành hai phần để xét.
- Theo Bổ đề 4.2 và Bổ đề 5.1, thu nhỏ chiều thứ hai của truy hồi xuống $O(\sqrt{n})$ trạng thái.
- Phân tích tính chất của truy hồi, rồi bỏ qua tính toán khi $p_i^2 > j$ để hạ độ phức tạp xuống $O(n^{3/4}/\log n)$.

Thực chất, các bước trên đã tách thừa số nguyên tố cho mọi số, và khi truy hồi thì các thừa số nguyên tố được đưa vào có thứ tự. Vì vậy thuật toán này có thể hoàn thành một số chức năng của sàng.

²Tác giả cảm ơn Mao Xiao đã cung cấp cách cài đặt này.

6.6.1. Ví dụ 5. Ví dụ 5 (UR13C Sanrd).

Tính tổng thừa số nguyên tố lớn thứ hai của mọi hợp số không vượt quá n . Ở đây thừa số nguyên tố lớn thứ hai được định nghĩa như sau: giả sử

$$n = \prod_{i=1}^m p_i,$$

trong đó các p_i đều là số nguyên tố và $p_i \leq p_{i+1}$. Thừa số nguyên tố lớn thứ hai là p_{m-1} ; nếu $m < 2$ thì không tính.

Tương tự, chia thừa số nguyên tố thành hai phần theo ngưỡng $\sqrt{n}$, và theo Bổ đề 6.1, thừa số nguyên tố lớn thứ hai của $x \leq n$ chắc chắn không vượt quá $\sqrt{n}$.

Nhắc lại thuật toán đầu tiên trong Mục 6.4 và Mục 6.5. Quy trình tạo thành mỗi số x là: trước hết thêm các thừa số nguyên tố không vượt quá $\sqrt{n}$ của x theo thứ tự tăng dần, cuối cùng mới tính thừa số nguyên tố lớn hơn $\sqrt{n}$ của x . Tức trong toàn bộ quá trình, các thừa số nguyên tố của x được liệt kê theo thứ tự tăng dần.

Khi tính các thừa số nguyên tố không vượt quá $\sqrt{n}$, có thể xem mỗi trạng thái là đang duy trì thông tin của một tập số. Một lần chuyển là nhân mọi số trong tập đó với thừa số nguyên tố đang được liệt kê, thu được một tập mới và nhập vào trạng thái đích. Vì thừa số nguyên tố được liệt kê có thứ tự, mỗi trạng thái sẽ không có số bị tính lặp, và còn bảo đảm các tập số giữa các trạng thái không giao nhau.

Dễ thấy khi chuyển, thừa số nguyên tố vừa thêm chắc chắn là thừa số nguyên tố lớn nhất của số mới sinh; thừa số nguyên tố lớn nhất của trạng thái được chuyển chính là thừa số nguyên tố lớn thứ hai. Vì vậy chỉ cần ghi tổng thừa số nguyên tố lớn nhất của tập số mà mỗi trạng thái biểu diễn, ta có thể thu được thừa số nguyên tố lớn thứ hai.

6.7. Tổng kết

Thuật toán này là nội dung chính bài viết muốn trình bày. Nó giải quyết bài toán tính tổng hàm nhân tính với dạng tương đối tổng quát. Các ràng buộc chính đối với hàm ban đầu như sau:

- Hàm cần tính tổng $F(n)$ phải là hàm nhân tính.
- $G(p)$ có thể phân tích thành tổng của một số hàm hoàn toàn nhân tính; do bước khởi tạo mảng truy hồi cần điều này, tổng tiền tố của các hàm hoàn toàn nhân tính ấy phải tính nhanh được. Một ví dụ đơn giản là $G(p)$ là đa thức bậc thấp theo p .
- Do chuyển ở Mục 6.5 yêu cầu, hệ số chuyển $G(p)$ và $T(p^c)$ phải tính nhanh được.

Các chuyển truy hồi trong Mục 6.4 và Mục 6.5 đều có thể dùng mảng cuộn; số nguyên tố cũng chỉ cần xử lý trước trong phạm vi $\sqrt{n}$, nên độ phức tạp không gian là $O(\sqrt{n})$.

Thuật toán này có thể giúp ta tính tổng đa số hàm nhân tính trong

$$O\left(\frac{n^{3/4}}{\log n}\right),$$

và có ưu thế lớn với những bài toán tính tổng hàm nhân tính không có tính chất đặc biệt hoặc khó suy luận.

Lời cảm ơn

Xin cảm ơn Hiệp hội Máy tính đã cung cấp nền tảng học tập và giao lưu.

Xin cảm ơn các thầy Chen Heli và Dong Yehua của Trường Trung học số 1 Thiệu Hưng vì sự quan tâm và chỉ dẫn trong nhiều năm.

Xin cảm ơn các anh Yu Yili, Dong Honghua, Zhang Hengjie và Wang Jianhao của Đại học Thanh Hoa vì đã giúp đỡ tôi.

Xin cảm ơn bạn Mao Xiao đã thảo luận thuật toán này với tôi và cung cấp một cách cài đặt ngắn gọn.

Xin cảm ơn các thầy cô và bạn học khác từng giúp đỡ và gợi mở cho tôi.

Tài liệu tham khảo

1. Jia Zhipeng, *Sàng tuyển tính và hàm nhân tính*, trao đổi học viên WC2012.
2. Ji Ruyi, <http://jiruyi910387714.i11r.com/posts/195270.html>, jiry_2's Blog.
3. Project Euler forum, Problem 521.

Bài 2. Một số phương pháp mô hình hóa bằng luồng mạng

Jiang Zhihao, Trường Trung học số 1 Shengli, Đông Doanh

Tóm tắt

Luồng mạng có phạm vi ứng dụng rộng trong thi tin học, và nhiều mô hình luồng mạng được xây dựng bằng những cách rất tinh tế. Bài viết này tổng kết một số phương pháp mô hình hóa luồng mạng thường dùng.

1. Dẫn nhập

Luồng mạng là một phương pháp giải quyết vấn đề dựa trên phép tương tự với dòng chảy. Trong thi tin học, nó xuất hiện rất thường xuyên. Các bài toán thường gặp gồm luồng cực đại, luồng cực đại chi phí nhỏ nhất, luồng có chặn trên và chặn dưới, v.v. Điểm tinh tế của bài toán luồng mạng thường không nằm ở quá trình cài đặt thuật toán, mà nằm ở cách mô hình hóa bài toán thành một mạng luồng.

Bài viết này tổng kết một số cách mô hình hóa luồng mạng tương đối thông dụng, chia thành bốn nhóm: mô hình hóa từ góc độ luồng cực đại, từ góc độ cắt nhỏ nhất, từ góc độ luồng chi phí, và từ tư tưởng cân bằng luồng. Hy vọng các phương pháp này ít nhiều có thể giúp ích cho các bạn tham gia thi tin học.

2. Mô hình hóa từ góc độ luồng cực đại

Nói chung, mô hình hóa từ góc độ luồng cực đại là trực quan nhất. Ta thường dùng một đơn vị luồng đi từ nguồn S tới đích T để biểu diễn một phương án. Chẳng hạn, khi dùng luồng cực đại để tìm khớp cực đại trong đồ thị hai phía, một đường tăng $S \rightarrow T$ biểu diễn một cặp ghép.

2.1. Ví dụ mô hình hóa

Ví dụ 1 (Soldier Occupation). Cho một bàn cờ $n \times m$, một số ô là chướng ngại vật. Ta cần chọn một số ô để đặt lính; mỗi ô nhiều nhất đặt một lính và ô chướng ngại vật không được đặt lính. Gọi một cách

đặt lính là chiếm được toàn bộ bàn cờ nếu hàng thứ i có ít nhất r_i lính và cột thứ j có ít nhất c_j lính. Hãy dùng ít lính nhất để chiếm bàn cờ.

$$1 \leq n, m \leq 100.$$

Phương án tệ nhất là đặt riêng đủ số lính cho từng hàng, rồi lại đặt riêng đủ số lính cho từng cột. Khi đó mỗi lính chỉ có đóng góp 1. Một số lính có thể vừa đóng góp cho hàng vừa đóng góp cho cột, tức đóng góp 2. Số lính loại này càng nhiều thì tổng số lính cần dùng càng ít, nên ta xét cách làm cho số lính có đóng góp 2 lớn nhất.

Số lính đóng góp 2 trên hàng i không thể vượt quá r_i , nếu không sẽ có lính không cần thiết cho hàng đó. Tương tự, số lính đóng góp 2 trên cột j không thể vượt quá c_j . Ta dựng đồ thị như sau:

- Với mỗi hàng lập một đỉnh A_i , nối $S \rightarrow A_i$ với dung lượng r_i .
- Với mỗi cột lập một đỉnh B_j , nối $B_j \rightarrow T$ với dung lượng c_j .
- Nếu ô (i, j) có thể đặt lính, nối $A_i \rightarrow B_j$ với dung lượng 1.

Trong đồ thị này, một lính có đóng góp 2 tương ứng với một đơn vị luồng $S \rightarrow T$. Dung lượng của cạnh giữa hàng và cột bảo đảm một ô nhiều nhất có một lính, còn dung lượng cạnh kề nguồn và kề đích bảo đảm số lính đóng góp 2 của mỗi hàng, mỗi cột không vượt quá yêu cầu. Vì vậy, luồng cực đại là số lính đóng góp 2 lớn nhất, và số lính tối thiểu bằng

$$\sum_i r_i + \sum_j c_j - \text{maxflow}.$$

Ví dụ 2 (Dining). Có f loại thức ăn và d loại đồ uống. Mỗi loại thức ăn hoặc đồ uống chỉ có thể phục vụ một con bò, và mỗi con bò chỉ dùng một loại thức ăn cùng một loại đồ uống. Có n con bò; mỗi con bò có danh sách các loại thức ăn và đồ uống nó thích. Hỏi nhiều nhất có bao nhiêu con bò đồng thời được dùng thức ăn và đồ uống mà nó thích.

$$1 \leq n, f, d \leq 100.$$

Ta dùng một đơn vị luồng $S \rightarrow T$ để biểu diễn việc thỏa mãn một con bò. Dựng đồ thị như sau:

- Với mỗi loại thức ăn lập đỉnh A_i , nối $S \rightarrow A_i$ dung lượng 1. Với mỗi loại đồ uống lập đỉnh B_i , nối $B_i \rightarrow T$ dung lượng 1.
- Với mỗi con bò lập hai đỉnh C_i, D_i , nối $C_i \rightarrow D_i$ dung lượng 1.
- Nếu bò i thích thức ăn j , nối $A_j \rightarrow C_i$ dung lượng 1. Nếu bò i thích đồ uống j , nối $D_i \rightarrow B_j$ dung lượng 1.

Việc tách một con bò thành hai đỉnh và đặt cạnh ở giữa có dung lượng 1 nhằm giới hạn mỗi con bò chỉ được thỏa mãn một lần.

Một số bài toán không thể mô hình hóa trực tiếp. Khi đó cần phân tích đề bài, biến đổi nó thành dạng có thể giải bằng luồng mạng.

Ví dụ 3 (Collector's Problem). Bob và $n - 1$ người bạn cùng sưu tầm nhãn dán. Có m loại nhãn dán khác nhau. Mỗi người có một số nhãn dán, có thể có nhãn dán trùng lặp, và bạn của Bob chỉ chịu đổi lấy loại nhãn dán mà họ chưa có. Mỗi lần đổi là một đổi một.

Bob thông minh hơn các bạn, vì cậu nhận ra rằng nếu chỉ đổi lấy loại mình chưa có thì chưa chắc tối ưu. Trong một số trường hợp, đổi về một nhãn dán trùng lặp lại có lợi hơn. Giả sử các bạn chỉ đổi với

Bob, không đổi với nhau, và họ chỉ đưa ra những nhãn dán thừa để đổi lấy loại khác mà họ chưa có. Hãy tính số loại nhãn dán khác nhau lớn nhất Bob có thể có cuối cùng.

$$2 \leq n \leq 10, \quad 5 \leq m \leq 25.$$

Với một người bạn F , Bob chỉ có thể đưa cho F một loại nhãn dán mà F chưa có, và mỗi loại nhiều nhất đưa một lần. Nếu F có $k \geq 2$ bản sao của một loại nhãn dán, F nhiều nhất có thể đưa Bob $k - 1$ tấm loại đó. Vì vậy vai trò của một người bạn là biến một loại nhãn dán trong tay Bob thành một loại nhãn dán khác.

Dựng đồ thị như sau:

- Với mỗi loại nhãn dán lập đỉnh A_i . Nối $S \rightarrow A_i$ với dung lượng bằng số nhãn dán loại i ban đầu Bob có. Nối $A_i \rightarrow T$ với dung lượng 1, biểu diễn việc cuối cùng Bob chỉ cần giữ một tấm của mỗi loại.
- Với mỗi người bạn lập đỉnh B_j . Nếu bạn j không có nhãn dán loại i , nối $A_i \rightarrow B_j$ dung lượng 1, biểu diễn Bob đưa loại đó cho bạn ấy. Nếu bạn j có $k \geq 2$ nhãn dán loại i , nối $B_j \rightarrow A_i$ dung lượng $k - 1$, biểu diễn số nhãn dán thừa loại đó có thể đưa lại cho Bob.

Một đơn vị luồng $S \rightarrow T$ trước hết đi từ S tới một đỉnh nhãn dán A_i , biểu diễn một nhãn dán ban đầu của Bob. Sau đó nó đi qua không hoặc nhiều đoạn $A \rightarrow B \rightarrow A$, biểu diễn các lần đổi với bạn. Cuối cùng nó đi từ một đỉnh nhãn dán tới T , biểu diễn loại nhãn dán Bob giữ được sau khi đổi.

2.2. Tiêu kết

Đặc điểm của mô hình luồng cực đại là trực quan, dễ hiểu. Một đơn vị luồng $S \rightarrow T$ thường có ý nghĩa thực tế rõ ràng, biểu diễn một phương án hoặc một chuỗi thao tác. Tuy nhiên biến thể của bài toán luồng cực đại rất nhiều. Có những lúc phải phân tích kỹ bản chất bài toán, đơn giản hóa hoặc chuyển hóa nó, rồi mới tìm được mô hình luồng phù hợp.

3. Mô hình hóa từ góc độ cắt nhỏ nhất

3.1. Dùng cạnh dung lượng vô cùng để biểu diễn xung đột

Ví dụ 4 (NOI2006, Maximum Profit). Công ty truyền thông có n địa điểm có thể xây trạm trung chuyển tín hiệu. Xây trạm thứ i tốn chi phí p_i . Có m nhóm khách hàng tiềm năng; nhóm thứ i dùng hai trạm a_i, b_i để liên lạc và đem lại lợi nhuận c_i . Công ty có thể chọn một số trạm để xây, rồi phục vụ những nhóm khách hàng có đủ hai trạm cần thiết. Hãy tối đa hóa lãi ròng, tức tổng lợi nhuận thu được trừ tổng chi phí xây trạm.

$$1 \leq n \leq 5000, \quad 1 \leq m \leq 50000.$$

Xem trạm và nhóm khách hàng là các đỉnh. Với trạm i , lập đỉnh A_i và nối $S \rightarrow A_i$ dung lượng p_i ; cắt cạnh này nghĩa là chịu chi phí xây trạm i . Với nhóm khách hàng i , lập đỉnh B_i và nối $B_i \rightarrow T$ dung lượng c_i ; cắt cạnh này nghĩa là không phục vụ nhóm đó và mất lợi nhuận c_i .

Nếu nhóm khách hàng i dùng trạm j , thì cạnh $S \rightarrow A_j$ và cạnh $B_i \rightarrow T$ không được đồng thời giữ lại trong cắt theo cách mâu thuẫn với việc phục vụ nhóm đó. Ta nối $A_j \rightarrow B_i$ bằng cạnh dung lượng $+\infty$. Như vậy cắt nhỏ nhất không thể chọn phương án vừa giữ lợi nhuận của khách hàng vừa không trả chi phí xây trạm cần thiết.

Tổng lợi nhuận của mọi nhóm khách hàng trừ đi giá trị cắt nhỏ nhất chính là lãi ròng lớn nhất. Trong bài toán này, cạnh dung lượng vô cùng giúp các lợi ích xung đột không thể đồng thời được bảo lưu. Cạnh dung lượng vô cùng cũng có thể dùng để hạn chế nhiều loại xung đột khác.

Ví dụ 5 (Codechef Dec14, Course Selection). Một sinh viên có n môn học, mỗi môn phải hoàn thành trong một trong m học kỳ. Có k quan hệ tiên quyết; a là môn tiên quyết của b nghĩa là học kỳ hoàn thành a phải đứng trước học kỳ hoàn thành b . Điểm của môn i nếu hoàn thành ở học kỳ j là $x_{i,j}$; nếu $x_{i,j} = -1$ thì học kỳ j không mở môn i . Hãy tối đa hóa điểm trung bình.

$$1 \leq n, m \leq 100, \quad 0 \leq k \leq 100, \quad -1 \leq x_{i,j} \leq 100.$$

Điểm trung bình lớn nhất tương đương tổng điểm lớn nhất. Giả sử điểm tối đa là 100, ta chuyển sang tối thiểu hóa tổng điểm bị trừ. Gọi $y_{i,j}$ là số điểm bị trừ khi học môn i ở học kỳ j . Nếu $x_{i,j} = -1$ thì $y_{i,j} = +\infty$; ngược lại $y_{i,j} = 100 - x_{i,j}$.

Với mỗi cặp môn học kỳ (i, j) , lập một đỉnh. Dựng một chuỗi cho từng môn:

- Nối $S \rightarrow (i, 1)$ dung lượng $y_{i,1}$.
- Với $2 \leq j \leq m$, nối $(i, j-1) \rightarrow (i, j)$ dung lượng $y_{i,j}$.
- Nối $(i, m) \rightarrow T$ dung lượng $+\infty$.

Cắt cạnh đi vào (i, j) biểu diễn môn i được học ở học kỳ j . Cắt nhỏ nhất của các chuỗi trên chính là tổng điểm bị trừ nhỏ nhất nếu tạm bỏ qua các quan hệ tiên quyết.

Nếu a là môn tiên quyết của b , vị trí cắt của chuỗi a phải đứng trước vị trí cắt của chuỗi b . Ta thêm các cạnh dung lượng $+\infty$:

$$S \rightarrow (b, 1), \quad (a, j-1) \rightarrow (b, j) \quad (2 \leq j \leq m).$$

Các cạnh này không xuất hiện trong cắt nhỏ nhất, nên chúng ép phương án thỏa mãn mọi ràng buộc tiên quyết. Bản chất ở đây là dùng cạnh vô cùng để cấm đồng thời hai điều kiện: “ S liên thông với đầu cạnh” và “cuối cạnh liên thông với T ”.

3.2. Mô hình cắt nhỏ nhất từ quan hệ giữa hai điểm

Ví dụ 6 (happiness). Sơ đồ chỗ ngồi của lớp là một lưới $n \times m$. Sau một học kỳ, hai học sinh ngồi kề nhau theo bốn hướng trở thành bạn tốt. Mỗi học sinh chọn bạn tự nhiên sẽ có độ vui a_i , chọn bạn xã hội sẽ có độ vui b_i . Với một cặp bạn tốt, nếu cả hai cùng chọn tự nhiên, họ được thêm độ vui chung v_2 ; nếu cùng chọn xã hội, họ được thêm độ vui chung v_1 . Hãy phân ban để tổng độ vui của cả lớp lớn nhất.

$$1 \leq n, m \leq 100.$$

Xét trước quan hệ của hai học sinh kề nhau x, y . Giả sử giữ cạnh nối với S biểu diễn chọn tự nhiên, giữ cạnh nối với T biểu diễn chọn xã hội. Với cặp x, y , gọi v_1 là lợi ích nếu cả hai cùng chọn xã hội, v_2 là lợi ích nếu cả hai cùng chọn tự nhiên. Ta cần các dung lượng sao cho mất mát của bốn trường hợp là:

$$\begin{aligned} a + b &= v_1 & (x, y \text{ đều chọn xã hội}), \\ c + d &= v_2 & (x, y \text{ đều chọn tự nhiên}), \\ a + d + e &= v_1 + v_2 & (x \text{ chọn xã hội, } y \text{ chọn tự nhiên}), \\ b + c + e &= v_1 + v_2 & (x \text{ chọn tự nhiên, } y \text{ chọn xã hội}). \end{aligned}$$

Lấy hai phương trình sau trừ hai phương trình đầu được

$$2e = v_1 + v_2, \quad e = \frac{v_1 + v_2}{2}, \quad a = b = \frac{v_1}{2}, \quad c = d = \frac{v_2}{2}.$$

Sau khi giải được dung lượng, cộng thêm lợi ích riêng của từng học sinh, mô hình là:

- Với mỗi học sinh lập một đỉnh. Nối S tới đỉnh đó với dung lượng bằng lợi ích chọn tự nhiên của học sinh này, cộng một nửa tổng lợi ích chung khi học sinh này và từng bạn kề nó cùng chọn tự nhiên.
- Nối đỉnh đó tới T với dung lượng bằng lợi ích chọn xã hội, cộng một nửa tổng lợi ích chung khi học sinh này và từng bạn kề nó cùng chọn xã hội.
- Với hai học sinh kề nhau x, y , nối một cạnh hai chiều giữa hai đỉnh với dung lượng bằng trung bình cộng của lợi ích “cùng xã hội” và “cùng tự nhiên” của cặp đó.

Tổng mọi lợi ích trừ đi cắt nhỏ nhất chính là tổng độ vui lớn nhất. Bài toán này có lợi ích liên quan nhiều nhất tới hai người. Vì vậy ta phân tích trước quan hệ giữa hai người, rồi tổng hợp mọi quan hệ vào đồ thị.

Ví dụ 7 (Google Code Jam 2008 Final E, The Year of Code Jam). Sphinnny đang xem lịch của năm mới. Trong thế giới của cô, một năm có n tháng, mỗi tháng có đúng m ngày. Mỗi ngày được đánh dấu bằng một trong ba loại:

1. Trắng: ngày đó cô không tham gia cuộc thi.
2. Xanh: ngày đó cô tham gia một cuộc thi.
3. Dấu hỏi: ngày đó có cuộc thi dự định sẵn, nhưng cô chưa quyết định có tham gia hay không.

Độ vui của một cuộc thi được tính như sau: ban đầu là 4, và với mỗi ngày kề cạnh cũng có tham gia thi, độ vui giảm 1. Sphinnny sẽ đổi mọi dấu hỏi thành trắng hoặc xanh sao cho tổng độ vui lớn nhất.

$$1 \leq n, m \leq 50.$$

Bài toán này cũng bắt đầu từ quan hệ giữa hai điểm. Khởi tạo ans bằng độ vui khi mọi dấu hỏi đều là trắng, rồi cộng thêm 4 lần số dấu hỏi. Khi đó:

- Nếu một dấu hỏi đổi thành trắng, ans giảm 4.
- Nếu một dấu hỏi đổi thành xanh, và xung quanh nó có tot ô ban đầu đã xanh, ans giảm $2 \cdot tot$.
- Nếu hai dấu hỏi kề nhau $?_1$ và $?_2$ đều đổi thành xanh, ans giảm 2.

Ta thử giải dung lượng cạnh như ví dụ trước nhưng khó thu được hệ phương trình đơn giản. Cách làm là tô hai màu bàn cờ lịch, vì hai ô kề cạnh luôn thuộc hai phía khác nhau của đồ thị hai phía.

Với một phía, dùng kiểu $?_1$: giữ cạnh với S nghĩa là đổi thành trắng, giữ cạnh với T nghĩa là đổi thành xanh. Với phía còn lại, dùng kiểu $?_2$: giữ cạnh với S nghĩa là đổi thành xanh, giữ cạnh với T nghĩa là đổi thành trắng. Với mỗi dấu hỏi kiểu $?_1$, nối $S \rightarrow ?_1$ dung lượng $2tot$ và $?_1 \rightarrow T$ dung lượng 4. Với mỗi dấu hỏi kiểu $?_2$, nối $S \rightarrow ?_2$ dung lượng 4 và $?_2 \rightarrow T$ dung lượng $2tot$. Với hai dấu hỏi kề nhau thuộc hai phía, nối $?_2 \rightarrow ?_1$ dung lượng 2. Chỉ khi cả hai dấu hỏi đều đổi thành xanh thì cạnh giữa chúng mới bị cắt, làm ans giảm 2.

Do đó ans trừ đi giá trị cắt nhỏ nhất chính là tổng độ vui lớn nhất.

3.3. Tiểu kết

Cắt nhỏ nhất có thể giải một số bài toán tối đa hóa lợi ích khi các lợi ích xung đột nhau. Trong đồ thị luồng, trên mọi đường đi từ S tới T đều phải cắt ít nhất một cạnh. Tận dụng tính chất này, ta biểu diễn xung đột giữa các lợi ích ngay trong mô hình luồng mạng.

4. Mô hình hóa từ góc độ luồng chi phí

Thêm một yếu tố “chi phí” vào mạng luồng, ta có bài toán luồng chi phí. So với luồng cực đại đơn thuần, luồng chi phí giải được nhiều kiểu bài toán hơn và linh hoạt hơn.

4.1. Ví dụ mô hình hóa

Ví dụ 8 (Sequence). Cho dãy số nguyên dương A_i độ dài n . Chọn một dãy con sao cho trong bất kỳ đoạn liên tiếp độ dài m nào của dãy gốc, số phần tử được chọn không vượt quá k . Hãy tối đa hóa tổng các phần tử được chọn.

$$1 \leq n \leq 1000, \quad 1 \leq m, k \leq 100.$$

Điều kiện “mỗi đoạn độ dài m chọn nhiều nhất k phần tử” có thể chuyển thành: không phải chọn một lần, mà chọn k lần; trong mỗi lần chọn, bất kỳ đoạn liên tiếp độ dài m nào chọn nhiều nhất một phần tử. Một phương án hợp lệ của bài toán gốc có phương án tương đương trong bài toán sau khi chuyển hóa, và ngược lại.

Dựng đồ thị như sau. Lập nguồn S , đích T , và với phần tử thứ i lập đỉnh P_i .

- Nối $S \rightarrow P_1$ dung lượng k , chi phí 0.
- Với $1 \leq i < n$, nối $P_i \rightarrow P_{i+1}$ dung lượng k , chi phí 0.
- Nối $P_n \rightarrow T$ dung lượng k , chi phí 0.
- Với $1 \leq i \leq n - m$, nối $P_i \rightarrow P_{i+m}$ dung lượng 1, chi phí A_i .
- Với $n - m + 1 \leq i \leq n$, nối $P_i \rightarrow T$ dung lượng 1, chi phí A_i .

Tìm luồng cực đại có chi phí cực đại. Chi phí cực đại chính là tổng phần tử được chọn lớn nhất. Bài toán này cần chuyển bài toán gốc thành dạng để mô hình hóa hơn, rồi mới dùng luồng mạng để giải.

Ví dụ 9 (WC2007, Rock Paper Scissors). Trong một số trò chơi đối kháng một chọi một, ví dụ cờ, bóng bàn hay cầu lông, ta thường gặp quan hệ A thắng B , B thắng C , nhưng C lại thắng A . Gọi đó là một tình huống kéo búa bao. Có n người tham gia; giữa hai người bất kỳ có đúng một trận, tổng cộng $n(n - 1)/2$ trận. Một số trận đã có kết quả. Hãy sắp xếp kết quả các trận còn lại để số bộ ba vô thứ tự (A, B, C) tạo thành tình huống kéo búa bao là lớn nhất.

$$1 \leq n \leq 100.$$

Đếm trực tiếp số bộ ba tạo thành kéo búa bao không thuận tiện. Ta xét phần bù: nếu (A, B, C) không tạo thành kéo búa bao, thì trong ba người có một người thắng 2 trận, một người thắng 1 trận, và một người thắng 0 trận. Nếu người i thắng tổng cộng w_i trận, thì

$$\sum_i \frac{w_i(w_i - 1)}{2}$$

là số bộ ba (A, B, C) không tạo thành kéo búa bao. Vì vậy tối đa hóa số tình huống kéo búa bao tương đương tối thiểu hóa tổng trên.

Trước hết mô hình hóa việc chọn kết quả các trận chưa đấu. Lập nguồn S và đích T . Với mỗi người lập đỉnh P_i . Với mỗi trận chưa đấu giữa i, j , lập đỉnh $C_{i,j}$, nối $S \rightarrow C_{i,j}$ dung lượng 1, rồi nối $C_{i,j} \rightarrow P_i$ và $C_{i,j} \rightarrow P_j$, mỗi cạnh dung lượng 1. Luồng đi vào P_i biểu diễn số trận chưa đấu mà người i được sắp thắng. Cuối cùng nối $P_i \rightarrow T$.

Để tối thiểu hóa $\sum_i w_i(w_i - 1)/2$, ta đặt chi phí trên các cạnh $P_i \rightarrow T$. Nhưng khi lưu lượng trên cạnh này tăng thêm 1, chi phí biên không cố định mà tăng dần. Có thể tách cạnh thành nhiều cạnh dung lượng 1, với chi phí lần lượt là chi phí phát sinh khi số trận thắng của người đó tăng thêm một. Nếu người i đã thắng b_i trận trong các kết quả có sẵn, các cạnh tách ra có chi phí

$$b_i, b_i + 1, b_i + 2, \dots$$

Vì ta tìm luồng cực đại chi phí nhỏ nhất, thuật toán sẽ ưu tiên tăng qua các cạnh rẻ trước, đúng với hàm lồi $\binom{w_i}{2}$.

Bài toán này dùng tư tưởng chuyển sang phần bù. Đồng thời, khi chi phí của một cạnh không cố định theo lưu lượng, có thể giải quyết bằng cách tách cạnh.

4.2. Tiểu kết

Luồng chi phí giải được nhiều loại bài toán hơn luồng cực đại, và cũng linh hoạt hơn. Trong nhiều trường hợp, cần chuyển hóa bài toán gốc trước, rồi dùng luồng chi phí để giải.

5. Tư tưởng cân bằng luồng

Có những bài toán khó mô hình hóa trực tiếp bằng trực giác, nhưng ta có thể thu được các quan hệ giữa biến trong bài toán, thường viết được thành một số đẳng thức. Trong đồ thị luồng, ngoài nguồn và đích, mọi đỉnh đều thỏa mãn cân bằng luồng. Điều kiện cân bằng luồng cũng có thể viết dưới dạng đẳng thức. Vì vậy một số hệ đẳng thức trong bài toán có thể được cấu tạo thành đồ thị luồng, sao cho các đẳng thức ấy chính là phương trình cân bằng tại các đỉnh.

5.1. Ví dụ mô hình hóa

Ví dụ 10 (NOI2008, Volunteer Recruitment). Một dự án cần n ngày để hoàn thành; ngày thứ i cần ít nhất a_i tình nguyện viên. Có m loại tình nguyện viên có thể tuyển; loại thứ j làm việc từ ngày s_j tới ngày t_j , chi phí tuyển mỗi người là c_j . Hãy tuyển đủ người với tổng chi phí nhỏ nhất.

$$1 \leq n \leq 1000, \quad 1 \leq m \leq 10000.$$

Xét một ví dụ nhỏ có ba ngày. Giả sử có ba loại tình nguyện viên: làm từ ngày 1 tới 3, từ ngày 2 tới 3, và từ ngày 1 tới 2; số người tuyển tương ứng là b_1, b_2, b_3 . Điều kiện nhu cầu có thể viết thành bất đẳng thức:

$$p_1 = b_1 + b_3 \geq a_1,$$

$$p_2 = b_1 + b_2 + b_3 \geq a_2,$$

$$p_3 = b_1 + b_2 \geq a_3.$$

Thêm biến dư $d_i \geq 0$, ta được

$$p_1 = b_1 + b_3 = a_1 + d_1,$$

$$p_2 = b_1 + b_2 + b_3 = a_2 + d_2,$$

$$p_3 = b_1 + b_2 = a_3 + d_3.$$

Lấy hiệu các đẳng thức kề nhau, rồi thêm hai phương trình biên, ta có một hệ đẳng thức mà mỗi biến xuất hiện đúng hai lần, một lần với dấu dương và một lần với dấu âm:

$$b_1 + b_3 - a_1 - d_1 = 0,$$

$$b_2 - a_2 + a_1 - d_2 + d_1 = 0,$$

$$-b_3 - a_3 + a_2 - d_3 + d_2 = 0,$$

$$-b_1 - b_2 + a_3 + d_3 = 0.$$

Trong đồ thị luồng, nếu quy ước lưu lượng chảy ra là dương và lưu lượng chảy vào là âm, tổng đại số lưu lượng tại mỗi đỉnh bằng 0. Do đó, mỗi đẳng thức trên có thể tương ứng với một đỉnh; mỗi biến xuất hiện một lần dương và một lần âm nên tương ứng với một cạnh giữa hai đỉnh.

Từ cách nhìn này, mô hình tổng quát là:

- Lập nguồn S , đích T , và các đỉnh $1, 2, \dots, n+1$.
- Với $2 \leq i \leq n+1$, nối $i \rightarrow i-1$ dung lượng $+\infty$, chi phí 0. Các cạnh này tương ứng với biến dư.
- Với loại tình nguyện viên j làm từ ngày s_j tới ngày t_j và chi phí c_j , nối $s_j \rightarrow t_j + 1$ dung lượng $+\infty$, chi phí c_j .
- Đặt $a_0 = a_{n+1} = 0$. Với mỗi i , nếu $a_i - a_{i-1} \geq 0$, nối $S \rightarrow i$ dung lượng $a_i - a_{i-1}$, chi phí 0; nếu $a_i - a_{i-1} < 0$, nối $i \rightarrow T$ dung lượng $a_{i-1} - a_i$, chi phí 0.

Vì các a_i là hằng số, mọi cạnh đi ra từ S và đi vào T đều phải đầy lưu lượng. Luồng cực đại trong đồ thị thỏa mãn điều kiện này. Để tối thiểu hóa chi phí tuyến, ta tìm luồng cực đại chi phí nhỏ nhất.

Ví dụ 11 (World Finals 2011, Chips Challenge). Trong một lưới $n \times n$, ta đặt các linh kiện. Một số ô đã bắt buộc có linh kiện, một số ô không được đặt, còn các ô khác có thể đặt hoặc không. Yêu cầu tổng số linh kiện trên hàng i bằng tổng số linh kiện trên cột i . Để bảo đảm tản nhiệt, số linh kiện trên bất kỳ hàng hoặc cột nào không được vượt quá một tỉ lệ $\frac{A}{B}$ của tổng số linh kiện. Hãy đặt được nhiều linh kiện nhất.

$$1 \leq n \leq 40.$$

Gọi $a_{i,j}$ cho biết ô hàng i , cột j có đặt linh kiện hay không, trong đó 1 là đặt và 0 là không đặt. Gọi t_i là tổng số linh kiện trên hàng i , đồng thời cũng là tổng số linh kiện trên cột i . Ta có

$$\begin{aligned} t_i &= a_{i,1} + a_{i,2} + \dots + a_{i,n} \quad (1 \leq i \leq n), \\ t_i &= a_{1,i} + a_{2,i} + \dots + a_{n,i} \quad (1 \leq i \leq n). \end{aligned}$$

Sắp xếp lại:

$$\begin{aligned} a_{i,1} + a_{i,2} + \dots + a_{i,n} - t_i &= 0 \quad (1 \leq i \leq n), \\ t_i - a_{1,i} - a_{2,i} - \dots - a_{n,i} &= 0 \quad (1 \leq i \leq n). \end{aligned}$$

Ta còn có giới hạn tản nhiệt. Nếu liệt kê tổng số linh kiện tot , ta xác định được số linh kiện tối đa trên mỗi hàng hoặc cột, gọi là $maxt$. Với mỗi giá trị tot , tìm tổng số linh kiện nhiều nhất ans trong giới hạn $maxt$; nếu $ans \geq tot$, phương án cho tot là hợp lệ.

Sau khi cố định $maxt$, dựng đồ thị như sau:

- Lập đỉnh A_i cho đẳng thức của hàng $a_{i,1} + a_{i,2} + \dots + a_{i,n} - t_i = 0$.
- Lập đỉnh B_i cho đẳng thức của cột $t_i - a_{1,i} - a_{2,i} - \dots - a_{n,i} = 0$.
- Nối $B_i \rightarrow A_i$ với chặn dưới 0, chặn trên $maxt$, chi phí 1. Cạnh này biểu diễn biến t_i .
- Nếu ô (i,j) bắt buộc đặt linh kiện, nối $A_i \rightarrow B_j$ với chặn dưới và chặn trên đều bằng 1, chi phí 0.
- Nếu ô (i,j) tùy chọn, nối $A_i \rightarrow B_j$ với chặn dưới 0, chặn trên 1, chi phí 0. Nếu ô bị cấm thì không thêm cạnh tương ứng.

Một luồng khả thi trong mạng này chính là một phương án đặt linh kiện hợp lệ với giới hạn $maxt$. Luồng khả thi có chi phí lớn nhất cho ta phương án tốt nhất dưới giới hạn đó. Liệt kê mọi giá trị $maxt$ và lấy phương án hợp lệ tốt nhất làm đáp án cuối cùng.

5.2. Tiểu kết

Ta biểu diễn quan hệ giữa các biến dưới dạng đẳng thức, rồi liên hệ chúng với các phương trình cân bằng luồng trong đồ thị. Nhờ đó có thể ánh xạ bài toán gốc sang một mạng luồng.

Đa số mô hình luồng mạng đều có thể được hiểu bằng tư tưởng cân bằng luồng, nên tư tưởng này có thể dùng để giải phần lớn các bài toán luồng mạng. Nhược điểm của nó là không đủ trực quan, đôi khi còn làm bài toán trở nên phức tạp. Tuy vậy, với những bài khó hiểu hoặc khó nghĩ ra mô hình trực tiếp, tư tưởng cân bằng luồng có thể phát huy tác dụng rất lớn.

6. Tổng kết

Góc nhìn mô hình hóa luồng mạng rất đa dạng: từ luồng cực đại, từ cắt nhỏ nhất, từ luồng chi phí, từ cân bằng luồng, v.v.

Các kỹ thuật dùng khi mô hình hóa luồng mạng cũng rất nhiều, chẳng hạn tách đỉnh, tách cạnh, xây đồ thị phân tầng.

Khi mô hình hóa bằng luồng mạng, đôi khi cần phân tích bài toán và chuyển bài toán gốc thành một bài toán dễ mô hình hóa hơn.

Đôi khi mô hình luồng mạng phải kết hợp với thuật toán hoặc phép biến đổi khác mới giải được bài toán, chẳng hạn tìm kiếm nhị phân hoặc chuyển sang bài toán phần bù.

Tóm lại, nội dung mô hình hóa luồng mạng rất phong phú. Hy vọng có bạn đọc có thể lấy cảm hứng từ bài viết này để phát hiện những mô hình luồng mạng mới và tinh tế hơn.

Tài liệu tham khảo

1. Liu Rujia, Chen Feng, *Sách nhập môn kinh điển về thi thuật toán: hướng dẫn luyện tập*, Nhà xuất bản Đại học Thanh Hoa.
2. PHVA, *Ứng dụng mô hình cắt nhỏ nhất trong thi tin học*.

Bài 3. Bàn sơ lược về quy hoạch tuyến tính và bài toán đối ngẫu

Dong Kefan, Trường Trung học số 1 Phúc Châu, Phúc Kiến

Tóm tắt

Quy hoạch tuyến tính là một mô hình toán học quan trọng và có phạm vi ứng dụng rộng. Trong thi tin học, nhiều bài toán có thể được biểu diễn trực quan bằng quy hoạch tuyến tính. Bài viết này thảo luận tính chất của lớp bài toán đó từ góc nhìn quy hoạch tuyến tính, rồi dùng nguyên lý đối ngẫu của quy hoạch tuyến tính để chuyển hóa mô hình, qua đó giải bài toán hiệu quả hơn.

1. Dẫn nhập

Trong thi tin học, các lời giải của một loạt bài toán quy hoạch tuyến tính mà đại diện là luồng mạng thường rất linh hoạt. Bài viết này biểu diễn những bài toán như vậy dưới dạng quy hoạch tuyến tính để hiểu chúng ở tầng sâu hơn, với hy vọng đem lại phần nào gợi ý cho các bạn tham gia thi tin học.

Phần đầu của bài viết giới thiệu định nghĩa quy hoạch tuyến tính, phương pháp đơn hình để giải quy hoạch tuyến tính tổng quát, và cách biểu diễn bài toán thành mô hình quy hoạch tuyến tính. Phần sau

giới thiệu đối ngẫu; thông qua các ví dụ, ta mô tả một hướng suy nghĩ mới: lấy mô hình quy hoạch tuyến tính làm điểm xuất phát, dùng tính đối ngẫu để chuyển đổi mô hình, từ đó giải bài toán hiệu quả.

2. Quy hoạch tuyến tính

2.1. Định nghĩa

Trong các bài toán tối ưu hóa, có một lớp bài toán mà mọi ràng buộc cũng như hàm mục tiêu đều là hàm tuyến tính theo các biến. Ta gọi đó là bài toán quy hoạch tuyến tính. Để tiện trình bày, trước hết đưa ra một vài định nghĩa.

Định nghĩa 2.1. Cho các số thực $a_1, a_2, \dots, a_n$ và các biến $x_1, x_2, \dots, x_n$. Một hàm tuyến tính trên các biến này được định nghĩa là

$$f(x_1, x_2, \dots, x_n) = \sum_{i=1}^n a_i x_i.$$

Các công thức

$$f(x_1, x_2, \dots, x_n) = b, \quad f(x_1, x_2, \dots, x_n) \leq b, \quad f(x_1, x_2, \dots, x_n) \geq b$$

được gọi là các **ràng buộc tuyến tính**.

Bài toán quy hoạch tuyến tính yêu cầu cực đại hóa hoặc cực tiểu hóa một hàm tuyến tính chịu sự chi phối của một số hữu hạn ràng buộc tuyến tính.

Một nghiệm thỏa mọi ràng buộc được gọi là **ng nghiệm khả thi**; nghiệm khả thi làm hàm mục tiêu đạt tối ưu được gọi là **ng nghiệm tối ưu**; miền gồm mọi nghiệm khả thi được gọi là **không gian nghiệm**.

2.2. Tính chất của quy hoạch tuyến tính

Để mô tả tính chất và thuật toán của quy hoạch tuyến tính, ta cần dùng một dạng chuẩn hơn.

Định nghĩa 2.2 (dạng chuẩn). Quy hoạch tuyến tính dạng chuẩn có dạng

$$\begin{aligned} &\text{cực đại hóa} && \sum_{j=1}^n c_j x_j, \\ &\text{thỏa} && \sum_{j=1}^n a_{ij} x_j \leq b_i, \quad i = 1, 2, \dots, m, \\ &&& x_j \geq 0, \quad j = 1, 2, \dots, n. \end{aligned}$$

Dễ thấy sau một vài phép biến đổi đơn giản, mọi bài toán quy hoạch tuyến tính đều có thể mô tả bằng dạng chuẩn. Nếu hàm mục tiêu cần lấy cực tiểu, ta đổi dấu nó để thành cực đại. Ràng buộc $f(x_1, x_2, \dots, x_n) = b$ có thể tách thành $f(x_1, x_2, \dots, x_n) \leq b$ và $-f(x_1, x_2, \dots, x_n) \leq -b$. Ràng buộc $f(x_1, x_2, \dots, x_n) \geq b$ có thể viết thành $-f(x_1, x_2, \dots, x_n) \leq -b$. Một biến không bị chặn dấu x có thể tách thành hai biến không âm x_0, x_1 sao cho $x = x_0 - x_1$.

Dạng chuẩn cũng có thể viết bằng ma trận:

$$\begin{aligned} &\text{cực đại hóa} && c^T x, \\ &\text{thỏa} && Ax \leq b, \\ &&& x \geq 0. \end{aligned} \tag{2.2.1}$$

Ở đây $x \leq y$ nghĩa là với mọi tọa độ tương ứng đều có $x_i \leq y_i$.

Dạng chuẩn ngắn gọn, rõ ràng. Nhưng để giải quy hoạch tuyến tính, ta muốn các ràng buộc đều là đẳng thức, nhờ đó tránh vấn đề đổi chiều dấu bất đẳng thức khi xử lý hệ số âm. Vì vậy ta giới thiệu một dạng biểu diễn khác: dạng nổi lồi.

Định nghĩa 2.3 (dạng nổi lồi). Quy hoạch tuyến tính dạng nổi lồi có dạng

$$\begin{aligned} \text{cực đại hóa} \quad & \sum_{j=1}^n c_j x_j, \\ \text{thỏa} \quad & x_{i+n} = b_i - \sum_{j=1}^n a_{ij} x_j, \quad i = 1, 2, \dots, m, \\ & x_j \geq 0, \quad j = 1, 2, \dots, n + m. \end{aligned}$$

Việc chuyển dạng chuẩn sang dạng nổi lồi là tự nhiên. Với một ràng buộc $f_i(x_1, x_2, \dots, x_n) \leq b_i$, thêm biến $x_{i+n} = b_i - f_i(x_1, x_2, \dots, x_n)$; khi đó bất đẳng thức ban đầu biến thành đẳng thức $x_{i+n} = b_i - f_i(x_1, x_2, \dots, x_n) \geq 0$, và mọi biến vẫn lấy giá trị thực không âm.

Không gian nghiệm của một quy hoạch tuyến tính là giao của nhiều miền nghiệm bất đẳng thức tuyến tính. Mỗi miền nghiệm của một bất đẳng thức tuyến tính là một miền lồi,³ nên bằng quy nạp, không gian nghiệm của quy hoạch tuyến tính cũng là một miền lồi.

Vì không gian nghiệm là miền lồi, giá trị của nghiệm tối ưu cục bộ là duy nhất⁴ và bằng giá trị tối ưu toàn cục. Khi ta đi tham lam trong miền này bằng một thuật toán kiểu leo núi để tìm giá trị tối ưu, ta không cần lo thuật toán dừng ở một nghiệm tối ưu cục bộ sai. Tính chất đó dẫn tới một phương pháp tổng quát để giải quy hoạch tuyến tính: phương pháp đơn hình.

3. Đơn hình

3.1. Mô tả thuật toán

Ý tưởng cơ bản của phương pháp đơn hình là dùng phép thay biến để di chuyển trong không gian nghiệm dọc theo biên, theo hướng làm tăng hàm mục tiêu. Thao tác lõi là thao tác **xoay trục** (*pivot*), tức thay đổi vai trò của các biến.

Trước hết đặt một vài khái niệm:

- **Biến cơ sở:** mọi biến nằm ở vế trái của các đẳng thức trong dạng nổi lồi.
- **Biến phi cơ sở:** mọi biến nằm ở vế phải của các đẳng thức trong dạng nổi lồi.

Một tập biến cơ sở và phi cơ sở của quy hoạch tuyến tính ngầm xác định một **ng nghiệm cơ bản**: mọi biến cơ sở lấy giá trị bằng hằng số ở vế phải,⁵ mọi biến phi cơ sở lấy giá trị 0. Trong thuật toán đơn hình, ta chỉ xét các nghiệm cơ bản.

Thuật toán đơn hình có hai thao tác chính: thao tác xoay trục và thao tác simplex. Xoay trục chọn một biến cơ sở x_B và một biến phi cơ sở x_N , rồi hoán đổi chúng; x_N gọi là biến vào, x_B gọi là biến ra. Cụ thể, nếu ban đầu có

$$x_B = b_i - \sum_{j=1}^n a_{ij} x_j,$$

³Một cách hiểu trực quan: với hai điểm bất kỳ trong miền, mọi điểm trên đoạn thẳng nối chúng cũng thuộc miền.

⁴Tạm thời không xét trường hợp quy hoạch tuyến tính không bị chặn, tức giá trị tối ưu có thể tiến tới vô hạn. Nghiệm tối ưu cục bộ có thể không duy nhất, chẳng hạn là cả một biên song song với hàm mục tiêu, nhưng giá trị của chúng nhất định bằng nhau.

⁵Trường hợp hằng số âm sẽ được xét ở phần sau.

thì

$$x_N = \frac{b_i - \sum_{j \neq N} a_{ij}x_j - x_B}{a_{iN}}.$$

Thay x_N trong hàm mục tiêu và mọi ràng buộc khác bằng biểu thức này, khi đó vế phải không còn xuất hiện x_N . Dĩ nhiên khi chọn x_N phải bảo đảm $a_{iN} \neq 0$.

Thao tác simplex là quá trình chính của thuật toán đơn hình: xuất phát từ một nghiệm cơ bản, qua một dãy phép xoay trục để đi tới nghiệm tối ưu. Bằng cách chọn biến vào và biến ra thích hợp, mỗi phép xoay trục đều làm tăng hàm mục tiêu cho tới khi đạt tối ưu toàn cục.

Xét ví dụ sau:

$$\begin{array}{ll} \text{cực đại hóa} & 3x_1 + x_2 + 2x_3, \\ \text{thỏa} & x_4 = 30 - x_1 - x_2 - 3x_3, \\ & x_5 = 24 - 2x_1 - 2x_2 - 5x_3, \\ & x_6 = 36 - 4x_1 - x_2 - 2x_3, \\ & x_j \geq 0, \quad j = 1, 2, \dots, 6. \end{array} \quad (3.1)$$

Nghiệm cơ bản ban đầu là

$$(x_1, x_2, \dots, x_6) = (0, 0, 0, 30, 24, 36).$$

Gọi hàm mục tiêu là z , khi đó $z = 0$.

Bước đầu chọn x_1 làm biến vào, vì hệ số của x_1 trong hàm mục tiêu dương. Do mọi biến đều không âm, x_1 không thể tăng vô hạn. Ba ràng buộc lần lượt cho $x_1 \leq 30$, $x_1 \leq 12$, $x_1 \leq 9$. Ràng buộc chặt nhất là ràng buộc thứ ba, nên chọn x_6 làm biến ra. Viết lại ràng buộc thứ ba rồi thế vào các biểu thức còn lại, ta được

$$\begin{array}{ll} \text{cực đại hóa} & 27 + \frac{x_2}{4} + \frac{x_3}{2} - \frac{3x_6}{4}, \\ \text{thỏa} & x_4 = 21 - \frac{3x_2}{4} - \frac{5x_3}{2} + \frac{x_6}{4}, \\ & x_5 = 6 - \frac{3x_2}{2} - 4x_3 + \frac{x_6}{2}, \\ & x_1 = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4}, \\ & x_j \geq 0, \quad j = 1, 2, \dots, 6. \end{array}$$

Hệ mới vẫn là dạng nói lỏng tương đương với hệ cũ, và nghiệm cơ bản mới là $(9, 0, 0, 21, 6, 0)$, làm $z = 27$.

Vì trong hàm mục tiêu vẫn còn biến có hệ số dương, ta tiếp tục tăng. Chọn x_2 làm biến vào; ràng buộc chặt nhất khi đó đưa x_5 ra. Sau phép xoay trục, lại chọn x_3 làm biến vào. Ba ràng buộc lúc này cho các chặn trên lần lượt là $\infty, 4, 13/2$, nên x_2 ra. Kết quả:

$$\begin{array}{ll} \text{cực đại hóa} & 28 - \frac{x_3}{6} - \frac{x_5}{6} - \frac{2x_6}{3}, \\ \text{thỏa} & x_4 = 18 - \frac{5x_5}{6}, \\ & x_2 = 4 - \frac{8x_3}{3} - \frac{2x_5}{3} + \frac{x_6}{3}, \\ & x_1 = 8 + \frac{x_3}{6} + \frac{x_5}{6} - \frac{x_6}{3}, \\ & x_j \geq 0, \quad j = 1, 2, \dots, 6. \end{array}$$

Nghiệm cơ bản tương ứng là $(8, 4, 0, 18, 0, 0)$. Lúc này mọi hệ số của biến trong hàm mục tiêu đều không dương, nên ta đã tìm được một điểm tối ưu cục bộ; theo tính chất của quy hoạch tuyến tính, đó chính là nghiệm tối ưu toàn cục.

Dưới đây là giả mã của thuật toán đơn hình; A, b, c có ý nghĩa như trong (2.2.1).

Algorithm 1 Simplex(A, b, c)

```

1: initialization(A,b,c)
2: while exists e such that c_e > 0 do
3:     find l with A_{l,e} > 0 minimizing b_l / A_{l,e}
4:     if no such l exists then
5:         return Unbounded
6:     else
7:         pivot(A,b,c,l,e)
8:     endif
9: end while

```

Dòng 4–5 xử lý trường hợp biến vào không bị ràng buộc, khi đó quy hoạch tuyến tính không bị chặn. Nếu không, ta tìm ràng buộc chặt nhất và xoay trục. Thao tác *initialization* dùng để tìm một nghiệm cơ bản ban đầu. Thông thường nếu $b_i \geq 0$ với mọi i , ta có thể bỏ qua bước này và bắt đầu từ nghiệm cơ bản $(0, 0, \dots, 0, b_1, b_2, \dots, b_m)$. Vì mỗi bước xoay trục đều chọn ràng buộc chặt nhất, sau xoay trục vẫn có $b_i \geq 0$ với mọi i .

Tư tưởng cơ bản của khởi tạo là đưa vào một quy hoạch tuyến tính phụ:

$$\begin{array}{ll}
 \text{cực đại hóa} & -x_0, \\
 \text{thỏa} & x_{i+n} = b_i - \sum_{j=1}^n a_{ij}x_j + x_0, \quad i = 1, 2, \dots, m, \\
 & x_j \geq 0, \quad j = 0, 1, \dots, n + m.
 \end{array}$$

Nếu quy hoạch tuyến tính phụ có nghiệm tối ưu $x_0 = 0$, xóa x_0 khỏi mô hình sẽ không ảnh hưởng tới các ràng buộc; sau đó đổi hàm mục tiêu về hàm ban đầu, ta nhận được một quy hoạch tuyến tính tương đương với bài toán gốc và có $b_i \geq 0$. Như vậy hoàn tất khởi tạo.

Nghiệm ban đầu của quy hoạch phụ rất dễ dựng. Vì x_0 có hệ số +1 trong mọi ràng buộc, chọn x_0 làm biến vào và chọn ràng buộc có b_i nhỏ nhất làm biến ra, rồi xoay trục một lần. Khi đó quy hoạch phụ có một nghiệm cơ bản khả thi, và ta chỉ cần chạy simplex.

Ví dụ:

$$\begin{array}{ll}
 \text{cực đại hóa} & 2x_1 - x_2, \\
 \text{thỏa} & x_3 = 2 - 2x_1 + x_2, \\
 & x_4 = -4 - x_1 + 5x_2, \\
 & x_j \geq 0, \quad j = 1, 2, \dots, 4.
 \end{array} \tag{3.2}$$

Mô hình này không có nghiệm cơ bản ban đầu khả thi, nên đưa vào bài toán phụ với x_0 . Giá trị b_i nhỏ nhất là -4 , vì thế xoay trục với x_0 vào và x_4 ra. Sau đó chạy simplex cho bài toán phụ; khi nghiệm tối ưu có $x_0 = 0$, xóa x_0 và khôi phục hàm mục tiêu $2x_1 - x_2$. Ta thu được một dạng nói lỏng có nghiệm cơ bản khả thi, tương đương với (3.2). Trong ví dụ, nghiệm cơ bản tương ứng với bài toán gốc là $(x_1, x_2, x_3, x_4) = (0, 2, 4, 0)$, thỏa mọi ràng buộc của (3.2).

Nếu cuối cùng x_0 là biến phi cơ sở thì có thể xóa trực tiếp. Nếu x_0 là biến cơ sở, cần xoay trục thêm một lần để đưa x_0 ra; vì $x_0 = 0$, có thể chọn tùy ý một biến phi cơ sở ở vế phải của đẳng thức chứa x_0 .

3.2. Độ phức tạp của phương pháp đơn hình

Khởi tạo và simplex có cùng bậc độ phức tạp, nên chỉ cần xét simplex. Một phép pivot là phép thế và thay biến, có độ phức tạp $O(nm)$. Tuy nhiên số lần pivot không bị chặn bởi một đa thức. Dù vậy trong thực tế, thuật toán đơn hình thường tìm nghiệm tối ưu rất nhanh.

Cần chú ý rằng phần lớn đề OI sinh ra các ràng buộc có cấu trúc riêng. Khởi tạo có thể phá hỏng cấu trúc đó, làm tăng thời gian chạy không cần thiết, lại khá rườm rà. Vì vậy tác giả không khuyến nghị dùng khởi tạo trong thi đấu; ở các ví dụ sau sẽ giới thiệu một số cách tránh bước này.

Điều đó không có nghĩa là quy hoạch tuyến tính không giải được trong thời gian đa thức. Thuật toán ellipsoid và thuật toán điểm trong đều là các thuật toán đa thức cho quy hoạch tuyến tính. Ta nêu định lý sau mà không chứng minh:

Định lý 3.1. Bài toán quy hoạch tuyến tính tổng quát có thể giải trong thời gian đa thức.

Dù vậy, thuật toán đơn hình dễ cài đặt và thời gian chạy thực tế không kém rõ rệt so với các thuật toán trên, nên vẫn là một phương pháp tốt để giải quy hoạch tuyến tính.

4. Biểu diễn bài toán thành quy hoạch tuyến tính

4.1. Một ví dụ

Ví dụ 1 (precious stones). Hai người A, B tìm được n viên đá trong hang. Viên thứ i có giá trị A_i đối với A , và B_i đối với B . Đá có thể cắt, và cả hai đều coi giá trị của đá là đồng đều, tức giá trị tỉ lệ với thể tích. Họ muốn chia đá sao cho giá trị mỗi người nhận được bằng nhau. Hỏi giá trị lớn nhất mỗi người có thể nhận là bao nhiêu.

$$n \leq 50, \quad 0 < A_i, B_i < 100.$$

Giả sử phần p_i của viên đá thứ i được chia cho A , phần còn lại cho B . Khi đó

$$V_A = \sum_{i=1}^n p_i A_i, \quad V_B = \sum_{i=1}^n (1 - p_i) B_i.$$

Ta muốn cực đại hóa giá trị chung khi $V_A = V_B$. Có thể nói lỏng thành $V_A \leq V_B$: nếu $V_A < V_B$, chuyển một phần đá mà B nhận được sang A sẽ không làm giảm A 's value và không vi phạm ràng buộc, nên mục tiêu vẫn tăng hoặc không giảm. Do đó nhận được quy hoạch tuyến tính

$$\begin{array}{ll} \text{cực đại hóa} & \sum_{i=1}^n p_i A_i, \\ \text{thỏa} & \sum_{i=1}^n p_i (A_i + B_i) \leq \sum_{i=1}^n B_i, \\ & p_i \leq 1, \quad i = 1, 2, \dots, n, \\ & p_i \geq 0, \quad i = 1, 2, \dots, n. \end{array}$$

Mô hình đã nói lỏng này không cần khởi tạo; chỉ cần chạy trực tiếp thuật toán đơn hình.

4.2. Bài toán luồng cực đại

Trong bài toán luồng cực đại, mọi đỉnh trừ nguồn và đích đều thỏa bảo toàn luồng: tổng luồng vào bằng tổng luồng ra. Mỗi cạnh có luồng không vượt quá dung lượng. Gọi $c(u, v)$ là dung lượng cạnh (u, v) , $f(u, v)$ là lưu lượng. Để bài toán gọn hơn, ta thêm một cạnh từ đích t về nguồn s có dung lượng vô hạn; khi đó bảo toàn luồng đúng với mọi đỉnh, và $f(t, s)$ là giá trị luồng từ nguồn tới đích. Với tập đỉnh V , tập cạnh E , ta có:

$$\begin{array}{ll} \text{cực đại hóa} & f(t, s), \\ \text{thỏa} & f(u, v) \leq c(u, v), \quad (u, v) \in E, \\ & \sum_v f(v, u) = \sum_v f(u, v), \quad u \in V, \\ & f(u, v) \geq 0, \quad (u, v) \in E \cup \{(t, s)\}. \end{array}$$

4.3. Bài toán luồng chi phí nhỏ nhất

Ở đây ta chỉ yêu cầu chi phí nhỏ nhất, không cần ràng buộc luồng cực đại. Mỗi cạnh có chi phí $w(u, v)$:

$$\begin{aligned} \text{cực tiểu hóa} \quad & \sum_{(u,v) \in E} f(u, v)w(u, v), \\ \text{thỏa} \quad & f(u, v) \leq c(u, v), \quad (u, v) \in E, \\ & \sum_v f(v, u) = \sum_v f(u, v), \quad u \in V - \{s, t\}, \\ & f(u, v) \geq 0, \quad (u, v) \in E \cup \{(t, s)\}. \end{aligned}$$

Trong mô hình luồng mạng, mỗi biến $f(u, v)$ xuất hiện trong ràng buộc dung lượng và trong các phương trình bảo toàn luồng của hai đầu nút. Trong hai phương trình bảo toàn đó, hệ số của $f(u, v)$ lần lượt là $+1$ và -1 .

4.4. Bài toán luồng nhiều loại hàng

Xét biến thể sau của luồng cực đại. Có n cặp nguồn–đích (s_i, t_i) , $i = 1, 2, \dots, n$. Mỗi cặp ứng với một loại hàng khác nhau, nhưng tất cả dùng chung một mạng vận chuyển. Với mỗi cặp (s_i, t_i) , yêu cầu luồng giữa chúng không nhỏ hơn d_i . Hỏi có nghiệm khả thi hay không.

Với bài toán này, mọi đỉnh phải bảo toàn luồng riêng cho từng loại hàng; trên mỗi cạnh, tổng luồng của mọi loại hàng không vượt quá dung lượng. Vì chỉ cần kiểm tra khả thi, hàm mục tiêu là rỗng:

$$\begin{aligned} \text{cực đại hóa} \quad & 0, \\ \text{thỏa} \quad & \sum_i f_i(u, v) \leq c(u, v), \quad (u, v) \in E, \\ & \sum_v f_i(v, u) = \sum_v f_i(u, v), \quad u \in V, i = 1, 2, \dots, n, \\ & f_i(t_i, s_i) \geq d_i, \quad i = 1, 2, \dots, n, \\ & f_i(u, v) \geq 0, \quad (u, v) \in E \cup \{(t_i, s_i) \mid 1 \leq i \leq n\}. \end{aligned}$$

Lời giải đa thức duy nhất được biết cho bài toán này là biểu diễn nó thành quy hoạch tuyến tính rồi giải quy hoạch tuyến tính đó trong thời gian đa thức.

5. Quy hoạch tuyến tính nguyên

Trong quy hoạch tuyến tính có một lớp đặc biệt mà biến chỉ được lấy giá trị nguyên thay vì số thực. Đó là **quy hoạch tuyến tính nguyên**.

Quy hoạch tuyến tính nguyên là bài toán NP-hard, nên trong thi tin học việc giải quy hoạch tuyến tính nguyên tổng quát là không thực tế. Tuy nhiên, có một số bài toán đặc biệt bảo đảm đáp án khi biến lấy giá trị nguyên trùng với đáp án khi biến lấy giá trị thực. Ví dụ luồng cực đại và luồng chi phí nhỏ nhất với dung lượng nguyên: tuy phát biểu bài toán không yêu cầu lưu lượng là số nguyên, do cấu trúc đặc biệt của bài toán, luôn tồn tại nghiệm tối ưu mà mọi cạnh có lưu lượng nguyên. Điều này có thể suy ra từ các bước của thuật toán luồng mạng.

Nhưng không phải bài toán nào cũng có tính chất đó. Nếu khi ấy ta dùng đơn hình để giải quy hoạch tuyến tính tổng quát, nghiệm thu được thường không đạt được dưới các ràng buộc của bài toán gốc. Sau đây là một ví dụ mà tác giả cho rằng cần hoài nghi.

Ví dụ 2 (Volunteer Recruitment bản tăng cường). Một cuộc thi kéo dài N ngày. Có M loại tình nguyện viên; thời gian làm việc của mỗi loại gồm vài đoạn liên tiếp. Loại i có thể làm việc trong p_i đoạn,

mỗi đoạn từ ngày s_{ij} tới ngày t_{ij} ; tuyển một người loại này tốn chi phí c_i . Ngày thứ i cần số tình nguyện viên làm việc không nhỏ hơn w_i . Hãy tìm chi phí nhỏ nhất.

$$N \leq 1000, \quad M \leq 10000.$$

Đề bài rõ ràng yêu cầu số tình nguyện viên tuyển vào là số nguyên, nên đây là một quy hoạch tuyến tính nguyên. Bài gốc *Volunteer Recruitment* chỉ có mỗi loại tình nguyện viên làm việc trên một đoạn liên tiếp, có thể chuyển thành mô hình luồng mạng, nên khi dùng đơn hình vẫn bảo đảm nghiệm thực bằng nghiệm của bài gốc. Nhưng bài tăng cường này không có bảo đảm đó.

Chẳng hạn cuộc thi có ba ngày và ba loại tình nguyện viên. Loại thứ nhất làm ngày $[1, 1]$ và $[2, 2]$, chi phí 1. Loại thứ hai làm ngày $[1, 1]$ và $[3, 3]$, chi phí 1. Loại thứ ba làm ngày $[2, 2]$ và $[3, 3]$, chi phí 1. Mỗi ngày cần ít nhất một người.

Nghiệm nguyên tối ưu là tuyển hai loại bất kỳ, mỗi loại một người, chi phí 2. Nhưng nếu nói lỏng biến sang số thực, nghiệm tối ưu là tuyển mỗi loại 0.5 người, chi phí 1.5. Nghiệm này đúng là làm mỗi ngày có tổng số tình nguyện viên bằng 1, nhưng không phù hợp với phát biểu bài toán gốc.

Tác giả không có cách tốt để giải quyết vấn đề này. Tuy nhiên nhìn vào tình hình nộp bài, phần lớn người giải đều dùng đơn hình để qua bài này; tác giả không hiểu rõ nguyên nhân.

6. Bài toán đối ngẫu

6.1. Dẫn nhập

Xét quy hoạch tuyến tính sau:

$$\begin{array}{ll} \text{cực tiểu hóa} & 7x_1 + x_2 + 5x_3, \\ \text{thỏa} & x_1 - x_2 + 3x_3 \geq 10, \\ & 5x_1 + 2x_2 - x_3 \geq 6, \\ & x_i \geq 0, \quad i = 1, 2, 3. \end{array}$$

Dễ thấy

$$7x_1 + x_2 + 5x_3 \geq x_1 - x_2 + 3x_3 \geq 10,$$

do so sánh từng hệ số biến. Nghĩa là giá trị hàm mục tiêu ít nhất là 10.

Có thể dùng cùng phương pháp để nhận được một cận dưới chặt hơn không? Nếu xét đồng thời hai ràng buộc:

$$7x_1 + x_2 + 5x_3 \geq (x_1 - x_2 + 3x_3) + (5x_1 + 2x_2 - x_3) \geq 10 + 6.$$

Ta muốn tối đa hóa cận dưới thu được theo cách này. Cận dưới đó có dạng

$$\begin{aligned} f(y_1, y_2) &= y_1(x_1 - x_2 + 3x_3) + y_2(5x_1 + 2x_2 - x_3) \\ &= (y_1 + 5y_2)x_1 + (-y_1 + 2y_2)x_2 + (3y_1 - y_2)x_3, \quad y_1, y_2 \geq 0. \end{aligned}$$

Nếu

$$y_1 + 5y_2 \leq 7, \quad -y_1 + 2y_2 \leq 1, \quad 3y_1 - y_2 \leq 5,$$

thì nhờ tính không âm của biến, ta có

$$7x_1 + x_2 + 5x_3 \geq f(y_1, y_2) \geq 10y_1 + 6y_2.$$

Để cận dưới lớn nhất, cần cực đại hóa $10y_1 + 6y_2$, và bài toán này lại là một quy hoạch tuyến tính:

$$\begin{array}{ll} \text{cực đại hóa} & 10y_1 + 6y_2, \\ \text{thỏa} & y_1 + 5y_2 \leq 7, \\ & -y_1 + 2y_2 \leq 1, \\ & 3y_1 - y_2 \leq 5, \\ & y_i \geq 0, \quad i = 1, 2. \end{array}$$

Bài toán mới này gọi là **bài toán đối ngẫu** của bài toán ban đầu. Nếu lấy đối ngẫu của bài toán đối ngẫu, ta trở lại bài toán gốc; quá trình đối ngẫu là tương hỗ. Một bài toán cực đại có thể đối ngẫu thành một bài toán cực tiểu, và ngược lại.

Định nghĩa 6.1 (bài toán đối ngẫu). Cho một quy hoạch tuyến tính nguyên thủy:

$$\begin{array}{ll} \text{cực tiểu hóa} & \sum_{j=1}^n c_j x_j, \\ \text{thỏa} & \sum_{j=1}^n a_{ij} x_j \geq b_i, \quad i = 1, 2, \dots, m, \\ & x_j \geq 0, \quad j = 1, 2, \dots, n. \end{array}$$

Đối ngẫu của nó là

$$\begin{array}{ll} \text{cực đại hóa} & \sum_{i=1}^m b_i y_i, \\ \text{thỏa} & \sum_{i=1}^m a_{ij} y_i \leq c_j, \quad j = 1, 2, \dots, n, \\ & y_i \geq 0, \quad i = 1, 2, \dots, m. \end{array}$$

Dùng ma trận, có thể viết trực quan hơn:

$$\begin{array}{ll} \text{cực tiểu hóa} & c^T x \\ \text{thỏa} & Ax \geq b \\ & x \geq 0 \end{array} \quad \begin{array}{ll} \text{cực đại hóa} & b^T y, \\ \text{thỏa} & A^T y \leq c, \\ & y \geq 0. \end{array}$$

Hai bài toán trên đối ngẫu với nhau; A^T là ma trận chuyển vị của A .

6.2. Tính đối ngẫu của quy hoạch tuyến tính

Trước hết phát biểu hình thức hóa kết luận ở phần trên.

Định lý 6.1 (đối ngẫu yếu). Giả sử $x = (x_1, \dots, x_n)$ là một nghiệm khả thi của bài toán nguyên thủy, và $y = (y_1, \dots, y_m)$ là một nghiệm khả thi của bài toán đối ngẫu. Khi đó

$$\sum_{j=1}^n c_j x_j \geq \sum_{i=1}^m b_i y_i.$$

Chứng minh. Vì y khả thi cho bài toán đối ngẫu,

$$c_j \geq \sum_{i=1}^m a_{ij} y_i.$$

Lại có $x_j \geq 0$, nên

$$\sum_{j=1}^n c_j x_j \geq \sum_{j=1}^n \left(\sum_{i=1}^m a_{ij} y_i \right) x_j.$$

Tương tự, vì x khả thi cho bài toán nguyên thủy và $y_i \geq 0$,

$$\sum_{i=1}^m \left(\sum_{j=1}^n a_{ij} x_j \right) y_i \geq \sum_{i=1}^m b_i y_i.$$

Mệnh đề suy ra từ đẳng thức hiển nhiên

$$\sum_{j=1}^n \left(\sum_{i=1}^m a_{ij} y_i \right) x_j = \sum_{i=1}^m \left(\sum_{j=1}^n a_{ij} x_j \right) y_i.$$

Định lý trên chỉ nói rằng bài toán đối ngẫu cho ta một cận dưới của nghiệm tối ưu nguyên thủy. Định lý tiếp theo nói rằng cận dưới đó luôn đạt được.

Định lý 6.2 (đối ngẫu mạnh). Giả sử $x^* = (x_1^*, \dots, x_n^*)$ và $y^* = (y_1^*, \dots, y_m^*)$ lần lượt là nghiệm tối ưu của bài toán nguyên thủy và bài toán đối ngẫu. Khi đó

$$\sum_{j=1}^n c_j x_j^* = \sum_{i=1}^m b_i y_i^*.$$

Chứng minh định lý này khá rườm rà, ở đây bỏ qua; tài liệu tham khảo [1] có chứng minh chi tiết.

Từ hai định lý trên, nếu x, y lần lượt là nghiệm tối ưu của hai bài toán đối ngẫu, thì trong chứng minh đối ngẫu yếu mọi dấu bằng đều phải xảy ra. Điều này dẫn tới định lý sau.

Định lý 6.3 (bù trừ lỏng). Giả sử x, y lần lượt là nghiệm khả thi của bài toán nguyên thủy và đối ngẫu. Khi đó x, y đều tối ưu khi và chỉ khi đồng thời thỏa:

- Với mọi $1 \leq j \leq n$: hoặc $x_j = 0$, hoặc $\sum_i a_{ij} y_i = c_j$.
- Với mọi $1 \leq i \leq m$: hoặc $y_i = 0$, hoặc $\sum_j a_{ij} x_j = b_i$.

Nguyên lý đối ngẫu nổi một bài toán cực đại với một bài toán cực tiểu tương ứng. Trong quá trình đối ngẫu, mỗi ràng buộc của bài toán gốc, tức mỗi hàng của ma trận gốc, sinh ra một biến trong bài toán đối ngẫu, tức một cột của ma trận chuyển vị; mỗi biến của bài toán gốc biến thành một ràng buộc trong bài toán đối ngẫu. Định lý 6.2 cho biết hai giá trị tối ưu bằng nhau, còn Định lý 6.3 cung cấp quan hệ giữa nghiệm tối ưu của hai phía.

Xét thêm những ràng buộc không hoàn toàn chuẩn. Nếu bài toán gốc có ràng buộc $\sum_j a_{rj} x_j = b_r$, ta có thể tách nó thành $\sum_j a_{rj} x_j \geq b_r$ và $-\sum_j a_{rj} x_j \geq -b_r$. Giả sử hai ràng buộc này sinh ra các biến đối ngẫu y_r, y'_r , đều không âm. Trong ràng buộc thứ j của bài toán đối ngẫu sẽ xuất hiện

$$\sum_{i \neq r} a_{ij} y_i + a_{rj} y_r - a_{rj} y'_r = \sum_{i \neq r} a_{ij} y_i + a_{rj} (y_r - y'_r).$$

Nếu đặt $w_r = y_r - y'_r$, thì w_r không bị chặn dấu. Nghĩa là một ràng buộc đẳng thức sau khi đối ngẫu trở thành một biến không bị chặn dấu. Tương tự, ràng buộc $\sum_j a_{rj} x_j \leq b_r$ sau khi đối ngẫu sinh ra một biến không dương.

6.3. Đối ngẫu của một số bài toán kinh điển

6.3.1. Luồng cực đại. Người đã học luồng mạng đều biết đối ngẫu của luồng cực đại là cắt nhỏ nhất. Ta chứng minh điều này.

Mô hình tuyến tính của luồng cực đại đã nêu ở Mục 4.2:

$$\begin{aligned} &\text{cực đại hóa} && f(t, s), \\ &\text{thỏa} && f(u, v) \leq c(u, v), \quad (u, v) \in E, \\ & && \sum_v f(v, u) = \sum_v f(u, v), \quad u \in V, \\ & && f(u, v) \geq 0, \quad (u, v) \in E \cup \{(t, s)\}. \end{aligned}$$

Bài toán cắt nhỏ nhất có thể biểu diễn thành quy hoạch tuyến tính:

$$\begin{array}{ll} \text{cực tiểu hóa} & \sum_{(u,v) \in E} c(u,v)d_{uv}, \\ \text{thỏa} & d_{uv} - p_u + p_v \geq 0, \quad (u,v) \in E, \\ & p_s - p_t \geq 1, \\ & p_u, d_{uv} \in \{0, 1\}. \end{array}$$

Một lát cắt chia tập đỉnh thành hai tập rời nhau S, T , trong đó $s \in S, t \in T$. Đặt $p_u = [u \in S]$ và $d_{uv} = \max\{p_u - p_v, 0\}$, lát cắt tương ứng với một nghiệm khả thi của mô hình trên. Ngược lại, trong nghiệm tối ưu, nếu $p_u = 1, p_v = 0$ thì phải có $d_{uv} = 1$, còn các trường hợp khác có thể lấy $d_{uv} = 0$. Vì thế nghiệm tối ưu của quy hoạch tuyến tính này tương ứng với một lát cắt nhỏ nhất.

Xét bài toán đối ngẫu của luồng cực đại. Gọi d_{uv} là biến sinh ra từ ràng buộc dung lượng của cạnh (u, v) , và p_u là biến sinh ra từ bảo toàn luồng tại đỉnh u . Bài toán đối ngẫu là

$$\begin{array}{ll} \text{cực tiểu hóa} & \sum_{(u,v) \in E} c(u,v)d_{uv}, \\ \text{thỏa} & d_{uv} - p_u + p_v \geq 0, \quad (u,v) \in E, \\ & p_s - p_t \geq 1, \\ & d_{uv} \geq 0, \quad (u,v) \in E. \end{array}$$

So với mô hình cắt nhỏ nhất, nó chỉ thiếu ràng buộc $p_u, d_{uv} \in \{0, 1\}$. Có thể chứng minh luôn tồn tại một nghiệm tối ưu thỏa ràng buộc nguyên này; phần chứng minh không liên quan trực tiếp tới chủ đề chính nên được đưa vào phụ lục. Ta nhận được kết luận:

Định lý 6.4 (định lý luồng cực đại–cắt nhỏ nhất). Trong mạng luồng có nguồn s và đích t , giá trị luồng cực đại từ s tới t bằng dung lượng cắt nhỏ nhất giữa s và t .

6.3.2. Ghép cặp trọng số lớn nhất trên đồ thị hai phía. Bài toán ghép cặp trọng số lớn nhất trên đồ thị hai phía có thể viết:

$$\begin{array}{ll} \text{cực đại hóa} & \sum_{(u,v) \in E} c_{uv}d_{uv}, \\ \text{thỏa} & \sum_{v \in Y} d_{uv} \leq 1, \quad u \in X, \\ & \sum_{u \in X} d_{uv} \leq 1, \quad v \in Y, \\ & d_{uv} \in \{0, 1\}. \end{array}$$

Ở đây d_{uv} biểu thị cạnh (u, v) có được chọn hay không, c_{uv} là trọng số ghép, còn X, Y là hai phía của đồ thị. Tương tự cắt nhỏ nhất, bỏ ràng buộc nguyên không làm thay đổi đáp án.⁶

Gọi p_u, p_v là các biến đối ngẫu tương ứng với hai loại ràng buộc trên. Đối ngẫu là

$$\begin{array}{ll} \text{cực tiểu hóa} & \sum_{u \in X} p_u + \sum_{v \in Y} p_v, \\ \text{thỏa} & p_u + p_v \geq c_{uv}, \quad u \in X, v \in Y, \\ & p_u, p_v \geq 0. \end{array}$$

Diễn giải tổ hợp của bài toán này là: gán cho mỗi đỉnh một **nhãn đỉnh**; với mỗi cạnh, tổng nhãn của hai đầu mút không nhỏ hơn trọng số cạnh; mọi nhãn không âm; cực tiểu hóa tổng nhãn. Ta gọi đây là bài toán tổng nhãn đỉnh nhỏ nhất. Do đồ thị không đổi, có kết luận:

⁶Vì có thể tìm được một nghiệm nguyên có giá trị hàm mục tiêu bằng nghiệm tối ưu.

Định lý 6.5. Trong một đồ thị hai phía có trọng số, trọng số ghép cặp lớn nhất bằng tổng nhân đỉnh nhỏ nhất.

Xét phiên bản yếu hơn, khi mọi cạnh có trọng số 1, bài toán gốc là ghép cặp cực đại hai phía. Nếu nghiệm đối ngẫu được yêu cầu nguyên, ràng buộc đối ngẫu nói rằng với mỗi cạnh, ít nhất một trong hai đầu mút phải có nhãn 1, tức ít nhất một đỉnh được chọn để **phủ** cạnh đó. Vì thế:

Định lý 6.6. Trong đồ thị hai phía, kích thước ghép cặp cực đại bằng kích thước phủ đỉnh nhỏ nhất.

Bài toán ghép cặp trọng số lớn nhất hai phía có thể giải hiệu quả bằng thuật toán KM hoặc trực tiếp bằng luồng chi phí nhỏ nhất. Thông thường ta có xu hướng chuyển bài toán tổng nhân đỉnh nhỏ nhất thành bài toán ghép cặp trọng số lớn nhất để giải.

Ví dụ 3 (AMS minimum spanning tree). Cho một đồ thị vô hướng có trọng số G và một cây khung T trên đồ thị đó. Có thể sửa trọng số của một cạnh; chi phí bằng trị tuyệt đối của độ thay đổi trọng số. Hãy sửa sao cho T trở thành một cây khung nhỏ nhất của G , với tổng chi phí nhỏ nhất.

$$n < 50, \quad m < 800.$$

Để T trở thành cây khung nhỏ nhất, thao tác trên cạnh cây chỉ cần giảm trọng số, còn trên cạnh ngoài cây chỉ cần tăng trọng số. Xét một cạnh ngoài cây e nối hai đỉnh u, v . Nếu T là cây khung nhỏ nhất, trọng số của e phải không nhỏ hơn trọng số của bất kỳ cạnh nào trên đường đi $u-v$ trong cây. Nếu cạnh cây t nằm trên đường đi đó, cần

$$V(e) + A_e \geq V(t) - A_t,$$

tức

$$A_t + A_e \geq V(t) - V(e).$$

Xem mỗi cạnh của đồ thị gốc là một đỉnh, chia theo việc cạnh đó có nằm trong T hay không; đây là một đồ thị hai phía. Nếu coi A_t, A_e là nhãn đỉnh, ràng buộc trên yêu cầu tổng nhân không nhỏ hơn $V(t) - V(e)$. Do đó bài toán trở thành tổng nhân đỉnh nhỏ nhất, rồi chuyển thành ghép cặp trọng số lớn nhất trên đồ thị hai phía.

6.4. Ứng dụng nguyên lý đối ngẫu trong thi tin học

6.4.1. Tìm nghiệm cơ bản ban đầu.

Ví dụ 4 (Flight Distance). Cho một đồ thị có hướng n đỉnh, m cạnh; cạnh (u, v) có độ dài w_{uv} . Có thể tăng hoặc giảm trọng số một cạnh, chi phí bằng độ thay đổi trọng số. Sau thao tác, yêu cầu với mọi cạnh trực tiếp nối hai đỉnh, độ dài cạnh đó không vượt quá độ dài đường đi ngắn nhất giữa hai đỉnh. Hãy tìm chi phí nhỏ nhất và in dưới dạng phân số.

$$n < 10, \quad m < 20.$$

Gọi E là tập cạnh. Với mỗi cạnh (u, v) , dùng t_{uv}^+ và t_{uv}^- lần lượt biểu diễn lượng tăng và giảm trọng số. Sau thao tác, trọng số cạnh là $w_{uv} + t_{uv}^+ - t_{uv}^-$. Gọi d_{uv} là độ dài đường đi ngắn nhất từ u tới v sau thao tác. Vì đề bài yêu cầu cạnh trực tiếp không dài hơn đường đi ngắn nhất, đồng thời đường đi ngắn nhất giữa hai đầu mút cũng không thể nhỏ hơn chính cạnh trực tiếp, ta có

$$d_{uv} = w_{uv} + t_{uv}^+ - t_{uv}^-.$$

Ngoài ra các khoảng cách ngắn nhất phải thỏa

$$d_{uv} \leq w_{ui} + t_{ui}^+ - t_{ui}^- + d_{iv} \quad (u \neq v, (u, i) \in E).$$

Từ đó viết được quy hoạch tuyến tính:

$$\begin{array}{ll} \text{cực tiểu hóa} & \sum_{(u,v) \in E} (t_{uv}^+ + t_{uv}^-), \\ \text{thỏa} & d_{uv} - t_{uv}^+ + t_{uv}^- = w_{uv}, \quad (u, v) \in E, \\ & -d_{uv} + t_{uv}^+ - t_{uv}^- = -w_{uv}, \quad (u, v) \in E, \\ & t_{ui}^+ - t_{ui}^- + d_{iv} - d_{uv} \geq -w_{ui}, \quad u \neq v, (u, i) \in E, \\ & d_{uv}, t_{uv}^+, t_{uv}^- \geq 0. \end{array}$$

Nếu giải trực tiếp, cần khởi tạo đơn hình. Nhưng vì mọi hệ số trong hàm mục tiêu đều không âm, sau khi lấy đối ngẫu, mọi hằng số ràng buộc đều không âm; vì vậy có thể chạy đơn hình trên bài toán đối ngẫu mà không cần khởi tạo. Đáp án cần dạng phân số, nên khi cài đặt cần dùng một lớp phân số.

6.4.2. Chuyển mô hình quy hoạch tuyến tính thành giao nửa mặt phẳng. Khi một quy hoạch tuyến tính chỉ có hai biến, mỗi ràng buộc giới hạn tập nghiệm vào một nửa mặt phẳng trong mặt phẳng hai chiều. Không gian nghiệm của quy hoạch tuyến tính khi đó là giao của các nửa mặt phẳng tương ứng.

Ví dụ 5 (Equations). Cho ba mảng độ dài n : a_i, b_i, c_i , và m truy vấn. Mỗi truy vấn cho hai tham số s, t . Hãy tìm các số thực không âm x_i thỏa

$$\sum_{i=1}^n a_i x_i = s, \quad \sum_{i=1}^n b_i x_i = t,$$

đồng thời cực đại hóa $\sum_i c_i x_i$. Với mỗi truy vấn, in giá trị lớn nhất đó hoặc kết luận vô nghiệm.

$$n \leq 10^5, \quad m \leq 10^4, \quad 1 \leq a_i, b_i, c_i, s, t \leq 10^4.$$

Với mỗi truy vấn, mô hình nguyên thủy là

$$\begin{array}{ll} \text{cực đại hóa} & \sum_{i=1}^n c_i x_i, \\ \text{thỏa} & \sum_{i=1}^n a_i x_i = s, \\ & \sum_{i=1}^n b_i x_i = t, \\ & x_i \geq 0, \quad i = 1, 2, \dots, n. \end{array}$$

Mô hình này chỉ có hai ràng buộc, nên sau khi đối ngẫu chỉ có hai biến, gọi là x, y :

$$\begin{array}{ll} \text{cực tiểu hóa} & sx + ty, \\ \text{thỏa} & a_i x + b_i y \geq c_i, \quad i = 1, 2, \dots, n, \\ & x, y \in \mathbb{R}. \end{array}$$

Mỗi ràng buộc $a_i x + b_i y \geq c_i$ là một nửa mặt phẳng. Tiền xử lý giao nửa mặt phẳng của toàn bộ các ràng buộc; với mỗi truy vấn, cần tìm điểm trong giao này làm $sx + ty$ nhỏ nhất. Nếu $sx + ty$ không bị chặn dưới, bài toán gốc vô nghiệm. Tổng độ phức tạp có thể đạt $O((n + m) \log n)$.

6.4.3. Chuyển mô hình quy hoạch tuyến tính thành luồng mạng. Trong bài toán luồng chi phí nhỏ nhất, ma trận quy hoạch tuyến tính thỏa rằng mỗi cột có đúng hai vị trí khác không, và hai giá trị đó lần lượt là $+1$ và -1 . Khi đó mỗi biến có thể xem là lưu lượng trên một cạnh, còn mỗi hàng là một phương trình bảo toàn luồng tại một đỉnh. Cách dựng cụ thể được trình bày trong ví dụ sau.

Thông thường, sau khi dựng được mô hình luồng mạng, dùng thuật toán luồng mạng sẽ nhanh hơn giải trực tiếp bằng đơn hình, vì các thuật toán luồng khai thác cấu trúc đặc biệt của bài toán, còn đơn hình tổng quát bỏ qua cấu trúc đó. Đây là lý do ta muốn chuyển quy hoạch tuyến tính thành luồng mạng.

Ví dụ 6 (Chefbook). Cho một đồ thị có hướng n đỉnh, m cạnh. Cạnh (u, v) ban đầu có trọng số L_{uv} , cùng các giới hạn S_{uv}, T_{uv} . Mỗi thao tác có thể chọn một trong hai loại: tăng toàn bộ trọng số các cạnh ra khỏi một đỉnh u thêm P_u , hoặc giảm toàn bộ trọng số các cạnh vào một đỉnh u đi Q_u , trong đó P_u, Q_u là số dương tùy ý. Mỗi loại thao tác tại mỗi đỉnh chỉ được thực hiện một lần, và có thể thực hiện cả hai loại tại cùng một đỉnh. Gọi trọng số cuối cùng của cạnh là L_{uv}^* ; yêu cầu $S_{uv} \leq L_{uv}^* \leq T_{uv}$, đồng thời cực đại hóa $\sum L_{uv}^*$ và in phương án.

$$n \leq 100, \quad m < n^2.$$

Mỗi ràng buộc viết được thành

$$S_{uv} \leq L_{uv} + P_u - Q_v \leq T_{uv}, \quad (u, v) \in E,$$

và cần cực đại hóa $\sum_{(u,v) \in E} (L_{uv} + P_u - Q_v)$. Bỏ qua hằng số trong hàm mục tiêu, gộp hạng tử theo đỉnh, hệ số của P_u là bậc ra $\deg_{\text{out}}[u]$, còn hệ số của Q_u là âm bậc vào $-\deg_{\text{in}}[u]$. Ta có

$$\begin{aligned} \text{cực đại hóa} \quad & \sum_u (P_u \deg_{\text{out}}[u] - Q_u \deg_{\text{in}}[u]), \\ \text{thỏa} \quad & P_u - Q_v \leq T_{uv} - L_{uv}, \quad (u, v) \in E, \\ & -P_u + Q_v \leq L_{uv} - S_{uv}, \quad (u, v) \in E, \\ & P_u, Q_u \geq 0, \quad u \in V. \end{aligned}$$

Ma trận của quy hoạch này có đúng hai hệ số khác không trên mỗi hàng và chúng lần lượt là $+1, -1$. Lấy đối ngẫu. Dùng x_{uv} cho loại ràng buộc thứ nhất, y_{uv} cho loại thứ hai:

$$\begin{aligned} \text{cực tiểu hóa} \quad & \sum_{(u,v) \in E} ((T_{uv} - L_{uv})x_{uv} + (L_{uv} - S_{uv})y_{uv}), \\ \text{thỏa} \quad & \sum_v (x_{uv} - y_{uv}) \geq \deg_{\text{out}}[u], \quad u \in V, \\ & \sum_u (-x_{uv} + y_{uv}) \geq -\deg_{\text{in}}[v], \quad v \in V, \\ & x_{uv}, y_{uv} \geq 0, \quad (u, v) \in E. \end{aligned}$$

Đưa bất đẳng thức về đẳng thức bằng biến phụ:

$$\begin{aligned} \sum_v (x_{uv} - y_{uv}) - \deg_{\text{out}}[u] - a_u &= 0, \\ \sum_u (-x_{uv} + y_{uv}) + \deg_{\text{in}}[v] - b_v &= 0. \end{aligned}$$

Sau biến đổi này, mỗi biến có thể xem là một cạnh: giá trị biến là lưu lượng, chi phí cạnh là hệ số của biến trong hàm mục tiêu, mỗi ràng buộc là một đỉnh. Nếu biến có hệ số $+1$ trong một ràng buộc, cạnh đi vào đỉnh đó; nếu hệ số -1 , cạnh đi ra. Như vậy mô hình tương đương với luồng chi phí.

Cách dựng tổng quát là: với mỗi cạnh $(u, v) \in E$, thêm cạnh $u \rightarrow v + n$ chi phí $T_{uv} - L_{uv}$, dung lượng vô hạn ứng với x_{uv} , và cạnh $v + n \rightarrow u$ chi phí $L_{uv} - S_{uv}$, dung lượng vô hạn ứng với y_{uv} . Các biến phụ chỉ xuất hiện một lần thì nối tới nguồn hoặc đích với chi phí 0. Các hằng số $-\deg_{\text{out}}[u]$ và $\deg_{\text{in}}[v]$ được xử lý bằng các cạnh có chặn dưới lưu lượng. Khi đó bài toán trở thành luồng chi phí cực đại có chặn dưới.

Bài này còn có cách xử lý khéo hơn. Vì ma trận thỏa tổng mỗi cột bằng 0, kể cả cột hằng số, mọi nghiệm khả thi thực ra phải thỏa dấu bằng trong hai ràng buộc:

$$\sum_v (x_{uv} - y_{uv}) = \deg_{\text{out}}[u], \quad \sum_u (-x_{uv} + y_{uv}) = -\deg_{\text{in}}[v].$$

Do tổng về phải của mọi ràng buộc là 0 và tổng về trái cũng là 0, nếu có bất kỳ ràng buộc nào chặt hơn dấu bằng thì cộng tất cả sẽ mâu thuẫn. Vì thế không cần thêm biến phụ. Hơn nữa tổng dung lượng ra và vào phù hợp, ngoài các cạnh này không có luồng khả thi khác, nên chỉ cần chạy luồng chi phí cực đại để bảo đảm mọi đẳng thức, không cần đặt chặn dưới; mô hình ít cạnh hơn và chạy nhanh hơn.

Sau khi tìm được đáp án, còn phải in phương án. Ta đã biết một nghiệm tối ưu của bài toán đối ngẫu, tức lưu lượng trên các cạnh. Theo Định lý 6.3, nếu $x_{uv} > 0$ thì nghiệm tối ưu của bài toán gốc phải thỏa $P_u - Q_v = T_{uv} - L_{uv}$; nếu $y_{uv} > 0$ thì phải thỏa $P_u - Q_v = L_{uv} - S_{uv}$. Ngoài ra còn có các ràng buộc ban đầu

$$L_{uv} - S_{uv} \leq P_u - Q_v \leq T_{uv} - L_{uv}.$$

Đây là một hệ ràng buộc hiệu, có thể giải bằng đường đi ngắn nhất. Với $P_u - Q_v \leq T_{uv} - L_{uv}$, nối cạnh tương ứng trong đồ thị ràng buộc hiệu; với ràng buộc ngược cũng nối tương tự. Đẳng thức được tách thành hai bất đẳng thức. Chạy thuật toán đường đi ngắn nhất từ một điểm tùy ý, nhãn khoảng cách của các đỉnh sẽ cho một nghiệm tối ưu. Để bảo đảm P, Q không âm, cộng cùng một hằng số lớn vào mọi P, Q ; điều này không ảnh hưởng tới đáp án và các ràng buộc.

Với bài này, tác giả so sánh thời gian chạy giữa đơn hình và luồng mạng trên 10 bộ dữ liệu $n = 50, m = 1000$: thuật toán luồng chi phí mất khoảng 60 ms, trong khi đơn hình mất khoảng 1630 ms. Điều đó cho thấy cấu trúc đặc biệt của bài toán ảnh hưởng rất lớn tới thời gian chạy.

Ví dụ 7 (Orz the MST). Cho một đồ thị vô hướng liên thông n đỉnh, m cạnh. Với mỗi cạnh, tăng trọng số thêm 1 tốn chi phí a_i , giảm trọng số đi 1 tốn chi phí b_i . Đã chỉ định một cây khung của đồ thị. Hãy sửa trọng số sao cho cây khung đó trở thành cây khung nhỏ nhất, với tổng chi phí nhỏ nhất.

$$n < 300, \quad m < 1000.$$

Tương tự Ví dụ 3, chia cạnh theo việc có thuộc cây T hay không. Gọi tập cạnh cây là T , cạnh ngoài cây là $E \setminus T$. Giả sử cạnh cây i giảm x_i , cạnh ngoài cây j tăng y_j . Nếu i được cạnh j phủ, tức i nằm trên đường đi trong cây giữa hai đầu của j , thì cần

$$w_i - x_i \leq w_j + y_j, \quad \text{tức} \quad x_i + y_j \geq w_i - w_j.$$

Đặt $\text{cover}(i, j)$ cho quan hệ phủ, mô hình là

$$\begin{aligned} &\text{cực tiểu hóa} \quad \sum_{i \in T} b_i x_i + \sum_{j \in E \setminus T} a_j y_j, \\ &\text{thỏa} \quad \begin{aligned} &x_i + y_j \geq w_i - w_j, \quad i \in T, j \in E \setminus T, \text{ cover}(i, j) = 1, \\ &x_i, y_j \geq 0. \end{aligned} \end{aligned}$$

Lấy đối ngẫu với biến s_{ij} :

$$\begin{aligned} &\text{cực đại hóa} \quad \sum_{\text{cover}(i, j)=1} (w_i - w_j) s_{ij}, \\ &\text{thỏa} \quad \begin{aligned} &\sum_{j: \text{cover}(i, j)=1} s_{ij} \leq b_i, \quad i \in T, \\ &\sum_{i: \text{cover}(i, j)=1} s_{ij} \leq a_j, \quad j \in E \setminus T, \\ &s_{ij} \geq 0. \end{aligned} \end{aligned}$$

Xem ràng buộc thứ nhất là chặn bậc ra của phía trái, ràng buộc thứ hai là chặn bậc vào của phía phải. Với đỉnh trái i , nối nguồn tới i dung lượng b_i , chi phí 0. Với đỉnh phải j , nối j tới đích dung lượng a_j , chi phí 0. Nếu i được j phủ, nối $i \rightarrow j$ chi phí $w_i - w_j$, dung lượng vô hạn, rồi chạy luồng chi phí cực đại. Mô hình đối ngẫu này chính là bài toán ghép cặp nhiều lần tối ưu trên đồ thị hai phía.

Ví dụ 8 (Defense Line). Chiến tuyến là một dãy độ dài n . Cần xây tháp trên dãy; xây một tháp tại vị trí i tốn chi phí c_i , và mỗi vị trí có thể xây tùy ý nhiều tháp. Có m ràng buộc, mỗi ràng buộc có dạng: trên đoạn $[l_i, r_i]$ phải có ít nhất d_i tháp. Hãy tìm chi phí nhỏ nhất thỏa mọi ràng buộc.

$$n < 1000, \quad m < 10000.$$

Mỗi ràng buộc là tổng số tháp trên một đoạn. Để dùng mô hình luồng, đặt biến tiền tố $S_i = \sum_{j=1}^i x_j$. Khi đó tổng trên đoạn biến thành hiệu của hai biến:

$$\begin{aligned} \text{cực tiểu hóa} \quad & \sum_{j=1}^n c_j (S_j - S_{j-1}), \\ \text{thỏa} \quad & S_{r_i} - S_{l_i-1} \geq d_i, \quad i = 1, 2, \dots, m, \\ & S_j - S_{j-1} \geq 0, \quad j = 1, 2, \dots, n, \\ & S_j \geq 0, \quad j = 0, 1, \dots, n. \end{aligned}$$

Đối ngẫu của mô hình này có cấu trúc mỗi biến xuất hiện trong hai ràng buộc, có thể chuyển thành bài toán luồng mạng trên các đỉnh tiền tố.

6.4.4. Tiểu kết. Khâu quan trọng nhất để giải hiệu quả một số bài toán quy hoạch tuyến tính là khai thác tính chất riêng của mô hình cụ thể. Ví dụ, quy hoạch tuyến tính chỉ có hai biến có thể giải bằng giao nửa mặt phẳng; mô hình mà mỗi biến chỉ xuất hiện trong hai ràng buộc và tổng hệ số bằng 0 có thể giải bằng luồng mạng. Nguyên lý đối ngẫu là một công cụ để khai thác các tính chất như vậy.

7. Tổng kết

Nhìn các bài toán thi tin học từ góc độ quy hoạch tuyến tính làm mô tả bài toán rõ ràng hơn, đồng thời đem lại một cách giải mới. Nhưng sau khi tổng quát hóa bài toán, ta khó tránh khỏi việc đánh mất tính chất riêng của bài toán, khiến việc giải hiệu quả trở nên khó hơn.

Bằng cách tổng kết biểu diễn quy hoạch tuyến tính của những bài toán có cấu trúc đặc biệt, như luồng mạng, ta có thể chuyển một số quy hoạch tuyến tính đặc biệt thành các bài toán khác, rồi dùng thuật toán chuyên biệt cho các bài toán đó để giải hiệu quả. Như vậy ta vừa có thể nhìn bài toán từ góc độ tổng quát, vừa tận dụng tốt cấu trúc riêng để đạt mục tiêu giải bài nhanh.

Lời cảm ơn

Xin cảm ơn cha mẹ đã nuôi dưỡng. Xin cảm ơn cô Chen Ying, huấn luyện viên Yu Linyun, đàn anh Huang Haoshuo, đàn anh Zhang Ruicai đã chỉ dẫn; cảm ơn Zhou Qihao và các bạn học khác đã góp ý và giúp đỡ bài viết này. Xin cảm ơn CCF đã cung cấp cơ hội học tập và giao lưu.

Tài liệu tham khảo

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, second edition.

2. Dimitris Bertsimas, John N. Tsitsiklis, *Introduction to Linear Optimization*.
3. Luca Trevisan, Stanford University CS261: Optimization, Handout 15.
4. Vijay V. Vazirani, *Approximation Algorithms*: Chapter 10, Introduction to LP-Duality; Chapter 13, The primal-dual schema.
5. Hu Botao, *Ứng dụng mô hình cắt nhỏ nhất trong thi tin học*.
6. Cao Qinxiang, *Quy hoạch tuyến tính và luồng mạng*.

Phụ lục

Quan hệ giữa nghiệm tối ưu của quy hoạch tuyến tính đối ngẫu luồng cực đại và lát cắt

Quy hoạch tuyến tính đối ngẫu của luồng cực đại là

$$\begin{array}{ll} \text{cực tiểu hóa} & \sum_{(u,v) \in E} c_{uv} d_{uv}, \\ \text{thỏa} & d_{uv} - p_u + p_v \geq 0, \quad (u, v) \in E, \\ & p_s - p_t \geq 1, \\ & d_{uv} \geq 0. \end{array}$$

Gọi nghiệm tối ưu là (d_{uv}^*, p_u^*) , giá trị tối ưu là

$$z^* = \sum_{(u,v) \in E} c_{uv} d_{uv}^*.$$

Để chứng minh tồn tại một lát cắt có dung lượng z^* , trước hết cần chứng minh tồn tại nghiệm (d_{uv}, p_u) sao cho

$$\sum c_{uv} d_{uv} = \sum c_{uv} d_{uv}^* \quad \text{và} \quad 0 \leq p_u \leq 1.$$

Vì p_u không xuất hiện trong hàm mục tiêu, và trong ràng buộc p_u chỉ xuất hiện qua hiệu $p_u - p_v$, có thể giả sử $p_t^* = 0$. Xét nghiệm

$$(d_{uv}, p_u) = (d_{uv}^*, \min\{p_u^*, 1\}).$$

Giá trị hàm mục tiêu không đổi. Ta kiểm tra ràng buộc $d_{uv} - p_u + p_v \geq 0$:

- Nếu $p_u^* > 1, p_v^* > 1$, ràng buộc tương đương $d_{uv} \geq 0$.
- Nếu $p_u^* > 1, p_v^* \leq 1$, do $p_u^* > p_u$, suy ra $d_{uv} - p_u + p_v > d_{uv}^* - p_u^* + p_v^* \geq 0$.
- Nếu $p_u^* \leq 1, p_v^* > 1$, có $p_u \leq p_v$, nên $d_{uv} \geq 0 \geq p_u - p_v$.

Các trường hợp khác tương tự, nên nghiệm mới vẫn khả thi và tối ưu. Tương tự có thể chứng minh tồn tại nghiệm tối ưu thỏa $p_u \geq 0$.

Bây giờ chứng minh tồn tại lát cắt $\text{cut}(s, t)$ sao cho $\text{cut}(s, t) \leq z^*$. Một lát cắt có thể biểu diễn bằng tập S thỏa $s \in S, t \notin S$, chia tập đỉnh thành S và $V \setminus S$. Lấy x là biến ngẫu nhiên phân bố đều trên $(0, 1]$, và định nghĩa

$$S_x = \{u \mid p_u \geq x\}.$$

Tập S_x ứng với một lát cắt $\text{cut}_x(s, t)$. Ta xét kỳ vọng dung lượng lát cắt này.

Nhờ tính tuyến tính của kỳ vọng, xét riêng từng cạnh (u, v) . Cạnh được tính vào lát cắt khi và chỉ khi $u \in S_x$ và $v \notin S_x$. Nếu $p_u \geq p_v$, đóng góp kỳ vọng là $c_{uv}(p_u - p_v)$; nếu không thì bằng 0. Trong mô hình cắt nhỏ nhất tuyến tính có ràng buộc

$$d_{uv} \geq \max\{0, p_u - p_v\},$$

nên

$$\mathbb{E}(|\text{cut}_x(s, t)|) \leq \sum_{(u,v) \in E} c_{uv} d_{uv} = z^*.$$

Vì kỳ vọng là trung bình của mọi phương án, tồn tại $x \in (0, 1]$ sao cho $|\text{cut}_x(s, t)| \leq z^*$. Chứng minh hoàn tất.

Bài 4. Bàn sơ lược về một số thuật toán và ứng dụng cho bài toán cắt nhỏ nhất trên đồ thị vô hướng

Wang Wentao, Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Bài viết nghiên cứu một lớp bài toán cắt nhỏ nhất định nghĩa trên đồ thị vô hướng có trọng số cạnh không âm trong thi tin học. Tác giả giới thiệu một số thuật toán xử lý lớp bài toán này, gồm thuật toán dựng cây Gomory-Hu (cây cắt nhỏ nhất), thuật toán dựng cây luồng tương đương của Gusfield, thuật toán Stoer-Wagner và thuật toán Karger. Qua một vài ví dụ, bài viết cũng trình bày ngắn gọn cách đưa những bài toán thi về mô hình cắt nhỏ nhất vô hướng.

1. Dẫn nhập

Trong thi tin học và trong một số bài toán thực tế, ta không chỉ gặp bài toán tìm luồng cực đại hoặc cắt nhỏ nhất giữa một nguồn và một đích trong đồ thị có hướng với trọng số cạnh không âm, mà còn gặp bài toán tìm luồng cực đại hoặc cắt nhỏ nhất giữa hai đỉnh bất kỳ trên đồ thị vô hướng có trọng số cạnh không âm. Với trường hợp đồ thị có hướng, các luận văn trước đã nghiên cứu khá nhiều thuật toán và dạng bài liên quan; còn trường hợp sau tạm thời chưa có nhiều khảo cứu sâu. Bài viết này triển khai theo hướng đó.

Trong Mục 2, để thuận tiện cho phần mô tả sau, tác giả định nghĩa một số khái niệm liên quan đến lát cắt trên đồ thị vô hướng.

Mục 3 giới thiệu cây Gomory-Hu, còn gọi là cây cắt nhỏ nhất, và cây luồng tương đương dùng để giải bài toán cắt nhỏ nhất giữa hai đỉnh bất kỳ, cùng các thuật toán dựng chúng.

Mục 4 giới thiệu thuật toán tất định Stoer-Wagner và thuật toán ngẫu nhiên Karger để tìm cắt nhỏ nhất toàn cục.

Với các thuật toán này, tác giả đưa ra chứng minh và cách cài đặt tóm lược.

Mục 5 quan sát qua vài ví dụ cách áp dụng các thuật toán đó trong quá trình giải bài thi tin học.

2. Các định nghĩa liên quan đến lát cắt

Cho đồ thị vô hướng có trọng số $G = (V, E, c)$, trong đó $c : E \rightarrow \mathbb{R}_{\geq 0}$ là ánh xạ trọng số. Trong bài viết này, mọi đồ thị đều mặc định là đồ thị vô hướng có trọng số không âm như vậy.

Với một tập cạnh $S \subseteq E$, định nghĩa

$$c(S) = \sum_{e \in S} c(e).$$

Với hai tập đỉnh rời nhau A, B , định nghĩa $c(A, B)$ là tổng trọng số của mọi cạnh (u, v) thỏa $u \in A, v \in B$.

Một **lát cắt** của đồ thị G là một phân hoạch $(U, V \setminus U)$ của tập đỉnh V . Ta gọi U và $V \setminus U$ là hai phía của lát cắt.

Tập cạnh của lát cắt này là tập mọi cạnh (u, v) thỏa $u \in U, v \in V \setminus U$, ký hiệu là $\delta(U)$.

Trọng số của lát cắt là $c(\delta(U))$, hoặc ký hiệu $c(U, V \setminus U)$.

Nếu $u \in U$ và $v \in V \setminus U$, ta gọi $(U, V \setminus U)$ là một lát cắt u, v . Khi đó U là phía u , còn $V \setminus U$ là phía v .

Định nghĩa **cắt nhỏ nhất** u, v là lát cắt u, v có trọng số nhỏ nhất. Ký hiệu một tập cạnh cắt nhỏ nhất như vậy là $\alpha(u, v)$, và ký hiệu trọng số của nó là $\lambda(u, v)$.

Dễ thấy cắt nhỏ nhất u, v tương đương với cắt nhỏ nhất v, u .

Chú ý rằng khuyên không ảnh hưởng tới lát cắt, nên trong mọi đồ thị dưới đây ta mặc định đã bỏ khuyên.

3. Cắt nhỏ nhất giữa hai đỉnh bất kỳ

Bài toán ở đây là tìm cắt nhỏ nhất cho mọi cặp đỉnh trong đồ thị.

Cách dễ nghĩ nhất là liệt kê trực tiếp mọi cặp đỉnh rồi chạy thuật toán luồng cực đại một lần cho mỗi cặp. Nhưng như vậy cần $\Theta(|V|^2)$ lần luồng cực đại, hiệu quả khá thấp. Vì thế ta cần khai thác tính chất của cắt nhỏ nhất để giảm số lần tính luồng cực đại.

3.1. Cây cắt nhỏ nhất

Định nghĩa một cây $T = (V, E_T)$ là **cây Gomory-Hu**, hay **cây cắt nhỏ nhất**, nếu với mọi cạnh $(s, t) \in E_T$, tập $\delta(W)$ là một $\alpha(s, t)$, trong đó W là một trong hai thành phần liên thông sinh ra sau khi xóa cạnh (s, t) khỏi cây T .

Tất nhiên, hiện tại ta chưa biết cây Gomory-Hu có tồn tại hay không. Nhưng phần sau sẽ chứng minh tính đúng đắn của thuật toán dựng cây, qua đó cũng chứng minh sự tồn tại. Vì vậy tạm thời có thể giả sử cây này tồn tại.

Định lý 1. Với ba đỉnh khác nhau bất kỳ a, b, c ,

$$\lambda(a, b) \geq \min(\lambda(a, c), \lambda(c, b)).$$

Chứng minh. Với ba đỉnh khác nhau bất kỳ a, b, c , xét cặp nhỏ nhất trong $\lambda(a, b), \lambda(b, c), \lambda(c, a)$; không mất tính tổng quát, giả sử cặp đó là $\lambda(a, b)$. Xét c nằm ở phía nào của $\alpha(a, b)$; giả sử c nằm ở phía b , phía còn lại làm tương tự. Khi đó có $\lambda(a, c) \leq \lambda(a, b)$. Mặt khác, do $\lambda(a, b)$ là cặp nhỏ nhất nên $\lambda(a, c) \geq \lambda(a, b)$. Suy ra $\lambda(a, c) = \lambda(a, b)$. Vậy với ba đỉnh khác nhau bất kỳ, trong ba giá trị $\lambda(a, b), \lambda(b, c), \lambda(c, a)$, luôn có hai giá trị nhỏ bằng nhau và một giá trị lớn hơn hoặc bằng chúng. Mệnh đề được chứng minh.

Thực ra ta còn có thể mở rộng định lý này thành:

$$\lambda(u, v) \geq \min(\lambda(u, w_1), \lambda(w_1, w_2), \dots, \lambda(w_k, v))$$

với hai đỉnh khác nhau bất kỳ u, v .

Định lý 2. Với mọi $u, v \in V$, giả sử (s, t) là cạnh có $\lambda(s, t)$ nhỏ nhất trên đường đi duy nhất từ u đến v trong cây Gomory-Hu T . Khi đó

$$\lambda(u, v) = \lambda(s, t).$$

Nói cách khác, cắt nhỏ nhất s, t chính là cắt nhỏ nhất u, v , và cắt nhỏ nhất u, v bằng trọng số nhỏ nhất trên đường đi từ u đến v trong cây Gomory-Hu.

Chứng minh. Từ phần mở rộng của Định lý 1, có $\lambda(u, v) \geq \lambda(s, t)$. Theo định nghĩa, u và v nằm ở hai phía của $\alpha(s, t)$, nên $\lambda(u, v) \leq \lambda(s, t)$. Do đó $\lambda(u, v) = \lambda(s, t)$.

Định nghĩa 1. Với một tập V , xét hàm $f(U)$ trên các tập con $U \subseteq V$. Nếu với mọi $A, B \subseteq V$,

$$f(A) + f(B) \geq f(A \cup B) + f(A \cap B),$$

thì f được gọi là **hàm dưới mô-đun** (submodular).

Định lý 3. Hàm trọng số lát cắt $c(\delta(U))$ là hàm dưới mô-đun.

Chứng minh. Với hai tập đỉnh A, B , chia mọi đỉnh thành bốn phần:

$$A \cap B, \quad A \setminus B, \quad B \setminus A, \quad V \setminus (A \cup B).$$

Đếm đóng góp của các cạnh giữa bốn phần này vào hai vế của bất đẳng thức dưới mô-đun cho thấy mỗi loại cạnh ở vế trái xuất hiện không ít hơn ở vế phải. Vì vậy

$$c(\delta(A)) + c(\delta(B)) \geq c(\delta(A \cup B)) + c(\delta(A \cap B)).$$

Định nghĩa 2. Với một hàm f trên các tập con của V , nếu

$$f(U) = f(V \setminus U),$$

thì f được gọi là **hàm đối xứng**.

Định nghĩa 3. Với một hàm f trên các tập con của V , nếu với mọi $A, B \subseteq V$,

$$f(A) + f(B) \geq f(A \setminus B) + f(B \setminus A),$$

thì f được gọi là **hàm phản mô-đun** (posi-modular). Tác giả nguồn không tìm được dịch ngữ tương ứng, nên dùng tên “phản mô-đun” để thay thế.

Định lý 4. Nếu f vừa là hàm dưới mô-đun, vừa là hàm đối xứng, thì f là hàm phản mô-đun.

Chứng minh.

$$\begin{aligned} f(A) + f(B) &= f(V \setminus A) + f(B) \\ &\geq f((V \setminus A) \cap B) + f((V \setminus A) \cup B) \\ &= f(B \setminus A) + f(V \setminus (A \setminus B)) \\ &= f(B \setminus A) + f(A \setminus B). \end{aligned}$$

Hiển nhiên, hàm trọng số lát cắt cũng là hàm phản mô-đun.

Định lý 5. Xét một cắt nhỏ nhất $\alpha(s, t)$ của hai đỉnh s, t . Gọi W là một phía của $\alpha(s, t)$. Khi đó với mọi $u, v \in W$, $u \neq v$, tồn tại một cắt nhỏ nhất u, v là $\delta(X)$ sao cho $X \subseteq W$.

Chứng minh. Không mất tính tổng quát, giả sử $s \in W$, và chọn một cắt nhỏ nhất u, v là $\delta(X)$ sao cho $u \in X, v \notin X$. Các trường hợp khác đều có thể đổi ký hiệu để đưa về dạng này.

Xét hai trường hợp.

Nếu $t \notin X$, do hàm trọng số lát cắt là dưới mô-đun, có

$$c(\delta(X)) + c(\delta(W)) \geq c(\delta(X \cap W)) + c(\delta(X \cup W)).$$

Ở đây $\delta(X \cap W)$ là một lát cắt u, v , nên $c(\delta(X \cap W)) \geq c(\delta(X))$. Đồng thời, $\delta(X \cup W)$ là một lát cắt s, t , nên $c(\delta(X \cup W)) \geq c(\delta(W))$. Vì thế mọi bất đẳng thức ở trên đều phải lấy dấu bằng, và $\delta(X \cap W)$ là một cắt nhỏ nhất u, v .

Nếu $t \in X$, do hàm trọng số lát cắt là phản mô-đun, có

$$c(\delta(X)) + c(\delta(W)) \geq c(\delta(W \setminus X)) + c(\delta(X \setminus W)).$$

Ở đây $\delta(W \setminus X)$ là một lát cắt u, v , nên $c(\delta(W \setminus X)) \geq c(\delta(X))$. Đồng thời, $\delta(X \setminus W)$ là một lát cắt s, t , nên $c(\delta(X \setminus W)) \geq c(\delta(W))$. Vì vậy mọi bất đẳng thức cũng lấy dấu bằng, và $\delta(W \setminus X)$ là một cắt nhỏ nhất u, v .

Kết hợp hai trường hợp, mệnh đề được chứng minh.

Đây là định lý quan trọng và cốt lõi nhất để dựng cây Gomory-Hu. Nó cho thấy các cắt nhỏ nhất có thể được chọn để “không giao nhau”: những cặp đỉnh nằm trong cùng một phía của một cắt nhỏ nhất có cắt nhỏ nhất không liên quan tới phía kia, từ đó sinh ra cấu trúc đệ quy.

Định nghĩa 4. Cho đồ thị $G = (V, E)$ và tập đỉnh $R \subseteq V$. Định nghĩa cây Gomory-Hu của R trong G là một cặp (T, C) , trong đó $T = (R, E_T)$, còn $C = (C_r \mid r \in R)$ là một phân hoạch của V theo tập đỉnh R , thỏa:

- Với mọi $r \in R$, có $r \in C_r$.
- Với mọi cạnh $(s, t) \in E_T$, hai thành phần của $T - (s, t)$ biểu diễn một cắt nhỏ nhất s, t trong G . Cụ thể, nếu một thành phần của $T - (s, t)$ là X , thì

$$\delta\left(\bigcup_{r \in X} C_r\right)$$

là một cắt nhỏ nhất s, t trong G .

Chú ý rằng cây Gomory-Hu của G chính là trường hợp $R = V$.

Thuật toán. Mã giả của cách dựng đệ quy:

```
Gomory-Hu(G, R)
  if |R| = 1 then
    T = ({r}, empty)
    C_r = V
    return (T, C)
  end if

  choose r1, r2 in R, r1 != r2
  let delta(W) be a minimum r1,r2 cut, with r1 in W
  G1 = graph obtained from G by contracting V \ W into v1
  R1 = R cap W
  G2 = graph obtained from G by contracting W into v2
```

```

R2 = R \ W

(T1, (C^1_r | r in R1)) = Gomory-Hu(G1, R1)
(T2, (C^2_r | r in R2)) = Gomory-Hu(G2, R2)

choose r' with v1 in C^1_{r'}
choose r'' with v2 in C^2_{r''}
T = (R1 union R2, E_T1 union E_T2 union {(r', r'')})

for all r in R1 do C_r = C^1_r end for
for all r in R2 do C_r = C^2_r end for
C_{r'} = C_{r'} \ {v1}
C_{r''} = C_{r''} \ {v2}
return (T, C)

```

Chứng minh tính đúng đắn. Theo định nghĩa cây Gomory-Hu, ta cần chứng minh mọi cạnh $(s, t) \in E_T$ đều thỏa: hai thành phần của $T - (s, t)$ biểu diễn một cắt nhỏ nhất s, t trong G . Khi $|R| = 1$ điều này hiển nhiên.

Với trường hợp tổng quát, nếu $(s, t) \in T_1$ hoặc $(s, t) \in T_2$, theo Định lý 5, các đỉnh ngoài T_1 hoặc T_2 không ảnh hưởng đến cắt nhỏ nhất giữa các đỉnh bên trong, nên T_1 và T_2 đều là cây Gomory-Hu hợp lệ.

Vì vậy chỉ cần chứng minh cạnh mới thêm (r', r'') là hợp lệ. Gọi $\delta(W)$ là cắt nhỏ nhất r_1, r_2 được thuật toán tìm, với $r_1 \in W$. Ta cần chứng minh $\delta(W)$ cũng là cắt nhỏ nhất r', r'' . Do $\delta(W)$ đã là một lát cắt r', r'' , chỉ cần chứng minh $\lambda(r', r'') \geq \lambda(r_1, r_2)$.

Giả sử $r_1 \neq r'$ và $r_2 \neq r''$; các trường hợp còn lại tương tự. Nếu $\lambda(r_1, r') < \lambda(r_1, r_2)$, thì do trong G_1 tập $V \setminus W$ bị co thành một đỉnh v_1 , và $v_1 \in C_{r'}^1$, cắt nhỏ nhất r_1, r' cũng sẽ là một lát cắt r_1, r_2 , mâu thuẫn với việc $\lambda(r_1, r_2)$ là cắt nhỏ nhất r_1, r_2 . Do đó $\lambda(r_1, r') \geq \lambda(r_1, r_2)$. Tương tự, $\lambda(r_2, r'') \geq \lambda(r_1, r_2)$.

Theo Định lý 1,

$$\lambda(r', r'') \geq \min\{\lambda(r', r_1), \lambda(r_1, r_2), \lambda(r_2, r'')\} = \lambda(r_1, r_2).$$

Mệnh đề được chứng minh. Nếu $r' = r_1$ hoặc $r'' = r_2$, chỉ là bớt một số hạng trong biểu thức min, kết luận vẫn giống hệt.

Hệ quả 1. Để dựng cây Gomory-Hu của $R \subseteq V$ trong đồ thị G , chỉ cần tính $|R| - 1$ lần cắt nhỏ nhất trên đồ thị có kích thước không vượt quá G .

Khi $R = V$, cây dựng được là cây Gomory-Hu của G . Vì vậy toàn bộ thuật toán chỉ cần $\Theta(|V|)$ lần luồng cực đại.

Cài đặt không đệ quy. Trong thuật toán đệ quy ở trên, việc nối cạnh cần dùng phân hoạch C , mà phân hoạch này khá khó ghi lại trong cài đặt cụ thể. Vì thế ta xét cách cài đặt không đệ quy.

Thứ tự đệ quy không quan trọng. Có thể nhìn thuật toán như sau: tại mỗi thời điểm có nhiều tập đỉnh. Mỗi lần chọn tùy ý một tập có kích thước lớn hơn 1, tách nó thành hai tập, và tiếp tục cho đến khi mọi tập đều có kích thước 1. Các cạnh của cây Gomory-Hu cũng có thể được duy trì động: tại mỗi thời điểm, mỗi cạnh không nối hai đỉnh đơn lẻ mà nối hai tập đỉnh. Nếu xem mỗi tập đỉnh là một đỉnh lớn, các cạnh đó tạo thành một cây trên các đỉnh lớn, còn những đỉnh bị co trong thuật toán đệ quy chính là các cây con kề với mỗi tập đỉnh. Khi tách một tập đỉnh, phân hoạch của những đỉnh bị co cũng chính là phân hoạch các cạnh kề với tập đó.

Ta cài đặt thuật toán không đệ quy như sau. Với mỗi tập đỉnh, chọn đỉnh có chỉ số nhỏ nhất làm đại diện. Với mỗi đỉnh x , ghi một đỉnh mà nó trở tới là f_x . Nếu x là đại diện của tập chứa nó, cạnh từ x đến

fa_x biểu diễn một cạnh thật của cây Gomory-Hu, nối tập có đại diện x với tập có đại diện fa_x , đồng thời đã có trọng số. Nếu x không phải đại diện của tập chứa nó, fa_x biểu diễn quan hệ thuộc tập; fa_x là đại diện của tập đó. Ban đầu mọi đỉnh nằm trong cùng một tập, nên ngoài đỉnh 1, mọi fa_x đều bằng 1.

Sau đó liệt kê $u = 2, 3, \dots, |V|$. Nếu fa_u đang biểu diễn quan hệ thuộc tập, ta tách tập chứa u . Đặt $v = fa_u$, chọn v và u làm hai đỉnh dùng để tách theo cắt nhỏ nhất. Sau khi tính cắt nhỏ nhất, ta chia tập đỉnh thành hai phần, đồng thời sửa mọi cạnh cây kề với tập hiện tại. Cụ thể, mọi đỉnh x được chia về phía u và thỏa $fa_x = v$ đều được đổi thành $fa_x = u$, trọng số không đổi. Bước này vừa xử lý các đỉnh trong tập, vừa xử lý các cạnh cây thật. Riêng với cạnh giữa u và v , nếu tập mà fa_v đại diện cũng bị chia về phía u , cần đặt $fa_u = fa_v$, chuyển trọng số cũ sang u , rồi đặt $fa_v = u$ với trọng số bằng giá trị cắt nhỏ nhất. Ngược lại, đặt $fa_u = v$ với trọng số bằng giá trị cắt nhỏ nhất. Như vậy một lần tách hoàn tất.

Mã giả:

```
Gomory-Hu Incremental Method(G)
w_1, w_2, ..., w_|V| = NULL
fa_1 = NULL
fa_2, fa_3, ..., fa_|V| = 1

for u = 2 to |V| do
    v = fa_u
    compute a minimum v,u cut after contracting other current sets
    obtain alpha(v,u) and lambda(v,u)

    for x = 1 to |V| do
        if x != u and fa_x = v and x is on the u side then
            fa_x = u
        end if
    end for

    if fa_v is on the u side then
        fa_u = fa_v
        w_u = w_v
        fa_v = u
        w_v = lambda(v,u)
    else
        fa_u = v
        w_u = lambda(v,u)
    end if
end for

E = empty
for u = 1 to |V| do
    if fa_u != NULL then
        add edge (fa_u, u) to E with weight w_u
    end if
end for
return T = (V, E, c)
```

Mỗi lần tách chỉ biến cạnh $u \rightarrow fa_u$ từ một cạnh biểu diễn quan hệ thuộc tập thành cạnh thật của cây. Vì thế trong quá trình liệt kê, những đỉnh đã liệt kê trước đó đều có cạnh fa là cạnh cây, còn những đỉnh chưa liệt kê đều có cạnh fa biểu diễn quan hệ thuộc tập. Do đó khi liệt kê tới một đỉnh, chắc chắn ta sẽ thực hiện đúng một lần tách cho nó.

3.2. Thuật toán dựng cây luồng tương đương của Gusfield

Đôi khi ta không yêu cầu cây Gomory-Hu phản ánh cụ thể cắt nhỏ nhất là gì, mà chỉ cần biết trọng số cắt nhỏ nhất. Nói cách khác, ta cần một cây sao cho cắt nhỏ nhất u, v bằng trọng số nhỏ nhất trên đường đi giữa u và v của cây; với mỗi cạnh cây, trọng số cạnh là trọng số cắt nhỏ nhất giữa hai đầu cạnh, nhưng không yêu cầu hai thành phần liên thông sau khi xóa cạnh đó biểu diễn đúng lát cắt nhỏ nhất của hai đầu cạnh. Ta gọi cây như vậy là **cây luồng tương đương** (equivalent flow tree).

Thuật toán Gusfield để dựng cây luồng tương đương đơn giản hơn nhiều so với cây Gomory-Hu. Ta chỉ cần sửa nhẹ thuật toán dựng cây Gomory-Hu: trong quá trình đệ quy, không chọn hai đỉnh theo cách cũ để nối cạnh, mà trực tiếp nối một cạnh có trọng số bằng cắt nhỏ nhất tương ứng giữa nguồn s và đích t . Chứng minh cũng đơn giản hơn. Trước hết, hai phần đệ quy đều dựng ra cây luồng tương đương hợp lệ. Sau khi nối thêm cạnh ở giữa, ta chỉ cần chứng minh mọi đường đi từ một đỉnh u ở phía s tới một đỉnh v ở phía t đều thỏa tính chất cắt nhỏ nhất tương đương:

$$\lambda(u, v) = \min\{\text{trọng số nhỏ nhất trên đường } u \rightarrow s, \lambda(s, t), \text{ trọng số nhỏ nhất trên đường } t \rightarrow v\}.$$

Từ Định lý 1, vế trái không nhỏ hơn vế phải. Vì vậy chỉ cần chứng minh vế trái không lớn hơn vế phải.

Hiển nhiên, cắt nhỏ nhất s, t cũng là một lát cắt u, v , nên $\lambda(u, v) \leq \lambda(s, t)$. Xét cắt nhỏ nhất được biểu diễn bởi cạnh nhẹ nhất trên đường $u \rightarrow s$. Theo Định lý 5, cắt nhỏ nhất này chắc chắn không cắt rời tập đỉnh phía t ; vì vậy toàn bộ tập phía t hoặc nằm ở phía s , hoặc nằm ở phía u của lát cắt này. Nếu nằm ở phía s , lát cắt đó cũng là lát cắt u, v . Nếu nằm ở phía u , lát cắt đó cũng là lát cắt s, t , nên trọng số không thể nhỏ hơn $\lambda(s, t)$. Lập luận với cạnh nhẹ nhất trên đường $t \rightarrow v$ cũng tương tự. Kết hợp lại, ta chứng minh được kết luận trên.

Chú ý rằng cắt nhỏ nhất s, t tìm trong bước đệ quy có thể giao với cắt nhỏ nhất bên ngoài tập đỉnh đang xử lý. Nếu có giao, theo các định lý trước luôn có thể chỉnh thành một cắt nhỏ nhất cùng trọng số và không giao, còn cách nó chia tập đỉnh hiện tại không đổi. Vì vậy thuật toán Gusfield không cần co đỉnh, dễ cài đặt bằng mã hơn.

Trên thực tế, thuật toán Gusfield có thể cài đặt đệ quy hoặc không đệ quy. Mã giả dạng không đệ quy:

```
Gusfield's Algorithm(G)
  fa_1 = NULL
  fa_2, fa_3, ..., fa_|V| = 1

  for u = 2 to |V| do
    v = fa_u
    compute alpha(v, u), lambda(v, u)
    w_u = lambda(v, u)
    for x = u + 1 to |V| do
      if fa_x = v and x is on the u side then
        fa_x = u
      end if
    end for
  end for

  E = empty
  for u = 2 to |V| do
    add edge (fa_u, u) to E with weight w_u
  end for
  return T = (V, E, c)
```

4. Cắt nhỏ nhất toàn cục

Bài toán ở đây là tìm một lát cắt nhỏ nhất trong toàn bộ đồ thị, tức cắt nhỏ nhất toàn cục.

Một cách làm là dựng cây Gomory-Hu, rồi lấy cạnh có trọng số nhỏ nhất trên cây; cạnh đó chính là cắt nhỏ nhất toàn cục. Nhưng có những thuật toán đạt độ phức tạp tốt hơn.

4.1. Thuật toán Stoer-Wagner

Với hai đỉnh bất kỳ trong đồ thị, chúng có thể nằm cùng phía hoặc khác phía trong cắt nhỏ nhất toàn cục. Nếu nằm cùng phía, hiển nhiên ta có thể co hai đỉnh đó thành một đỉnh mà không ảnh hưởng đến đáp án. Nếu nằm khác phía, cắt nhỏ nhất giữa hai đỉnh đó chính là cắt nhỏ nhất toàn cục. Vì thế, chỉ cần lặp thao tác: mỗi lần chọn hai đỉnh bất kỳ, tìm cắt nhỏ nhất giữa chúng để cập nhật đáp án, rồi co hai đỉnh đó lại; khi chỉ còn một đỉnh thì đã tìm được cắt nhỏ nhất toàn cục.

Vậy tìm một cặp đỉnh thích hợp trong đồ thị bằng cách nào? Thuật toán sau đưa ra một phương án khả thi.

Tìm kiếm kề lớn nhất. Ta có một tập đỉnh A . Ban đầu A chứa một đỉnh khởi đầu tùy ý. Mỗi lần, chọn một đỉnh $v \notin A$ sao cho $c(A, \{v\})$ lớn nhất, rồi thêm v vào A . Lặp đến khi mọi đỉnh đều đã được thêm vào A .

Định lý. Gọi đỉnh thứ i được thêm vào A là v_i . Khi đó $(\{v_n\}, V \setminus \{v_n\})$ là cắt nhỏ nhất giữa hai đỉnh cuối cùng v_{n-1} và v_n trong tìm kiếm kề lớn nhất.

Chứng minh. Khi $n = 2$, mệnh đề hiển nhiên. Giả sử $n > 2$, xét ba đỉnh cuối cùng được thêm vào là v_{n-2}, v_{n-1}, v_n . Trước hết có thể xóa cạnh (v_{n-1}, v_n) , vì mọi lát cắt v_{n-1}, v_n đều cắt qua cạnh này; việc xóa nó không làm thay đổi thứ tự tương đối giữa các lát cắt đó.

Ta chứng minh

$$c(\{v_1, \dots, v_{n-2}, v_n\}, \{v_{n-1}\}) \leq \lambda(v_{n-2}, v_{n-1}).$$

Xét đồ thị sau khi bỏ v_n cùng các cạnh kề nó. Nếu chạy lại tìm kiếm kề lớn nhất theo thứ tự cũ, v_{n-2} và v_{n-1} sẽ trở thành hai đỉnh cuối cùng. Theo giả thiết quy nạp, $(\{v_1, \dots, v_{n-2}\}, \{v_{n-1}\})$ là cắt nhỏ nhất v_{n-2}, v_{n-1} .

Thêm v_n cùng các cạnh kề nó trở lại. Hiển nhiên cắt nhỏ nhất v_{n-2}, v_{n-1} không thể tốt hơn trước. Vì v_n không còn cạnh nối tới v_{n-1} , có thể thêm v_n vào phía của v_{n-2} trong lát cắt cũ mà không đổi trọng số. Do đó $(\{v_1, \dots, v_{n-2}, v_n\}, \{v_{n-1}\})$ vẫn là một cắt nhỏ nhất v_{n-2}, v_{n-1} .

Theo lựa chọn ở bước áp chót của tìm kiếm kề lớn nhất,

$$c(\{v_1, \dots, v_{n-2}\}, \{v_n\}) \leq c(\{v_1, \dots, v_{n-2}\}, \{v_{n-1}\}).$$

Do v_{n-1} không có cạnh nối tới v_n sau bước xóa cạnh đã nói ở trên, suy ra

$$c(\{v_1, \dots, v_{n-1}\}, \{v_n\}) \leq c(\{v_1, \dots, v_{n-2}, v_n\}, \{v_{n-1}\}),$$

tức

$$c(\{v_1, \dots, v_{n-1}\}, \{v_n\}) \leq \lambda(v_{n-2}, v_{n-1}).$$

Tương tự, bằng cách bỏ tạm v_{n-1} , có thể chứng minh

$$c(\{v_1, \dots, v_{n-1}\}, \{v_n\}) \leq \lambda(v_{n-2}, v_n).$$

Kết hợp hai phần,

$$\lambda(v_{n-1}, v_n) \leq c(\{v_1, \dots, v_{n-1}\}, \{v_n\}) \leq \min(\lambda(v_{n-1}, v_n), \lambda(v_{n-2}, v_n)).$$

Lại theo Định lý 1,

$$\lambda(v_{n-1}, v_n) \geq \min(\lambda(v_{n-1}, v_n), \lambda(v_{n-2}, v_n)).$$

Vì vậy mọi bất đẳng thức ở trên đều lấy dấu bằng. Nói cách khác,

$$\lambda(v_{n-1}, v_n) = c(\{v_1, \dots, v_{n-1}\}, \{v_n\}).$$

Mệnh đề được chứng minh.

Cài đặt. Với tìm kiếm kề lớn nhất, ta có thể dùng một heap để duy trì mọi giá trị $c(A, \{v\})$. Nếu dùng heap nhị phân, độ phức tạp là $O(|E| \log |V|)$; dùng heap Fibonacci có thể đạt $O(|E| + |V| \log |V|)$ cho mỗi pha.

4.2. Thuật toán Karger

Đây là thuật toán ngẫu nhiên, không bảo đảm đúng tuyệt đối, nhưng có thể tăng xác suất đúng bằng cách chạy nhiều lần.

Đồ thị không trọng số. Với đồ thị không trọng số, thuật toán như sau: mỗi lần chọn ngẫu nhiên đều một cạnh của đồ thị, giả sử cạnh đó không nằm trên cắt nhỏ nhất toàn cục, rồi co hai đầu của cạnh thành một đỉnh lớn. Sau khi co, bỏ mọi khuyên. Lặp quá trình co cạnh cho đến khi đồ thị chỉ còn hai đỉnh; các cạnh giữa hai đỉnh đó tạo thành một lát cắt thu được.

Xét xác suất đúng. Nếu kết quả cuối cùng là cắt nhỏ nhất toàn cục, thì mọi cạnh bị co trong quá trình đều không được nằm trên cắt nhỏ nhất. Giả sử cắt nhỏ nhất toàn cục có k cạnh. Khi đó bậc của mọi đỉnh đều không nhỏ hơn k ; nếu có một đỉnh bậc nhỏ hơn k , riêng đỉnh đó đã tạo thành một lát cắt nhỏ hơn. Vì vậy tổng số cạnh ít nhất là $k|V|/2$. Xác suất để một lần chọn ngẫu nhiên rơi vào cắt nhỏ nhất toàn cục không vượt quá

$$\frac{k}{|E|} \leq \frac{2}{|V|}.$$

Do đó xác suất không chọn vào cắt nhỏ nhất ở bước có i đỉnh ít nhất là $1 - 2/i$. Mỗi lần co cạnh làm số đỉnh giảm 1, nên xác suất cuối cùng thu được cắt nhỏ nhất toàn cục không nhỏ hơn

$$\prod_{i=|V|}^3 \left(1 - \frac{2}{i}\right) = \frac{2}{|V|(|V| - 1)}.$$

Vậy xác suất một lần chạy không thu được cắt nhỏ nhất toàn cục là

$$1 - \frac{2}{|V|(|V| - 1)}.$$

Do $1 - x < e^{-x}$, nếu chạy thuật toán m lần rồi lấy đáp án tốt nhất, xác suất đúng ít nhất là

$$1 - \left(1 - \frac{2}{|V|(|V| - 1)}\right)^m > 1 - e^{-2m/(|V|(|V|-1))}.$$

Chọn $m = |V|(|V| - 1)/2$ thì đạt xác suất đúng ít nhất $1 - e^{-1}$. Nói cách khác, chỉ cần chạy $O(|V|^2)$ lần là có được một xác suất đúng hằng số.

Cài đặt. Với quá trình chọn cạnh ngẫu nhiên, có thể sinh trước một hoán vị ngẫu nhiên của mọi cạnh, rồi liệt kê cạnh theo thứ tự trong hoán vị đó để thử co cạnh cho tới khi đồ thị chỉ còn hai đỉnh. Nếu cạnh đang xét đã trở thành khuyên trong đồ thị hiện tại, bỏ qua nó. Dễ thấy cách chọn này tương đương với cách chọn ở trên.

Khó khăn chính nằm ở thao tác co đỉnh. Nếu lưu cạnh bằng danh sách kề hoặc ma trận kề, mỗi lần co đỉnh cần $O(|V|)$ thời gian, tổng thời gian khá lớn. Cần tối ưu hóa bước này.

Nếu dùng danh sách liên kết để lưu cạnh, có thể giống thuật toán Kruskal cho cây khung nhỏ nhất: duy trì một cấu trúc hợp nhất-tìm kiếm cho các đỉnh. Khi co hai đỉnh u, v , nối trực tiếp danh sách cạnh của v vào sau danh sách cạnh của u , rồi trong cấu trúc hợp nhất-tìm kiếm cho v trở tới u . Tại mọi thời điểm, với một cạnh trong đồ thị, hai đầu cạnh đều được tra qua cấu trúc hợp nhất-tìm kiếm. Khi đó độ phức tạp cho một lần chạy là $O(|E|\alpha(|V|))$.

Còn có một cách đạt độ phức tạp tốt hơn. Trên hoán vị cạnh, nhị phân thời điểm mà đồ thị bắt đầu chỉ còn hai đỉnh. Ví dụ ở bước đầu, tính xem nếu lần lượt thao tác trên nửa đầu $|E|/2$ cạnh thì đồ thị có chỉ còn hai đỉnh không; điều này có thể kiểm tra bằng DFS. Nếu có, nhị phân ở nửa trái. Nếu không, co trực tiếp trên đồ thị những đỉnh được nối bởi các cạnh trong nửa trái, tính lại hai đầu của mọi cạnh ở nửa phải trong đồ thị mới, rồi nhị phân ở nửa phải. Các thao tác này đều giải quyết được trong $O(|E|)$, và vì số cạnh mỗi lần giảm một nửa, độ phức tạp cuối cùng vẫn là $O(|E|)$.

Đồ thị có trọng số. Với một cạnh có trọng số, có thể xem nó như nhiều cạnh song song trọng số đơn vị, từ đó chuyển về đồ thị không trọng số. Khi chọn cạnh ngẫu nhiên, cần sửa phân bố xác suất: xác suất chọn một cạnh phải bằng trọng số của cạnh đó chia cho tổng trọng số của mọi cạnh còn lại. Vì vậy cần duy trì một tập có thứ tự các cạnh để hỗ trợ thao tác chọn cạnh. Mỗi lần sinh ngẫu nhiên một số trong khoảng tổng trọng số mọi cạnh còn lại, tìm cạnh đầu tiên sao cho tổng trọng số từ đầu tới cạnh đó không nhỏ hơn số vừa sinh, chọn cạnh đó để co, rồi xóa cạnh đó khỏi tập. Tập này có thể duy trì bằng cây Fenwick. Độ phức tạp là $O(|E| \log |E|)$ cho một lần chạy.

5. Ứng dụng

5.1. Ví dụ 1: [ZJOI2011] Cắt nhỏ nhất

Cho một đồ thị vô hướng có trọng số. Có nhiều truy vấn; mỗi truy vấn cho một số x , hỏi có bao nhiêu cặp đỉnh có dung lượng cắt nhỏ nhất giữa hai đỉnh không vượt quá x .

Cách làm. Đây là một bài toán về cắt nhỏ nhất giữa hai đỉnh bất kỳ. Ta có thể dựng cây Gomory-Hu. Sau đó xử lý các cạnh cây theo trọng số tăng dần; mỗi lần dùng cấu trúc hợp nhất-tìm kiếm để hợp nhất hai thành phần liên thông mà cạnh đó nối, đồng thời cộng tích kích thước của hai thành phần vào đáp án. Như vậy ta tìm được số cặp đỉnh có cắt nhỏ nhất không vượt quá từng trọng số có thể xuất hiện. Khi trả lời truy vấn, chỉ cần dùng `lower_bound` trên mảng này.

5.2. Ví dụ 2: Codeforces Round #200 Div.1 E. Pumping Stations

Cho một đồ thị vô hướng có trọng số. Hãy tìm một hoán vị của mọi đỉnh sao cho tổng luồng cực đại giữa hai đỉnh kề nhau trong hoán vị là lớn nhất.

Cách làm. Trước hết, luồng cực đại bằng cắt nhỏ nhất. Tương tự, ta dựng cây Gomory-Hu. Sau đây chứng minh kết luận: đáp án của hoán vị tối ưu bằng tổng trọng số mọi cạnh trên cây Gomory-Hu.

Một hoán vị tương ứng với một đường đi qua các đỉnh trên cây. Xét cạnh có trọng số nhỏ nhất trên cây, nếu có nhiều cạnh thì chọn tùy ý một cạnh. Khi đi từ một đỉnh ở một phía của cạnh này sang một đỉnh ở phía kia, giá trị chắc chắn bằng trọng số của cạnh đó. Nếu một đường đi vượt qua cạnh này nhiều hơn một lần, ta có thể điều chỉnh để đáp án không xấu đi: cắt đường đi tại mọi vị trí vượt qua cạnh này, ta được nhiều đoạn đường không vượt qua cạnh đó. Giả sử vượt qua k lần, ta thu được $k + 1$ đoạn. Ghép đầu cuối các đoạn ở cùng một phía lại với nhau, ta được một đường đi không vượt cạnh ở mỗi phía. Sau đó nối hai đường đi của hai phía lại, lúc này bắt buộc chỉ vượt qua cạnh giữa đúng một lần. Vì cạnh ở giữa là cạnh nhỏ nhất, việc không đi qua nó không thể kém hơn việc đi qua nó. Do đó điều chỉnh này không làm đáp án xấu đi.

Sau đó, áp dụng lập luận này đệ quy cho hai phía của cạnh. Như vậy ta chứng minh được kết luận, đồng thời trong quá trình đệ quy cũng dựng được một nghiệm tối ưu.

5.3. Ví dụ 3: UVALive 5099 Nubulsa Expo

Cho một đồ thị vô hướng. Với một nguồn đã cố định, hãy tìm giá trị nhỏ nhất của luồng cực đại từ nguồn đó tới mọi đích có thể.

Cách làm. Trước hết, luồng cực đại bằng cắt nhỏ nhất. Hiển nhiên đáp án chính là cắt nhỏ nhất toàn cục. Một cắt nhỏ nhất toàn cục chia đồ thị thành hai phần; tất yếu một phần chứa nguồn đã cho. Chỉ cần chọn một đỉnh bất kỳ ở phần còn lại làm đích, cắt nhỏ nhất toàn cục sẽ là cắt nhỏ nhất giữa nguồn và đích đó.

5.4. Ví dụ 4: Chu trình nhỏ nhất trong đồ thị phẳng có trọng số cạnh không âm

Cho một đồ thị phẳng, mỗi cạnh có trọng số không âm. Hãy tìm chu trình có tổng trọng số nhỏ nhất.

Một cách làm. Dựng đồ thị đối ngẫu của đồ thị phẳng. Khi đó một chu trình trong đồ thị phẳng tương ứng với một lát cắt trong đồ thị đối ngẫu, và trọng số chu trình chính là trọng số lát cắt. Vì vậy chỉ cần tìm cắt nhỏ nhất toàn cục của đồ thị đối ngẫu.

Có thể tồn tại thuật toán có độ phức tạp tốt hơn.

6. Tổng kết

Khi gặp lớp bài toán này trong thi tin học, ta thường trước hết cần dựa vào điều kiện đề bài để chuyển mô hình về cắt nhỏ nhất trên đồ thị vô hướng. Sau đó hoặc trực tiếp áp dụng cấu trúc hay thuật toán phù hợp, hoặc khai thác các tính chất của cắt nhỏ nhất vô hướng để tiếp tục chuyển bài toán thành dạng có thể giải bằng thuật toán hoặc cấu trúc dữ liệu khác.

Với lớp bài toán này, giới học thuật còn có nhiều nghiên cứu sâu hơn, cùng nhiều thuật toán khác có thể giải hiệu quả. Bài viết này chỉ trình bày một phần rất nhỏ; bạn đọc quan tâm có thể tiếp tục nghiên cứu sâu hơn. Hy vọng bài viết có thể đóng vai trò gợi mở.

7. Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Xin cảm ơn các thầy cô Chen Heli, Dong Yehua, Shao Hongxiang và You Guangmiao của Trường Trung học số 1 Thiệu Hưng đã nhiều năm quan tâm và chỉ dẫn tôi.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Yu Linyun và Chen Xumin đã chỉ đạo.

Xin cảm ơn các đàn anh Yu Yili, Dong Honghua, He Qizheng, Zhang Hengjie, Wang Jianhao, Jia Yuekai ở Đại học Thanh Hoa đã giúp đỡ tôi.

Xin cảm ơn các bạn Ren Zhizhou, Hong Huadun, Tao Yuanzheng, Ye Zhaodao, Li Yang và nhiều bạn học khác ở Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở cho tôi.

Xin cảm ơn những thầy cô và bạn học khác đã từng giúp đỡ hoặc gợi ý cho tôi.

Xin cảm ơn cha mẹ đã quan tâm và ủng hộ.

Tài liệu tham khảo

1. R. E. Gomory, T. C. Hu. Multi-terminal network flows. *Journal of the Society for Industrial and Applied Mathematics*, vol. 9, 1961.
2. Dan Gusfield (1990). “Very Simple Methods for All Pairs Network Flow Analysis”. *SIAM J. Comput.* 19 (1): 143–155.
3. David R. Karger, *Global Min-cuts in RNC, and Other Ramifications of a Simple Min-Cut Algorithm*, Department of Computer Science, Stanford University.
4. David Morrison, Chandra Chekuri, “Gomory-Hu Trees”, CS 598CSC: Combinatorial Optimization Lecture 6.
5. Uri Zwick, lecture notes for “Analysis of Algorithms”: Global minimum cuts. School of Computer Science, Tel Aviv University.
6. Gomory-Hu tree – Wikipedia.
7. Karger’s algorithm – Wikipedia.
8. Ghi chú thuật toán – Cut.

Bài 5. Bàn sơ lược về ứng dụng của quy hoạch tuyến tính trong thi tin học

Zou Xiaoyao, Trường Trung học Trần Hải, Ninh Ba

Tóm tắt

Quy hoạch tuyến tính được dùng rộng rãi trong đời sống thực tế, nhưng hiện nay chưa thật phổ biến trong giới OI, vì vậy số bài liên quan cũng còn ít. Bài viết này giới thiệu ngắn gọn cách cài đặt quy hoạch tuyến tính, đồng thời nêu một số ứng dụng của quy hoạch tuyến tính trong các kiểu bài OI khác nhau.

Dẫn nhập

Trong các kỳ thi thuật toán thời kỳ đầu, quy hoạch tuyến tính ít được nhắc tới và thường chỉ xuất hiện dưới dạng bài mẫu. Vì phạm vi ứng dụng của quy hoạch tuyến tính rất rộng, những năm gần đây các kỳ thi OI đã có nhiều bài cần dùng hoặc có thể dùng quy hoạch tuyến tính để giải, hoặc ít nhất để lấy một phần điểm.

Thuật toán đơn hình cài đặt rất đơn giản, hiệu quả chạy cũng không tệ. Khi giải một số bài luồng mạng có mô hình hóa khá phức tạp, nó không hề kém các thuật toán khác. Đồng thời, phạm vi dùng được của quy hoạch tuyến tính rộng hơn luồng mạng. Với một số bài, tuy dùng trực tiếp quy hoạch tuyến tính là không đúng về mặt lý thuyết, nếu người ra đề không xây dữ liệu kỹ thì vẫn có thể may mắn được AC.

Do trên mạng đã có rất nhiều tài liệu tham khảo về chứng minh trong lĩnh vực này, bài viết lược bỏ phần lớn chứng minh; ở chỗ giới thiệu thuật toán, bài viết chỉ nói ngắn gọn cách cài đặt.

Bài viết gồm năm chương. Chương 1 giới thiệu định nghĩa quy hoạch tuyến tính. Chương 2 giới thiệu thuật toán thường dùng để giải quy hoạch tuyến tính: phương pháp đơn hình. Chương 3 tới chương 5 lần lượt giới thiệu ứng dụng của quy hoạch tuyến tính trong các kiểu bài OI khác nhau.

1. Quy hoạch tuyến tính là gì

Trong đời sống thực tế có rất nhiều bài toán có dạng sau: chúng cần cực đại hóa hoặc cực tiểu hóa một mục tiêu, đồng thời thường bị giới hạn bởi tài nguyên, thời gian và nhiều mặt khác. Nếu đơn giản hóa mục tiêu thành một hàm tuyến tính, và biểu diễn các hạn chế thành một số đẳng thức hoặc bất đẳng thức tuyến tính, thì các bài toán đó có thể được mô tả thành bài toán quy hoạch tuyến tính.

1.1. Quy hoạch tuyến tính tổng quát

Trong bài toán quy hoạch tuyến tính tổng quát, ta muốn tối ưu hóa một hàm tuyến tính thỏa một nhóm ràng buộc bất đẳng thức tuyến tính.

Định nghĩa hàm tuyến tính là hàm f thỏa

$$f(x_1, x_2, \dots, x_n) = \sum_{i=1}^n a_i x_i,$$

trong đó a_i là số thực, x_i là biến.

Định nghĩa đẳng thức tuyến tính là

$$f(x_1, x_2, \dots, x_n) = b,$$

trong đó b là số thực và f là hàm tuyến tính.

Định nghĩa bất đẳng thức tuyến tính là

$$f(x_1, x_2, \dots, x_n) \leq b \quad \text{hoặc} \quad f(x_1, x_2, \dots, x_n) \geq b,$$

trong đó b là số thực và f là hàm tuyến tính.

Phần sau dùng **ràng buộc tuyến tính**, gọi tắt là **ràng buộc**, để chỉ đẳng thức tuyến tính và bất đẳng thức tuyến tính.

Nói chính thức, quy hoạch tuyến tính là bài toán cần cực tiểu hóa hoặc cực đại hóa một hàm tuyến tính bị giới hạn bởi hữu hạn ràng buộc.

1.2. Dạng chuẩn và dạng nói lỏng

Sau khi hiểu định nghĩa quy hoạch tuyến tính, ta cần một cách chuẩn tắc hơn để biểu diễn nó.

Hai dạng thường dùng là dạng chuẩn và dạng nói lỏng. Nói đơn giản, trong dạng chuẩn, quy hoạch tuyến tính được biểu diễn thành bài toán cực đại hóa một hàm tuyến tính với các ràng buộc đều là bất đẳng thức tuyến tính. Trong dạng nói lỏng, quy hoạch tuyến tính được biểu diễn thành bài toán cực đại hóa một hàm tuyến tính với các ràng buộc đều là đẳng thức tuyến tính.

Thông thường ta viết dạng chuẩn như sau:

$$\begin{aligned} &\text{cực đại hóa} && \sum_{i=1}^n c_i x_i, \\ &\text{thỏa} && \sum_{j=1}^m a_{ij} x_j \leq b_i, \quad i = 1, 2, \dots, m, \\ &&& x_i \geq 0, \quad i = 1, 2, \dots, n. \end{aligned}$$

Cũng có thể viết gọn hơn:

$$\begin{array}{ll} \text{cực đại hóa} & c^T x, \\ \text{thỏa} & Ax \leq b, \\ & x_i \geq 0. \end{array}$$

Ở đây $A = (a_{ij})$ là ma trận $m \times n$, $b = (b_i)$ là véc-tơ m chiều, còn $c = (c_j)$ và $x = (x_j)$ là véc-tơ n chiều. Đôi khi ta cũng viết là

$$\{\max c^T x \mid Ax \leq b, x_i \geq 0\}.$$

Dạng tổng quát của dạng nói lỏng là

$$\begin{array}{ll} \text{cực đại hóa} & c^T x, \\ \text{thỏa} & Ax = b, \\ & x_i \geq 0. \end{array}$$

Với bất kỳ bài toán quy hoạch tuyến tính nào, một đẳng thức đều có thể thay bằng hai bất đẳng thức mà không đổi đáp án. Tương tự, với một bất đẳng thức, ta lập biến phụ y_i , biến

$$\sum_{j=1}^n a_{ij} x_j \leq b_i$$

thành

$$\sum_{j=1}^n a_{ij} x_j + y_i = b_i, \quad y_i \geq 0,$$

tức là một đẳng thức cộng một ràng buộc không âm.

Với một bài toán cực tiểu hóa tuyến tính, có thể thêm biến phụ p bằng giá trị cần cực tiểu, rồi chuyển thành cực đại hóa $-p$. Như vậy mọi bài toán quy hoạch tuyến tính đều có thể viết thành dạng chuẩn hoặc dạng nói lỏng; hai dạng này chỉ là hai cách biểu diễn chuẩn tắc hơn.

Để trình bày dễ hiểu, phần sau không luôn liệt kê quy hoạch tuyến tính theo dạng thật chuẩn; khi cần giải chỉ cần chuyển đổi lại.

1.3. Chuyển bài toán tổng quát thành quy hoạch tuyến tính

1.3.1. Bài toán đường đi ngắn nhất. Xét dạng đơn giản nhất của bài toán đường đi ngắn nhất: cho đồ thị có hướng có trọng số gồm n đỉnh và m cạnh; cạnh thứ i đi từ u_i tới v_i , có độ dài l_i . Cần tìm đường đi ngắn nhất từ s tới t .

Đặt n biến d_i biểu diễn khoảng cách ngắn nhất từ s tới i . Khi đó có thể liệt kê quy hoạch tuyến tính sau:

$$\begin{array}{ll} \text{cực đại hóa} & d_t, \\ \text{thỏa} & d_s = 0, \\ & d_{v_i} \leq d_{u_i} + l_i, \quad i = 1, 2, \dots, m. \end{array}$$

1.3.2. Luồng cực đại. Bài toán luồng cực đại: cho đồ thị có hướng gồm n đỉnh, giới hạn dung lượng từ i tới j là l_{ij} . Để tiện mô tả, giả sử dung lượng thỏa nếu $l_{ij} > 0$ thì $l_{ji} = 0$. Cần tìm luồng cực đại từ s tới t .

Đặt n^2 biến f_{ij} biểu diễn lưu lượng thực tế từ i tới j . Khi đó có thể liệt kê quy hoạch tuyến tính:

$$\begin{array}{ll} \text{cực đại hóa} & \sum_u f_{s,u}, \\ \text{thỏa} & f_{u,v} \leq l_{u,v}, \quad u, v = 1, 2, \dots, n, \\ & f_{u,v} = -f_{v,u}, \quad u, v = 1, 2, \dots, n, \\ & \sum_i f_{u,i} = 0, \quad u \in \{1, 2, \dots, n\} \setminus \{s, t\}. \end{array}$$

1.3.3. Luồng nhiều loại hàng. Cho đồ thị có hướng gồm n đỉnh, giới hạn dung lượng từ i tới j là l_{ij} . Để tiện mô tả, giả sử dung lượng thỏa nếu $l_{ij} > 0$ thì $l_{ji} = 0$.

Có p loại hàng cần vận chuyển; loại thứ k cần chuyển d_k đơn vị từ s_k tới t_k . Hỏi có tồn tại một phương án khả thi hay không.

Bài toán này giải bằng quy hoạch tuyến tính rất tiện, nhưng không có lời giải hiệu quả nào khác ngoài quy hoạch tuyến tính.

Đặt pn^2 biến $f_{k,i,j}$ biểu diễn lưu lượng thực tế của hàng k từ i tới j . Khi đó có thể liệt kê quy hoạch tuyến tính:

cực đại hóa 0 (chỉ cần nghiệm khả thi nên không cần hàm mục tiêu),

$$\begin{aligned} \text{thỏa} \quad & f_{k,u,v} \leq l_{u,v}, \quad u, v = 1, 2, \dots, n; k = 1, 2, \dots, p, \\ & \sum_k f_{k,u,v} \leq l_{u,v}, \quad u, v = 1, 2, \dots, n, \\ & f_{k,u,v} = -f_{k,v,u}, \quad u, v = 1, 2, \dots, n; k = 1, 2, \dots, p, \\ & \sum_i f_{k,u,i} = 0, \quad u \in \{1, 2, \dots, n\} \setminus \{s_k, t_k\}; k = 1, 2, \dots, p, \\ & \sum_i f_{k,s_k,i} = d_k, \quad k = 1, 2, \dots, p. \end{aligned}$$

Chỉ cần kiểm tra quy hoạch tuyến tính này có nghiệm hay không.

2. Thuật toán đơn hình

Trong chương này, n biểu diễn số biến, m biểu diễn số ràng buộc.

2.1. Giới thiệu thuật toán đơn hình

Thuật toán đơn hình là phương pháp cổ điển để giải quy hoạch tuyến tính. Trong trường hợp xấu nhất, độ phức tạp thời gian của nó là cấp số mũ. Tuy nhiên, để làm đơn hình đạt mức số mũ cần trọng số có kích thước cấp số mũ, trong khi các bài toán trong thi thuật toán thường có một mô hình cụ thể và trọng số phần lớn là số nguyên trong một phạm vi nhất định. Với nhiều điều kiện như vậy, khó có thể ép đơn hình chạy tới cấp số mũ.

Vì quy hoạch tuyến tính chưa phổ biến cao, việc “hack” quy hoạch tuyến tính lại khó hơn cài đặt nó rất nhiều; không thể giả sử người ra đề nào cũng tinh thông kỹ thuật làm khó đơn hình. Trong trường hợp ngẫu nhiên, số lần kỳ vọng gọi thủ tục pivot của đơn hình là $O(\text{số ràng buộc})$. Nói cách khác, trong thực tế nó có thể xử lý dữ liệu cỡ vài trăm trong 1 giây, đủ cho phần lớn nhu cầu. Khi dữ liệu thỏa nhiều hạn chế hơn, thậm chí chỉ cần số phép pivot ít hơn số ràng buộc đã có thể thu được nghiệm.

Thuật toán đơn hình nhận một quy hoạch tuyến tính dạng chuẩn

$$\{\max c^T x \mid Ax \leq b, x_i \geq 0\}$$

làm đầu vào, và trả về “vô nghiệm” (*Infeasible*), “không bị chặn” (*Unbounded*), hoặc một phương án đạt giá trị tối ưu.

Có thể xem mỗi cách gán biến là một điểm trong không gian n chiều; mỗi bất đẳng thức tương ứng với một siêu phẳng $n - 1$ chiều, còn tập nghiệm hợp lệ tương ứng với một đa diện lồi n chiều.

Thuật toán đơn hình trước hết chọn một nghiệm cơ bản khả thi ban đầu, có thể hiểu là tìm một điểm trên đa diện lồi; nếu không tìm được thì trả về vô nghiệm. Sau đó nó liên tục thực hiện thao tác xoay trục (*pivot*) trên nghiệm này, có thể hiểu là đi tới một đỉnh khác của đơn hình chứa điểm đó. Mỗi thao tác bảo đảm đáp án tăng; khi lợi ích khi đi theo mọi cạnh đều âm thì đã tìm được nghiệm tối ưu.

Có thể chứng minh thuật toán này chắc chắn dừng, vì số đỉnh của đa diện lồi là hữu hạn, và giá trị nó trả về chắc chắn đúng: nếu không thể đi tiếp thì điểm hiện tại nhất định là nghiệm tối ưu.

Nói cách khác, thuật toán đơn hình có thể chia thành các bước sau:

- Chuyển quy hoạch tuyến tính đầu vào thành dạng nói lỏng.
- Thực hiện quá trình khởi tạo để tìm một nghiệm ban đầu; nếu không tìm được thì trả về vô nghiệm.
- Thực hiện quá trình tối ưu hóa để tìm nghiệm tối ưu hoặc trả về không bị chặn.

Tiếp theo ta sẽ giới thiệu chi tiết từng bước.

2.2. Chuyển quy hoạch tuyến tính đầu vào thành dạng nói lỏng

Tạo mới m biến $x_{n+1}, x_{n+2}, \dots, x_{n+m}$, và đặt

$$x_{n+i} = b_i - \sum_{j=1}^n a_{ij}x_j, \quad x_{n+i} \geq 0.$$

Khi đó quy hoạch tuyến tính được viết thành

$$\begin{aligned} &\text{cực đại hóa} && \sum_{i=1}^n c_i x_i, \\ &\text{thỏa} && x_{n+i} = b_i - \sum_{j=1}^n a_{ij}x_j, \quad i = 1, 2, \dots, m, \\ &&& x_i \geq 0, \quad i = 1, 2, \dots, n+m. \end{aligned}$$

Trong đó $x_1, x_2, \dots, x_n$ gọi là biến phi cơ sở, $x_{n+1}, x_{n+2}, \dots, x_{n+m}$ gọi là biến cơ sở.

Ở cách biểu diễn dạng nói lỏng này, đặt mọi biến phi cơ sở bằng 0, mọi biến cơ sở bằng b_i , ta thu được nghiệm mà dạng nói lỏng này đại diện. Trong quá trình chạy, thuật toán đơn hình liên tục chọn các tập biến cơ sở khác nhau để biểu diễn cùng quy hoạch tuyến tính dưới các dạng nói lỏng khác nhau; cuối cùng, khi mọi c_i đều trở thành số âm thì đã tìm được nghiệm tối ưu.

Vì vậy, nếu ban đầu mọi b_i đều không âm thì $\{0, 0, \dots, 0\}$ là một nghiệm hợp lệ, dạng nói lỏng ban đầu là hợp lệ, nên có thể bỏ qua bước khởi tạo. Ngược lại, phải chạy quá trình khởi tạo để có một nghiệm cơ bản khả thi rồi mới chạy quá trình tối ưu hóa.

2.3. Thao tác xoay trục

Tác dụng của thao tác xoay trục là hoán đổi một biến cơ sở với một biến phi cơ sở. Cả quá trình khởi tạo lẫn quá trình tối ưu hóa đều cần dùng thao tác này.

Để tiện mô tả, giả sử trước mỗi thao tác ta bảo đảm $x_1, x_2, \dots, x_n$ là các biến phi cơ sở.

Thao tác xoay trục nhận hai số l, e làm đầu vào, rồi hoán đổi vị trí của x_{n+l} và x_e , tức là biến x_{n+l} trở thành biến phi cơ sở, còn x_e trở thành biến cơ sở.

Ban đầu có đẳng thức

$$x_{n+l} = b_l - \sum_j a_{lj}x_j.$$

Chuyển về được

$$a_{le}x_e = b_l - \sum_{j \neq e} a_{lj}x_j - x_{n+l}.$$

Chia hai vế cho a_{le} , ta được

$$x_e = \frac{b_l - \sum_{j \neq e} a_{lj}x_j - x_{n+l}}{a_{le}}.$$

Dùng đẳng thức này thay cho đẳng thức ban đầu, rồi lần lượt thế nó vào các đẳng thức khác và vào biểu thức cần tối ưu, ta nhận được biểu thức liên quan tới

$$\{x_1, x_2, \dots, x_n\} \setminus \{x_e\} \cup \{x_{n+l}\}.$$

Cuối cùng chỉ cần hoán đổi chỉ số của x_{n+l} và x_e .

Lưu ý: nếu cuối cùng cần xuất phương án, phải ghi lại mỗi biến hiện tại tương ứng với biến ban đầu nào.

2.4. Quá trình tối ưu hóa

Giả sử ta đã có một nghiệm cơ bản khả thi của quy hoạch tuyến tính. Khi đó chạy quá trình tối ưu hóa là đủ để thu được kết quả:

- Chọn một biến phi cơ sở x_e thỏa $c_e > 0$.
- Tìm biến cơ sở x_{n+l} sao cho $a_{le} > 0$ và b_l/a_{le} nhỏ nhất.
- Thực hiện pivot(l, e), rồi quay lại bước đầu.

Tức là mỗi lần chọn một biến mà nếu tăng nó lên thì trạng thái hiện tại trở nên tốt hơn, dù đáp án không nhất thiết tăng trong mọi cách viết trung gian; tăng nó cho tới khi có một biến cơ sở trở thành 0, rồi hoán đổi vị trí của hai biến đó.

Ở đây có thể thêm một tối ưu tham lam: liệt kê mọi cặp (l, e) thỏa điều kiện, chọn cặp làm đáp án tăng nhiều nhất để thực hiện pivot. Vì độ phức tạp của pivot là $O(mn)$, cách làm này không ảnh hưởng tới độ phức tạp tổng thể, nhưng có thể giảm đáng kể số lần pivot.

Nếu chọn tùy ý, có thể xuất hiện vòng lặp chết. Có thể tránh bằng cách mỗi lần chọn chỉ số e nhỏ nhất trong các e hợp lệ, đồng thời chọn chỉ số l nhỏ nhất trong các l hợp lệ.

Quá trình này sẽ dừng trong hai trường hợp:

- Không tìm được e hợp lệ; có thể chứng minh lúc này đã tìm được nghiệm tối ưu.
- Không tìm được l hợp lệ; khi đó nghiệm tối ưu là vô hạn, trả về “không bị chặn”.

2.5. Quá trình khởi tạo

Để tìm một nghiệm khả thi ban đầu, cần lập một quy hoạch tuyến tính phụ:

$$\begin{array}{ll} \text{cực đại hóa} & -x_0, \\ \text{thỏa} & \sum_{j=1}^n a_{ij}x_j \leq b_i + x_0, \quad i = 1, 2, \dots, m, \\ & x_i \geq 0, \quad i = 0, 1, \dots, n. \end{array}$$

Dễ thấy quy hoạch tuyến tính ban đầu có nghiệm khi và chỉ khi nghiệm tối ưu của quy hoạch phụ này là 0.

Chuyển quy hoạch phụ này thành dạng nổi lỏng, rồi tìm l sao cho b_l nhỏ nhất và thực hiện pivot($l, 0$). Như vậy ta có nghiệm ban đầu của quy hoạch phụ; chạy quá trình tối ưu hóa sẽ giải được nghiệm tối ưu của quy hoạch phụ.

Sau khi chạy xong quá trình khởi tạo, trực tiếp bỏ x_0 rồi tiếp tục chạy quá trình tối ưu hóa là được.

Khi cài đặt, không cần thật sự thêm một biến mới; có thể trực tiếp thực hiện thao tác xoay trục trên quy hoạch tuyến tính ban đầu.

2.6. Ví dụ

Ví dụ 1. Happiness. Cần mua cùng một loại suất ăn cho m người. Suất ăn gồm n loại thực phẩm. Biết đơn giá của mỗi loại thực phẩm, độ thỏa mãn mà mỗi người nhận được khi nhận thêm một đơn vị của từng loại thực phẩm, và giới hạn trên độ thỏa mãn của mỗi người. Hỏi trong điều kiện không vượt giới hạn thỏa mãn của bất kỳ ai, có thể tiêu nhiều nhất bao nhiêu tiền.

Bài này là bài mẫu của quy hoạch tuyến tính: xem mỗi loại thực phẩm là một biến, mỗi người là một bất đẳng thức.

Đồng thời $\{0, 0, \dots, 0\}$ là một nghiệm ban đầu thỏa điều kiện, nên bài này thậm chí không cần quá trình khởi tạo.

Ví dụ 2. FLYDIST. Cho đồ thị vô hướng có n đỉnh, m cạnh, trọng số cạnh là số nguyên. Cần biến trọng số của mỗi cạnh thành độ dài đường đi ngắn nhất giữa hai đỉnh mà nó nối, và sau khi thay đổi thì mỗi trọng số cạnh phải không âm. Gọi trọng số ban đầu là O_i , trọng số sau thay đổi là N_i , chi phí là $\sum_i |O_i - N_i|$. Cần cực tiểu hóa chi phí.

Ràng buộc: $n \leq 10$, $O_i \leq 20$. Nguồn: Uva 10498 và Codechef FEB12.

Trước hết xem đường đi ngắn nhất giữa mỗi cặp đỉnh là một biến, đặt là d_{ij} . Với mỗi cạnh k , đặt hai biến I_k, D_k , biểu diễn trọng số cuối cùng là $O_k + I_k - D_k$. Khi đó cần cực tiểu hóa

$$\sum_k (I_k + D_k).$$

Với cạnh k , giả sử hai đầu mút là x_k, y_k . Khi đó với mỗi j đều có

$$d_{x_k, j} \leq O_k + I_k - D_k + d_{y_k, j},$$

và chiều ngược lại cũng có một bất đẳng thức tương ứng. Đồng thời, để bảo đảm trọng số của mỗi cạnh không âm, cần thêm $O_k + I_k - D_k \geq 0$. Dùng đơn hình giải quy hoạch tuyến tính này là được.

3. Ứng dụng của quy hoạch tuyến tính trong bài toán luồng mạng

3.1. Ma trận hoàn toàn đơn mô-đun

Ma trận hoàn toàn đơn mô-đun (*totally unimodular matrix*) là ma trận mà với mọi ma trận con vuông đủ hạng, giá trị định thức đều là 1 hoặc -1 . Bản thân ma trận gốc không cần có số hàng bằng số cột.

Còn có một phương pháp thường dùng để nhận biết ma trận hoàn toàn đơn mô-đun (điều kiện đủ): nếu ma trận A đồng thời thỏa bốn điều kiện sau thì A là ma trận hoàn toàn đơn mô-đun:

- A chỉ gồm các phần tử $-1, 0, 1$.
- Mỗi cột của A chứa nhiều nhất hai phần tử khác 0.
- Các hàng của A có thể chia thành hai tập B, C .
- Nếu một cột chứa hai phần tử khác 0 cùng dấu, các hàng chứa chúng được chia vào hai tập khác nhau; nếu một cột chứa hai phần tử khác 0 trái dấu, các hàng chứa chúng được chia vào cùng một tập.

Ma trận hoàn toàn đơn mô-đun có một tính chất rất quan trọng: nếu trong quy hoạch tuyến tính

$$\{\max c^T x \mid Ax \leq b, x_i \geq 0\}$$

ma trận A là hoàn toàn đơn mô-đun, thì có thể chứng minh trong quá trình chạy đơn hình, mọi hệ số trong mọi ràng buộc được xét tới đều có giá trị thuộc $\{1, 0, -1\}$.

Nói cách khác, giới hạn mỗi biến là số thực hay giới hạn là 0 hoặc 1 đều cho ra cùng một đáp án. Vì vậy, trực tiếp chạy quy hoạch tuyến tính cũng có thể nhận được đáp án đúng của bài toán quy hoạch nguyên tương ứng.

Đồng thời, trong trường hợp này quá trình đơn hình chỉ cần dùng phép cộng trừ; có thể dùng danh sách liên kết để tối ưu, loại bỏ những hạng tử có giá trị 0, qua đó tối ưu hằng số và tăng tốc đáng kể.

Có nhiều bài toán có thể chuyển thành quy hoạch tuyến tính với ma trận hoàn toàn đơn mô-đun, chẳng hạn bất kỳ bài toán luồng cực đại nào, bất kỳ bài toán luồng cực đại chi phí nhỏ nhất nào, và một số bài DP rất đặc biệt.

Một số bài luồng mạng vốn không có cách dựng đồ thị hiển nhiên, mà cần trước hết liệt kê quy hoạch tuyến tính, sau đó qua đối ngẫu và một loạt biến đổi để cuối cùng chuyển bài toán quy hoạch tuyến tính thành luồng mạng. Nhưng nếu có thể dùng luồng mạng để giải thì ma trận dựng ra nhất định là hoàn toàn đơn mô-đun; lúc này quy hoạch tuyến tính là một cách giải tổng quát cho loại bài này, không cần phân tích sâu thêm mô hình của bài toán.

3.2. Tính đối ngẫu của quy hoạch tuyến tính

Tính đối ngẫu của quy hoạch tuyến tính (*linear-programming duality*) nói rằng với bất kỳ quy hoạch tuyến tính

$$\{\max c^T x \mid Ax \leq b, x_i \geq 0\},$$

nghiệm tối ưu của nó bằng nghiệm tối ưu của bài toán đối ngẫu tương ứng

$$\{\min b^T y \mid A^T y \geq c, y_i \geq 0\}.$$

Sau khi lần lượt biểu diễn bài toán luồng cực đại và bài toán cắt nhỏ nhất bằng quy hoạch tuyến tính, chúng trở thành hai bài toán đối ngẫu. Vì vậy định lý luồng cực đại - cắt nhỏ nhất có thể chứng minh bằng tính đối ngẫu của quy hoạch tuyến tính.

Trong phần lớn bài, giữa bài toán gốc và bài toán đối ngẫu thường có một bài toán tìm được nghiệm ban đầu trực tiếp, không cần khởi tạo, nhờ đó giảm lượng mã cài đặt.

3.3. Ví dụ

Ví dụ 3. Volunteer Recruitment. Cho một dãy độ dài n ($n \leq 1000$) và các giá trị $b_1, \dots, b_n$. Có m ($m \leq 10000$) loại đoạn; chọn một đoạn loại i sẽ làm mọi vị trí từ l_i tới r_i tăng 1 và tốn chi phí a_i . Cần làm cho cuối cùng giá trị ở mỗi vị trí trong dãy không nhỏ hơn b_i , với chi phí nhỏ nhất. Nguồn: NOI 2008.

Trước hết liệt kê quy hoạch tuyến tính:

$$\begin{array}{ll} \text{cực tiểu hóa} & \sum_{i=1}^m a_i x_i, \\ \text{thỏa} & \sum_{l_j \leq i \leq r_j} x_j \geq b_i, \quad i = 1, 2, \dots, n, \\ & x_i \geq 0, \quad i = 1, 2, \dots, m. \end{array}$$

Có thể thấy mỗi biến xuất hiện liên tiếp trong các ràng buộc. Sau khi lấy sai phân các ràng buộc, mỗi biến chỉ xuất hiện trong hai ràng buộc.

Lập một đỉnh cho mỗi ràng buộc, xem ràng buộc đó là cân bằng luồng của đỉnh này. Khi đó mỗi biến đại diện cho một cạnh, hằng số trong mỗi ràng buộc đại diện cho luồng bù từ nguồn và tới đích. Dựng đồ thị xong chạy luồng cực đại chi phí nhỏ nhất là được; đây là lời giải luồng chi phí chính thức của bài.

Nhưng cách dựng đồ thị này không quá hiển nhiên. Nếu không nhìn ra cách dựng, mà dùng trực tiếp đơn hình, chỉ cần chuyển quy hoạch tuyến tính này sang đối ngẫu:

$$\begin{array}{ll} \text{cực đại hóa} & \sum_{i=1}^n b_i x_i, \\ \text{thỏa} & \sum_{l_i \leq j \leq r_i} x_j \leq a_i, \quad i = 1, 2, \dots, m, \\ & x_i \geq 0, \quad i = 1, 2, \dots, n. \end{array}$$

Như vậy ta có một quy hoạch tuyến tính với n biến và m ràng buộc, hơn nữa $\{0, 0, \dots, 0\}$ là nghiệm ban đầu của nó, nên không cần viết quá trình khởi tạo. Sau khi thêm tối ưu phép cộng trừ và tối ưu danh sách liên kết, tốc độ chạy rất nhanh.

Về hiệu quả cài đặt, luồng chi phí kiểu zkw nhanh hơn đơn hình, đơn hình nhanh hơn luồng chi phí dùng đường đi ngắn nhất thô. Cả ba cách đều có thể qua mọi dữ liệu của bài này, tuy cài luồng chi phí đường đi ngắn nhất kém có thể TLE.

Ví dụ 4. Defense Line. Có một dãy độ dài n ($n \leq 1000$). Chi phí để tăng giá trị ở vị trí i lên 1 là a_i . Cho m ($m \leq 10000$) đoạn từ l_i tới r_i và giá trị yêu cầu b_i . Cần làm cho tổng giá trị trên mỗi đoạn l_i tới r_i cuối cùng không nhỏ hơn b_i , với chi phí nhỏ nhất. Nguồn: ZJOI 2013 Day1.

$$\begin{array}{ll} \text{cực tiểu hóa} & \sum_{i=1}^n a_i x_i, \\ \text{thỏa} & \sum_{l_i \leq j \leq r_i} x_j \geq b_i, \quad i = 1, 2, \dots, m, \\ & x_i \geq 0, \quad i = 1, 2, \dots, n. \end{array}$$

Vì $\{0, 0, \dots, 0\}$ không phải nghiệm ban đầu, dùng trực tiếp quy hoạch tuyến tính sẽ có hiệu quả thấp và mã phức tạp hơn.

Chuyển quy hoạch tuyến tính này sang đối ngẫu:

$$\begin{array}{ll} \text{cực đại hóa} & \sum_{i=1}^m b_i x_i, \\ \text{thỏa} & \sum_{l_j \leq i \leq r_j} x_j \leq a_i, \quad i = 1, 2, \dots, n, \\ & x_i \geq 0, \quad i = 1, 2, \dots, m. \end{array}$$

Sau đó giống bài trước, trực tiếp dùng đơn hình nguyên để giải là được.

3.4. Tổng kết

Những năm gần đây, các mô hình đơn giản của luồng mạng đã được mọi người nắm vững, vì vậy đề bài thường kiểm tra khả năng mô hình hóa. Hơn nữa, do tính đặc thù của bài toán luồng mạng, không có thuật toán vét cạn nào có độ phức tạp gần với lời giải chuẩn; đồng thời tốc độ chạy thực tế của các thuật toán luồng mạng vốn đã nhanh hơn nhiều so với cận lý thuyết. Số cạnh và số đỉnh trong đề thường khá lớn, người ra đề cũng không thể bảo đảm chương trình của mình nhất định chạy kịp trên mọi đồ thị trong giới hạn m, n , nên giới hạn thời gian của bài luồng mạng thường khá rộng, dữ liệu cũng thường không cố ý nhằm vào một thuật toán cụ thể, càng không cố ý siết hằng số.

Với dữ liệu ngẫu nhiên 1000×10000 của hai bài trên, số lần kỳ vọng thực hiện pivot của đơn hình chưa tới 100. Về hiệu quả và độ phức tạp mã, nó đều tốt hơn thuật toán luồng mạng đường đi ngắn nhất phổ biến nhất. Điểm tiếc là độ phức tạp không gian là $O(mn)$; nếu giới hạn bộ nhớ khá chặt thì không thể qua.

Dù vậy, vì đơn hình có thể tiết kiệm thời gian nghĩ cách dựng đồ thị, nó có lợi thế rất lớn khi thời gian làm bài bị giới hạn.

4. Ứng dụng của quy hoạch tuyến tính trong bài toán quy hoạch nguyên đặc biệt

Quy hoạch nguyên là quy hoạch tuyến tính trong đó mọi biến của nghiệm cuối cùng đều bắt buộc là số nguyên. Quy hoạch 0-1 là một dạng đặc biệt của quy hoạch nguyên, tức là yêu cầu mỗi biến thuộc $\{0, 1\}$.

Vì phần lớn bài toán trong đời sống khi biểu diễn thành quy hoạch tuyến tính đều cần nghiệm nguyên, quy hoạch nguyên có phạm vi ứng dụng còn rộng hơn.

Tuy nhiên, hiện nay quy hoạch nguyên chưa có cách giải đa thức tổng quát; ví dụ bài toán phủ chính xác có thể quy về quy hoạch 0-1. Vì vậy, trong thi OI, các bài có thể dùng trực tiếp quy hoạch tuyến tính để giải thường là những bài chuyển được thành ma trận hoàn toàn đơn mô-đun. Dù vậy, với những bài không chuyển được thành ma trận hoàn toàn đơn mô-đun, nếu dữ liệu không được xây cẩn thận mà chỉ dùng dữ liệu ngẫu nhiên, quy hoạch tuyến tính vẫn có thể đạt tỷ lệ đúng rất cao.

Ví dụ 5. Lắp đặt WiFi. Trên mặt phẳng có một hình chữ nhật dài vô hạn, rộng R , bên trong có n ($n \leq 100$) điểm. Có m ($m \leq 100$) đường tròn có tâm nằm ngoài hình chữ nhật, bán kính đều là R , đường tròn thứ i có trọng số a_i . Cần tìm phủ trọng số nhỏ nhất của các đường tròn lên các điểm. Với 40% dữ liệu, mọi tâm đường tròn đều nằm phía trên hình chữ nhật. Nguồn: ZJOI 2015 Day2.

Nếu tùy ý xác định mỗi đường tròn phủ những điểm nào, đây là bài toán phủ chính xác. Tuy nó là NPC, ta vẫn có thể dùng quy hoạch tuyến tính để giải nghiệm thực của nó:

$$\begin{array}{ll} \text{cực tiểu hóa} & \sum_{i=1}^m a_i x_i, \\ \text{thỏa} & \sum_{j \text{ phủ } i} x_j \geq 1, \quad i = 1, 2, \dots, n, \\ & x_i \geq 0, \quad i = 1, 2, \dots, m. \end{array}$$

Nếu ma trận của quy hoạch tuyến tính này là hoàn toàn đơn mô-đun, có thể chứng minh nghiệm tối ưu giải theo số thực bằng nghiệm tối ưu của bài toán gốc.

Với 40 điểm đầu, có thể chứng minh cách làm này là đúng. Cách chứng minh có thể mô phỏng lời giải DP 40 điểm: nhất định tồn tại một cách sắp xếp các điểm sao cho mỗi đường tròn chỉ phủ một đoạn liên tiếp các điểm. Khi đó bài toán biến thành bài phủ đoạn, dễ thấy ma trận là hoàn toàn đơn mô-đun.

Đáng tiếc là ma trận của bài này không phải hoàn toàn đơn mô-đun, nên sẽ có trường hợp sai. Tuy nhiên, nếu dữ liệu không được xây có chủ ý thì rất khó chặn đơn hình; trong kỳ thi, cài trực tiếp đơn hình nguyên có thể nhận được 70 điểm.

Ví dụ 6. Literature. Cho n điểm và m nửa mặt phẳng trên mặt phẳng; nửa mặt phẳng thứ i có trọng số a_i . Cần tìm phủ trọng số nhỏ nhất của các nửa mặt phẳng lên các điểm. Với 40% dữ liệu, $a_i = 1$. Nguồn: Tsinghua Training 2014.

Bài này có thể giống bài trước, trực tiếp chuyển thành mô hình phủ chính xác rồi liệt kê quy hoạch tuyến tính.

Nhưng cũng như bài trước, ma trận này không phải hoàn toàn đơn mô-đun. Một dữ liệu $n = 3, m = 3$ đã có thể hack dễ dàng: ba nửa mặt phẳng lần lượt phủ $\{1, 2\}, \{1, 3\}, \{2, 3\}$, trọng số đều là 2. Đáp án giải ra theo số thực là 3, còn đáp án đúng là 4.

Tuy nhiên khi đó người ra đề chưa chú ý tới cách dùng đơn hình, không tạo dữ liệu nhắm vào nó, nên trong phiên bản dữ liệu ban đầu đơn hình qua mọi test và được 100 điểm. Trong lúc thi, người ra đề phải tạo lại dữ liệu mới để chặn đơn hình xuống còn 80 điểm. Có thể thấy đơn hình vẫn là một công cụ lấy điểm rất hiệu quả.

Đồng thời người ra đề cho nhiều điểm thành phần với $a_i = 1$. Trong điều kiện này, trực tiếp xuất kết quả làm tròn từ nghiệm thực không dễ bị dữ liệu ngẫu nhiên hack; người ra đề cũng cho rằng chi phí chặn phần này quá lớn nên không chặn.

Ví dụ 7. Red and Black Tree. Cho một cây có n ($n \leq 500$) đỉnh; đỉnh thứ i có trọng số a_i ($a_i \in \{0, 1\}$), và đúng m đỉnh có trọng số 0, các đỉnh còn lại có trọng số 1. Nguồn: Codeforces 375E.

Cần chọn một tập đỉnh gồm m đỉnh sao cho với mọi đỉnh trên cây, tồn tại một đỉnh trong tập được chọn có khoảng cách tới nó không vượt quá k . Trên cơ sở đó, cần cực tiểu hóa tổng trọng số của m đỉnh được chọn.

Liệt kê trực tiếp quy hoạch tuyến tính theo đề:

$$\begin{array}{ll} \text{cực tiểu hóa} & \sum_{i=1}^n a_i x_i, \\ \text{thỏa} & \sum_i x_i = m, \\ & \sum_{\text{dis}(i,j) \leq k} x_j \geq 1, \quad i = 1, 2, \dots, n, \\ & x_i \geq 0, \quad i = 1, 2, \dots, n. \end{array}$$

Trước hết tùy ý chọn một đỉnh làm gốc, rồi chọn một lá xa gốc nhất. Có thể thấy trong các đỉnh có thể phủ lá này, các phạm vi phủ của chúng bao hàm nhau từng đôi một. Dùng tính chất này có thể chứng minh khá dễ rằng nghiệm do quy hoạch tuyến tính này cho ra bằng nghiệm của quy hoạch nguyên.

Nhưng hai đáp án bằng nhau không phải vì ma trận của quy hoạch tuyến tính này là hoàn toàn đơn mô-đun; chỉ là dưới điều kiện của bài, nghiệm tối ưu của quy hoạch tuyến tính nhất định bằng nghiệm tối ưu của quy hoạch nguyên. Nếu sửa đề thành yêu cầu xuất một phương án, sẽ không thể dùng quy hoạch tuyến tính để giải.

Nói cách khác, đa diện do các ràng buộc của quy hoạch tuyến tính này đại diện không phải có mọi đỉnh đều là điểm nguyên; nó chỉ bảo đảm rằng tồn tại ít nhất một điểm nguyên có đáp án bằng nghiệm tối ưu dưới ràng buộc số thực.

Bài này còn có một lời giải DP trên cây $O(n^3)$, nhưng hiệu quả thực tế chậm hơn thuật toán đơn hình rất nhiều.

5. Ứng dụng của quy hoạch tuyến tính trong bài toán trò chơi

Những bài đã giới thiệu ở trên đều có các cách làm khác khá tốt. Trong khi đó, độ phức tạp của quy hoạch tuyến tính khó ước lượng chính xác và tiêu tốn không gian lớn, khiến nó ít xuất hiện khi giải các loại bài này. Tuy nhiên, quy hoạch tuyến tính là một công cụ mạnh để giải lớp bài sắp được nhắc tới dưới đây.

5.1. Cân bằng Nash

Trong một trò chơi có hai hoặc nhiều người tham gia, nếu tại một trạng thái không ai có thể chỉ thay đổi quyết sách của chính mình để nhận được lợi ích cao hơn, thì tổ hợp chiến lược đó được gọi là cân bằng

Nash.

Nếu trong một tổ hợp chiến lược, chiến lược của mỗi người đều cố định, cân bằng Nash này gọi là cân bằng Nash chiến lược thuần. Nếu chiến lược của mỗi người là tổ hợp các chiến lược theo một xác suất nhất định, cân bằng Nash này gọi là cân bằng Nash chiến lược hỗn hợp.

Có thể chứng minh rằng với bất kỳ trò chơi tổng bằng không nào có số quyết sách hữu hạn, tức tổng lợi ích của mọi người bằng 0, đều tồn tại ít nhất một cân bằng Nash chiến lược hỗn hợp.

Với một trò chơi tổng bằng không hai người, ta rất dễ chuyển bài toán thành quy hoạch tuyến tính, rồi dùng đơn hình để tìm cân bằng Nash.

5.2. Ví dụ

Ví dụ 8. Đọc tâm trí. Có n ($n \leq 50$) lựa chọn; A và B lần lượt chọn một lựa chọn trong số đó. Cho ma trận lợi ích v của A trong mọi n^2 trường hợp; lợi ích của B là $-v_{ij}$. Bảo đảm $v_{ij} \in \{-1, 0, 1\}$ và $v_{ij} = -v_{ji}$. Nguồn: BJOI 2011 Day2.

Bây giờ A cần đưa ra một bảng xác suất chọn từng chiến lược, sao cho dù B chọn chiến lược nào, kỳ vọng lợi ích của A đều không nhỏ hơn 0.

Do ma trận được bảo đảm phản đối xứng, cân bằng Nash của A và B là như nhau; nói cách khác, chắc chắn tồn tại một phương án thỏa đề.

Đặt xác suất của mỗi quyết sách làm biến. Để thấy rằng buộc là lợi ích của đối phương với mỗi quyết sách không vượt quá 0, và tổng xác suất của mọi quyết sách bằng 1.

Sửa nhẹ một chút, ta có thể liệt kê quy hoạch tuyến tính:

$$\begin{array}{ll} \text{cực đại hóa} & \sum_{i=1}^n x_i, \\ \text{thỏa} & \sum_{i=1}^n x_i \leq 1, \\ & \sum_j v_{ij} x_j \geq 0, \quad i = 1, 2, \dots, n, \\ & x_i \geq 0, \quad i = 1, 2, \dots, n. \end{array}$$

Do tính đặc biệt của ma trận trong bài, quy hoạch tuyến tính chạy rất nhanh; dữ liệu với n khoảng 500 không có vấn đề gì. Tuy nhiên khi đó lời giải chính thức của bài là phương pháp điều chỉnh, nên giới hạn dữ liệu rất nhỏ.

Ví dụ 9. TheK Factor. Có một mảng độ dài n ($n \leq 50$). Bạn bắt đầu từ đầu trái của mảng và đi sang phải. Mỗi ô có một số nguyên v_i , biểu diễn lợi ích nếu cuối cùng dừng ở ô này. Nguồn: Topcoder SRM 682 Div1 Hard.

Hệ thống sẽ đặt một bức tường sau một ô nào đó từ 1 tới n . Tức là tồn tại i sao cho bạn không thể đi tới các ô bên phải i ; khi thử đi tới các ô đó, cuối cùng bạn sẽ dừng ở ô i .

Gọi xác suất tường cuối cùng ở i là a_i , thỏa với mọi j đều có

$$\sum_{i=1}^j a_i \leq \frac{j}{n}, \quad \sum_i a_i = 1.$$

Bây giờ bạn cần xác định một mảng b , biểu diễn xác suất cuối cùng chọn dừng ở mỗi ô; hiển nhiên phải thỏa $\sum_i b_i = 1$. Yêu cầu cực đại hóa điểm số nhỏ nhất của bạn trong trường hợp a được xác định tùy ý.

Liệt kê bất đẳng thức theo mọi mảng a có thể là không thực tế, vì vậy cần xét cách chuyển hóa bài toán.

Bài này là một mô hình trò chơi tổng bằng không hai người khá rõ. Sau khi chuyển hóa nhẹ và dùng kết luận ở trên, có thể chứng minh chắc chắn tồn tại một cân bằng Nash. Nói cách khác, đáp án nhận được khi liệt kê bất đẳng thức theo mọi mảng b có thể cũng giống đáp án giải theo mảng a .

Vì mảng b không bị ràng buộc tiền tố, chỉ cần bảo đảm các trường hợp mỗi b_i lần lượt bằng 1 đều thỏa điều kiện thì mọi mảng b đều thỏa.

Khi đó bất đẳng thức rất dễ liệt kê. Đặt ans là kỳ vọng điểm số lớn nhất của bạn, ta có

$$\begin{aligned} \text{cực tiểu hóa } & ans, \\ \text{thỏa } & \sum_{i=1}^n a_i = 1, \\ & \sum_{i=1}^j a_i v_i + \sum_{i=j+1}^n a_i v_j \leq ans, \quad j = 1, 2, \dots, n, \\ & \sum_{i=1}^j a_i \leq \frac{j}{n}, \quad j = 1, 2, \dots, n, \\ & a_i \geq 0, \quad i = 1, 2, \dots, n. \end{aligned}$$

Giải quy hoạch tuyến tính này là được.

Ví dụ 10. Quyết sách tối ưu 2. A và B lần lượt nhận được một số nguyên ngẫu nhiên khác nhau trong $1 \sim n$. Sau đó A chọn một số trong $1 \sim m$ để báo ra; B chọn một số để báo ra trong hai lựa chọn: “2” và “số A đã báo”. Nguồn: kiểm tra tương hỗ đội tuyển tập huấn năm 2016.

Nếu hai số được báo ra giống nhau, giả sử đều là x , thì người có số nguyên ngẫu nhiên lớn hơn nhận lợi ích x , người còn lại nhận $-x$.

Nếu hai số được báo ra khác nhau, giả sử là x và y , $x < y$, thì người chọn y nhận lợi ích x , người còn lại nhận $-x$.

Bây giờ cần xác định cho A một bảng chiến lược, tức khi nhận được một số i thì chọn mỗi lựa chọn với xác suất bao nhiêu, để cực đại hóa kỳ vọng lợi ích của A trong trường hợp xấu nhất.

Đặt $n \times m$ biến x_{ij} , biểu diễn khi nhận được i thì xác suất chọn j là bao nhiêu. Lại đặt m biến c_j , biểu diễn tổng lợi ích của B khi nhìn thấy chiến lược của A là j .

Vấn đề là tính c_j như thế nào. Chú ý rằng dù B nhìn thấy số nào, nó nhất định sẽ chọn một ngưỡng p ; nếu nhận được số nhỏ hơn p thì chọn bỏ cuộc, nếu không thì chọn tấn công. Vì vậy c_j là giá trị nhỏ nhất trong $n + 1$ phương án mà B có thể chọn.

Đặt $s_{i,j,k,p}$ là lợi ích của A khi A nhận số i và chọn lựa chọn j , B nhận số k và chọn tấn công ($p = 1$) hoặc bỏ cuộc ($p = 0$). Khi $i = k$, đặt $s_{i,j,k,p} = 0$. Khi đó có thể liệt kê quy hoạch tuyến tính:

$$\begin{aligned} \text{cực đại hóa } & \sum_i c_i, \\ \text{thỏa } & \sum_i \sum_j x_{ij} \left(\sum_{k=1}^{p-1} s_{i,j,k,0} + \sum_{k=p}^n s_{i,j,k,1} \right) \geq c_j, \\ & \quad j = 1, 2, \dots, m; \quad p = 1, 2, \dots, n + 1, \\ & \sum_j x_{ij} = 1, \quad i = 1, 2, \dots, n, \\ & x_{ij} \geq 0, \quad i = 1, 2, \dots, n; \quad j = 1, 2, \dots, m. \end{aligned}$$

Có quy hoạch tuyến tính này, ta có thể giải các trường hợp n, m khá nhỏ, từ đó nhận được một bảng với phạm vi tương đối lớn. Nhìn bảng đó khá dễ đoán ra kết luận cuối cùng và AC bài này.

Lời cảm ơn

Xin cảm ơn CCF đã cung cấp nền tảng học tập và giao lưu.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Chen Xumin và Yu Linyun vì sự tận tụy của thầy cô.

Xin cảm ơn cha mẹ đã quan tâm và chăm sóc tôi.

Xin cảm ơn bạn Zhou Ziyuan ở Trường Học Quân đã gợi mở cho tôi trong quá trình viết luận văn.

Xin cảm ơn bạn Mao Xiao ở Trường Yali đã đọc và kiểm tra bản thảo.

Xin cảm ơn tất cả các bạn học từng giúp đỡ và gợi mở cho tôi.

Tài liệu tham khảo

1. Liu Rujia, *Hướng dẫn huấn luyện kinh điển nhập môn thi thuật toán*, Nhà xuất bản Đại học Thanh Hoa.
2. Li Yujian, *Bàn sơ lược về quy hoạch tuyến tính trong thi tin học – cài đặt phương pháp đơn hình ngắn gọn và hiệu quả cùng ứng dụng*, luận văn đội tuyển tập huấn quốc gia IOI 2007.
3. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*.
4. [Trang Wikipedia tiếng Anh về ma trận đơn mô-đun](#).
5. [Trang Wikipedia tiếng Anh về cân bằng Nash](#).
6. [Trang Wikipedia tiếng Anh về thuật toán đơn hình](#).

Bài 6. Phép toán cực trị trên đoạn và bài toán cực trị lịch sử

Ji Ruyi, Trường Học Quân Hàng Châu

Tóm tắt

Bài viết này chủ yếu giới thiệu một loạt phép biến đổi và phương pháp xử lý liên quan tới phép toán cực trị trên đoạn và bài toán hồi cực trị lịch sử trong thi thuật toán. Hiện nay các bài về thao tác lấy cực trị trên đoạn và hồi cực trị lịch sử chưa xuất hiện nhiều, nhưng tần suất đang tăng dần; hơn nữa, do thí sinh ít tiếp xúc với hướng này, chúng thường có độ khó khá cao trong phòng thi. Bài viết phân loại và phân tích những bài tương tự đã xuất hiện, đồng thời giới thiệu một số cách tổng quát để chuyển loại bài này về các bài truyền thống hơn, chẳng hạn chuyển thao tác cực trị trên đoạn thành thao tác cộng/trừ trên đoạn, hoặc chuyển câu hỏi tổng cực trị lịch sử trên đoạn thành câu hỏi tổng đoạn thông thường. Hy vọng bài viết đưa lớp bài thú vị này vào tầm nhìn của thí sinh và gợi mở thêm nghiên cứu.

1. Lời nói đầu

Bài toán cấu trúc dữ liệu luôn là một loại bài xuất hiện với tần suất rất cao trong thi thuật toán. Nhưng vài năm gần đây, khi nghiên cứu của thí sinh về cấu trúc dữ liệu ngày càng sâu, không gian ra đề còn lại cũng dần thu hẹp. Vì vậy trong các kỳ thi trong nước Trung Quốc hiện nay, phần lớn bài cấu trúc dữ liệu hoặc chỉ không ngừng đóng gói lại bài cũ, hoặc có xu hướng đưa một bài dãy kinh điển lên cây hay xương rồng để cường ép tăng lượng mã. Tác giả cho rằng cách làm đó khá vô vị: nó phá hỏng tính chất gọn gàng, đẹp đẽ của nhiều bài cấu trúc dữ liệu, khiến đề bài trở nên nhạt nhòa.

Gần đây, các phép toán cực trị trên đoạn, cụ thể là lấy max hoặc lấy min trên đoạn, cùng các bài toán cực trị lịch sử, cụ thể là cực đại lịch sử, cực tiểu lịch sử và tổng các phiên bản lịch sử, dần bước vào tầm nhìn của mọi người. Thực ra ngay IOI 2014 đã có bài liên quan tới phép toán cực trị trên đoạn, còn nguyên mẫu của bài toán cực trị lịch sử có thể truy ngược tới năm năm trước đó; tuy nhiên khi ấy chúng chưa gây nhiều chú ý và cũng chưa có nhiều người nghiên cứu sâu. Tới mùa hè năm ngoái, trong đợt huấn luyện liên trường, bài có độ khó cao đầu tiên về phép toán cực trị trên đoạn xuất hiện; sau đó loại bài này dần nhiều hơn, và đã xuất hiện trong các kỳ tập huấn Thanh Hoa cũng như trên Universal Online Judge (UOJ).

Vì loại bài này còn khá mới, thí sinh thường khó giải khi lần đầu gặp. Do đó bài viết này giới thiệu một phương pháp biến đổi tổng quát cho phép toán cực trị trên đoạn, đồng thời phân tích có chủ đích vài loại bài toán cực trị lịch sử thường gặp, mong có ích cho người đọc.

Mục 2 ôn lại định nghĩa và tính chất của cây đoạn. Mục 3 và Mục 4 lần lượt thảo luận phép toán cực trị trên đoạn và bài toán cực trị lịch sử.

2. Cây đoạn

2.1. Giới thiệu

Cây đoạn là một cấu trúc dữ liệu rất thường gặp trong thi thuật toán. Mỗi nút của cây đoạn đại diện cho một đoạn; riêng nút gốc đại diện cho toàn đoạn, còn nút lá đại diện cho một vị trí cụ thể.

Mỗi nút không phải lá có hai con: con trái và con phải. Nếu nút đang xét đại diện cho đoạn $[l, r]$, thì con trái đại diện cho

$$\left[l, \left\lfloor \frac{l+r}{2} \right\rfloor \right],$$

còn con phải đại diện cho

$$\left[\left\lfloor \frac{l+r}{2} \right\rfloor + 1, r \right].$$

Chẳng hạn, cây đoạn dựng trên đoạn $[1, 8]$ là một cây nhị phân cân bằng, gốc là $[1, 8]$, tầng kế tiếp là $[1, 4]$ và $[5, 8]$, rồi tiếp tục chia đôi cho tới các lá $[1, 1], \dots, [8, 8]$.

Nhờ các tính chất của cấu trúc này, ta có thể thực hiện nhanh nhiều thao tác trên cây đoạn.

2.2. Thao tác

2.2.1. Thao tác điểm. Do cấu trúc chia đôi trị của cây đoạn, độ dài đoạn của nút con bằng một nửa độ dài đoạn của nút cha, nên chiều cao cây đoạn là $O(\log n)$.

Vì vậy với thao tác trên một điểm, ta có thể trực tiếp định vị nút lá đại diện cho vị trí đó, rồi cập nhật thông tin của nó và các nút cha. Độ phức tạp mỗi lần là $O(\log n)$.

2.2.2. Hỏi đoạn. Để giải quyết câu hỏi trên đoạn, ta cần định vị một đoạn $[L, R]$ trên cây đoạn.

Xét thuật toán sau. Bắt đầu tìm kiếm từ gốc. Giả sử nút hiện tại đại diện cho đoạn $[l, r]$. Khi đó có ba trường hợp:

1. Nếu $[l, r] \subseteq [L, R]$, đoạn hiện tại được lấy trọn và dừng tại nút này.
2. Nếu $[l, r] \cap [L, R] = \emptyset$, đoạn hiện tại không đóng góp gì và dừng tại nút này.
3. Nếu không thuộc hai trường hợp trên, tiếp tục tìm kiếm ở hai con của nút hiện tại.

Ví dụ, khi định vị đoạn $[4, 7]$ trên cây đoạn của $[1, 8]$, các nút được lấy trọn là $[4, 4]$, $[5, 6]$ và $[7, 7]$; đường tìm kiếm chỉ đi qua những nhánh giao với đoạn hỏi.

Ta chứng minh độ phức tạp của cách làm này là $O(\log n)$. Hai trường hợp đầu là điều kiện dừng; chỉ trường hợp thứ ba mới tiếp tục đệ quy. Vì mỗi nút của cây đoạn có nhiều nhất hai con, số nút thuộc hai trường hợp đầu không vượt quá hai lần số nút thuộc trường hợp thứ ba, nên chỉ cần xét số nút của trường hợp thứ ba.

Xét từng tầng của cây đoạn, tức các nút có cùng độ sâu. Các đoạn do các nút trên một tầng đại diện là rời nhau; sắp chúng theo đầu trái. Với đoạn hỏi $[L, R]$, mọi đoạn giao với $[L, R]$ tạo thành một dãy liên tiếp theo thứ tự này, và chỉ đoạn ngoài cùng bên trái cùng ngoài cùng bên phải mới có thể rơi vào trường hợp thứ ba. Vì cây đoạn có $O(\log n)$ tầng, mỗi tầng chỉ có $O(1)$ nút thuộc trường hợp thứ ba, nên thời gian là $O(\log n)$.

Sau khi định vị $[L, R]$, chỉ cần hợp nhất thông tin của các nút được lấy trọn là thu được đáp án.

2.2.3. Sửa đoạn. Với sửa đoạn, hiển nhiên không thể dùng sửa điểm để bạo lực thay đổi từng vị trí; khi đó cần đưa vào khái niệm **nhãn lười**.

Lấy phép cộng/trừ trên đoạn làm ví dụ. Với mỗi nút i của cây đoạn, ta ghi một nhãn lười a_i , biểu thị mọi số trong đoạn do nút i đại diện đều đã được cộng thêm a_i .

Giả sử cần cộng x cho $[L, R]$. Trước hết định vị đoạn này trên cây đoạn, rồi gán nhãn x cho các nút được lấy trọn: cộng x vào nhãn của nút, đồng thời cập nhật tổng đoạn bằng cách cộng kích thước đoạn nhân với x . Sau đó cập nhật thông tin các nút cha.

Với một nút bất kỳ, thông tin ghi tại nút chỉ chắc chắn đúng khi mọi nhãn của các tổ tiên đã bằng 0. Vì thế, bất kể thao tác sửa hay hỏi, mỗi khi truy cập tới một nút ta đều cần đẩy nhãn của nút đó xuống hai con. Đẩy nhãn của nút i nghĩa là gán nhãn a_i cho con trái và con phải, rồi đặt $a_i = 0$.

Độ phức tạp giống hỏi đoạn, mỗi lần $O(\log n)$.

Nhãn lười rất hữu dụng: chỉ cần nhãn có thể hợp nhất và khi gán nhãn có thể cập nhật nhanh thông tin nút, ta có thể dùng cây đoạn để duy trì, chẳng hạn cộng/trừ đoạn hỏi tổng đoạn, phủ đoạn hỏi tổng đoạn, phủ đoạn hỏi gcd đoạn. Nhưng nó cũng có giới hạn; ví dụ với bài lấy max với một số trên đoạn rồi hỏi tổng đoạn, nhãn lười cơ bản không xử lý được.

3. Phép toán cực trị trên đoạn

Với một dãy A , phép toán cực trị trên đoạn cụ thể là: cho ba số l, r, x , với mọi $i \in [l, r]$, thay A_i bằng $\max(A_i, x)$ hoặc $\min(A_i, x)$.

Sau đây ta dùng vài ví dụ để thảo luận cách xử lý loại bài này.

Ví dụ 1. Gorgeous Sequence. Cho một mảng A độ dài n . Có m thao tác thuộc ba loại:

1. Cho l, r, x ; với mọi $i \in [l, r]$, đặt $A_i \leftarrow \min(A_i, x)$.
2. Cho l, r ; hỏi giá trị lớn nhất của A_i trên $[l, r]$.
3. Cho l, r ; hỏi $\sum_{i=l}^r A_i$.

Dữ liệu: $n, m \leq 10^6$. Bài có thể tìm trên HDU 5306.

Như đã nói ở Mục 1, khi gán nhãn lấy min cho một nút, ta không thể cập nhật nhanh tổng đoạn, nên bài này không giải được bằng nhãn lười truyền thống.

Xét cách làm sau. Với mỗi nút của cây đoạn, ngoài tổng đoạn sum , ta duy trì thêm giá trị lớn nhất ma , giá trị lớn thứ hai nghiêm ngặt se , và số lần xuất hiện của giá trị lớn nhất t .

Giả sử cần lấy min với x trên $[L, R]$. Trước hết định vị đoạn này trên cây đoạn. Với mỗi nút được định vị, ta bắt đầu tìm kiếm bạo lực và xét ba trường hợp:

1. Nếu $ma \leq x$, thao tác không ảnh hưởng tới nút này, thoát ngay.
2. Nếu $se < x < ma$, thao tác chỉ ảnh hưởng tới mọi phần tử đang bằng giá trị lớn nhất. Khi đó cộng $t(x - ma)$ vào sum , cập nhật $ma \leftarrow x$, gắn nhãn tương ứng rồi thoát.
3. Nếu $se \geq x$, chưa thể cập nhật trực tiếp thông tin nút, nên tiếp tục tìm kiếm ở hai con.

Trong hình nguồn, tác giả minh họa một cây đoạn trên $[1, 4]$, mỗi nút ghi (ma, se) . Khi lấy min với 2 trên toàn đoạn, các nhánh cần đi sâu được tô đỏ; sau cập nhật, chỉ các giá trị lớn hơn 2 bị hạ xuống 2.

Ta có thể chứng minh độ phức tạp là $O(m \log n)$.

Trước hết xem giá trị lớn nhất như một nhãn. Với mỗi nút cây đoạn, ghi một giá trị nhãn bằng giá trị lớn nhất trong đoạn nó quản lý. Nếu nhãn của một nút bằng nhãn của cha nó, ta có thể bỏ nhãn ở nút đó; khi đó mỗi vị trí thực tế lấy giá trị của nhãn đầu tiên gặp được trên đường từ lá tương ứng đi lên. Hình nguồn minh họa cách chuyển một cây ghi giá trị lớn nhất thành cây chỉ ghi những nhãn cần thiết; sau chuyển đổi, cây có nhiều nhất $2n$ nhãn, và mọi nhãn đều lớn hơn mọi nhãn trong cây con của nó.

Quan sát thuật toán trên, giá trị lớn thứ hai của đoạn đúng bằng giá trị nhãn lớn nhất trong cây con. Tác dụng của DFS bạo lực là: sau khi gắn nhãn mới, để giữ tính chất nhãn giảm dần theo độ sâu, ta thu hồi các nhãn trong cây con có giá trị lớn hơn nhãn mới. Ta cần chứng minh độ phức tạp của quá trình thu hồi nhãn.

Định nghĩa **lớp nhãn** như sau:

1. Các nhãn sinh ra từ cùng một lần gắn nhãn lấy min trên đoạn thuộc cùng một lớp.
2. Nhãn mới sinh ra từ việc đẩy xuống cùng một nhãn thuộc cùng lớp với nhãn ban đầu.
3. Hai nhãn không thỏa hai điều kiện trên thuộc hai lớp khác nhau.

Định nghĩa trọng số của một lớp nhãn là kích thước cây ảo tạo bởi các nhãn của lớp đó và gốc cây đoạn, tức số nút có ít nhất một nhãn của lớp này trong cây con. Đặt thế năng $\Phi(x)$ là tổng trọng số của mọi lớp nhãn.

Xét một thao tác lấy cực trị trên đoạn. Ta thêm một lớp nhãn mới; trọng số của nó bằng số nút đi qua khi gắn nhãn, nên mỗi lần là $O(\log n)$. Xét một lần đẩy nhãn xuống; trọng số của lớp nhãn đó chỉ tăng $O(1)$. Xét quá trình DFS: một khi bắt đầu DFS bạo lực ở một nút, trong cây con của nút đó chắc chắn tồn tại nhãn cần thu hồi, và sau thu hồi thì cây con không còn nhãn của lớp ấy. Vì thế mỗi nút đi qua làm trọng số của ít nhất một lớp nhãn giảm 1, kéo theo thế năng giảm 1.

Đẩy nhãn chỉ xảy ra khi định vị đoạn trên cây đoạn, nên tổng số lần đẩy nhãn là $O(m \log n)$; tổng đóng góp khi gắn nhãn vào thế năng cũng là $O(m \log n)$. Vì vậy tổng biến thiên của thế năng chỉ là $O(m \log n)$, và độ phức tạp của thuật toán là $O(m \log n)$.

Ví dụ 2. Picks loves segment tree. Cho một dãy A độ dài n . Có m thao tác:

1. Với mọi $i \in [l, r]$, đặt $A_i \leftarrow \min(A_i, x)$.
2. Với mọi $i \in [l, r]$, cộng x vào A_i , trong đó x có thể âm.
3. Với mọi $i \in [l, r]$, hỏi tổng A_i .

Dữ liệu: $n, m \leq 3 \cdot 10^5$. Nguồn: bài tự sáng tác.

Vì có thêm cộng/trừ đoạn, giá trị lớn nhất, lớn thứ hai nghiêm ngặt và số lần xuất hiện của giá trị lớn nhất vẫn duy trì được, nên ta tiếp tục dùng cách làm vừa rồi.

Chứng minh độ phức tạp cần sửa lại. Do có thao tác cộng/trừ đoạn, chứng minh của Ví dụ 1 không còn dùng trực tiếp được, nhưng tư tưởng chính vẫn có thể kế thừa. Sửa định nghĩa lớp nhãn:

1. Các nhãn sinh ra từ cùng một lần lấy min thuộc cùng một lớp.
2. Nhãn mới sinh ra từ việc đẩy xuống cùng một nhãn thuộc lớp ban đầu.
3. Với một thao tác cộng/trừ đoạn, xét một lớp bất kỳ trong cây đoạn hiện tại. Nếu lớp này bị đoạn sửa chứa trọn hoặc rời hẳn đoạn sửa thì lớp không đổi; ngược lại, tách lớp thành hai lớp: phần bị sửa và phần còn lại.

Định nghĩa lại trọng số của một lớp nhãn là kích thước cây ảo do tất cả các nút có nhãn của lớp này tạo thành trên cây đoạn, lần này không tính gốc. Đặt hàm phụ $\Phi(x)$ bằng tổng trọng số mọi lớp nhãn.

Khi cộng/trừ đoạn, ảnh hưởng tới Φ chỉ gồm $O(\log n)$ lần đẩy nhãn; khi tách lớp, thế năng chỉ giảm. Mỗi lần đẩy nhãn làm Φ tăng $O(1)$; mỗi lần gán nhãn làm Φ tăng $O(\log n)$.

Xét DFS. Ta định vị một số nút trên cây đoạn, rồi DFS từ các nút đó. Với một nút bắt đầu và một lớp nhãn sẽ bị thu hồi: nếu lớp nhãn chỉ có một phần nằm trong cây con này, thì mỗi nút đi qua làm trọng số của lớp giảm 1. Nếu toàn bộ lớp nhãn nằm trong cây con, quá trình thu hồi có thể xem như trước hết tốn chi phí định vị LCA của lớp nhãn, rồi thu hồi trên cây ảo của lớp đó.

Vì vậy độ phức tạp gồm hai phần. Phần thứ nhất là thời gian gán với việc làm giảm hàm phụ, bằng $O(m \log n)$. Phần thứ hai là chi phí định vị LCA của một lớp nhãn; mỗi lớp có thể đóng góp $O(\log n)$, mà tổng số nhãn chỉ là $O(m \log n)$, nên số lớp cũng là $O(m \log n)$, và phần này có cận trên $O(m \log^2 n)$. Do đó thuật toán có cận $O(m \log^2 n)$. Tác giả nhận xét cận này khá lỏng: trên các dữ liệu đã dựng, tốc độ thực tế gần $O(m \log n)$ hơn, nhưng chưa có chứng minh hay dữ liệu cực hạn cho cận chặt hơn.

3.1. Tiểu kết

Qua hai ví dụ trên, ta có một phương pháp duy trì tổng đoạn trong bài có phép toán cực trị trên đoạn; thao tác lấy max tương tự thao tác lấy min.

Quan sát chứng minh độ phức tạp của Ví dụ 2, ta thấy trong chứng minh không dùng tính chất cụ thể của thao tác cộng/trừ đoạn. Vì vậy có thể thay cộng/trừ đoạn bằng các thao tác sửa khác, chứng minh vẫn thành lập.

Ví dụ 3. AcrossTheSky loves segment trees. Cho một dãy A độ dài n . Có m thao tác:

1. Với mọi $i \in [l, r]$, đặt $A_i \leftarrow \min(A_i, x)$.
2. Với mọi $i \in [l, r]$, đặt $A_i \leftarrow \max(A_i, x)$.
3. Với mọi $i \in [l, r]$, hỏi tổng A_i .

Dữ liệu: $n, m \leq 3 \cdot 10^5$. Nguồn: bài tự sáng tác.

Khi giải riêng thao tác lấy min và lấy max, ta cần duy trì giá trị nhỏ nhất, nhỏ thứ hai nghiêm ngặt, số lần xuất hiện của nhỏ nhất, giá trị lớn nhất, lớn thứ hai nghiêm ngặt và số lần xuất hiện của lớn nhất. Nay hai thao tác xuất hiện đồng thời, các thông tin này vẫn duy trì được.

Khi phân tích độ phức tạp, xét riêng hai loại nhãn. Ảnh hưởng của mỗi loại nhãn lên loại kia chỉ là đẩy nhãn xuống; trong trường hợp cực đoạn có thể thu hồi trực tiếp một phần nhãn trong một cây con, nhưng chi phí của lần thu hồi đó là $O(1)$, nên không ảnh hưởng. Vì thế chứng minh của Ví dụ 2 vẫn đúng, và thời gian là $O(m \log^2 n)$.

3.2. Chuyển phép toán cực trị trên đoạn thành cộng/trừ đoạn

Quan sát thuật toán cho Ví dụ 2, khi gán nhãn lấy min, thực ra chỉ các phần tử đang bằng giá trị lớn nhất của đoạn chịu ảnh hưởng; các số còn lại không đổi. Đồng thời, sau khi giá trị lớn nhất bị sửa, những

phần tử vốn là giá trị lớn nhất vẫn là giá trị lớn nhất, còn những phần tử vốn không phải giá trị lớn nhất thì vẫn không phải.

Vì vậy thông qua thuật toán trên, ta đã chuyển phép toán cực trị trên đoạn thành một loại thao tác cộng/trừ chỉ tác động lên giá trị lớn nhất hoặc nhỏ nhất của đoạn.

Do đó với mỗi nút cây đoạn, ta có thể chia các vị trí trong đoạn nút quản lý thành hai lớp: các vị trí đạt giá trị lớn nhất trong đoạn, và các vị trí còn lại. Dùng Ví dụ 2 làm mẫu, ta cần xử lý các thao tác: cộng/trừ một số cho mọi giá trị lớn nhất trong một nút cây đoạn, và cộng/trừ một số cho mọi số trong một đoạn. Vì vậy chỉ cần duy trì riêng nhãn và thông tin cho lớp giá trị lớn nhất và lớp không phải giá trị lớn nhất trong mỗi nút.

Khi đẩy nhãn xuống, cần xét giá trị lớn nhất của hai con. Nhãn của lớp giá trị lớn nhất chỉ được đẩy xuống con có giá trị lớn nhất lớn hơn; nếu hai con có giá trị lớn nhất bằng nhau thì đẩy xuống cả hai.

Như vậy ta thu được một phương pháp chuyển phép toán cực trị trên đoạn thành phép cộng/trừ đoạn, với chi phí thêm một nhân tử log n có hằng số nhỏ. Sau đây là ba ví dụ ở các hướng khác nhau.

Ví dụ 4. Mzl loves segment tree. Cho một dãy A độ dài n . Định nghĩa một mảng B , ban đầu mọi phần tử của B đều bằng 0. Có m thao tác:

1. Với mọi $i \in [l, r]$, đặt $A_i \leftarrow \min(A_i, x)$.
2. Với mọi $i \in [l, r]$, đặt $A_i \leftarrow \max(A_i, x)$.
3. Với mọi $i \in [l, r]$, cộng x vào A_i .
4. Với mọi $i \in [l, r]$, hỏi tổng B_i .

Sau mỗi thao tác, với mỗi vị trí i , nếu giá trị A_i thay đổi thì cộng 1 vào B_i ; nếu không thì B_i giữ nguyên. Dữ liệu: $n, m \leq 3 \cdot 10^5$. Nguồn: bài tự sáng tác.

Nếu chỉ có cộng/trừ đoạn, chỉ cần $x \neq 0$ thì mọi số trong đoạn sửa đều thay đổi, nên cộng 1 cho B trên đoạn đó là đủ.

Nay có thêm lấy min và lấy max, ta có thể chia các vị trí trong mỗi nút thành ba phần: giá trị lớn nhất, giá trị nhỏ nhất, và những giá trị không phải lớn nhất cũng không phải nhỏ nhất. Sau đó duy trì riêng từng lớp để khử tác động của thao tác lấy min và lấy max.

Cần chú ý: khi hai loại thao tác cực trị cùng tồn tại, tập vị trí do giá trị lớn nhất và giá trị nhỏ nhất kiểm soát có thể trùng nhau. Trường hợp này phải xử lý riêng để tránh cộng lặp vào B_i .

Ví dụ 5. ChiTuShaoNian loves segment tree. Cho hai dãy A, B cùng độ dài n . Có m thao tác:

1. Với mọi $i \in [l, r]$, đặt $A_i \leftarrow \min(A_i, x)$.
2. Với mọi $i \in [l, r]$, đặt $B_i \leftarrow \min(B_i, x)$.
3. Với mọi $i \in [l, r]$, cộng x vào A_i .
4. Với mọi $i \in [l, r]$, cộng x vào B_i .
5. Với mọi $i \in [l, r]$, hỏi giá trị lớn nhất của $A_i + B_i$.

Dữ liệu: $n, m \leq 3 \cdot 10^5$. Nguồn: bài tự sáng tác.

Trong bài này có hai đối tượng thao tác là mảng A và mảng B . Với một mảng đơn lẻ, trong mỗi nút ta cần chia vị trí thành hai lớp: giá trị lớn nhất của đoạn và không phải giá trị lớn nhất.

Mở rộng nhẹ sẽ được cách làm cho bài này. Trên cây đoạn, chia các vị trí trong mỗi nút thành bốn lớp: đồng thời là giá trị lớn nhất trong A và B ; là giá trị lớn nhất trong A nhưng không phải trong B ; là

giá trị lớn nhất trong B nhưng không phải trong A ; và đồng thời không phải giá trị lớn nhất của cả hai mảng. Duy trì đáp án và nhãn riêng cho bốn lớp này.

Về độ phức tạp, hai mảng độc lập hoàn toàn trong thao tác, hai bộ nhãn không ảnh hưởng lẫn nhau, nên thời gian vẫn là $O(m \log^2 n)$.

Cách làm này mở rộng được cho nhiều mảng. Nếu thao tác đồng thời trên k mảng, trong mỗi nút cần chia các vị trí thành 2^k lớp, nên thời gian là $O(2^k m \log^2 n)$ và không gian là $O(n 2^k)$.

Ví dụ 6. Dzy loves segment tree. Cho một dãy A độ dài n . Có m thao tác:

1. Với mọi $i \in [l, r]$, đặt $A_i \leftarrow \min(A_i, x)$.
2. Với mọi $i \in [l, r]$, cộng x vào A_i .
3. Hỏi $\gcd(A_l, A_{l+1}, \dots, A_r)$.

Dữ liệu: $n, m \leq 10^5$. Nguồn: bài tự sáng tác.

Trước hết xét trường hợp chỉ có cộng/trừ đoạn. Đây là bài kinh điển, vì

$$\gcd(a, b, c) = \gcd(a, b - a, c - b).$$

Ta có thể lấy sai phân mảng A , tức dùng $A_i - A_{i-1}$ để thay cho A_i . Sau sai phân, bài toán tách thành hai phần: cộng/trừ đoạn hỏi giá trị điểm, và sửa điểm hỏi gcd đoạn. Cả hai đều duy trì đơn giản bằng cây đoạn.

Nay xét thêm thao tác lấy min. Cách xử lý cực trị trên đoạn là chia các số thành hai lớp, nhưng như vậy thứ tự của các số bị phá vỡ, không thể trực tiếp sai phân theo thứ tự chỉ số ban đầu. Do đó xét một kiểu sai phân khác: tách riêng giá trị lớn nhất của đoạn để duy trì, rồi sai phân toàn bộ dãy các giá trị không phải lớn nhất.

Cụ thể, với mỗi nút cây đoạn, duy trì thêm các thông tin: giá trị lớn nhất ma , giá trị lớn thứ hai nghiêm ngặt se , phần đầu s và phần cuối t của dãy không phải giá trị lớn nhất, cùng gcd của dãy này sau sai phân, ký hiệu num .

Xét cập nhật thông tin từ hai con l và r . Chỉ nêu trường hợp $ma_l > ma_r$; các trường hợp khác tương tự. Khi đó

$$ma = ma_l, \quad se = \max(se_l, ma_r).$$

Khi cập nhật dãy sai phân, chú ý rằng thứ tự sai phân có thể tùy ý. Ta nối dãy không phải lớn nhất của con trái, dãy không phải lớn nhất của con phải, rồi giá trị lớn nhất của con phải, tức đặt

$$s = s_l, \quad t = ma_r,$$

và

$$num = \gcd(num_l, num_r, t_l - s_r, t_r - ma_r).$$

Khi gán nhãn cộng/trừ đoạn, vì dãy không phải giá trị lớn nhất đã được sai phân nên num không bị ảnh hưởng; chỉ cần sửa các thông tin còn lại, việc này rất đơn giản.

Nếu tính cả độ phức tạp của phép gcd, ta thu được cách làm $O(m \log^3 n)$.

3.3. Tổng kết

Trong mục này, ta thu được một phương pháp tổng quát để xử lý phép toán cực trị trên đoạn: bằng cách duy trì giá trị lớn nhất (hoặc nhỏ nhất) và giá trị lớn thứ hai (hoặc nhỏ thứ hai) nghiêm ngặt, ta có thể

chuyển phép toán cực trị trên đoạn thành phép cộng/trừ đoạn trong chi phí thêm một nhân tử $\log n$ có hằng số nhỏ.

Nhưng cách làm này cũng có giới hạn. Như trong Ví dụ 6, khi câu hỏi liên quan tới thứ tự chỉ số thì phương pháp khó áp dụng, vì khi tách một nút ra ta không xét tới thứ tự chỉ số. Chẳng hạn: cộng/trừ đoạn, lấy max trên đoạn, rồi hỏi trong một đoạn cho trước, tổng trọng số lớn nhất của mọi đoạn con. Bài này khó giải bằng phương pháp trên.

4. Bài toán cực trị lịch sử

4.1. Giới thiệu

Trong bài toán cấu trúc dữ liệu, ta thường cần thực hiện một số thao tác trên một mảng A cho trước, rồi trả lời một số câu hỏi.

Hiện nay phần lớn bài cấu trúc dữ liệu hỏi phiên bản hiện tại; một số ít cần hỏi phiên bản lịch sử, và đa số trong nhóm này khảo sát cấu trúc dữ liệu khả tri. Ngoài ra, có một loại đặc biệt liên quan tới phiên bản lịch sử mà ta gọi là **bài toán cực trị lịch sử**.

Câu hỏi trong bài toán cực trị lịch sử có thể chia đại khái thành ba loại.

4.1.1. Cực đại lịch sử. Định nghĩa một mảng phụ B , ban đầu hoàn toàn giống mảng A . Sau mỗi thao tác, với mọi $i \in [1, n]$, cập nhật

$$B_i = \max(B_i, A_i).$$

Khi đó B_i được gọi là cực đại lịch sử của vị trí i .

4.1.2. Cực tiểu lịch sử. Định nghĩa một mảng phụ B , ban đầu hoàn toàn giống mảng A . Sau mỗi thao tác, với mọi $i \in [1, n]$, cập nhật

$$B_i = \min(B_i, A_i).$$

Khi đó B_i được gọi là cực tiểu lịch sử của vị trí i .

4.1.3. Tổng phiên bản lịch sử. Định nghĩa một mảng phụ B , ban đầu mọi phần tử đều bằng 0. Sau mỗi thao tác, với mọi $i \in [1, n]$, cập nhật

$$B_i \leftarrow B_i + A_i.$$

Khi đó B_i được gọi là tổng các phiên bản lịch sử của vị trí i .

Tiếp theo, ta chia bài toán cực trị lịch sử thành bốn loại theo độ khó tăng dần.

4.2. Bài có thể xử lý bằng nhãn lười

Ví dụ 7. CPU Monitor. Cho một dãy A độ dài n , đồng thời định nghĩa mảng phụ B , ban đầu giống hệt A . Có m thao tác:

1. Với mọi $i \in [l, r]$, đặt $A_i \leftarrow x$.
2. Với mọi $i \in [l, r]$, cộng x vào A_i .
3. Với mọi $i \in [l, r]$, hỏi giá trị lớn nhất của A_i .
4. Với mọi $i \in [l, r]$, hỏi giá trị lớn nhất của B_i .

Sau mỗi thao tác, cập nhật $B_i = \max(B_i, A_i)$. Dữ liệu: $n, m \leq 10^5$. Nguồn: TYVJ 1518.

Khi mới tiếp xúc loại bài này, ví dụ này có thể khá khó, nên trước hết bỏ qua loại thao tác thứ nhất.

Xét dùng nhãn lười truyền thống. Nếu chỉ hỏi giá trị lớn nhất trên đoạn, một nhãn cộng/trừ đoạn, ký hiệu Add , là đủ.

Nay xét hỏi giá trị lớn nhất của cực đại lịch sử trên đoạn. Định nghĩa một nhãn lười mới: nhãn cộng/trừ lớn nhất trong lịch sử, ký hiệu Pre . Nhãn này là giá trị lớn nhất mà nhãn Add của nút từng đạt được trong khoảng thời gian từ lần đẩy nhãn gần nhất của nút đó tới hiện tại.

Khi đẩy nhãn của nút i xuống con l , các nhãn hợp nhất như sau:

$$Pre_l = \max(Pre_l, Add_l + Pre_i), \quad Add_l = Add_l + Add_i.$$

Việc cập nhật thông tin cực đại lịch sử của đoạn cũng tương tự: lấy giá trị lớn nhất hiện tại của đoạn cộng với Pre_i , rồi so với cực đại lịch sử cũ.

Quay lại bài gốc, quan sát biến đổi của một nút bị ảnh hưởng trong quá trình sửa. Nếu nút chưa bị đẩy nhãn, ban đầu nó chỉ chịu ảnh hưởng của cộng/trừ đoạn, có thể dùng nhãn Pre trên để ghi lại. Đến một thời điểm nào đó, nút bị nhãn phủ đoạn ảnh hưởng; khi đó mọi số trong nút trở nên hoàn toàn giống nhau, và sau đó mọi sửa cộng/trừ đoạn đối với nút này không khác gì thao tác phủ đoạn.

Vì vậy nhãn của mỗi nút có thể chia thành hai phần: phần đầu là cộng/trừ đoạn, phần sau là phủ đoạn. Dùng cặp (x, y) để biểu diễn nhãn cực trị lịch sử: trong giai đoạn thứ nhất, nhãn cộng/trừ lớn nhất là x ; trong giai đoạn thứ hai, nhãn phủ lớn nhất là y . Nhãn này rõ ràng có thể hợp nhất và cập nhật. Do đó bài được giải bằng nhãn lười truyền thống với thời gian $O(m \log n)$.

Ví dụ 8. V. Cho một dãy A độ dài n , đồng thời định nghĩa mảng phụ B , ban đầu giống hệt A . Có m thao tác:

1. Với mọi $i \in [l, r]$, đặt $A_i \leftarrow \max(A_i - x, 0)$.
2. Với mọi $i \in [l, r]$, cộng x vào A_i .
3. Với mọi $i \in [l, r]$, đặt $A_i \leftarrow x$.
4. Cho i , hỏi giá trị A_i .
5. Cho i , hỏi giá trị B_i .

Sau mỗi thao tác, cập nhật $B_i = \max(B_i, A_i)$. Dữ liệu: $n, m \leq 5 \cdot 10^5$. Nguồn: UOJ 164.

Định nghĩa nhãn (a, b) , nghĩa là trước hết cộng a cho mọi số trong đoạn, rồi lấy max với b . Ba loại sửa trong đề đều biểu diễn được bằng nhãn này:

$$(-x, 0), \quad (x, -\infty), \quad (-\infty, x),$$

trong đó ∞ là một hằng đủ lớn được đặt trước.

Hợp nhất hai nhãn (a, b) và (c, d) cho kết quả

$$(a + c, \max(b + c, d)).$$

Vì vậy để hỏi A_i , chỉ cần đẩy mọi nhãn trên đường từ gốc tới lá hỏi xuống lá là tính được đáp án trực tiếp.

Nay xét nhãn cực đại lịch sử. Có thể xem một nhãn như một hàm: giá trị hàm tại mỗi số là kết quả sau khi áp dụng nhãn lên số đó. Trong bài này, nhãn (a, b) tương ứng một hàm từng đoạn: đoạn đầu là đường thẳng song song trục x , đoạn sau là đường thẳng góc số góc 1.

Khi cập nhật nhãn cực đại lịch sử, ta đặt đồ thị của hai nhãn vào cùng hệ trục và lấy phần nằm phía trên. Hình nguồn minh họa một trường hợp hợp nhất: hai đồ thị cũ màu xanh, đồ thị mới màu đỏ. Vì sau

cập nhật, dạng hàm vẫn như cũ, nhãn cực đại lịch sử duy trì được. Do đó với câu hỏi loại 5, ta cũng chỉ cần đẩy các nhãn liên quan xuống là có đáp án. Thời gian $O(m \log n)$.

Ví dụ 9. Rikka with Sequences. Cho một dãy A độ dài n , đồng thời định nghĩa một mảng phụ hai chiều B . Với mỗi $l \leq r$, ban đầu $B_{l,r} = \sum_{i=l}^r A_i$. Có m thao tác:

1. Cho i, x , đặt $A_i \leftarrow x$.
2. Cho l, r với $l \leq r$, hỏi $B_{l,r}$.

Sau mỗi thao tác, cập nhật

$$B_{l,r} = \min \left(B_{l,r}, \sum_{i=l}^r A_i \right).$$

Dữ liệu: $n, m \leq 10^5$.

Bài này không tiện xử lý trực tuyến, nên xét thuật toán ngoại tuyến. Đọc tất cả câu hỏi trước, và xem mỗi câu hỏi (l, r) là một điểm trên mặt phẳng hai chiều. Xét một lần sửa vị trí i ; nó ảnh hưởng tới mọi câu hỏi có hoành độ không vượt quá i và tung độ không nhỏ hơn i , tức mọi đoạn chứa i .

Vì vậy bài toán trở thành: trên mặt phẳng có một số điểm, có các thao tác cộng một số cho mọi điểm trong một hình chữ nhật, hoặc hỏi cực tiểu lịch sử của trọng số một điểm. Đây là bài cực trị lịch sử trong hai chiều. Trường hợp một chiều có thể giải dễ bằng cây đoạn; ý tưởng tự nhiên là dùng cây đoạn hai chiều. Nhưng nhãn cực tiểu lịch sử phụ thuộc thời gian, không thể tách thành nhiều phần độc lập, nên cây đoạn hai chiều không phù hợp.

Ta cần một cấu trúc dữ liệu thỏa hai điều kiện:

1. Tác dụng của nhãn phải theo đúng thứ tự thời gian.
2. Các nhãn ảnh hưởng tới cùng một điểm phải ảnh hưởng tại cùng một nút của cấu trúc dữ liệu.

Trong hai chiều, một cấu trúc thỏa các điều kiện này là kd-tree. Dựng kd-tree trên toàn bộ điểm câu hỏi. Khi gán nhãn, làm tương tự cây đoạn: với mỗi nút kd-tree, duy trì hình chữ nhật bao các điểm trong cây con; nếu rời vùng gán nhãn thì thoát, nếu bị chứa trọn thì gán nhãn trực tiếp, nếu không thì đệ quy. Khi hỏi, cũng giống cây đoạn: đẩy nhãn xuống. Thời gian $O(m\sqrt{n})$.

4.3. Bài điểm không xử lý được bằng nhãn lười

Ở đây, bài điểm không chỉ gồm hỏi cực trị lịch sử tại một điểm, mà còn gồm hỏi cực tiểu của cực tiểu lịch sử trên đoạn và hỏi cực đại của cực đại lịch sử trên đoạn, vì hai loại hỏi đoạn này xử lý hoàn toàn giống hỏi điểm.

Ví dụ 10. Ông già năm mới và dãy số. Cho một dãy A độ dài n , đồng thời định nghĩa mảng phụ B , ban đầu giống hệt A . Có m thao tác:

1. Với mọi $i \in [l, r]$, đặt $A_i \leftarrow \max(A_i, x)$.
2. Với mọi $i \in [l, r]$, cộng x vào A_i .
3. Với mọi $i \in [l, r]$, hỏi giá trị nhỏ nhất của A_i .
4. Với mọi $i \in [l, r]$, hỏi giá trị nhỏ nhất của B_i .

Sau mỗi thao tác, cập nhật $B_i = \min(B_i, A_i)$. Dữ liệu: $n, m \leq 3 \cdot 10^5$. Nguồn: bài tự sáng tác, có trên UOJ 169.

Bài này có phép toán cực trị trên đoạn, vốn đã được nghiên cứu kỹ ở Mục 3.

Vì trong bài, hướng của phép toán cực trị và hướng của câu hỏi cực trị lịch sử trái ngược nhau: một bên lấy max, một bên là cực tiểu lịch sử, nên nhân không thể hợp nhất. Hình nguồn minh họa hai nhân màu xanh sau khi hợp nhất cho phần dưới là một đường gấp khúc màu cam; số đoạn của nó tăng tuyến tính theo số lần cập nhật, không thể chấp nhận. Vì vậy không thể dùng cách nhân lười của Ví dụ 8.

Ở Mục 3, ta đã có cách chuyển phép toán cực trị trên đoạn thành cộng/trừ đoạn. Dùng lại cách đó: chia các vị trí trong mỗi nút thành hai phần, giá trị nhỏ nhất và không phải giá trị nhỏ nhất. Khi ấy thao tác còn lại chỉ là cộng/trừ đoạn, còn nhân cực trị lịch sử thì duy trì rất dễ.

Do đó bài được giải với thời gian $O(m \log^2 n)$.

4.4. Bài đoạn không có phép toán cực trị trên đoạn

Tiêu đề này chủ yếu chỉ ba bài: cộng/trừ đoạn, hỏi tổng cực tiểu lịch sử trên đoạn; cộng/trừ đoạn, hỏi tổng cực đại lịch sử trên đoạn; và cộng/trừ đoạn, hỏi tổng phiên bản lịch sử trên đoạn.

Những bài này thoát nhìn cần duy trì thông tin lịch sử trên cả đoạn, nhưng có thể chuyển về câu hỏi tổng đoạn thông thường bằng cách xây dựng mảng phụ.

4.4.1. Cực tiểu lịch sử. Đặt A_i là mảng hiện tại, B_i là cực tiểu lịch sử. Định nghĩa mảng phụ C_i sao cho tại mọi thời điểm

$$C_i = A_i - B_i.$$

Khi cộng x cho A_i , nếu B_i không đổi thì tương đương cộng x cho C_i ; nếu B_i thay đổi thì C_i trở thành 0. Cụ thể, phép cộng/trừ đoạn tương đương

$$C_i \leftarrow \max(C_i + x, 0),$$

một thao tác rất quen thuộc.

Với câu hỏi tổng đoạn, chỉ cần tính được tổng đoạn của A_i và C_i , trừ nhau là ra đáp án. Tổng của A là bài cây đoạn cơ bản; tổng của C đã được nghiên cứu ở Mục 3. Do đó có thuật toán $O(m \log^2 n)$.

4.4.2. Cực đại lịch sử. Cách xử lý tương tự cực tiểu lịch sử. Đặt A_i là mảng hiện tại, B_i là cực đại lịch sử. Định nghĩa $C_i = A_i - B_i$. Khi đó cộng/trừ đoạn tương đương

$$C_i \leftarrow \min(C_i + x, 0),$$

rồi hỏi tổng trực tiếp. Thời gian $O(m \log^2 n)$.

4.4.3. Tổng phiên bản lịch sử. Đặt A_i là mảng hiện tại, B_i là tổng phiên bản lịch sử, và t là số thao tác đã kết thúc hiện tại, không tính thao tác đang xử lý. Định nghĩa mảng phụ

$$C_i = B_i - tA_i.$$

Với vị trí không bị cộng/trừ đoạn ảnh hưởng, B_i không thay đổi. Khi cộng x cho A_i , ta tương đương trừ xt khỏi C_i . Như vậy bài tổng phiên bản lịch sử được chuyển thành bài cộng/trừ đoạn hỏi tổng đoạn đơn giản, dùng nhân lười trực tiếp. Thời gian $O(m \log n)$.

4.5. Bài đoạn có phép toán cực trị trên đoạn

Sau các thảo luận trên, thuật toán cho loại cuối cùng đã có hình dạng: trước hết dùng phương pháp ở Mục 3 chuyển phép toán cực trị trên đoạn thành cộng/trừ đoạn, rồi dùng phương pháp ở mục con trước để giải.

Vì cách xử lý của loại bài này khá giống nhau, ở đây chỉ chọn một bài làm ví dụ phân tích.

Ví dụ 11. Cho một dãy A độ dài n , đồng thời định nghĩa mảng phụ B , ban đầu giống hệt A . Có m thao tác:

1. Với mọi $i \in [l, r]$, đặt $A_i \leftarrow \max(A_i, x)$.
2. Với mọi $i \in [l, r]$, cộng x vào A_i .
3. Với mọi $i \in [l, r]$, hỏi tổng B_i .

Sau mỗi thao tác, cập nhật $B_i = \min(B_i, A_i)$. Dữ liệu: $n, m \leq 10^5$. Nguồn: bài tự sáng tác.

Trước hết dùng phương pháp của mục con trước, chuyển câu hỏi thành duy trì tổng đoạn của mảng A và mảng C . Sau đó dùng phương pháp của Mục 3, duy trì giá trị nhỏ nhất và nhỏ thứ hai của A , từ đó chuyển thao tác lấy max trên đoạn thành thao tác cộng/trừ lên các giá trị nhỏ nhất của đoạn; như vậy duy trì được tổng đoạn của A .

Tiếp theo xét cách duy trì tổng đoạn cho C . Các thao tác ảnh hưởng tới C gồm:

- Với mọi $i \in [l, r]$, đặt $C_i \leftarrow \max(C_i + x, 0)$. Gọi đây là thao tác loại một.
- Với mọi $i \in [l, r]$, nếu A_i là giá trị nhỏ nhất của A trên đoạn này, đặt $C_i \leftarrow \max(C_i + x, 0)$. Gọi đây là thao tác loại hai.

Vì vậy vẫn dùng phương pháp của Mục 3: với mỗi nút cây đoạn, duy trì trong tập vị trí có A bằng giá trị nhỏ nhất các thông tin giá trị nhỏ nhất của C , giá trị nhỏ thứ hai của C , số lần xuất hiện của giá trị nhỏ nhất; đồng thời duy trì các thông tin tương tự cho tập vị trí có A không bằng giá trị nhỏ nhất. Khi sửa C , lần lượt DFS bạo lực trên hai phần là được.

Cuối cùng chứng minh độ phức tạp. Gọi phần mảng A là tầng thứ nhất, phần mảng C là tầng thứ hai. Tầng thứ nhất có độ phức tạp như Mục 3, chứng minh được là $O(m \log^2 n)$. Vì vậy chỉ xét tầng thứ hai.

Mô phỏng phương pháp ở Mục 3, trước hết chuyển giá trị nhỏ nhất ở tầng thứ hai thành nhãn. Có thể chuyển riêng các vị trí mà A bằng giá trị nhỏ nhất và các vị trí mà A không bằng giá trị nhỏ nhất thành nhãn; gọi chúng là cây thứ nhất và cây thứ hai. Hình nguồn minh họa: phía trên là cây gốc, hai bên lần lượt là giá trị nhỏ nhất của A và C ; phía dưới là hai cây sau khi tách.

Có thể thấy tại mọi thời điểm, bất kỳ nhãn nào xuất hiện trong cây thứ hai đều từng xuất hiện trong cây thứ nhất, nên cấp số lượng nhãn từng xuất hiện trên hai cây giống với cây thứ nhất. Vì vậy chỉ cần phân tích cây thứ nhất.

Định nghĩa của cây thứ nhất phụ thuộc vào mảng A . Nếu trong cây gốc, giá trị nhỏ nhất của một nút khác giá trị nhỏ nhất của cha nó, thì trong cây thứ nhất không tồn tại chuyển tiếp từ nút này lên cha; cần xóa cạnh đó. Do vậy sau chuyển đổi, cây thứ nhất bị chia thành một số thành phần liên thông; mỗi thành phần đều chứa ít nhất một lá và thỏa tính chất giá trị nhãn tăng theo độ sâu.

Với thao tác thu hồi nhãn và cộng/trừ đoạn trực tiếp trên cây thứ nhất, tình hình không khác mảng A , nên chỉ cần xét ảnh hưởng sinh ra khi DFS bạo lực trên mảng A . Trong DFS bạo lực, ta không chỉ thực hiện một số lần cộng/trừ cây con trên cây thứ nhất, mà còn hợp nhất một số thành phần liên thông. Nếu trong khi DFS ta đồng thời đẩy nhãn xuống, thì khi hợp nhất hai thành phần, trên đường giữa gốc của thành phần phía dưới và gốc của thành phần phía trên sẽ không còn nhãn nào; khi đó hợp nhất trực tiếp không phá vỡ ràng buộc nhãn tăng theo độ sâu.

Ở Mục 3, ta đã chứng minh cận trên cho số nút đi qua trong DFS bạo lực là $O(m \log n)$, nên số lần đẩy nhãn trong cây thứ nhất có cận $O(m \log^2 n)$, và tổng số nhãn là $O(m \log^2 n)$. Dùng lại chứng minh của thuật toán ở Mục 3, ta có một cận trên $O(m \log^3 n)$ cho bài này.

Cận này khá lỏng, tức hằng số thực tế rất nhỏ. Trong các dữ liệu đã dựng, số lần DFS bạo lực còn xa mới đạt cấp $O(m \log n)$. Tác giả phỏng đoán có thể tồn tại chứng minh cận thấp hơn; nếu phỏng đoán ở Mục 3 được chứng minh, theo chứng minh trên sẽ có cận $O(m \log^2 n)$. Tuy nhiên hiện chưa có chứng

minh tốt hơn hay cấu trúc dữ liệu cực hạn, nên tạm dùng cận khá lỏng $O(m \log^3 n)$ để đo hiệu quả chạy của thuật toán.

Thực chất bài này là lồng thuật toán của Mục 3 thêm một tầng. Chứng minh trên cũng dùng được cho nhiều tầng lồng hơn: nếu có k mảng lồng nhau, áp dụng thuật toán trên thu được độ phức tạp khoảng

$$O(2^k m \log^{k+1} n).$$

4.6. Tổng kết

Trong mục này, ta thảo luận sâu ba loại bài toán cực trị lịch sử, và giới thiệu cách xây dựng mảng phụ để chuyển câu hỏi tổng cực trị lịch sử trên đoạn thành câu hỏi tổng đoạn đơn giản.

Trong các kỳ OI trong nước Trung Quốc hiện nay, số bài liên quan tới cực trị lịch sử còn rất hạn chế, và phần lớn tập trung ở hai loại đầu có độ khó thấp. Vì vậy việc khai thác loại bài này vẫn còn nhiều không gian; những gì tác giả giải quyết chỉ là một phần nhỏ. Chẳng hạn bài cộng/trừ đoạn, hỏi cực tiểu lịch sử của tổng đoạn, tới nay tác giả vẫn chưa có lời giải tốt. Do đó tác giả hy vọng bài viết có thể gợi mở thêm nghiên cứu. Nếu ai có thu hoạch mới về loại bài này, tác giả cũng hoan nghênh trao đổi.

5. Lời cảm ơn

1. Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
2. Xin cảm ơn thầy Xu Xianyou ở Trường Học Quân đã quan tâm và chỉ dẫn.
3. Xin cảm ơn anh Xu Yinzhan đã gợi mở và chỉ dẫn.
4. Xin cảm ơn anh Peng Yuxiang, anh Lu Kaifeng, bạn Mao Xiao và bạn Luo Zhezhen đã giúp đỡ.
5. Xin cảm ơn bạn Zhou Ziyin và bạn Mao Xiaohan đã đọc kiểm tra bản thảo.
6. Xin cảm ơn tất cả những người cung cấp các tư liệu mà bài viết đã tham khảo.
7. Xin cảm ơn mọi người đã dành thời gian quý báu để đọc bài viết.

Tài liệu tham khảo

1. Xu Yinzhan, luận văn đội tuyển tập huấn quốc gia IOI 2014: *Ứng dụng của cây đoạn trên một lớp bài chia để trị*.
2. Trang Wikipedia tiếng Anh về cây đoạn: [Segment tree](#).
3. Lời giải bài *Gorgeous Sequence* trong đợt huấn luyện liên trường: blog.sina.com.cn.

Bài 7. Xét lại biến đổi Fourier nhanh

Mao Xiao, Trường Yali

Tóm tắt

Bài viết này chủ yếu giới thiệu cách cài đặt biến đổi Fourier nhanh cùng một số kỹ thuật và tối ưu hữu ích trong thi thuật toán. Biến đổi Fourier nhanh là một thuật toán đã thịnh hành từ lâu trong thi tin học, nên với mọi người nó không hề xa lạ; vì vậy bài viết được đặt tên là “xét lại” biến đổi Fourier nhanh. Nói là xét lại, dĩ nhiên bài viết vẫn giới thiệu nhiều phương pháp và kiến thức đã phổ biến, nhưng cũng dành một

phần đáng kể cho một số phương pháp hiện chưa quá phổ biến trong thi thuật toán, tuy rất đơn giản và hiệu quả. Chẳng hạn, khi nhân hai đa thức thực có bậc không quá 10^5 dưới modulo $10^9 + 7$, cách truyền thống hiện nay thường tính kết quả dưới ba modulo có thể dùng biến đổi số học, rồi dùng định lý phần dư Trung Quốc để thu được đáp án cuối cùng; cách đó cần 9 lần biến đổi Fourier rồi rạc theo nghĩa modulo. Nếu dùng phương pháp ở Mục 3.3 của bài viết này, ta chỉ cần 4 lần biến đổi Fourier rồi rạc trên số phức.

Lời nói đầu

Vài năm gần đây, đủ loại thuật toán dựa trên FFT nở rộ. Trong phần giao lưu học viên của trại mùa đông NOI 2014 đã xuất hiện nội dung về phép chia đa thức; sau đó Codeforces cũng có bài yêu cầu dùng phép chia đa thức. Rồi những thuật toán dựa trên lặp Newton lần lượt trở nên phổ biến, chẳng hạn trên Universal Online Judge có bài dùng lặp Newton để giải phương trình vi phân. Ở bài giảng thứ nhất của trại mùa đông NOI 2016, *Dẫn luận đa thức*, các thuật toán này lại được phổ biến thêm; gần đây UOJ còn có bài về tính giá trị đa điểm. Sự xuất hiện của các thuật toán dựa trên FFT đem lại rất nhiều thuận lợi cho việc giải bài.

Tuy vậy, tất cả các thuật toán ấy đều không tách khỏi một thứ: FFT. Nếu không nắm chắc thuật toán cơ bản nhất này, các thuật toán dựa trên FFT chắc chắn sẽ càng khó nắm hơn. Tựa như xây nhà cao tầng: nếu nhà xây rất cao nhưng móng không vững, nhà sập thì cao đến đâu cũng vô ích. Một cây lớn có cành lá sum suê nhưng rễ hỏng thì cũng chỉ còn lại một tán cành khô. Chỉ khi nền tảng vững thì mới đi xa được. Tác giả viết bài này chính từ suy nghĩ đó, mong cùng độc giả quay lại điểm xuất phát và nghiên cứu lại FFT, thuật toán vừa cơ bản vừa kỳ diệu này.

Thế nào là nắm vững một thuật toán? Theo tác giả, nắm vững không chỉ là đã viết, thậm chí học thuộc thuật toán đó, mà còn là hiểu nguyên lý cơ bản, biết các ứng dụng, tối ưu và kỹ thuật quanh nó.

1. Giới thiệu cơ bản

Biến đổi Fourier nhanh, tiếng Anh là Fast Fourier Transform (FFT), là một trong những thuật toán khá thịnh hành hiện nay. Thuật toán này được phổ biến từ năm 1965, nhưng một số phương pháp đã xuất hiện từ năm 1805. Thuật toán thực hiện việc chuyển đổi tín hiệu giữa miền gốc, thường là miền thời gian hoặc không gian, và miền tần số bằng cách tính biến đổi Fourier rời rạc (Discrete Fourier Transform, DFT) của một dãy. Trong thi thuật toán ngày nay, thuật toán này đã rất thường gặp, chủ yếu được dùng để tính tích chập của hai dãy.

1.1. Dẫn nhập

Xét bài toán sau.

Cho hai dãy hữu hạn a_i, b_i . Hãy tìm dãy c sao cho

$$c_i = \sum_{k=0}^i a_k b_{i-k}.$$

Rõ ràng độ dài của c bằng tổng độ dài của a, b trừ đi một. Trong bài viết này, chỉ số dãy đều bắt đầu từ 0.

1.2. Biến đổi Fourier rời rạc

Ta chọn một số n lớn hơn độ dài của c , đồng thời là một lũy thừa của 2. Vì sao cần lũy thừa của 2 sẽ được nhắc tới sau. Với một dãy s có độ dài $|s|$, mọi s_k với $k \notin [0, |s|)$ được xem là 0.

Khi đó

$$c_r = \sum_{p,q} [(p+q) \bmod n = r] a_p b_q. \quad (1.1)$$

Đặt $\omega^n = 1$, tức ω là căn đơn vị bậc n . Trong bài viết này, khi nói tới căn đơn vị, ta hiểu là căn nguyên thủy, nghĩa là n là số nhỏ nhất làm cho $\omega^n = 1$. Để biết rằng

$$\frac{1}{n} \sum_{k=0}^{n-1} \omega^{vk} = [v \bmod n = 0]. \quad (1.2)$$

Do đó

$$\begin{aligned} [(p+q) \bmod n = r] &= [(p+q-r) \bmod n = 0] \\ &= \frac{1}{n} \sum_{k=0}^{n-1} \omega^{(p+q-r)k} \\ &= \frac{1}{n} \sum_{k=0}^{n-1} \omega^{-rk} \omega^{pk} \omega^{qk}. \end{aligned}$$

Suy ra

$$\begin{aligned} c_r &= \sum_{p,q} [(p+q) \bmod n = r] a_p b_q \\ &= \frac{1}{n} \sum_{k=0}^{n-1} \omega^{-rk} \sum_p \omega^{pk} a_p \sum_q \omega^{qk} b_q. \end{aligned}$$

Ta thấy xuất hiện hai dạng quan hệ:

$$a_m = \sum_{k=0}^{n-1} \omega^{mk} b_k, \quad (1.3)$$

$$c_m = \frac{1}{n} \sum_{k=0}^{n-1} \omega^{-mk} d_k. \quad (1.4)$$

Nói cách khác, với mỗi dạng quan hệ, nếu biết về phải và tính nhanh được về trái thì bài toán tích chập sẽ được giải.

Xét tiếp đa thức

$$A(x) = \sum_{k=0}^{n-1} b_k x^k.$$

Khi đó $a_m = A(\omega^m)$. Điều này tương đương: cho biểu diễn hệ số của một đa thức, hãy tính giá trị của nó tại mọi điểm ω^k . Đây chính là quá trình biến đổi giữa biểu diễn hệ số và biểu diễn điểm giá trị, tức DFT.

Từ đây trở đi, khi nói tới quan hệ giữa đa thức và dãy hệ số, ta đều hiểu như quan hệ giữa $A(x)$ và b_k ở trên. Nói thực hiện DFT trên $A(x)$, thực ra là thực hiện DFT trên b_k ; nói DFT của một đối tượng thu được $A(x)$, thực ra là thu được dãy b_k .

1.3. Biến đổi Fourier nhanh

Mục trước đã nêu công thức DFT, nhưng nếu tính theo công thức thì hiển nhiên vẫn quá chậm. Lúc này cần dùng tính chất của căn đơn vị phức để tăng tốc DFT, tức dùng Fast Fourier Transform (FFT).

Sau đây chỉ xét công thức (1.3): biết b , cần tìm a .

Nhớ rằng n là lũy thừa của 2. Dùng ω_n^k để biểu thị căn đơn vị bậc n , ta có

$$\omega_{2n}^{2m} = \omega_n^m, \quad (1.5)$$

$$\omega_{2n}^{m+n} = -\omega_{2n}^m. \quad (1.6)$$

Gọi $A_0(x)$ là tổng các hạng tử bậc chẵn của $A(x)$, và $A_1(x)$ là tổng các hạng tử bậc lẻ. Khi đó

$$A(\omega_n^m) = A_0((\omega_n^m)^2) + \omega_n^m A_1((\omega_n^m)^2) \quad (1.7)$$

$$= A_0(\omega_{n/2}^m) + \omega_n^m A_1(\omega_{n/2}^m), \quad (1.8)$$

$$A(\omega_n^{m+n/2}) = A_0((\omega_n^m)^2) + \omega_n^{m+n/2} A_1((\omega_n^m)^2) \quad (1.9)$$

$$= A_0(\omega_{n/2}^m) - \omega_n^m A_1(\omega_{n/2}^m). \quad (1.10)$$

Vì vậy, chỉ cần tính DFT của $A_0(x)$ và $A_1(x)$, ta có thể tính DFT của $A(x)$ trong thời gian tuyến tính. Hai công thức (1.8), (1.10) thường được gọi là thao tác “bướm”.

Gọi độ phức tạp khi DFT một dãy độ dài n là $T(n)$. Khi đó

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n). \quad (1.11)$$

Theo định lý chủ, $T(n) = O(n \log n)$.

Nếu cài đặt đúng như mô tả trên, ta phải tách được phần bậc lẻ và bậc chẵn của một đa thức. Trong đệ quy điều này làm được, nhưng hằng số khá lớn. FFT là thuật toán được dùng thường xuyên; nếu hằng số cài đặt quá lớn thì không tốt.

Quan sát thứ tự hệ số đi vào mỗi lời gọi. Với độ dài 8, ta có:

$$\begin{aligned} & (a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7), \\ & (a_0, a_2, a_4, a_6) \quad (a_1, a_3, a_5, a_7), \\ & (a_0, a_4) \quad (a_2, a_6) \quad (a_1, a_5) \quad (a_3, a_7), \\ & (a_0) \quad (a_4) \quad (a_2) \quad (a_6) \quad (a_1) \quad (a_5) \quad (a_3) \quad (a_7). \end{aligned}$$

Thứ tự chỉ số là $(0, 4, 2, 6, 1, 5, 3, 7)$, có biểu diễn nhị phân

$$(000, 100, 010, 110, 001, 101, 011, 111).$$

Đây chính là thứ tự thu được bằng cách đảo bit từng biểu diễn nhị phân trong

$$(000, 001, 010, 011, 100, 101, 110, 111).$$

Thao tác này thường gọi là hoán vị đảo bit. Nếu trước hết đổi dãy sang thứ tự đảo bit, thì trong quá trình tính, mỗi lần xử lý đều là một đoạn liên tiếp, rất dễ cài đặt FFT không đệ quy, và tốc độ cũng nhanh hơn bản đệ quy.

Ngoài ra, để tối ưu hằng số hơn nữa, ta nên tiền xử lý căn đơn vị.

Có thể tiền xử lý căn đơn vị bằng truy hồi $\omega^{k+1} = \omega^k \omega$, nhưng sai số chính xác của cách này hơi lớn. Trong nhiều tình huống vẫn chấp nhận được, nhưng phương pháp FFT ở Mục 3 đòi hỏi độ chính xác tới khoảng 10^{-14} , nên cách tiền xử lý đó có thể gây vấn đề nghiêm trọng.

Một cách khác là với căn đơn vị bậc n , dùng trực tiếp

$$\omega_n^k = \cos \frac{2\pi k}{n} + i \sin \frac{2\pi k}{n},$$

trong đó $i = \sqrt{-1}$. Cách này chậm hơn truy hồi một chút, nhưng sai số nhỏ hơn và đáp ứng được yêu cầu độ chính xác 10^{-14} .

Quá trình làm tích chập chỉ là thực hiện DFT một lần cho hai mảng a, b , sau đó nhân từng điểm, rồi thực hiện một lần biến đổi ngược IDFT (Inverse Discrete Fourier Transform), tức công thức (1.4). Cách làm tương tự nên không trình bày chi tiết.

Có thể tốt hơn nữa không? Dĩ nhiên là có. Nhờ phương pháp ở Mục 3, ta có thể tính tích chập của dãy thực bằng hai lần DFT, thậm chí còn ít hơn theo cách đếm tương đương.

1.4. Biến đổi số học

Trong thi thuật toán hiện nay, bài toán thường yêu cầu lấy modulo một số. Nếu modulo đó là một số nguyên tố p , và $p - 1$ là bội của một lũy thừa của 2 lớn hơn độ dài DFT cần dùng, ta có thể tìm căn nguyên thủy g trong $\mathbb{F}_p$, rồi dùng $g^{(p-1)/n}$ để thay cho căn đơn vị phức ω . Đây là biến đổi số học, Number Theoretic Transform (NTT).

Nếu modulo không phải là số nguyên tố thỏa điều kiện trên, có thể tham khảo Mục 3.3.

Với phép nhân đa thức trên các trường khác, có thuật toán $O(n \log n \log \log n)$. Do giới hạn dung lượng, bài viết không trình bày thêm; độc giả quan tâm có thể xem nội dung liên quan trong bài giảng *Dẫn luận đa thức* ở trại mùa đông NOI 2016.

1.5. Thuật toán Bluestein

Trong công thức (1.1), nếu xét cả phần modulo, thực chất ta đang làm tích chập vòng độ dài n . Đôi khi đề bài yêu cầu đúng loại thao tác tích chập vòng này. Chẳng hạn, nếu cần lũy thừa nhanh trong nghĩa tích chập vòng, và nếu độ dài dãy là lũy thừa của 2, thì ta có thể DFT trực tiếp, lũy thừa nhanh từng điểm, rồi IDFT trở lại. Nhưng nếu đề yêu cầu tích chập vòng với độ dài n không phải lũy thừa của 2 thì sao?

Một cách là tính một tích chập thông thường để được đáp án c , cuối cùng cộng c_{k+n} vào c_k .

Chú ý cách này chỉ làm được một lần tích chập vòng. Nếu cần làm nhiều lần tích chập vòng, ta phải tính từng lần. Nếu còn cần lũy thừa nhanh trong nghĩa tích chập vòng thì lại thêm một hệ số $O(\log n)$, điều ta tất nhiên muốn tránh.

Nếu miền đang xét khá đặc biệt và tồn tại căn đơn vị bậc n , chẳng hạn $\mathbb{C}$ hoặc một số trường hữu hạn $\mathbb{F}_p$, ta có thể cân nhắc tính trực tiếp DFT của dãy độ dài n .

Ở đây chỉ giới thiệu thuật toán Bluestein, một thuật toán tương đối dễ hiểu và hiện vẫn chưa quá phổ biến trong thi thuật toán.

Tiếp tục xét công thức (1.3):

$$\begin{aligned} a_m &= \sum_{k=0}^{n-1} \omega^{mk} b_k \\ &= \omega^{m^2/2} \sum_{k=0}^{n-1} \omega^{k^2/2} b_k \omega^{-(m-k)^2/2}. \end{aligned}$$

Đặt

$$B_k = \omega^{k^2/2} b_k, \quad D_k = \omega^{-k^2/2}.$$

Khi đó

$$a_m = \omega^{m^2/2} \sum_{k=0}^{n-1} B_k D_{m-k} \tag{1.12}$$

$$= \omega^{m^2/2} \sum_{k=0}^{m+n} B_k D_{m-k}. \tag{1.13}$$

Việc mở rộng cận tổng dùng tính chất B_k chỉ có thể khác 0 khi $0 \leq k < n$. Như vậy, ta đã đưa bài toán về dạng tích chập. Nói cách khác, bình thường ta dùng DFT để làm tích chập, còn ở đây ta dùng tích chập để làm DFT.

Vì vậy, chỉ cần một lần FFT thông thường là có thể tính DFT độ dài tùy ý. Đây là quá trình của thuật toán Bluestein.

Thuật toán Bluestein thực ra tính Chirp Z-Transform (CZT), một dạng tổng quát của DFT. Nói đơn giản, thuật toán này không chỉ thực hiện DFT với độ dài tùy ý, mà còn có thể thay ω trong công thức (1.3) bằng một số bất kỳ khác, miễn các phép toán tương ứng có nghĩa.

2. Ứng dụng cơ bản

Tác dụng lớn nhất của FFT là tính tích chập. Tích chập không chỉ dùng trong nhân đa thức, mà còn xuất hiện trong nhiều dạng bài khác. Mục này giới thiệu một số ứng dụng thường gặp.

2.1. Ứng dụng cơ bản nhất

Ví dụ 1. Force. Cho n điểm, điểm thứ i có đại lượng q_i . Với mỗi i , cần tính một đại lượng có dạng

$$E_i = \sum_{j \neq i} \frac{q_j}{(i-j)^2}$$

với dấu phụ thuộc vào phía trái hay phía phải của i . Dữ liệu bảo đảm $n \leq 100000$. Nguồn: ZJOI 2014.

Biểu thức trong đề chỉ khác tích chập thông thường một chút. Một kỹ thuật là đặt một dãy phụ P theo chỉ số lệch:

$$P_i = \begin{cases} (i-n)^{-2}, & i > n, \\ -(i-n)^{-2}, & i < n, \\ 0, & i = n. \end{cases}$$

Khi đó, theo định nghĩa của P , giá trị cần tính có thể xem là một đoạn trong tích chập giữa dãy q và dãy P . Vì vậy dùng FFT là đủ.

Ví dụ 2. Fuzzy Search. Cho xâu mẹ và xâu mẫu, bảng chữ cái có kích thước 4. Cho thêm K . Xâu mẫu khớp tại một vị trí nếu với mọi vị trí trong xâu mẫu, trong đoạn dài K ký tự về hai phía của vị trí tương ứng trong xâu mẹ có một ký tự bằng nó. Hãy đếm số vị trí khớp. Độ dài xâu không vượt quá 200000.

Ta dễ tính trước, với mỗi vị trí của xâu mẹ và mỗi loại ký tự, vị trí đó có thể khớp với ký tự này hay không. Với mỗi ký tự, đặt các vị trí của xâu mẹ có thể khớp ký tự đó và các vị trí trong xâu mẫu bằng ký tự đó là 1, còn lại là 0. Nếu tại một vị trí cả xâu mẹ và xâu mẫu đều là 1, đóng góp là 1, ngược lại không đóng góp. Một vị trí khớp khi tổng đóng góp của bốn ký tự bằng độ dài xâu mẫu.

Kỹ thuật thường dùng tiếp theo là đảo ngược xâu mẫu hoặc xâu mẹ. Sau khi đảo, việc tính đóng góp trở thành một phép tích chập, nên có thể dùng FFT trực tiếp. Bài này thể hiện kỹ thuật quen thuộc: đảo xâu mẫu hoặc xâu mẹ trong bài toán khớp xâu để chuyển về tích chập.

2.2. Ứng dụng ở tầng cao hơn

Sau đây là một số ứng dụng FFT ở tầng cao hơn.

Ví dụ 3. Kyoya and Train. Cho đồ thị có n đỉnh và m cạnh, cùng đỉnh xuất phát và đỉnh kết thúc. Nếu không đến được đích trong thời gian T , ta phải trả một khoản phạt cho trước. Đi qua mỗi cạnh có một chi phí cho trước; thời gian đi qua cạnh là một biến ngẫu nhiên với phân bố xác suất cho trước. Cần cực tiểu hóa kỳ vọng chi phí phải trả. Dữ liệu bảo đảm $n \leq 50$, $m \leq 100$, $T \leq 20000$.

Trước hết xét quy hoạch động bạo lực. Đặt $DP_{x,t}$ là kỳ vọng chi phí nhỏ nhất để từ đỉnh x , khi thời gian đã trôi qua là t , đi tới đích. Gọi $P_{e,k}$ là xác suất cạnh e mất thời gian k . Với mỗi cạnh e đi tới đỉnh y , chuyển tiếp sẽ dùng

$$c_e + \sum_k DP_{y,t+k} P_{e,k}$$

để cập nhật đáp án. Nếu $t+k > T$, giá trị cụ thể của $t+k$ không còn ảnh hưởng, nên có thể đồng nhất tất cả về trạng thái $T+1$.

Với một cạnh $e : x \rightarrow y$, đặt

$$S_{e,t} = c_e + \sum_k DP_{y,t+k} P_{e,k}.$$

Khi đó $DP_{x,t}$ là giá trị nhỏ nhất của $S_{e,t}$ trên các cạnh đi ra từ x .

Xét tối ưu bằng chia để trị. Nếu cần tính mọi $DP_{x,t}$ và $S_{e,t}$ với $l \leq t < r$, đặt $mid = \lfloor (l+r)/2 \rfloor$. Trước hết tính xong các đáp án với $mid < t < r$. Sau đó chú ý tổng

$$\sum_k DP_{y,t+k} P_{e,k}$$

rất dễ chuyển thành dạng tích chập; có thể dùng FFT để tính đóng góp của đoạn DP bên phải vào mọi giá trị S ở bên trái, rồi đệ quy xử lý $l \leq t \leq mid$. Nếu $l = r$, cập nhật trực tiếp DP bằng S . Theo định lý chủ, độ phức tạp thời gian là $O(mT \log^2 T)$.

Bài này khéo léo dùng FFT trong chia để trị. Thực tế, chia để trị FFT là một phương pháp giải bài rất thường gặp. Nó cũng xuất hiện trong nội suy đa điểm, tính giá trị đa điểm và các bài toán tương tự; vì các nội dung đó lệch khỏi mục tiêu giới thiệu ứng dụng cơ bản của FFT trong bài này, ta không trình bày chi tiết.

Ví dụ 4. Transforming Sequence. Đếm số dãy k bit độ dài n sao cho các tiền tố theo phép OR bit đơn điệu tăng. Mỗi vị trí là một số nguyên trong đoạn $[0, 2^k - 1]$. Dữ liệu bảo đảm $k \leq 30000$, và đáp án cần xuất modulo $10^9 + 7$.

Tiếp tục bắt đầu từ thuật toán bạo lực. Có thể đặt $DP_{i,j}$ là số dãy hợp lệ độ dài i mà giá trị OR hiện tại đã có j bit bằng 1. Khi đó $DP_{i,j}$ đóng góp

$$DP_{i,j} \binom{k-j}{l}$$

cho $DP_{i+1,j+l}$.

Để tối ưu, giả sử đã biết mọi $DP_{i,x}$ và $DP_{j,y}$, và muốn tính nhanh mọi $DP_{i+j,z}$. Với một z cố định, đóng góp từ $DP_{i,x}$ tới $DP_{i+j,x+y}$ có dạng

$$DP_{i,x} \cdot 2^{jx} \cdot DP_{j,y} \cdot \binom{k-x}{y},$$

trong đó hệ số 2^{jx} xuất hiện vì các bit đã có trong phần đầu có thể được điền tùy ý ở phần sau. Đây là một dạng tích chập, nên có thể dùng FFT để tính nhanh.

Như vậy, bằng lũy thừa nhanh đơn giản và FFT, bài toán được giải trong $O(k \log^2 k)$.

Ứng dụng của FFT còn xa mới chỉ có từng ấy. Do giới hạn dung lượng, tác giả tạm thời chỉ giới thiệu đến đây.

2.3. Một số kỹ thuật tối ưu

2.3.1. Giảm số lần DFT. Xét việc tính

$$A(x) = B(x)C(x) + D(x)E(x).$$

Nếu làm riêng rẽ, ta cần tính tích $B(x)C(x)$ và $D(x)E(x)$. Nhưng dễ thấy rằng sau khi DFT, nếu kết hợp từng điểm bằng phép nhân, chia tương ứng rồi IDFT trở lại, ta sẽ thu được đúng phép kết hợp nhân, chia của các đa thức tương ứng, miễn độ dài không vượt quá giới hạn. Vì vậy ta chỉ cần DFT $B(x), C(x), D(x), E(x)$ để được các dãy b, c, d, e , rồi với mỗi k đặt

$$a_k = b_k c_k + d_k e_k,$$

cuối cùng IDFT dãy a để thu được $A(x)$.

Phương pháp này rất hữu ích trong các phép toán đa thức phức tạp. Nhược điểm là khó tương thích với kỹ thuật gộp DFT được giới thiệu ở Mục 3.

Còn một kỹ thuật đơn giản khác: nếu một dãy được dùng nhiều lần cho DFT, hiển nhiên có thể tiền xử lý DFT của nó, rồi mỗi lần dùng trực tiếp kết quả đã tiền xử lý.

2.3.2. Lợi dụng tích chập vòng. Xét hai dãy a, b độ dài n , cần tính các hệ số từ khoảng $[0.5n]$ tới $[1.5n]$ của tích chập c . Cách truyền thống là bù 0 để mở rộng thành dãy độ dài $2n$, rồi dùng FFT tính tích chập. Nhưng vì thứ FFT tính thực ra là tích chập vòng như đã nói ở Mục 1.5, nếu chỉ bù 0 tới độ dài lớn hơn $[1.5n]$ rồi gọi FFT, thì các hệ số từ $[0.5n]$ tới $[1.5n]$ không bị ảnh hưởng.

Hai tối ưu trên thường xuất hiện trong thuật toán dựa trên lặp Newton, và có tác dụng giảm hằng số khá lớn.

2.3.3. Bảo lực phạm vi nhỏ trong chia để trị FFT. Trong thuật toán chia để trị FFT, do FFT có hằng số tương đối lớn, khi độ dài dãy nhỏ, chuyển sang bạo lực thường đem lại tối ưu đáng kể. Thực ra tối ưu này không chỉ dùng trong chia để trị FFT, mà còn hữu dụng trong mọi thuật toán chia để trị.

Tối ưu trên có tác dụng lớn trong các thuật toán như nội suy đa điểm và tính giá trị đa điểm.

2.3.4. Tối ưu số lần nhân trong lũy thừa nhanh. Nếu cần tính lũy thừa bậc n của một đa thức, trong điều kiện cho phép có thể lấy ln đa thức, nhân với n , rồi lấy exp đa thức; độ phức tạp giảm đi một hệ số $O(\log n)$. Phương pháp tính ln và exp đa thức không thuộc trọng tâm ứng dụng cơ bản của FFT trong bài này, nên không trình bày.

Nếu đối tượng cần lũy thừa nhanh không thể tối ưu bằng cách trên, có một kỹ thuật hiện chưa quá phổ biến: dùng addition chain.

Định nghĩa một addition chain là một dãy bắt đầu bằng 1, và hiệu giữa một phần tử với phần tử đứng ngay trước nó bằng một phần tử nào đó xuất hiện trước phần tử đó trong dãy. Ta nói addition chain này có thể thu được mọi số trong dãy.

Ví dụ, 1, 2, 4, 6 là một addition chain: $6 - 4 = 2$ đã xuất hiện trước 6, và $4 - 2 = 2$ đã xuất hiện trước 4. Chuỗi này thu được các số $\{1, 2, 4, 6\}$. Từ đó ta thiết kế được cách dùng phép nhân để thu các lũy thừa 1, 2, 4, 6: muốn có lũy thừa bậc 6, bắt đầu từ bậc 1, nhân bậc 1 với bậc 1 để được bậc 2, nhân bậc 2 với bậc 2 để được bậc 4, rồi nhân bậc 4 với bậc 2 để được bậc 6.

Tìm addition chain tối ưu cho một số là bài toán NP-khó, nhưng có các thuật toán xấp xỉ tốt. Chẳng hạn, có thể BFS và ghi lại addition chain ứng với mỗi số. Ban đầu từ 1, addition chain của 1 chỉ gồm số 1. Mỗi lần xét số hiện tại x , liệt kê mọi số y trong addition chain của x ; nếu $x + y$ chưa được thăm thì đưa nó vào hàng đợi, và đặt addition chain của nó bằng addition chain của x nối thêm $x + y$. Trường hợp xấu nhất, phương pháp này tốt hơn lũy thừa nhanh rất nhiều; nhược điểm là cần tiền xử lý tuyến tính.

Nếu cần tính lũy thừa bậc n với n rất lớn, ví dụ 10^{1000} , có thể dùng Method of Four Russians để giảm số lần nhân từ khoảng $2 \log_2 n + C$ xuống còn $\log_2 n + O(\sqrt{\log n})$. Cụ thể, đặt $m = \sqrt{\log n}$, tiền xử lý mọi lũy thừa từ 0 tới $2^m - 1$, độ phức tạp là $O(2^m) = O(\sqrt{n})$ theo cách đếm trên số bit. Sau đó mỗi lần nhân đôi theo khối; sau mỗi khối, nhân thêm lũy thừa ứng với phần dư hiện tại modulo 2^m . Số lần nhân trong phần nhân đôi là $\log_2 n + C$, còn phần nhân thêm có cấp $O(\log n/m) = O(\sqrt{\log n})$.

Nếu số lần hỏi rất nhiều thì có thể chọn m lớn hơn.

Những phương pháp trên tuy chỉ là tối ưu hằng số, nhưng cài đặt rất đơn giản và hiệu quả thực tế khá tốt.

3. Một kỹ thuật mới

Tiếp theo, ta nghiên cứu một kỹ thuật có thể giảm đáng kể số lần DFT, từ đó tối ưu hằng số rất mạnh.

Thực ra kỹ thuật này hiện chưa hoàn toàn phổ biến trong thi thuật toán, nhưng cũng đã có một số người từng nghe qua. Tác giả giới thiệu ở đây với hy vọng giúp phổ biến thêm kỹ thuật này.

Kỹ thuật này là cách tối ưu được dùng trong bài CodeChef MGCH3D. Trên thực tế, enot1.10, tức Vladimir Smykalov từ Codeforces Nga, từng nói với tác giả trong một cuộc trao đổi về một phương pháp tối ưu trông có vẻ khác. Nhưng sau vài suy luận có thể thấy phương pháp ấy về bản chất gần giống với cách tối ưu trong lời giải CodeChef MGCH3D. Vì vậy ở đây chỉ mô tả theo lời giải của CodeChef MGCH3D; bản gốc được liệt kê trong tài liệu tham khảo.

Khi thảo luận số lần DFT, đôi khi ta không phân biệt DFT và IDFT; chẳng hạn 3 lần IDFT cộng 3 lần DFT được gọi là 6 lần DFT.

3.1. Giới thiệu cơ bản

Xét việc DFT hai đa thức thực $A(x), B(x)$ độ dài n . Giả sử n đã được chỉnh thành lũy thừa của 2.

Định nghĩa

$$P(x) = A(x) + iB(x), \quad (3.1)$$

$$Q(x) = A(x) - iB(x). \quad (3.2)$$

Ở đây $i = \sqrt{-1}$. Gọi $F_P[k] = P(\omega^k)$, $F_Q[k] = Q(\omega^k)$ là hai dãy thu được sau khi DFT P, Q , trong đó ω là căn đơn vị bậc $2L$. Sau đây suy ra quan hệ giữa hai dãy này. Do vấn đề trình bày, dùng X để thay cho góc trong biểu thức lượng giác; ý nghĩa của mỗi X ứng với chỉ số k có thể thấy từ ngữ cảnh. Ký hiệu $\text{conj}(z)$ là số phức liên hợp của z .

$$\begin{aligned} F_P[k] &= A(\omega^k) + iB(\omega^k) \\ &= \sum_j (A_j + iB_j)(\cos X + i \sin X), \end{aligned}$$

trong khi

$$\begin{aligned} F_Q[k] &= A(\omega^k) - iB(\omega^k) \\ &= \sum_j (A_j - iB_j)(\cos X + i \sin X) \\ &= \text{conj} \left(\sum_j (A_j + iB_j)(\cos(-X) + i \sin(-X)) \right) \\ &= \text{conj}(F_P[2L - k]). \end{aligned}$$

Vì vậy chỉ cần 1 lần DFT là có thể tính được cả hai dãy F_P, F_Q .

Gọi $\text{DFT}(A)[k]$ là phần tử thứ k của dãy thu được sau khi DFT A . Ta có

$$\text{DFT}(A)[k] = \frac{F_P[k] + F_Q[k]}{2}, \quad (3.3)$$

$$\text{DFT}(B)[k] = \frac{F_P[k] - F_Q[k]}{2i}. \quad (3.4)$$

Như vậy, ta gộp 2 lần DFT thành 1 lần DFT, giảm được một lần biến đổi.

3.2. Tích chập nhanh hơn

Có thể tiếp tục tối ưu xuống dưới 2 lần DFT không? Dĩ nhiên là có.

Ở trên đã nhắc tới kỹ thuật tách đa thức thành phần bậc chẵn và bậc lẻ. Tiếp tục dùng kỹ thuật này để tối ưu.

Theo định nghĩa trước đó, đặt $A_0(x)$ là tổng các hạng tử bậc chẵn của $A(x)$, $A_1(x)$ là tổng các hạng tử bậc lẻ. Khi đó

$$A(x) = A_0(x^2) + xA_1(x^2).$$

Giả sử cần tính tích của $A(x)$ và $B(x)$, trong đó cả hai đa thức đều có bậc không quá $n - 1$. Tương tự viết $B(x) = B_0(x^2) + xB_1(x^2)$. Khi đó

$$\begin{aligned} A(x)B(x) &= (A_0(x^2) + xA_1(x^2))(B_0(x^2) + xB_1(x^2)) \\ &= A_0(x^2)B_0(x^2) + x(A_0(x^2)B_1(x^2) + A_1(x^2)B_0(x^2)) \\ &\quad + x^2A_1(x^2)B_1(x^2). \end{aligned} \tag{3.5}$$

Giả sử sau khi tách chẵn lẻ, mỗi đa thức con có bậc không quá $m - 1$. Ban đầu ta cần 4 lần DFT độ dài m ; gộp từng đôi theo Mục 3.1 thì giảm còn 2 lần.

Xét IDFT. Thoạt nhìn cần 3 lần IDFT, nhưng thực ra không phải. Hai hạng đầu và cuối ở vế phải (3.5) chỉ có hệ số ở bậc chẵn, còn hạng giữa chỉ có hệ số ở bậc lẻ. Vì vậy ta xét gộp hạng đầu và hạng cuối.

Việc gộp hạng đầu và hạng cuối cũng có thể xem là một đa thức theo x^2 . Điều cần làm là từ DFT của $A(x)$, suy ra DFT của $xA(x)$. Dễ thấy DFT của $xA(x)$ chính là dãy thu được bằng cách nhân phần tử thứ k trong DFT của $A(x)$ với ω^k .

Vì vậy chỉ cần 2 lần IDFT độ dài m ; nếu gộp từng đôi thì còn 1 lần.

Xét tất cả các dãy cần IDFT. Các phần tử của chúng không nhất thiết đều là số thực, nhìn qua có vẻ khó gộp. Nhưng chú ý kết quả sau IDFT của chúng là dãy thực. Dù dãy trước IDFT có thuần thực hay không, IDFT vẫn là nghịch đảo của DFT. Từ hai công thức (3.3), (3.4), biết vế phải thì tính được vế trái; biết vế trái cũng khôi phục được vế phải. Ta chỉ cần với mọi k khôi phục các dãy tương ứng, rồi phục hồi $P(x)$ hoặc $Q(x)$. Vì kết quả IDFT là thực, có thể trực tiếp lấy phần thực và phần ảo để thu được kết quả IDFT. Như vậy, số lần IDFT giảm còn 1 lần.

Tóm lại, ta chỉ cần 3 lần DFT độ dài m , tức độ dài đơn. Trong khi đó cách trước cần 2 lần DFT độ dài gấp đôi. Nếu vẫn dùng độ dài gấp đôi làm đơn vị đo, thậm chí có thể mô tả tối ưu vừa nêu là “1.5 lần DFT”.

3.3. Tích chập modulo tùy ý

Tiếp theo, xét cách làm tích chập dưới modulo tùy ý. Ở đây giả sử modulo M có cỡ 10^9 , và độ dài dãy cần tích chập có cỡ 10^5 .

Xét tích chập của hai dãy độ dài cỡ 10^5 . Nếu xét kết quả tích chập trên trường số thực, mỗi hệ số có thể có cỡ 10^{23} . Kiểu số thực thông thường hiển nhiên có sai số. Ta có thể dùng số thực độ chính xác cao hơn, chẳng hạn `decimal` trong Python, nhưng Python có hiệu suất thấp và hiện không được dùng trong các kỳ NOI trong nước Trung Quốc; C++ và nhiều ngôn ngữ khác lại không có kiểu số thực chính xác cao tương tự `decimal`. Nếu tự viết thì rất phiền và hiệu suất cũng không khả quan.

3.3.1. NTT ba modulo. Một cách là tìm ba modulo cỡ 10^9 thỏa tính chất NTT, lần lượt tính kết quả tích chập dưới ba modulo này. Vì mỗi hệ số có cỡ 10^{23} , theo định lý phần dư Trung Quốc, ta có thể xác định duy nhất từng hệ số. Nội dung cụ thể của định lý phần dư Trung Quốc không nhắc lại ở đây.

Cài đặt cụ thể có hai hướng. Một là dùng số nguyên 128 bit hoặc số nguyên lớn. Số nguyên 128 bit nhiều nơi không dùng được, còn số nguyên lớn vừa rườm rà vừa chậm.

Hướng khác khéo hơn là dùng định lý phần dư Trung Quốc theo từng bước. Giả sử ba modulo lần lượt là $\text{mod}_0, \text{mod}_1, \text{mod}_2$. Trước hết hợp nhất hai modulo đầu, tức tìm kết quả dưới modulo mod_0mod_1 . Sau đó dùng nghịch đảo để biểu diễn kết quả dưới modulo $\text{mod}_0\text{mod}_1\text{mod}_2$ thành dạng

$$x + k \text{mod}_0\text{mod}_1.$$

Ta không cần thật sự tính số khổng lồ này, mà chỉ cần tính nó modulo M . Như vậy chỉ cần số nguyên 64 bit; kiểu này hầu như lúc nào cũng dùng được và nhanh hơn nhiều so với hai cách trước.

Nhưng phương pháp trên cần 9 lần DFT theo nghĩa modulo, nên hiệu suất chỉ có thể xem là bình thường.

3.3.2. FFT tách hệ số. Đặt $M_0 = \lfloor \sqrt{M} \rfloor$. Với mỗi hệ số x , viết

$$x = \left\lfloor \frac{x}{M_0} \right\rfloor M_0 + (x \bmod M_0).$$

Lần lượt gọi hai phần cao và thấp của các dãy a, b là a_h, a_l, b_h, b_l .

Ta cần tính bốn tích chập:

$$a_h * b_h, \quad a_h * b_l, \quad a_l * b_h, \quad a_l * b_l.$$

Do mỗi hệ số sau khi tách chỉ có cỡ $\sqrt{M}$, các hệ số trung gian của những tích chập này nhỏ hơn nhiều so với tích chập ban đầu; dùng phương pháp tiền xử lý căn đơn vị theo lượng giác ở Mục 1.3 là đủ chính xác. Sau khi tính xong, nhân từng kết quả với hệ số $M_0^2, M_0, M_0, 1$ tương ứng rồi cộng vào đáp án. Như kỹ thuật ở Mục 2.3.1, có thể tiền xử lý DFT của bốn dãy, rồi gộp trực tiếp các tích chập có cùng hệ số. Khi đó cần 7 lần DFT.

Có thể giảm số lần DFT tiếp không? Dĩ nhiên là có.

Trong quá trình vừa mô tả, bước đầu tiên là DFT bốn dãy riêng rẽ. Ta thấy có thể dùng kỹ thuật gộp DFT ở trên để gộp từng đôi, giảm tổng số lần DFT xuống 5.

Phần IDFT vốn cần 3 lần; gộp từng đôi thì giảm xuống 2 lần.

Như vậy tổng số lần DFT được giảm thành công xuống 4 lần.

Bản cài đặt C++ của tác giả có thể xem tại <http://uoj.ac/submission/49836>.

Tương tự, phương pháp tối ưu tích chập thông thường xuống 3 lần DFT độ dài đơn ở trên rõ ràng cũng có thể áp dụng cho tích chập modulo tùy ý, tức đạt cái gọi là “3.5 lần DFT” cho tích chập modulo tùy ý. Tuy nhiên công thức quá phức tạp mà hiệu quả tối ưu không nổi bật, nên ở đây không trình bày thêm.

3.4. Tổng kết

Mục này giới thiệu một phương pháp giảm số lần DFT rất mạnh nhưng hiện vẫn chưa thật phổ biến. Cuối cùng ta thấy rằng, hai DFT có thể gộp nếu dãy đầu vào của chúng đều là dãy thực; hai IDFT có thể gộp nếu kết quả của chúng đều là dãy thực. Tối ưu này gần như không làm tăng lượng mã, nhưng đem lại cải thiện hiệu suất rất đáng kể.

Tuy nhiên tối ưu này cũng có hạn chế: nó chỉ áp dụng cho FFT trên trường số thực, không áp dụng được cho NTT trên $\mathbb{F}_p$. Nếu có ai tìm được cách làm cho phương pháp này dùng được với NTT, tác giả rất hoan nghênh trao đổi.

Lời cảm ơn

1. Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
2. Xin cảm ơn thầy Wang Xingming ở Trường Yali đã quan tâm và chỉ dẫn trong nhiều năm.
3. Xin cảm ơn bạn Du Yuhao đã giúp đỡ rất nhiều trong quá trình tác giả viết bài này.
4. Xin cảm ơn bạn Jin Ce đã đọc kiểm tra bản thảo.
5. Xin cảm ơn các bạn khác trong phòng máy đã giúp rà lỗi và sửa sai.
6. Xin cảm ơn Vladimir Smykalov, tác giả bài MGCH3D và những người đầu tiên đề xuất kỹ thuật gộp DFT được nêu trong bài.
7. Xin cảm ơn tất cả những người cung cấp các tư liệu mà bài viết đã tham khảo.
8. Xin cảm ơn những thầy cô và bạn học khác từng giúp đỡ và gợi mở cho tác giả.
9. Xin cảm ơn cha mẹ đã quan tâm và chăm sóc.
10. Xin cảm ơn mọi người đã dành thời gian quý báu để đọc bài viết.

Tài liệu tham khảo

1. Bài viết của vflea về phép chia đa thức và ứng dụng: <http://vfleaking.blog.uoj.ac/blog/87>.
2. *Fast Fourier Transform*, phần 2: <http://picks.logdown.com/posts/177631-fast-fourier-transform>.
3. Lời giải CodeChef MGCH3D: <https://discuss.codechef.com/questions/74772/mgch3d-editorial>.
4. Trang Wikipedia tiếng Anh về CZT: [Chirp Z-transform](#).

Bài 8. Từ *Unknown* bàn về một lớp phương pháp duy trì thông tin đoạn có hỗ trợ chèn và xóa ở cuối

Luo Zhezheng, Trường THPT trực thuộc Đại học Sư phạm An Huy

Tóm tắt

Unknown là bài tự sáng tác của tác giả trong kỳ kiểm tra chéo đội tuyển tập huấn, tên đầy đủ là “Chúng ta vẫn chưa biết tên cấu trúc dữ liệu đã nhìn thấy hôm ấy”. Loại bài yêu cầu hỗ trợ chèn, xóa ở cuối và truy vấn đoạn xuất hiện khá thường xuyên trong các cuộc thi tin học những năm gần đây. Bài viết này thông qua việc giới thiệu chi tiết quá trình giải từng bước của bài *Unknown*, phân loại và phân tích một số nhánh khác nhau của loại bài này, tổng kết và đề xuất một số thuật toán tổng quát, đồng thời phân tích hiệu năng của chúng. Trọng tâm của bài viết là tổng kết một phương pháp chung để xử lý việc chèn, xóa tại đầu mút và duy trì thông tin đoạn trong trường hợp thông tin không dễ hợp nhất nhanh. Hy vọng bài viết có ích cho mọi người và gợi ra thêm suy nghĩ về loại bài này.

1. Ý chính đề bài

Cho một dãy S gồm các phần tử là vector, chỉ số bắt đầu từ 1. Ban đầu S rỗng. Cần hỗ trợ ba thao tác:

1. Thêm một phần tử (x, y) vào cuối S .

2. Xóa phần tử cuối của S .

3. Hỏi trong các phần tử có chỉ số thuộc đoạn $[l, r]$, giá trị lớn nhất của $(x, y) \times S_i$.

Ở đây $\times$ biểu thị tích có hướng của vector:

$$(x_1, y_1) \times (x_2, y_2) = x_1 y_2 - x_2 y_1.$$

Bài này được cải biên từ bài *SDOI 2014 Vector Set*: <http://www.lydsy.com/JudgeOnline/problem.php?id=3533>.

2. Định dạng nhập

Dòng đầu là một số nguyên tp , biểu thị kiểu dữ liệu. Sau đó có nhiều bộ dữ liệu, không quá 3 bộ, kết thúc bởi $m = 0$.

Với mỗi bộ dữ liệu, dòng đầu là một số nguyên m , biểu thị số thao tác. Tiếp theo là m dòng, mỗi dòng thuộc một trong ba dạng:

- 1 $\times$ y : thêm phần tử (x, y) vào cuối S .
- 2: xóa phần tử cuối của S .
- 3 l r $\times$ y : hỏi giá trị lớn nhất của $(x, y) \times S_i$ với $i \in [l, r]$.

3. Định dạng xuất

Với mỗi bộ dữ liệu, cần lấy kết quả của mỗi truy vấn modulo

$$M = 998244353 = 7 \cdot 17 \cdot 2^{23} + 1,$$

trong đó M là số nguyên tố, rồi XOR các kết quả đó và in ra.

Chú ý kết quả modulo theo nghĩa toán học nằm trong đoạn $[0, M)$, chẳng hạn $-1 \bmod M = M - 1$.

4. Phạm vi dữ liệu và quy ước

Các điểm kiểm tra có phạm vi như sau:

Điểm	Quy mô m	Điều kiện khác
1	$m \leq 1000$	
2		Không có
3	$m \leq 80000$	
4		Không có thao tác loại 2; truy vấn loại 3 luôn hỏi toàn bộ đoạn
5	$m \leq 300000$	Mọi thao tác loại 2 đều nằm sau thao tác loại 1; truy vấn loại 3 luôn hỏi toàn bộ đoạn
6		Mọi thao tác loại 3 đều nằm sau các thao tác loại 1 và loại 2
7		Với mọi thao tác loại 3, $l = 1$; giới hạn bộ nhớ 128 MB
8		Không có
9	$m \leq 500000$	Giới hạn bộ nhớ 128 MB
10		Giới hạn bộ nhớ 64 MB

Với toàn bộ dữ liệu, gọi n là độ dài dãy tại một thời điểm bất kỳ. Khi đó $n \leq 300000$, $m \leq 500000$. Số thao tác loại 1 không vượt quá 300000; các giá trị nhập vào thỏa

$$-10^9 \leq x \leq 10^9, \quad 1 \leq y \leq 10^9.$$

Số thao tác loại 3 không vượt quá 300000; trong thao tác này,

$$1 \leq x \leq 10^9, \quad -10^9 \leq y \leq 10^9, \quad 1 \leq l \leq r \leq n.$$

Giới hạn thời gian là 5 giây trên Tsinsen hoặc 3 giây trên UOJ, giới hạn bộ nhớ 512 MB, trừ khi điểm kiểm tra có ghi riêng.

5. Giới thiệu thuật toán

5.1. Thuật toán 1: vét cạn

Trước hết xét cách viết vết cạn. Nhìn ban đầu, bài này là một bài cấu trúc dữ liệu; vết cạn cho bài cấu trúc dữ liệu thường khá dễ viết, chỉ cần mô phỏng theo đề. Với truy vấn đoạn $[l, r]$, ta trực tiếp liệt kê mọi phần tử S_i trong đoạn và thống kê giá trị lớn nhất của $(x, y) \times S_i$.

Phân tích độ phức tạp: vì n và m cùng cấp, khi tính độ phức tạp ta không phân biệt nữa. Số truy vấn là $O(n)$, độ dài đoạn cũng có thể đạt cấp $O(n)$, nên thuật toán vét cạn có độ phức tạp thời gian $O(n^2)$, dự kiến đạt 20 điểm.

5.2. Quan sát tính chất của đáp án

Tích vô hướng đơn giản hơn tích có hướng. Ta viết tích có hướng thành tích vô hướng: nếu $S_i = (a_i, b_i)$, thì

$$(x, y) \times S_i = (-y, x) \cdot (a_i, b_i).$$

Sau khi biến đổi (x, y) thành $(-y, x)$, bài toán trở thành tìm giá trị lớn nhất của $(x, y) \cdot (a_i, b_i)$. Các phần sau không dùng trực tiếp tích có hướng nữa.

Do

$$\overrightarrow{OA} \cdot \overrightarrow{OB} = |OA| \cdot |OB| \cos \angle AOB,$$

đáp án tỉ lệ với độ dài hình chiếu của OB theo hướng OA . Vì thế có thể tìm điểm cho tích vô hướng lớn nhất như sau: dùng một đường thẳng vuông góc với OA , quét từ vô cực về phía gốc O ; điểm đầu tiên mà đường thẳng đụng tới chính là đáp án.

Trong hình minh họa của nguồn, với một tập điểm được đánh dấu, đường quét đầu tiên chạm vào điểm A , nên đáp án là $\overrightarrow{OQ} \cdot \overrightarrow{OA}$. Tiếp tục quan sát phạm vi góc của các vector truy vấn: góc cực của chúng nằm trong đoạn $(0, \pi)$. Dễ thấy đáp án chắc chắn nằm trên bao lồi trên của các phần tử trong đoạn.

Nếu ta có thể nhanh chóng lấy bao lồi trên của đoạn truy vấn, thì có thể nhị phân hệ số góc trên bao lồi để trả lời trong $O(\log n)$. Như vậy bài toán được chuyển thành duy trì bao lồi trên của đoạn.

5.3. Duy trì một lớp thông tin đoạn mà bao lồi là đại diện

Khi duy trì thông tin đoạn, cần chú ý một số tính chất của chính thông tin đó; chúng ảnh hưởng trực tiếp tới cách duy trì và độ khó.

5.3.1. Có hỗ trợ hợp nhất nhanh hay không. Với hai đoạn $[l, mid]$ và $[mid + 1, r]$, nếu biết thông tin của hai đoạn này và có thể hợp nhất chúng trong $O(1)$ hoặc $\text{poly}(\log(r - l))$, ta nói loại thông tin này hỗ trợ **hợp nhất nhanh**.

Ví dụ về thông tin hỗ trợ hợp nhất nhanh:

- Cực trị đoạn, với độ phức tạp hợp nhất $O(1)$.
- Bao lồi theo chỉ số đơn điệu. Nếu dùng cây cân bằng duy trì và nhị phân hệ số góc thì có thể hợp nhất trong $O(\log^2 n)$.

Ví dụ về thông tin không hỗ trợ hợp nhất nhanh:

- Bao lồi của tập điểm trong đoạn.
- mex của tập số trong đoạn.

Các thông tin hỗ trợ hợp nhất nhanh đều có thể dùng những cấu trúc dữ liệu đoạn kiểu chia để trị, chẳng hạn cây đoạn. Ví dụ cây đoạn duy trì giá trị lớn nhất hoặc nhỏ nhất trên đoạn là kỹ thuật quen thuộc.

5.3.2. Có hỗ trợ chèn, xóa nhanh hay không. Với một đoạn $[l, r]$, nếu biết thông tin của đoạn này và có thể tính thông tin của đoạn $[l, r + 1]$ trong $O(1)$ hoặc $\text{poly}(\log(r - l))$, ta nói loại thông tin này hỗ trợ **chèn nhanh**. Nếu có thể tính thông tin của đoạn $[l, r - 1]$ trong thời gian như vậy, ta nói nó hỗ trợ **xóa nhanh**.

Một kết luận hiển nhiên là: thông tin hỗ trợ hợp nhất nhanh chắc chắn hỗ trợ chèn nhanh, chẳng hạn chèn ở đầu phải có thể xem là hợp nhất hai đoạn $[l, r]$ và $[r + 1, r + 1]$.

Ví dụ hỗ trợ chèn nhanh nhưng không hỗ trợ xóa nhanh:

- Bao lồi của tập điểm trong đoạn, duy trì bằng cây cân bằng.
- Cây ảo của tập điểm trong đoạn, duy trì theo thứ tự DFS tương đối bằng cây cân bằng.

Ví dụ hỗ trợ xóa nhanh nhưng không hỗ trợ chèn nhanh:

- mex.

Ví dụ đồng thời hỗ trợ chèn và xóa nhanh:

- Hàm đa thức bậc hằng của tổng đoạn, chẳng hạn bình phương của tổng đoạn.
- Số lượng phần tử khác nhau trong đoạn.

Những thông tin đoạn hỗ trợ chèn, xóa nhanh đều có thể xử lý bằng thuật toán Mo. Vì thuật toán Mo không phải trọng tâm thảo luận của bài này, ở đây không mở rộng.

5.4. Thuật toán 2: chia khối

Vì bản thân bao lồi không hỗ trợ hợp nhất nhanh và xóa nhanh, còn bài này lại có thao tác xóa và truy vấn đoạn, nên các cấu trúc dữ liệu đoạn truyền thống kiểu chia để trị như cây đoạn không thể duy trì trực tiếp. Ta xét chia khối.

Chia dãy thành S khối, mỗi khối có kích thước xấp xỉ n/S , và với mỗi khối duy trì bao lồi của các phần tử trong khối. Khi chèn và xóa, xây dựng lại bao lồi trên của khối cuối, nên độ phức tạp chèn và xóa đều là

$$O\left(\frac{n}{S}\right).$$

Với truy vấn đoạn $[l, r]$, ta cần liệt kê các khối nằm hoàn toàn trong đoạn truy vấn và nhị phân trên bao lỗi của chúng, độ phức tạp

$$O(S \log n).$$

Vì vậy tổng độ phức tạp là

$$O\left(\frac{n^2}{S} + nS \log n\right).$$

Lấy $S = \sqrt{n / \log n}$ thì thời gian nhỏ nhất là

$$O\left(n\sqrt{n \log n}\right).$$

5.5. Thuật toán 3: phân nhóm nhị phân

Tính chất đặc biệt của điểm kiểm tra 4: không có thao tác loại 2, và truy vấn loại 3 luôn hỏi toàn bộ đoạn.

Trong điểm này, ta chỉ cần duy trì bao lỗi của toàn bộ phần tử, đồng thời chỉ cần hỗ trợ thao tác chèn.

Bài toán này tương đối kinh điển và có thể dùng cây cân bằng duy trì bao lỗi. Ở đây bài viết giới thiệu một cách không truyền thống: dùng phân nhóm nhị phân.

Ta chia dãy thành một số nhóm liên tiếp, mỗi nhóm duy trì bao lỗi của nó. Ban đầu dãy rỗng. Mỗi khi chèn một phần tử mới, tách riêng phần tử này thành một nhóm; hiển nhiên bao lỗi của nhóm đó chính là phần tử này. Nếu hai nhóm cuối có kích thước bằng nhau, hợp nhất hai nhóm cuối thành một nhóm.

Dễ chứng minh cách phân nhóm này khiến kích thước mỗi nhóm đều là một lũy thừa của 2, và kích thước các nhóm giảm nghiêm ngặt, nên tại mọi thời điểm số nhóm không vượt quá $O(\log n)$.

Xét việc hợp nhất hai nhóm có kích thước bằng nhau, giả sử kích thước là m . Vì bao lỗi của hợp hai tập điểm chắc chắn nằm trong hợp hai bao lỗi, ta có thể trực tiếp trộn hai bao lỗi theo khóa là tọa độ x , rồi dùng thuật toán Graham tính lại bao lỗi trong $O(m)$.

Phân tích độ phức tạp của quá trình hợp nhất: xét đóng góp của mỗi phần tử. Mỗi khi một phần tử bị hợp nhất một lần, kích thước nhóm chứa nó chắc chắn tăng gấp đôi. Vì thế đóng góp của mỗi phần tử cho độ phức tạp không vượt quá $O(\log n)$, nên độ phức tạp là $O(n \log n)$.

Chú ý thao tác truy vấn phải nhị phân một lần trên bao lỗi của từng nhóm, nên độ phức tạp là $O(n \log^2 n)$. Do đó tổng độ phức tạp của thuật toán này là $O(n \log^2 n)$.

5.6. Thuật toán 4: đảo ngược thời gian

Tính chất đặc biệt của điểm kiểm tra 5: mọi thao tác loại 2 đều nằm sau thao tác loại 1, và truy vấn loại 3 luôn hỏi toàn bộ đoạn.

Phân nhóm nhị phân chỉ liên quan tới tạo mới và hợp nhất, không xử lý được xóa trực tiếp. Quan sát tính chất của kiểu dữ liệu này, mọi thao tác xóa đều nằm sau thao tác chèn; điều đó gợi ý ta chia dãy thao tác thành hai phần.

Với nửa trước, dùng cách phân nhóm nhị phân của điểm kiểm tra 4. Với nửa sau, ta đảo ngược thời gian; khi đó thao tác xóa biến thành thao tác chèn, vẫn có thể áp dụng phân nhóm nhị phân.

Độ phức tạp thời gian là $O(n \log^2 n)$. Ý tưởng chia dãy thành hai phần gợi ý ta suy nghĩ theo hướng chia để trị.

5.7. Thuật toán 5: cây đoạn duy trì dãy tĩnh

Tính chất đặc biệt của điểm kiểm tra 6: mọi thao tác loại 3 đều nằm sau các thao tác loại 1 và loại 2.

Sau khi xử lý xong mọi thao tác chèn, xóa, các truy vấn nằm trên một dãy tĩnh S . Với truy vấn đoạn trên dãy tĩnh, ta có thể dùng cấu trúc dữ liệu đoạn kinh điển là cây đoạn.

Với mỗi đoạn $[a, b]$ trên cây đoạn, ta duy trì bao lồi của mọi phần tử trong đoạn đó. Khi xây cây, có thể trực tiếp dùng cách hợp nhất bao lồi tuyến tính đã nêu trong điểm kiểm tra 4. Phần này có độ phức tạp

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) = O(n \log n).$$

Tiếp theo xét truy vấn một đoạn $[l, r]$. Một đoạn có thể biểu diễn thành hợp của $O(\log n)$ đoạn ứng với các nút cây đoạn; ta chỉ cần nhị phân trên bao lồi lưu ở các nút này. Độ phức tạp thời gian là $O(n \log^2 n)$.

5.8. Thuật toán 6: chia để trị trên cây thao tác

Tính chất đặc biệt của điểm kiểm tra 7: với mọi thao tác loại 3, $l = 1$, và giới hạn bộ nhớ là 128 MB.

5.8.1. Cách dùng cấu trúc dữ liệu. Bài không yêu cầu bắt buộc trực tuyến. Vì vậy với một truy vấn $[l, r]$, chắc chắn tồn tại một thời điểm mà độ dài dãy vừa đúng bằng r và dãy khi đó giống hệt dãy tại thời điểm truy vấn. Ta ghi truy vấn vào thời điểm đó. Lại vì $l = 1$, việc này tương đương với mỗi lần truy vấn toàn bộ dãy; ta chỉ cần duy trì bao lồi của mọi phần tử.

Xét dùng Splay để duy trì. Chú ý khi chèn một điểm, các điểm bị thay khỏi bao lồi cũ chắc chắn tạo thành một đoạn trên bao lồi cũ. Nếu ghi lại hệ số góc trên các nút, ta có thể nhị phân hai đầu của đoạn này, rồi tách đoạn đó khỏi Splay và lưu lại. Khi xóa phần tử cuối, ta hợp nhất lại đoạn đã tách ra.

Trong hình minh họa của nguồn, sau khi thêm điểm H , ba điểm B, C, D bị thay khỏi bao lồi và được lưu lại. Vì đã ghi hệ số góc, truy vấn có thể nhị phân trực tiếp trên Splay; do đó độ phức tạp của mọi thao tác đều là $O(\log n)$, tổng cộng $O(n \log n)$.

5.8.2. Cách làm trên dãy. Chèn và xóa có cấu trúc hơi khó xét, nên trước tiên xét trường hợp dãy tĩnh, trong đó mỗi truy vấn hỏi một tiền tố $[1, p]$.

Ta có thể dùng chia để trị. Xét xử lý một đoạn $[l, r]$, đặt $mid = (l + r)/2$. Lấy ra mọi truy vấn có $p \geq mid$; các truy vấn này đều phủ đoạn $[l, mid]$. Ta xử lý một lượt đóng góp của các phần tử trong đoạn $[l, mid]$ đối với các truy vấn có p nằm trong $[mid, r]$. Sau khi xử lý xong đóng góp, dựa vào $p < mid$ hay $p \geq mid$ để chia truy vấn thành hai phần và đệ quy sang trái, phải. Khi chia để trị tới đoạn chỉ còn một phần tử, dùng trực tiếp phần tử đó cập nhật mọi truy vấn tại điểm này.

Tiếp theo xét cách nhanh chóng xử lý đóng góp của các phần tử trong $[l, mid]$ đối với các truy vấn trong $[mid, r]$.

Cách A. Ta dựng bao lồi cho các phần tử trong đoạn $[l, mid]$, rồi liệt kê từng truy vấn bên phải và nhị phân trên bao lồi đã dựng ở bên trái. Một lần tính đóng góp có độ phức tạp $O(n \log n)$.

Cách B. Tương tự, trước hết dựng bao lồi cho các phần tử trong đoạn $[l, mid]$. Sau đó sắp xếp các truy vấn trong đoạn $[mid, r]$ theo thứ tự góc cực. Khi xử lý truy vấn từ trước ra sau, điểm tối ưu trên bao lồi di chuyển đơn điệu. Vì vậy chỉ cần đặt hai con trỏ i, j lần lượt quét trên dãy truy vấn và trên bao lồi. Giả

mã như sau:

```

for  $i = 0$  to  $|Q| - 1$  do
  while  $j + 1 < |H|$  and  $H_{j+1} \cdot Q_i > H_j \cdot Q_i$  do
     $j = j + 1$ 
  end while
   $\text{ans}_i = \max(\text{ans}_i, H_j \cdot Q_i)$ 
end for.

```

Một lần quét có độ phức tạp $O(n)$. Vì cần sắp xếp, độ phức tạp thực tế là $O(n \log n)$. Chú ý việc sắp xếp có thể hoàn thành đồng thời với chia để trị bằng sắp xếp trộn, nên tổng độ phức tạp là

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) = O(n \log n).$$

Trên đây tính, thuật toán chia để trị không ưu thế hơn cách dùng cây cân bằng duy trì bao lỗi, nhưng nó có khả năng mở rộng tốt hơn.

5.8.3. Mở rộng lên cây thao tác. Quan sát cách dùng cấu trúc dữ liệu, ta thấy nó rất giống quá trình DFS trên một cây. Vì bài toán có thể làm ngoại tuyến, ta xây hẳn cây thao tác. Cách xây như sau:

1. Ban đầu tạo một nút gốc $root$, đặt con trở $p = root$.
2. Với thao tác chèn, tạo nút mới x , đặt $\text{father}_x = p$, rồi đặt $p = x$.
3. Với thao tác xóa, đặt $p = \text{father}_p$.

Vì chỉ tạo nút khi chèn, số nút của cây thao tác dựng được không vượt quá số thao tác chèn. Tiếp đó ta nhận ra đoạn truy vấn chính là một chuỗi đi từ trên xuống dưới trên cây. Trong điểm kiểm tra này, đầu trên của chuỗi là $root$; ta ghi truy vấn cho chuỗi $root \rightarrow p$ tại nút p .

Ta xét mở rộng thuật toán chia để trị. Trên cây thao tác, có thể dùng trực tiếp chia để trị theo trọng tâm. Để tiện, trong chia để trị theo trọng tâm, với một khối liên thông trên cây, định nghĩa nút có độ sâu nhỏ nhất trong khối làm gốc của khối, ký hiệu là R , và trọng tâm ký hiệu là G .

Tại mỗi tầng của cấu trúc chia để trị, ta xử lý một lượt đóng góp của các phần tử trên chuỗi $R \rightarrow G$ đối với mọi truy vấn trong cây con của G . Việc này có thể áp dụng trực tiếp Cách A hoặc Cách B ở trên. Vì cấu trúc chia để trị trên cây không dễ dùng sắp xếp trộn để tối ưu, nên một lần tính đóng góp theo Cách A hoặc Cách B đều có độ phức tạp $O(n \log n)$.

Do trọng tâm chia cây thành ít nhất hai khối liên thông, mỗi khối có kích thước không quá $n/2$, tổng độ phức tạp là

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n \log n) = O(n \log^2 n).$$

5.9. Thuật toán 7: nghiệm đầy đủ A

Qua việc suy nghĩ trường hợp $l = 1$, tức điểm kiểm tra 7, ta có được một công cụ mạnh: chia để trị trên cây. Bây giờ xét bài toán gốc.

Trước hết áp dụng thuật toán 6 để dựng cây thao tác, rồi chia để trị theo trọng tâm trên cây thao tác. Điểm khác duy nhất nằm ở việc tính đóng góp của các phần tử trên chuỗi $R \rightarrow G$ đối với mọi truy vấn trong cây con của G . Vì điều kiện $l = 1$ không còn thỏa mãn, đoạn truy vấn trong cây con của G không nhất thiết chứa trọn chuỗi $R \rightarrow G$, mà có thể là một hậu tố của chuỗi này.

Tách riêng vấn đề này ra sẽ thấy nó tương đương với bài toán truy vấn tiền tố trên dãy ở Mục 5.8.2. Do đó ta lấy chuỗi $R \rightarrow G$, đảo ngược nó, thêm các truy vấn vào rồi dùng cách ở Mục 5.8.2. Nếu chọn Cách B có tối ưu sắp xếp trộn, mỗi lần tính có độ phức tạp $O(n \log n)$.

Vì vậy độ phức tạp thời gian của thuật toán là $O(n \log^2 n)$. Chia để trị trên cây và sắp xếp trộn đều có độ phức tạp không gian $O(n)$, nên độ phức tạp không gian là $O(n)$. Thuật toán này có thể đạt 100 điểm.

5.10. Thuật toán 8: nghiệm đầy đủ B

Quan sát bản chất bài toán: sau khi các thao tác chèn, xóa được dựng thành một cây thao tác, đoạn truy vấn thực chất là một đường đi từ trên xuống dưới trên cây.

Ta vẫn xét chia để trị theo trọng tâm, mỗi lần xử lý mọi truy vấn đi qua trọng tâm G . Trước hết tách một truy vấn $u \rightarrow v$ đi qua G thành hai chuỗi truy vấn $G \rightarrow u$ và $G \rightarrow v$. Quan sát bài toán này, nếu lấy G làm gốc thì nó hoàn toàn giống trường hợp ở điểm kiểm tra 7. Dùng trực tiếp cách cấu trúc dữ liệu trong Mục 5.8.1 có thể nhận được thuật toán $O(n \log n)$ cho một tầng.

Do đó tổng độ phức tạp là $O(n \log^2 n)$. Chia để trị trên cây và Splay đều có độ phức tạp không gian $O(n)$, nên không gian cũng là $O(n)$. Thuật toán này có thể đạt 100 điểm.

5.11. Thuật toán 9: nghiệm đầy đủ C

Phân chia để trị trên cây giống thuật toán 7. Khi tính đóng góp của các phần tử trên chuỗi $R \rightarrow G$ đối với mọi truy vấn trong cây con của G , ta dùng cấu trúc dữ liệu để duy trì. Bắt đầu từ G , đi ngược lên R và lần lượt thêm các nút, dùng cấu trúc dữ liệu duy trì bao lồi của các nút đã thêm. Với mỗi truy vấn hậu tố, khi thêm tới vị trí tương ứng, ta nhị phân trên cấu trúc dữ liệu để lấy đáp án.

Vì cấu trúc dữ liệu ở đây chỉ cần hỗ trợ thêm điểm, mà khi thêm điểm có thể duy trì bao lồi bằng bao lực với bảo đảm độ phức tạp, nên phần lớn cây cân bằng hoặc trực tiếp set đều làm được.

Độ phức tạp thời gian là $O(n \log^2 n)$, đủ đạt 100 điểm.

5.12. Thuật toán 10: phân rã chuỗi nặng nhẹ

Trong Mục 5.8.3, ta đã chuyển đoạn truy vấn thành chuỗi trên cây. Đa số bài toán chuỗi trên cây, ngoài chia để trị theo trọng tâm, còn có thể giải bằng phân rã chuỗi nặng nhẹ. Bản thân thuật toán phân rã chuỗi nặng nhẹ đã khá phổ biến, ở đây không nhắc lại.

Sau khi phân rã chuỗi nặng nhẹ, một chuỗi trên cây bị tách thành $O(\log n)$ đoạn thứ tự DFS. Quan sát tính chất của các đoạn này, ngoài đoạn có độ sâu nhỏ nhất, mọi đoạn còn lại đều là tiền tố của một chuỗi nặng nào đó.

Ta xây cây đoạn theo thứ tự DFS như thuật toán 5, rồi dùng cây đoạn truy vấn cập nhật các đoạn không phải tiền tố chuỗi nặng. Tiếp theo xét mọi chuỗi nặng: với mỗi chuỗi nặng, chèn nút từ trên xuống dưới và dùng cấu trúc dữ liệu duy trì.

Tương tự, chỉ cần hỗ trợ chèn, nên phần lớn cây cân bằng hoặc set đều có thể hoàn thành.

Xét độ phức tạp thời gian: xây cây đoạn là $O(n \log n)$; với mỗi truy vấn, phần không phải tiền tố chuỗi nặng được hỏi trên cây đoạn trong $O(\log^2 n)$; quét từng chuỗi nặng và duy trì bao lồi bằng Splay có độ phức tạp $O(n \log n)$. Vì mỗi truy vấn bị tách thành $O(\log n)$ tiền tố chuỗi nặng, và mỗi tiền tố cần một lần nhị phân, tổng độ phức tạp là

$$O(n \log^2 n).$$

Do cần xây cây đoạn và các bao lồi, độ phức tạp không gian của thuật toán là $O(n \log n)$, nên có thể đạt khoảng 90 điểm.

5.13. Thuật toán 11: chia để trị bằng cây đoạn

Tiếp tục xét thuật toán 10. Một truy vấn bị tách thành $O(\log n)$ đoạn thứ tự DFS. Với cập nhật hoặc truy vấn đoạn trên dãy, ta có thể xử lý ngoại tuyến bằng cây đoạn; phương pháp này gọi là chia để trị bằng cây đoạn. Trước hết xét một ví dụ.

5.13.1. Ví dụ: Vector. Cần duy trì một tập vector, hỗ trợ các thao tác:

1. Thêm một vector (x, y) .
2. Xóa vector được thêm ở lần thứ i .
3. Hỏi giá trị lớn nhất của tích vô hướng giữa tập hiện tại và (x, y) . Nếu tập hiện tại rỗng thì xuất 0.

Cho phép làm ngoại tuyến. Với 100% dữ liệu, $n \leq 200000$ và $1 \leq x, y \leq 10^9$. Nguồn bài: đề mô phỏng NOI 2015 của HEB.

Với mỗi vector, ta có thể tiền xử lý thời điểm nó được thêm vào tập và thời điểm nó bị xóa khỏi tập. Như vậy mỗi vector tương ứng một đoạn trên trục thời gian.

Xét xây cây đoạn trên trục thời gian. Đưa một vector vào cây đoạn có thể phân tách thành $O(\log n)$ nút trên cây đoạn.

Tiếp đó, với mỗi nút, dựng bao lồi của các vector thuộc nút ấy. Với mỗi truy vấn, chỉ cần hỏi từ gốc xuống dưới qua các đoạn chứa nó. Độ phức tạp là $O(n \log^2 n)$.

Có thể tối ưu thêm: sắp xếp truy vấn theo góc cực, rồi dùng cách tuyến tính ở Mục 5.8.2 để lấy đáp án. Khi đó nút thất nằm ở sắp xếp.

Thực ra, nếu đưa mọi vector vào theo thứ tự tọa độ x , và đưa mọi truy vấn vào theo thứ tự góc cực, thì tại các nút cây đoạn không cần sắp xếp nữa. Tổng độ phức tạp trở thành

$$O(n \log n + Q \log Q + Q \log n).$$

5.13.2. Chia để trị bằng cây đoạn trong bài này. Chú ý bài *Vector* khác bài này ở chỗ: truy vấn trong *Vector* là một điểm thời gian, còn các vector trong tập phủ các đoạn; trong bài này, ta truy vấn thông tin đoạn của một dãy vector.

Ta vẫn có thể dùng cách chia để trị bằng cây đoạn. Trước hết đưa các truy vấn vào cây đoạn theo thứ tự góc cực, rồi DFS trên cây đoạn. Với mỗi nút cây đoạn, dùng hợp nhất để tính bao lồi của đoạn mà nút đại diện; tiếp đó vẫn dùng thuật toán ở Mục 5.8.2 để cập nhật đáp án.

Độ phức tạp như vậy là $O(n \log n + Q \log Q)$. Vì ta dùng phân rã chuỗi nặng nhẹ tách một truy vấn thành $O(\log n)$ truy vấn nhỏ, nên $Q = O(n \log n)$, tổng độ phức tạp là $O(n \log^2 n)$.

Do phải lưu $O(n \log^2 n)$ truy vấn trên cây đoạn, độ phức tạp không gian là $O(n \log^2 n)$. Cách này có thể đạt khoảng 70–90 điểm.

5.14. Thuật toán 12: xét lại phân nhóm nhị phân

Các thuật toán 6–11 đều yêu cầu cho phép làm ngoại tuyến. Vậy có cách nào hỗ trợ trực tuyến hay không?

Trong thuật toán 3, ta đã đưa vào một cách xử lý chèn trực tuyến: phân nhóm nhị phân. Bây giờ xét cải tiến thuật toán này.

Ta xây $t + 1$ tầng nhóm, trong đó t là số nguyên nhỏ nhất thỏa $2^t > n$, với n là độ dài đoạn hiện tại. Kích thước mỗi nhóm ở tầng i ($0 \leq i \leq t$) là 2^i , và số nhóm ở tầng đó là $\lceil n/2^i \rceil$. Chú ý mỗi nhóm ở tầng i được chia đều thành hai đoạn cùng độ dài ở tầng $i - 1$, nên đây là một cấu trúc cây đoạn; vì vậy có thể định nghĩa quan hệ con trái, con phải và cha như trên cây đoạn.

Nếu chỉ có chèn thì cách làm vẫn như cũ. Trước hết thêm một nhóm ở tầng 0. Sau đó bắt đầu từ tầng $i = 1$; nếu tầng i có thể tăng thêm một nhóm, thì hợp nhất hai nhóm con trái, phải để nhận được nhóm mới được thêm.

Thao tác xóa khó xử lý hơn, vì xóa một phần tử có thể khiến thông tin nhóm cuối của vài tầng đầu trở nên sai. Nếu mỗi lần đều xây lại bạo lực thì độ phức tạp là $O(n)$, không chấp nhận được. Ta đưa vào tư tưởng của cây scapegoat: khi thông tin của một nhóm bị sai, ta không xây lại ngay mà đánh dấu nhóm đó. Khi truy vấn tới nhóm này, ta đệ quy xuống hai con của nó, cho tới khi gặp nhóm chưa bị đánh dấu thì mới nhị phân trong nhóm để tính đáp án. Tuy nhiên cũng không thể mãi không cập nhật, nếu không khi truy vấn, số nút đệ quy xuống tận đáy có thể đạt $O(n)$, cũng không chấp nhận được. Vì vậy xét cơ chế cập nhật sau:

ở mỗi tầng, nhiều nhất chỉ cho phép nhóm cuối cùng bị đánh dấu.

Như vậy, khi tầng i cần tăng thêm một nhóm, ta mới cập nhật bạo lực nhóm cuối cũ; khi xóa, chỉ cần đánh dấu từ dưới lên trên.

Tiếp theo tính độ phức tạp của thuật toán này.

5.14.1. Độ phức tạp của tái xây dựng. Với tầng i , ta thống kê hai loại sự kiện:

1. Nhóm cuối bị đánh dấu, độ phức tạp $O(1)$.
2. Tái xây dựng nhóm cuối và thêm một nhóm mới có đánh dấu, độ phức tạp $O(2^i)$.

Gọi len là tổng độ dài của các nhóm không bị đánh dấu ở tầng i , và đặt $B_i = n - len$. Khi sự kiện 1 xảy ra, B_i tăng từ 0 thành 2^i ; khi sự kiện 2 xảy ra, B_i giảm từ 2^{i+1} xuống 2^i . Một thao tác chèn hoặc xóa chỉ làm B_i thay đổi nhiều nhất 1.

Xét hàm thế năng

$$\Phi(i) = |B_i - 2^i|.$$

Một thao tác chèn hoặc xóa đóng góp nhiều nhất 1 vào $\Phi(i)$, còn một sự kiện làm giảm thế năng 2^i . Vì vậy giữa hai lần sự kiện phải có ít nhất 2^i thao tác, nên độ phức tạp bình quân là $O(n)$ trên toàn bộ dãy thao tác của tầng đang xét.

Vì có t tầng, tổng độ phức tạp tái xây dựng là $O(tn)$.

5.14.2. Độ phức tạp truy vấn. Như đã nói, cấu trúc này là một cây đoạn. Trước hết tách đoạn truy vấn $[l, r]$ thành $O(\log n)$ nhóm. Với mỗi nhóm, có ba trường hợp:

1. Không bị đánh dấu. Khi đó trực tiếp truy vấn nhóm này.
2. Bị đánh dấu nhưng hai con đều không bị đánh dấu. Khi đó truy vấn lần lượt trong hai con, chỉ làm hằng số nhân 2.
3. Bị đánh dấu và con phải cũng bị đánh dấu. Khi đó truy vấn ở con trái rồi đệ quy xuống con phải. Chú ý con trái ở đây chắc chắn là nhóm áp chót của một tầng nào đó; những nhóm như vậy chỉ có $O(\log n)$.

Vì vậy số đoạn thực sự được truy vấn là $O(\log n)$. Do đó độ phức tạp của một thao tác truy vấn là $O(\log^2 n)$. Tổng độ phức tạp là

$$O(tn + n \log^2 n) = O(n \log^2 n).$$

Vì cần duy trì t tầng, độ phức tạp không gian là

$$O(tn) = O(n \log n).$$

Cách này có thể đạt khoảng 90 điểm.

6. Tóm tắt ý tưởng

Bài này tương đối phức tạp; nếu suy nghĩ trực tiếp sẽ khá khó. Có thể chia đại khái thành vài bước.

Trước hết phân tích thông tin cần duy trì. Trong bài này, ta cần duy trì bao lỗi của tập truy vấn, và bao lỗi là ví dụ điển hình của loại thông tin không dễ hợp nhất nhanh.

Tiếp đó xét điểm từng phần và lấy gợi ý từ chúng.

Phần chỉ có chèn và truy vấn toàn cục có thể xử lý trực tuyến bằng phân nhóm nhị phân.

Phần không có sửa đổi có thể dùng cây đoạn.

Phần $l = 1$ là phần khá then chốt, đồng thời khó hơn. Ta xét tình huống trên đây và đưa ra cách chia để trị. Sau đó đưa vào khái niệm cây thao tác và mở rộng thuật toán chia để trị lên cây, tức thuật toán chia để trị theo trọng tâm.

Sau khi có công cụ chia để trị theo trọng tâm, ta thử giải bài gốc. Quan sát điểm khác giữa bài gốc và phần $l = 1$: truy vấn trong cây con không nhất thiết là chuỗi từ gốc tới trọng tâm, mà có thể là một hậu tố của chuỗi này. Tiếp tục quan sát sẽ thấy đây chính là bài toán trên cây của phần $l = 1$, và có thể tiếp tục dùng chia để trị để giải. Như vậy ta thu được một thuật toán tốt với thời gian $O(n \log^2 n)$ và không gian $O(n)$.

Có thể thấy, tuy đề bài có tên “Chúng ta vẫn chưa biết tên cấu trúc dữ liệu đã nhìn thấy hôm ấy”, thuật toán đạt điểm tối đa mà tác giả đưa ra lại không dùng cấu trúc dữ liệu phức tạp, mà dùng hai lần chia để trị để đơn giản hóa bài toán.

Bản chất thật ra là đặt tư tưởng CDQ chia để trị lên cây: chia để trị trên cây trở thành chia để trị theo trọng tâm; quan sát rằng thống kê đóng góp ở mỗi tầng chia để trị là một bài con đặc biệt trên cây và tiếp tục áp dụng chia để trị chính là mấu chốt để giải bài này.

Các thuật toán trên yêu cầu làm ngoại tuyến. Nếu xét phiên bản tăng cường bắt buộc trực tuyến, yêu cầu không gian được nói tới $O(n \log n)$.

Điều này khiến ta nhớ lại phần điểm chỉ có chèn, vốn hỗ trợ bắt buộc trực tuyến. Vì vậy ta cải tiến phân nhóm nhị phân thành cấu trúc cây đoạn và đưa vào tư tưởng cây scapegoat.

Cuối cùng, ta quy định mỗi tầng nhiều nhất chỉ có đoạn cuối bị đánh dấu, và phân tích được độ phức tạp duy trì thông tin là $O(n \log n)$. Do truy vấn không đủ tốt, tổng độ phức tạp là $O(n \log^2 n)$.

Có thể nói đây là thuật toán trực tuyến khá tổng quát cho loại bài này, với độ phức tạp không gian $O(n \log n)$.

Tới đây bài toán đã được giải quyết trọn vẹn, nhưng ý nghĩa của bài không chỉ dừng ở đó. Những tư tưởng và phương pháp dùng trong quá trình giải bài có giá trị thực dụng và khả năng mở rộng rất lớn.

7. Mở rộng đơn giản

7.1. Chèn, xóa ở hai đầu

Các thảo luận ở trên đều giới hạn trong bài toán chèn, xóa phần tử ở cuối. Thực ra có thể mở rộng rất dễ sang bài toán chèn, xóa ở cả hai đầu.

7.1.1. Mở rộng thuật toán 7. Với thuật toán 7, ta xét cải tiến cách xây cây thao tác:

1. Ban đầu tạo một nút gốc *root*, đặt hai con trở $p = q = \text{root}$.
2. Với thao tác chèn ở đầu, tạo nút mới x , đặt $\text{father}_x = p$, rồi đặt $p = x$.
3. Với thao tác chèn ở cuối, tạo nút mới x , đặt $\text{father}_x = q$, rồi đặt $q = x$.

4. Với thao tác xóa ở đầu, đặt $p = \text{father}_p$.
5. Với thao tác xóa ở cuối, đặt $q = \text{father}_q$.

Với mỗi truy vấn đoạn $[l, r]$, chuỗi tương ứng trên cây là chuỗi gồm các nút từ nút thứ l tới nút thứ r trên đường đi $p \rightarrow q$. Tuy nhiên không thể trực tiếp dùng thuật toán 7, vì thuật toán 7 yêu cầu chuỗi truy vấn là một đoạn đi từ trên xuống dưới trên cây, còn cây thao tác sau cải tiến không đảm bảo điểm này.

Do đó cần cải tiến thêm: chỉ cần tách một chuỗi truy vấn $u \rightarrow v$ thành hai chuỗi $w \rightarrow u$ và $w \rightarrow v$, trong đó $w = \text{LCA}(u, v)$.

Tương tự, thuật toán 9 cũng có thể mở rộng lên cây thao tác cải tiến. Thuật toán 8 vốn xử lý các truy vấn đi qua gốc, nên không cần tách truy vấn thành hai chuỗi từ trên xuống; có thể xử lý trực tiếp. Cách mở rộng chi tiết tương tự thuật toán 7, ở đây không trình bày thêm.

7.1.2. Mở rộng thuật toán 12. Phân nhóm nhị phân chỉ hỗ trợ thao tác ở một đầu. Xét phân nhóm nhị phân cải tiến trong thuật toán 12: về bản chất nó là một cấu trúc cây đoạn, nên có thể dùng trực tiếp cây đoạn để thay thế.

Ban đầu xây cây đoạn có độ dài 2^l , và đặt vị trí ban đầu ở chính giữa. Ta sửa cơ chế cập nhật thành: mỗi tầng nhiều nhất cho phép hai nhóm bị đánh dấu, một ở trái và một ở phải. Khi chèn, xóa thì dùng trực tiếp phương pháp xử lý của thuật toán 12. Mọi phân tích độ phức tạp vẫn đúng ở đây; ta chỉ cần tính riêng thể năng bên trái và bên phải. Riêng lúc truy vấn, vì nhóm áp chót của cả hai phía đều có thể bị truy cập, hằng số sẽ nhân thêm 2.

7.2. Duy trì những thông tin đoạn khác không thể hợp nhất nhanh

Trên thực tế, khi thông tin không dễ hợp nhất nhanh, các bài yêu cầu hỗ trợ chèn, xóa ở cuối và truy vấn đoạn phần lớn đều có thể dùng các thuật toán được trình bày trong bài này. Xét một ví dụ.

7.2.1. Ví dụ: cây Ji Li. Cho một cây n đỉnh có trọng số cạnh. Cần duy trì một dãy và hỗ trợ ba thao tác:

1. Chèn một điểm x vào cuối.
2. Xóa điểm cuối.
3. Hỏi trong đoạn $[l, r]$, điểm xa x nhất có khoảng cách tới x là bao nhiêu.

Số thao tác là m , với $m \leq 100000$.

Subtask 1: cho phép làm ngoại tuyến. Subtask 2: bắt buộc trực tuyến. Nguồn bài: Ji Ruyi.

Xét thông tin cần duy trì. Nếu cho trước một cây, làm sao tìm điểm xa x nhất? Có thể dùng quy hoạch động trên cây để tiền xử lý điểm xa nhất cho mỗi đỉnh, độ phức tạp là $O(n)$.

Nếu cho một cây và một tập đỉnh S , cần tìm khoảng cách từ x tới điểm xa nhất trong S , giải thế nào? Vẫn có thể dùng quy hoạch động trên cây để tiền xử lý, độ phức tạp vẫn là $O(n)$. Tuy nhiên khi số truy vấn lớn, độ phức tạp này chưa đủ tốt. Ta xét dùng thuật toán cây ảo. Thuật toán cây ảo đã khá phổ biến trong thi tin học, nên ở đây không mở rộng.

Ta biết xây cây ảo có độ phức tạp $O(|S| \log n)$. Sau đó, trên cây ảo, dùng quy hoạch động trên cây để tiền xử lý với mỗi điểm z trên cây ảo khoảng cách d_z tới điểm xa nhất trong tập. Với mỗi điểm truy vấn x , nhị phân trên thứ tự DFS để tìm hai điểm gần x nhất trên cây ảo, gọi là u, v . Hiển nhiên u, v là hai đầu của một cạnh trên cây ảo. Giả sử u là cha của v , đặt $t = \text{LCA}(v, x)$.

Điểm xa nhất của x nằm ở một trong hai phía, u hoặc v . Nếu phía đó là s , đáp án có dạng

$$\text{dist}(x, t) + d_s - \text{dist}(s, t).$$

Vì vậy

$$\begin{aligned} \text{ans}(x) = \max(\text{dist}(x, t) + d_u - \text{dist}(u, t), \\ \text{dist}(x, t) + d_v - \text{dist}(v, t)). \end{aligned}$$

Hình nguồn minh họa trường hợp $d_v = \text{dist}(v, w)$, khi đó

$$\text{ans}(x) = \text{dist}(x, w) = \text{dist}(x, t) + d_v - \text{dist}(v, t).$$

Lời giải Subtask 1. Trước hết dựng cây thao tác để biến đoạn truy vấn thành một chuỗi trên cây.

Nếu dùng thuật toán 7, tức chia để trị theo trọng tâm lồng chia để trị, vấn đề cần giải là hợp nhất thông tin đoạn.

Việc dựng cây ảo dựa trên thứ tự DFS. Vì vậy chỉ cần duy trì kết quả sắp xếp các phần tử trong đoạn theo thứ tự DFS. Hợp nhất hai mảng có thứ tự là $O(n)$. Do nút thất khi dựng cây ảo là tìm LCA, mà LCA có thể được tối ưu bằng dãy Euler cộng RMQ để truy vấn $O(1)$, tổng độ phức tạp cho một lần hợp nhất là $O(n)$.

Như vậy có thể giải bài con này với độ phức tạp $O(n \log^2 n)$.

Xét cách khác, chẳng hạn thuật toán 8; khi đó cần giải thao tác chèn phần tử x .

Ta dùng cây cân bằng để duy trì các nút cây ảo theo thứ tự DFS. Khi chèn phần tử, nhị phân trong cây cân bằng để tìm cạnh gần x nhất trên cây ảo, từ đó tìm điểm y gần x nhất trên cạnh cây ảo. Đưa cả x và y vào tập. Như vậy mỗi lần chèn chỉ cần thêm nhiều nhất hai nút vào cây cân bằng, số lần sửa cạnh cây ảo cũng là $O(1)$, độ phức tạp $O(\log n)$. Khi DFS, dùng ngăn xếp để lưu các sửa đổi và hoàn tác.

Lời giải theo thuật toán 9 tương tự thuật toán 8, không nhắc lại.

Lời giải Subtask 2. Thuật toán 12 chỉ cần hợp nhất đoạn, mà trong Mục 7.2.1 thông tin đoạn đã có thể hợp nhất với độ phức tạp $O(n)$. Do đó trực tiếp thay thông tin đoạn mà thuật toán 12 duy trì bằng cây ảo là có thể giải bài này. Phân tích độ phức tạp hoàn toàn giống bài *Unknown*: tái xây dựng cập nhật có độ phức tạp bình quân $O(n \log n)$, truy vấn $O(n \log^2 n)$, đủ giải bài.

8. Ý tưởng ra đề và tổng kết

8.1. Ý tưởng ra đề *Unknown*

Những năm gần đây xuất hiện rất nhiều bài yêu cầu duy trì thông tin đoạn. Loại bài này đa số dùng cấu trúc dữ liệu đoạn để duy trì. Trong các thông tin tương đối khó duy trì, bao lồi trên xuất hiện thường xuyên nhất; những bài cần dùng các phương pháp không truyền thống như chia để trị theo thời gian, phân nhóm nhị phân để giải cũng ngày càng nhiều.

Bài thứ ba ngày 2 của NOI 2014, *Gou Piao*, là một bài điển hình cần duy trì bao lồi và hỗ trợ truy vấn đoạn để tối ưu DP. Anh Lu Kaifeng tại hiện trường đã dùng thuật toán chia để trị trên cây để AC bài này, và không lâu sau đó chia sẻ tư tưởng này cho tác giả. Tư tưởng đó gợi mở rất nhiều cho tác giả và khơi dậy hứng thú nghiên cứu các phương pháp duy trì loại thông tin này.

Bài kiểm tra chéo đội tuyển tập huấn mà tác giả nghiên cứu và ra đề, *Chúng ta vẫn chưa biết tên cấu trúc dữ liệu đã nhìn thấy hôm ấy*, là một bài có độ khó cao. Các lời giải điểm từng phần của nó bao phủ nhiều nhánh dùng phương pháp không truyền thống để giải bài cấu trúc dữ liệu truyền thống, như phân nhóm nhị phân, đảo ngược thời gian, chia để trị bằng cây đoạn. Để tìm được lời giải đầy đủ, thí sinh cần

quan sát nhạy bén, phân tích nghiêm cẩn và hiểu sâu, nắm vững thuật toán chia để trị. Đồng thời bài này cũng yêu cầu khả năng cài đặt khá cao; dự kiến thí sinh có thể cần 2–4 giờ mới lấy được toàn bộ điểm.

8.2. Tổng kết phương pháp

Cuối bài, ta tổng kết quy trình suy nghĩ và phương pháp giải chung cho loại bài chèn, xóa ở đầu mút và duy trì thông tin đoạn không dễ hợp nhất nhanh.

Nếu cho phép ngoại tuyến, hãy dựng cây thao tác và xét chia để trị. Nếu có thể hợp nhất đoạn trong $O(n)$, tiếp tục lồng chia để trị như thuật toán 7; nếu không, quan sát thông tin có đồng thời hỗ trợ thêm và xóa phần tử hay không. Nếu có, dùng cấu trúc dữ liệu DFS từ trọng tâm để thống kê như thuật toán 8; nếu chỉ hỗ trợ một trong hai thao tác chèn nhanh hoặc xóa nhanh, có thể quét chuỗi từ gốc tới trọng tâm theo một thứ tự nào đó như thuật toán 9.

Các thuật toán trên đều có độ phức tạp thời gian $O(n \log^2 n)$, độ phức tạp không gian $O(n)$.

Nếu bắt buộc trực tuyến, xét đưa tư tưởng cây scapegoat vào phân nhóm nhị phân như thuật toán 12. Trong phân tích độ phức tạp, nút thất không nằm ở tái xây dựng cập nhật mà nằm ở truy vấn. Vì vậy ta có thể chấp nhận độ phức tạp tái xây dựng $O(n \log n)$, miễn là đảm bảo truy vấn $O(n \log^2 n)$. Với một số bài, ta có thể dùng cách truy vấn tốt hơn, chẳng hạn định vị bằng hợp nhất, để tối ưu truy vấn xuống $O(\log n)$, từ đó giảm tổng độ phức tạp xuống $O(n \log n)$.

Do phải xây cấu trúc phân nhóm $O(\log n)$ tầng, độ phức tạp không gian là $O(n \log n)$.

Unknown chỉ là một đại diện trong rất nhiều bài thuộc loại này. Có nhiều cách giải khác nhau, mỗi cách có ưu điểm riêng và đều có tính thực dụng nhất định. Vì vậy tác giả cho rằng ý nghĩa của việc khơi gợi thí sinh suy nghĩ về các thuật toán này thậm chí còn lớn hơn bản thân đề bài. Hy vọng bài này có thể đóng vai trò gợi mở, khiến mọi người quan tâm hơn tới loại bài này và đề xuất những thuật toán hiệu quả hơn, thuận tiện hơn. Nếu có thu hoạch nghiên cứu trong hướng này, rất hoan nghênh mọi người trao đổi với tác giả.

9. Lời cảm ơn

- Xin cảm ơn thầy Ye Guoping đã chỉ dẫn và quan tâm trong học tập, đời sống.
- Xin cảm ơn CCF đã cung cấp nền tảng và cơ hội giao lưu.
- Xin cảm ơn anh Lu Kaifeng đã cung cấp cảm hứng ra đề.
- Xin cảm ơn các bạn Ji Ruyi, Yang Jiaqi, Wu Zuofan và Mao Xiao đã giúp đỡ.
- Xin cảm ơn các bạn Wu Zuofan, Ji Ruyi, Ni Xingyu và Wang Biyun đã đọc kiểm tra bản thảo.

Tài liệu tham khảo

1. Lời giải bài SDOI 2014 *Vector Set*: <http://www.cnblogs.com/joyouth/p/5350714.html>.
2. Xu Haoran, *Bàn sơ lược về vài cách giải phi kinh điển cho bài cấu trúc dữ liệu*, tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2013.
3. Chen Danqi, *Từ Cash bàn về ứng dụng của một lớp thuật toán chia để trị*.

Bài 9. Báo cáo ra đề mảng hậu tố của Tiểu C

Hong Huadun, Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Bài viết trình bày ý tưởng ra đề và phân tích một bài chuỗi truyền thống trong đợt kiểm tra chéo lần thứ ba của đội tuyển tập huấn năm 2016.

1. Ý chính đề bài

Tiểu C viết một chương trình dùng mảng hậu tố để tìm tiền tố chung dài nhất lcp của hai hậu tố. Quy trình thuật toán của cậu như sau:

1. Đọc vào một chuỗi chữ thường $a[1..n]$ có độ dài n .
2. Định nghĩa mảng $sa[1..n]$. $sa[i]$ biểu thị rằng sau khi sắp xếp tất cả hậu tố, hậu tố nhỏ thứ i là $a[sa[i]..n]$. Cách so sánh hai chuỗi là so từng ký tự; nếu một chuỗi là tiền tố của chuỗi kia thì chuỗi ngắn hơn nhỏ hơn.
3. Định nghĩa mảng $rank[1..n]$, sao cho $rank[sa[i]] = i$.
4. Với truy vấn l, r , đặt $x = \min(rank[l], rank[r])$ và $y = \max(rank[l], rank[r])$.
5. Với mọi i thỏa $x \leq i < y$, tính $\text{lcp}(a[sa[i]..n], a[sa[i+1]..n])$; giá trị nhỏ nhất là đáp án.

Lưu ý rằng lcp ở bước 5 được tính bằng một thuật toán vét cạn, không liên quan tới cách tính mảng *height* trong mảng hậu tố truyền thống; không cần quan tâm nó được cài đặt ra sao.

Ban đầu đây là một thuật toán khá hoàn chỉnh, nhưng có một cậu nhóc nghịch ngợm chèn thêm một bước giữa bước 2 và bước 3: xáo trộn ngẫu nhiên mảng sa , tức sa trở thành một hoán vị sinh ngẫu nhiên.

Bây giờ, với chuỗi và các truy vấn đã cho, Tiểu C muốn biết kỳ vọng đầu ra của thuật toán sau khi bị xáo trộn là bao nhiêu. Để tránh sai số số thực, cần nhân đáp án với $n!$ rồi lấy modulo 998244353 để xuất ra.

2. Định dạng vào

Dòng đầu gồm hai số nguyên n, Q , lần lượt là độ dài chuỗi và số truy vấn.

Dòng thứ hai là một chuỗi chữ thường độ dài n .

Tiếp theo là Q dòng, mỗi dòng gồm hai số nguyên dương l, r , bảo đảm $l \neq r$.

3. Định dạng ra

Xuất Q dòng, mỗi dòng một số nguyên biểu thị đáp án.

4. Phạm vi dữ liệu

- Với 10% dữ liệu, $1 \leq n \leq 10, 1 \leq Q \leq 10$.
- Với 15% dữ liệu, $1 \leq n \leq 100, 1 \leq Q \leq 2$.
- Với 15% dữ liệu, $1 \leq n \leq 100, 1 \leq Q \leq 100$.
- Với 10% dữ liệu, $1 \leq n \leq 3 \cdot 10^3, 1 \leq Q \leq 3 \cdot 10^3$.
- Với 20% dữ liệu, $1 \leq n \leq 10^5, Q = 1$.
- Với 30% dữ liệu, $1 \leq n \leq 10^5, 1 \leq Q \leq 10^5$.

Các nhóm dữ liệu trên đôi một không giao nhau.

5. Phân tích tổng thể

5.1. Cơ duyên ra đề

Ra đề là một phần rất quan trọng trong thi OI. Chất lượng đề của một cuộc thi lớn trực tiếp ảnh hưởng tới tính công bằng của cuộc thi đó, vì vậy tìm tòi phương pháp ra đề cũng là việc một tuyển thủ cần làm.

Bài này thử một hướng ra đề mới: với một thuật toán truyền thống quen thuộc rộng rãi, nếu ngẫu nhiên hóa hoàn toàn một bộ phận của nó thì kết quả sẽ ra sao?

Có lần tôi làm một bài trên TopCoder. Bài đó cho một đoạn mã sắp xếp trộn, trong đó hàm so sánh trả ngẫu nhiên 0 hoặc 1, rồi hỏi xác suất xuất hiện của một hoán vị. Tôi đã thử rất nhiều phương pháp mới giải được, và cảm thấy đó là một bài hay.

Vì vậy tôi bắt chước ý tưởng đó, đem ngẫu nhiên hóa các thuật toán quen thuộc trong thi đấu, chẳng hạn cây hậu tố, luồng mạng, cây đoạn, mảng hậu tố. Cuối cùng tôi thấy ngẫu nhiên hóa mảng hậu tố là thú vị nhất và đáng suy nghĩ nhất.

Ở giai đoạn đầu suy nghĩ bài này, tôi thử giải theo cách tính lcp của mảng hậu tố. Sau nhiều ngày thử nghiệm, tôi chỉ làm được độ phức tạp $O(nQ + n \log n)$; đó cũng là phiên bản đầu của bài.

Một ngày, khi đang chuẩn bị viết lời giải chuẩn, trong đầu tôi chợt lóe lên một ý tưởng: nếu khi giải bằng mảng hậu tố ta duy trì cây Descartes liên quan tới các giá trị lcp, thì sao không dùng trực tiếp cây hậu tố?

Sau đó tôi nhận ra nếu dùng cây hậu tố thì không cần kết luận cốt lõi trước đó. Trong cây hậu tố, lcp của nhiều chuỗi có thể biểu diễn trực tiếp bằng LCA, nên chỉ cần liệt kê LCA. Việc tính đáp án cũng chỉ cần vài phép biến đổi bằng phần bù tương đối đơn giản, cách làm khéo hơn nhiều so với thuật toán trước. Lúc ấy tôi rất hứng khởi và ra bài này. Tôi cho rằng một ý tưởng đơn giản và táo bạo như ngẫu nhiên hóa các thuật toán cuối cùng lại cần kết hợp nhiều thuật toán kinh điển để giải quyết, điều đó khá đẹp.

Bài này cũng gợi cho ta một cách nghĩ khi làm bài: nếu dùng một cấu trúc dữ liệu mà gặp nút thắt, hãy thử một cấu trúc dữ liệu tương tự; có thể nó sẽ có tính chất đẹp giúp giải bài.

5.2. Tình hình điểm số

Trong đội tuyển tập huấn, đa số tuyển thủ đã AC bài này.

6. Giới thiệu thuật toán

6.1. Thuật toán 1

Đề bài nói rằng mảng thứ tự bị xáo trộn ngẫu nhiên, nên ta có thể liệt kê $O(n!)$ mảng thứ tự, sau đó tìm vị trí của l, r và tính lcp từng cặp kề nhau. Độ phức tạp thời gian là $O(Q \cdot n! \cdot n^2)$.

Có thể thấy ta tốn rất nhiều thời gian vào việc tính tiền tố chung dài nhất. Ta có thể tiền xử lý lcp cho n^2 cặp hậu tố, từ đó tối ưu độ phức tạp xuống $O(Q \cdot n! \cdot n)$.

6.2. Thuật toán 2

Với bài này, ta có một suy luận sau. Giả sử thứ hạng từ điển của hậu tố $s[i..n]$ là $\text{rank}[i]$, và đặt $sa[\text{rank}[i]] = i$.

Xét mảng thứ tự, trong đoạn $[l, r]$, gọi hậu tố lớn nhất theo thứ tự từ điển là y , nhỏ nhất là x . Khi đó giá trị nhỏ nhất của các lcp từng cặp chính là tiền tố chung dài nhất của $s[x..n]$ và $s[y..n]$.

Ta chứng minh suy luận này. Ký hiệu $\text{lcp}(x, y)$ là tiền tố chung dài nhất của $s[x..n]$ và $s[y..n]$, đặt

$$h[i] = \text{lcp}(sa[i], sa[i + 1]).$$

Theo thuật toán mảng hậu tố kinh điển, ta có

$$\text{lcp}(x, y) = \text{lcp}(y, x) \quad (\text{rank}[x] > \text{rank}[y]),$$

và

$$\text{lcp}(x, y) = \min(h[\text{rank}[x]], h[\text{rank}[x] + 1], \dots, h[\text{rank}[y] - 1]) \quad (\text{rank}[x] < \text{rank}[y]).$$

Giả sử các hậu tố nằm giữa l và r trong mảng thứ tự lần lượt là $a_1, \dots, a_m$. Với mỗi i thỏa $\text{rank}[x] \leq i < \text{rank}[y]$, chỉ cần chứng minh tồn tại một j sao cho $h[i]$ từng đóng góp vào $\text{lcp}(a_j, a_{j+1})$.

Trước hết tìm $a_j = y$. Trong hai phần tử kề a_{j-1} và a_{j+1} , gọi phần tử có rank nhỏ hơn là p . Khi đó $h[\text{rank}[p]], \dots, h[\text{rank}[y] - 1]$ đều đã xuất hiện. Vì vậy có thể bỏ y đi rồi quy nạp tiếp. Cuối cùng mọi $h[i]$ thỏa điều kiện trên đều chắc chắn đã xuất hiện.

Do đó ta chỉ cần liệt kê hậu tố nhỏ nhất x và hậu tố lớn nhất y , rồi tính số phương án sao cho trong dãy ngẫu nhiên, cực trị từ điển của đoạn $[l, r]$ đúng là x, y .

Ở đây chỉ thảo luận trường hợp l, r, x, y đôi một khác nhau; các trường hợp có phần tử trùng nhau xử lý tương tự. Số phương án là

$$\sum_{k=0}^{\text{rank}[y]-\text{rank}[x]-3} \binom{\text{rank}[y]-\text{rank}[x]-3}{k} (k+2)!(n-k-3)! \cdot 2.$$

Vì vậy, sau khi tiền xử lý số tổ hợp và giai thừa, ta liệt kê x, y rồi tính trực tiếp. Độ phức tạp thời gian là $O(Qn^3 + n^2)$.

6.3. Thuật toán 3

Có thể nhận thấy giá trị của biểu thức

$$\sum_{k=0}^{\text{rank}[y]-\text{rank}[x]-3} \binom{\text{rank}[y]-\text{rank}[x]-3}{k} (k+2)!(n-k-3)! \cdot 2$$

chỉ phụ thuộc vào $\text{rank}[y] - \text{rank}[x] - 3$.

Đặt

$$f[x] = \sum_{k=0}^x \binom{x}{k} (k+2)!(n-k-3)! \cdot 2.$$

Ta có thể dùng biến đổi Fourier nhanh trên miền số nguyên để tính $f[x]$ trong $O(n \log n)$. Cụ thể, đặt

$$A[k] = \frac{(k+2)!(n-k-3)!}{k!}, \quad B[x] = \frac{2}{x!}.$$

Chập A, B sẽ cho

$$c[x] = \frac{f[x]}{x!}.$$

Vì vậy độ phức tạp thời gian giảm xuống $O(Qn^2 + n \log n)$.

6.4. Thuật toán 4

Trong thuật toán 3, cách tính đáp án là cộng dồn

$$\text{lcp}(x, y) \cdot f[\text{rank}[y] - \text{rank}[x] - 3],$$

nên độ phức tạp là $O(Qn^2)$. Ta có thể nhận thấy theo tính chất của mảng hậu tố, các giá trị $\text{lcp}(x, y)$ khác nhau nhiều nhất chỉ có $O(n)$ loại.

Vì vậy bây giờ ta không liệt kê x, y , mà đổi sang liệt kê $\text{lcp}(x, y)$. Cụ thể, liệt kê $\text{lcp}(x, y) = h[i]$, tìm pl, pr sao cho $h[i]$ là giá trị nhỏ nhất trong đoạn $h[pl..pr]$.

Đoạn này có đóng góp khi và chỉ khi

$$\begin{aligned} \text{rank}[x] &\in [pl, pr + 1], \\ \text{rank}[y] &\in [pl, pr + 1], \\ \min(\text{rank}[x], \text{rank}[y]) &\leq i < \max(\text{rank}[x], \text{rank}[y]). \end{aligned}$$

Khi đó miền lấy giá trị của cực tiểu là $[pl, \min(\text{rank}[l], \text{rank}[r])]$, còn miền lấy giá trị của cực đại là $(\max(\text{rank}[l], \text{rank}[r]), pr + 1]$. Bài toán trở thành tính

$$\sum_{i=1}^A \sum_{j=1}^B f[i + j + D],$$

trong đó D là một hằng số.

Đặt

$$f^2[x] = \sum_{i=1}^x f[i], \quad f^3[x] = \sum_{i=1}^x f^2[i].$$

Khi đó

$$\begin{aligned} \sum_{i=1}^A \sum_{j=1}^B f[i + j + D] &= \sum_{i=1}^A (f^2[i + B + D] - f^2[i + D]) \\ &= f^3[A + B + D] - f^3[B + D] - f^3[A + D] + f^3[D]. \end{aligned}$$

Do đó sau tiền xử lý, ta có thể tính số phương án trong $O(1)$. Độ phức tạp thời gian là $O(Qn)$.

6.5. Thuật toán 5

Các thuật toán trước đều xây dựng trên cơ sở mảng hậu tố. Như đã biết, mảng hậu tố là phiên bản giản hóa của cây hậu tố, nên ta có thể xét bài toán trên cây hậu tố.

Trước hết có thể dùng thuật toán hậu tố tự động kinh điển để dựng cây hậu tố. Giả sử trong dãy thứ tự, các hậu tố thuộc đoạn $l..r$ tương ứng với các nút $a_1, a_2, \dots, a_m$ trên cây hậu tố. Trên cây hậu tố, tiền tố chung dài nhất của hai chuỗi chính là độ sâu của tổ tiên chung gần nhất của hai nút tương ứng.

Để tiện trình bày, bên dưới $\text{LCA}(l, r)$ chỉ LCA của các nút đại diện cho hai hậu tố l, r . Nút nông nhất trong $\text{LCA}(a_i, a_{i+1})$ hiển nhiên chính là $\text{LCA}(a_1, \dots, a_m)$.

Vì trong a chắc chắn có các nút tương ứng với hậu tố l, r , ta có thể liệt kê các tổ tiên của $\text{LCA}(l, r)$, rồi tính đóng góp.

Cách tính đóng góp như sau. Với một tổ tiên x , giả sử trong cây con của nó có $\text{size}[x]$ nút hậu tố; giả sử con y của nó cũng là một tổ tiên của $\text{LCA}(l, r)$.

Trước hết hai nút tương ứng với l, r là bắt buộc chọn. Nhưng để x trở thành LCA , ta phải chọn thêm một số điểm. Ở đây có thể dùng bao hàm - loại trừ: chọn tùy ý các điểm khác trong cây con x ; với phương

án không hợp lệ, mọi điểm được chọn đều nằm trong cây con y , nên trừ đi. Vì vậy số phương án là

$$2 \sum_{k=0}^{\text{size}[x]-2} \binom{\text{size}[x]-2}{k} k!(n-k-1)! - 2 \sum_{k=0}^{\text{size}[y]-2} \binom{\text{size}[y]-2}{k} k!(n-k-1)!.$$

Đặt

$$g[x] = 2 \sum_{k=0}^{x-2} \binom{x-2}{k} k!(n-k-1)!.$$

Mảng g có thể được tính bằng biến đổi Fourier nhanh trong $O(n \log n)$. Sau khi tiền xử lý g , ta liệt kê từng tổ tiên và tính trực tiếp theo số nút hậu tố trong cây con. Độ phức tạp thời gian là $O(Qn + n \log n)$.

6.6. Thuật toán 6

Nút thất của các thuật toán trên là mỗi lần đều phải liệt kê mọi tổ tiên và tính đáp án cho từng nút. Khi cây suy biến thành một đường thẳng, độ phức tạp sẽ rất xấu.

Nếu ta có thể khiến đóng góp của một nút chỉ liên quan tới bản thân nó và độc lập với các nút khác, thì chỉ cần lấy tổng trên một đường đi.

Có thể thấy một đóng góp được cấu thành từ hai phần: một nút cha và một nút con. Đóng góp này có thể ánh xạ lên cạnh nối chúng.

Vì vậy khi tính đáp án, chỉ cần tìm LCA, rồi lấy tổng đóng góp trên mọi cạnh từ LCA tới gốc. Ta có thể tiền xử lý trong $O(n \log n)$, sau đó khi truy vấn dùng nhân đôi để tìm LCA. Như vậy bài toán được giải quyết hoàn chỉnh.

Độ phức tạp thời gian là $O(Q \log n + n \log n)$.

6.7. Thuật toán 7

Nút thất của thuật toán trên là cần dùng biến đổi Fourier nhanh để tính hệ số; như mọi người đều biết, hằng số của biến đổi Fourier nhanh khá lớn.

Ta tổng quát hóa bài toán: tương đương với việc các điểm $1..n$ có ba màu 0, 1, 2, trong đó chỉ có hai điểm màu 2, có x điểm màu 0, và có $n - x - 2$ điểm màu 1. Cần đếm số hoán vị sao cho giữa hai điểm màu 2 toàn là điểm màu 0.

Trước hết xếp các điểm màu 1 và màu 2. Hai điểm màu 2 bắt buộc kề nhau, nên buộc chúng thành một khối; số cách xếp là

$$2 \cdot (n - x - 1)!.$$

Sau đó xét chèn các điểm màu 0. Dù chèn thế nào thì giữa hai điểm màu 2 chắc chắn chỉ có điểm màu 0.

Vì vậy trước hết chọn vị trí cho $n - x$ điểm kia, có $\binom{n}{n-x}$ cách, rồi nhân thêm $x!$ cách hoán vị các điểm màu 0.

Bạn Mao Xiao cho tôi một cách hiểu đơn giản hơn đối với công thức này: chỉ xét dãy con gồm các điểm màu 1 và màu 2, hai điểm màu 2 chỉ cần kề nhau. Nói cách khác, điểm màu 2 thứ nhất đặt tùy ý, điểm màu 2 thứ hai có vị trí cố định, nên số cách là

$$\frac{2n!}{n - x + 1}.$$

Do đó ta có thể dùng thuật toán Ukkonen để dựng cây hậu tố; LCA cũng có thể cài đặt $O(1)$ bằng RMQ. Tổng độ phức tạp là $O(n + Q)$.

7. Lời cảm ơn

- Xin cảm ơn Hội Tin học đã cung cấp nền tảng học tập và giao lưu.
- Xin cảm ơn các thầy Chen Heli và Dong Yehua của Trường Trung học số 1 Thiệu Hưng đã nhiều năm quan tâm và chỉ dẫn.
- Xin cảm ơn các huấn luyện viên đội tuyển tập huấn quốc gia Yu Linyun và Chen Xumin đã chỉ dẫn.
- Xin cảm ơn các anh Zhang Hengjie, Wang Jianhao, Ye Yuekai của Đại học Thanh Hoa và bạn Ren Zhizhou của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ tôi.
- Xin cảm ơn bạn Sun Biancan của Trường Trung học số 1 Thiệu Hưng đã đọc kiểm tra bản thảo.
- Xin cảm ơn các bạn học ở Trường Trung học số 1 Thiệu Hưng và bạn Chen Qingyun của Trường Ngoại ngữ Hàng Châu đã kiểm thử bài.
- Xin cảm ơn bạn Mao Xiao của Trường Yali đã trao đổi với tôi về các cách giải khác của bài này.

Tài liệu tham khảo

1. Xie Chao, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa.
2. Liu Rujia, *Kinh điển nhập môn thi lập trình thuật toán*, Nhà xuất bản Đại học Thanh Hoa.
3. Wang Jianhao, *Bàn sơ lược về vài phương pháp khớp chuỗi*, tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015.
4. Zhang Tianyang, *Ô-tô-mát hậu tố và ứng dụng*, tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015.

Bài 10. Báo cáo ra đề *Xiaoxiaokan*

Zhang Haowei, Trường Trung học Dư Diêu, Chiết Giang

1. Đặc điểm của bài nộp đáp án

Những năm gần đây, bài nộp đáp án đã dần đi vào tầm nhìn của mọi người, từ các kỳ thi lớn như CTSC cho tới các kỳ thi nhỏ như ZJOI. Loại bài này biến hóa rất nhiều. Khác biệt lớn nhất của nó so với các dạng bài khác là dữ liệu vào được công khai, hoặc đề chỉ yêu cầu thí sinh xây dựng một nghiệm thỏa mãn điều kiện, chẳng hạn bài *UR7 Shui Ti Chu Ti Ren*.

Đối với người ra đề, ưu điểm rõ ràng của bài nộp đáp án so với bài truyền thống là khả năng phân hóa lớn: người ra đề có thể dựa vào độ tốt của đáp án mà thí sinh xuất ra để đặt nhiều mốc điểm.

Trong quá trình làm bài, do cách chấm đặc biệt, thí sinh không cần nộp chương trình. Trí tuệ con người vì thế có thể tạo ra những hiệu quả bất ngờ.

Xiaoxiaokan là một bài nộp đáp án thuộc loại phân tích dữ liệu. Bài này không chỉ kiểm tra năng lực viết mã và năng lực thuật toán của thí sinh; năng lực phân tích dữ liệu cũng trở nên đặc biệt quan trọng.

Bài viết này dùng bài *Xiaoxiaokan* để giúp người đọc hiểu sâu hơn về dạng bài nộp đáp án dựa trên phân tích dữ liệu.

2. Phân tích đề

2.1. Ý chính đề bài

Cho một hình $n \times m$ gồm các ô vuông có màu khác nhau. Mỗi lần có thể xóa một khối liên thông bốn hướng cùng màu có kích thước lớn hơn 1. Sau khi xóa, các ô phía trên có thể rơi xuống, và các ô bên phải cũng có thể dịch sang trái. Cả số ô bị xóa trong mỗi lần lẫn số ô còn lại cuối cùng đều đem lại điểm tương ứng. Có thể kết thúc trò chơi bất cứ lúc nào; mục tiêu là làm cho điểm số lớn nhất có thể.

2.2. Phân tích sơ bộ

Ta nhận thấy nếu dữ liệu hoàn toàn ngẫu nhiên và không có tính chất gì, thì số trạng thái của bài này hiển nhiên lớn đến mức không thể có lời giải đa thức. Vì vậy ta có lý do để tin rằng mỗi test đều có đặc điểm riêng; dựa vào những đặc điểm ấy, ta có thể thu được một cách làm khá tốt.

Tuy nhiên, ngoại trừ test đầu tiên, lượng dữ liệu của các test còn lại đều rất lớn, không thể trực tiếp quan sát để nắm được đặc điểm của chúng. Do đó ta có thể cân nhắc lấy mẫu để thu được một số thông tin hữu ích.

2.3. Phương pháp phân tích dữ liệu

Với các bài nộp đáp án có dữ liệu vào mang dạng hình học, có vài phương pháp thường gặp sau.

1. Quan sát từng hàng.
2. Quan sát từng cột.
3. Quan sát các hình chữ nhật con.

Riêng với bài này, còn có khái niệm màu; mỗi lần xóa chỉ có thể xóa một khối liên thông cùng màu, và còn có khái niệm điểm số. Vì vậy còn có các phương pháp sau.

4. Quan sát vị trí của các ô cùng màu.
5. Quan sát đặc trưng của từng khối liên thông cùng màu.
6. Quan sát quy luật xấp xỉ của điểm nhận được khi xóa ô và khi còn dư ô.

Thông qua những phương pháp này, ta có thể lần lượt phán đoán đặc điểm của từng test.

2.4. Phân tích sâu

Trước hết, vì đề bài không cung cấp *checker* để kiểm tra đáp án tự xuất có đúng hay không, ta phải tự viết một *checker* để mô phỏng sự biến đổi của các ô sau mỗi lần xóa.

2.4.1. Cách viết checker. Giữa các cột, duy trì một danh sách liên kết hai chiều để xử lý trường hợp một cột bị xóa hết.

Giữa các hàng trong mỗi cột, duy trì một danh sách liên kết hai chiều để xử lý trường hợp ô rơi xuống.

Đồng thời cần ghi lại ô cuối cùng của mỗi cột đang nằm ở hàng nào.

Khi đó, nếu muốn tìm ô ở hàng x , cột y , ta tương đương với việc xuất phát từ góc trái dưới, đi theo liên kết y bước, rồi đi lên $n - x$ bước.

Với mỗi ô, ô phía trên và phía dưới của nó có thể tìm trong $O(1)$. Với ô bên trái và bên phải, cần đi xuống đáy, rồi đi sang trái hoặc phải, rồi đi lên một số bước để tới nơi.

Do mỗi ô nhiều nhất chỉ bị xóa một lần, cách làm này có độ phức tạp thời gian $O(n^2m)$. Dĩ nhiên cũng có thể dùng cây cân bằng để tối ưu, đưa độ phức tạp xuống cỡ $O(nm \log^2(nm))$, nhưng không khuyến khích viết cách đó trong phòng thi.

2.4.2. Test 1, 2, 3, 4, 5. Năm test này có một điểm chung: xóa bao nhiêu ô cũng không nhận được điểm, còn số ô còn lại càng ít thì điểm càng cao.

2.4.3. Test 6, 8, 9. Điểm chung của ba test này là điểm nhận được khi xóa ô tăng theo bậc bình phương, nên xóa các ô cùng màu cùng một lúc luôn là tối ưu.

2.4.4. Test 1. Chỉ cần xem dữ liệu là có thể phát hiện đây chẳng qua là một ma trận 5×5 . Dựa vào thao tác thủ công và một chút “trí tuệ con người”, ta có thể trong vài phút tìm được lời giải để không còn ô nào.

Test này dùng để thí sinh kiểm nghiệm *checker* tự viết có đúng hay không, đồng thời là test tặng điểm.

2.4.5. Test 2. Quan sát ma trận 10×10 ở góc trái trên của dữ liệu, ta có thể phát hiện kích thước của mỗi khối liên thông nhiều nhất là 2. Chỉ cần viết thêm một đoạn mã để kiểm chứng phỏng đoán này, ta sẽ thấy ngoài một bộ rất lớn ở dưới cùng, các phần còn lại đều là khối liên thông kích thước 2, và mỗi màu chỉ có 2 ô. Khi đó ta chỉ cần xóa ô từ trên xuống dưới, từ trái sang phải, sao cho không ô nào rơi xuống và không ô nào dịch sang trái, là có thể xóa hết mọi ô.

Test này kiểm tra năng lực phân tích dữ liệu sơ bộ của thí sinh; có thể thấy tính chất của nó tương đối hiển nhiên.

2.4.6. Test 3. Test này khó quan sát ra quy luật hơn. Thực ra chỉ cần xem ma trận 10×10 ở góc trái trên, ta có thể thấy các cột 1 đến 7, trừ cột 5, đều đối xứng qua trục. Từ đó có thể đoán rằng cột 5 có lẽ chỉ dùng để gây nhiễu.

Quan sát kỹ hơn, ta biết được toàn bộ ma trận có hơn hai mươi cột gây nhiễu. Chỉ cần xóa các cột này, cả ma trận sẽ được chia thành nhiều khối, trong đó mỗi khối đều đối xứng trục.

Dựa vào tính chất đối xứng trục, ta chỉ cần xóa từ giữa ra ngoài, mỗi khối đều có thể bị xóa sạch. Việc còn lại chỉ là mô phỏng quá trình này.

Khi thiết kế test này, tác giả cân nhắc rằng trong các test không cho điểm khi xóa ô cũng nên có một test kiểm tra năng lực phân tích dữ liệu. Điểm khó của test này nằm ở phân tích dữ liệu và phỏng đoán kết luận; ngay cả khi đã biết tính chất của test, thí sinh vẫn cần viết một bộ mô phỏng để xuất được phương án.

2.4.7. Test 4, 5. Hai test này rất dễ thấy là chỉ có 3 hoặc 4 màu khả dĩ. Vì vậy chúng không cần năng lực phân tích dữ liệu cao siêu, mà kiểm tra trình độ vét cạn và thử nghiệm ngẫu nhiên của thí sinh.

Với test 4, mỗi lần ta có thể chọn ngẫu nhiên một khối liên thông để xóa; chỉ cần lặp ngẫu nhiên đủ nhiều lần, cuối cùng sẽ có thể xóa hết.

Với test 5, ta phát hiện xác suất xóa hết theo cách ấy rất thấp. Do đó có thể đặt một tham số x ; khi số ô còn lại không vượt quá x , ta vét cạn mọi phương án xóa, rồi dùng một số cắt tỉa nhất định để tìm ra đáp án.

2.4.8. Test 6. Quan sát mọi màu ngoài màu 1, ta có thể phát hiện mỗi màu đều nằm trên cùng một hàng hoặc cùng một cột, trong đó có thể có vài ô đã biến mất. Điều này rất dễ gợi ý rằng chúng bị các màu khác phủ lên.

Dĩ nhiên ta cũng có thể quan sát từng hàng và từng cột: có một màu xuất hiện nhiều lần, và có nhiều màu xuất hiện một lần. Điều này cũng dẫn đến kết luận trên.

Vì vậy ta chỉ cần lặp vài lượt; mỗi lần tìm một màu có thể phủ trọn một hàng hoặc một cột, rồi xóa nó.

Khi thiết kế test này, mục đích chính là để thí sinh thích ứng với thay đổi từ “xóa ô không có điểm” sang “xóa ô có điểm”.

2.4.9. Test 7. Test này là một ma trận một hàng m cột. Ta nhận thấy gần như mọi ô cùng màu đều kề nhau, nhưng ở giữa có thể tồn tại các điểm nhiễu vụn vặt; trong số các điểm nhiễu ấy chỉ có một điểm là không thể gộp được. Vì vậy, nếu các ô của cùng một màu không nằm trong cùng một khối liên thông, ta có thể mặc nhiên xem chúng là hai màu khác nhau.

Ta phát hiện các ô còn lại cuối cùng cũng có thể đem lại điểm khá tốt. Khi đó bài toán trở thành một bài ba lô 0/1 kinh điển.

Số ô của mỗi màu có thể xem là thể tích, điểm sau khi xóa có thể xem là giá trị, và ta có thể làm DP.

Thông qua mô hình *Xiaoxiaokan*, test này kiểm tra một bài toán kinh điển. Tính chất của nó tương đối hiển nhiên, nên nó chủ yếu kiểm tra độ quen thuộc của thí sinh với bài toán kinh điển.

2.4.10. Test 8. Đặc trưng của test này khó quan sát hơn. Điều dễ thấy là mỗi màu đều nằm trong cùng một khối liên thông, nhưng chỉ quan sát được điều này thì chưa nói lên được nhiều. Nếu ta in ra tọa độ vị trí của từng màu, ta sẽ thấy mỗi màu chỉ có thể tạo thành dạng chữ L , dạng một đường thẳng, hoặc một điểm đơn.

Nếu nhìn ra tính chất này thì test trở nên rất thuận tiện. Ta bỏ qua các điểm đơn, vì cuối cùng chắc chắn chúng không thể bị xóa. Sau đó ta chọn một nhóm ô mà sau khi xóa sẽ không làm khối liên thông bị tách ra; có thể chứng minh rằng luôn tồn tại nhóm ô như vậy.

Nhờ đó ta có thể xóa sạch mọi khối liên thông và đạt điểm cao nhất. Trong test này, điểm cao nhất vẫn là điểm âm.

So với bảy test phía trước, độ khó của test này tăng lên một bậc, nhằm phân biệt thí sinh giỏi về bài nộp đáp án với thí sinh nghiệp dư.

2.4.11. Test 9. Tính chất của test này tương đối dễ quan sát. Quan sát từng cột, ta thấy số hiệu màu đều liên tiếp; còn giữa hai cột khác nhau thì không xuất hiện ô cùng màu. Vì vậy rất dễ phát hiện rằng các cột không ảnh hưởng lẫn nhau, và nếu cuối cùng còn lại x ô thì sẽ mất $x + 1$ điểm.

Như vậy, ta có thể nén một bảng $n \times m$ thành m bài toán con dạng $1 \times n$, rồi làm DP.

Đặt $dp_{i,j}$ là điểm lớn nhất có thể nhận được sau khi xóa hết các ô từ i đến j ; nếu không xóa hết được thì giá trị là âm vô cùng.

Đặt f_i là điểm lớn nhất khi chỉ có các ô từ 1 đến i . Khi đó có một chuyển tiếp hiển nhiên:

$$f_i = \max\{f_{i-1} + 1, f_{j-1} + dp_{j,i}\},$$

trong đó lấy cực đại theo các j hợp lệ.

Vậy ta tính các giá trị dp như thế nào?

Ta có thể chia đoạn thành hai phần:

$$dp_{i,j} = \max_k \{dp_{i,k} + dp_{k+1,j}\}.$$

Ngoài ra, cũng có thể xảy ra trường hợp ô i và ô j có cùng màu. Khi đó ta xóa hết một phần trong đoạn $i..j$, để các ô còn lại đều có màu giống i và j , rồi các ô cùng màu này sẽ tụ lại và có thể xóa cùng nhau. Gọi màu của ô i là c .

Ở đây ta đặt thêm mảng g . Trong đó $g_{i,j}$ biểu diễn điểm lớn nhất khi đang xét tới ô thứ i , cuối cùng có j ô màu c tụ lại với nhau, và ô i nằm trong j ô đó.

Chỉ khi màu của ô i là c mới có chuyển tiếp:

$$g_{i,j} = \max_k \{g_{k,j-1} + dp_{k+1,i-1}\}.$$

Như vậy ta có thể dùng mảng g để thu được mảng dp .

Độ phức tạp thời gian xấu nhất là $O(n^5m)$. Thực tế thường còn xa mới chạm tới mức này, và với bài nộp đáp án thì chỉ cần chạy ra kết quả trong thời gian thi là được.

Test này kiểm tra nền tảng DP và năng lực viết mã của thí sinh. Tính chất của nó tương đối hiển nhiên, nhưng DP này hơi cao hơn độ khó NOIP.

2.4.12. Test 10. Test này cho một ma trận vừa rộng vừa thấp. Nếu ta in ra mọi vị trí của từng màu, ta sẽ phát hiện mỗi màu đều tạo thành một hình chữ nhật rộng.

Thực ra cấu trúc này tạo thành một cây. Xem mỗi hình chữ nhật là một đỉnh; nếu hình chữ nhật i chứa được hình chữ nhật j , thì i là tổ tiên của j . Quá trình dựng cây rất đơn giản nên không trình bày thêm.

Trong điểm nhận được khi xóa ô, ta thấy ngoài một mức điểm là 23333333, các mức còn lại đều bằng 0. Điều này có nghĩa là ta cần cố gắng xóa được một khối có đúng kích thước ấy.

Quan sát đặc trưng của các hình chữ nhật: với hai đỉnh bất kỳ i, j trên cây, nếu i không phải tổ tiên của j , thì xóa hình chữ nhật j sẽ không làm hỏng hình chữ nhật i . Vì vậy, trong quá trình xóa, chỉ cần bắt đầu từ gốc; khi đó xóa một hình chữ nhật tương đương với việc thả ra một số ô màu 1.

Bài toán vì thế được chuyển thành một cây có trọng số ở đỉnh: tìm một khối liên thông chứa gốc sao cho tổng trọng số đỉnh đúng bằng giá trị cho trước. Đây thực chất là một bài DP trên cây.

Test này kiểm tra trình độ tổng hợp của thí sinh: cần có năng lực phân tích dữ liệu để nhìn ra tính chất của test, cần quen thuộc với bài toán kinh điển để nhìn ra mô hình sau khi chuyển hóa, và cần năng lực viết mã cao để xuất phương án. Phải nói rằng thí sinh vượt qua được test này trong phòng thi đã hiểu rất sâu về bài nộp đáp án.

2.4.13. Thử nghiệm ngẫu nhiên. Trên thực tế, khi thí sinh không có thời gian phân tích từng test một, xét tới việc bài này có 10 mức điểm, ta có thể viết một cách làm tổng quát để lấy được lượng điểm lớn.

Cụ thể, vì mọi test của bài này đều không có tình huống xóa ô bị trừ điểm, và ngoài test 7 và test 10 thì số ô còn lại càng ít càng tốt, ta có thể áp dụng nguyên tắc “xóa được thì xóa”, mỗi lần chọn ngẫu nhiên một khối liên thông để xóa.

Chỉ cần mô phỏng quá trình này là có thể đạt khoảng 50 điểm; điều này đòi hỏi thí sinh tích lũy kinh nghiệm khi luyện tập và khi thi.

3. Ước lượng điểm

Với các test khác nhau, thời gian suy nghĩ và thời gian viết mã hơi khác nhau; điểm số nhìn chung tỉ lệ thuận với thời gian làm bài.

Điểm cao nhất nên nằm trong khoảng 80 đến 90, còn điểm trung bình khoảng 60.

4. Tình hình điểm trong kỳ thi chéo đội tuyển tập huấn

Điểm số dao động từ 18 đến 69, trung bình là 44 điểm, hơi thấp hơn ước lượng dự kiến.

5. Tổng kết

Bài nộp đáp án thuộc loại phân tích dữ liệu thường xuất hiện vài tình huống khác nhau. Với bài này, có hai tình huống: “xóa ô có điểm” và “xóa ô không có điểm”. Dựa vào từng tình huống, người ra đề thường sẽ đặt cả test dễ lẫn test khó. Nếu thí sinh nắm được điểm này và không sa lầy vào các test phía trước, chẳng hạn test 3 trong bài này, thì có thể nổi bật lên.

Thực ra phương pháp quan sát dữ liệu của bài này vẫn áp dụng được cho bài *Xiaochu Youxi* của NOI 2014. Với các bài nộp đáp án phân tích dữ liệu khác nhau, cách quan sát dữ liệu có thể hơi khác, nhưng bản chất có thể tóm gọn trong một câu: mạnh dạn phỏng đoán, cẩn thận luận chứng.

6. Lời cảm ơn

- Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Xin cảm ơn tất cả các thầy cô đã hướng dẫn tôi và các bạn học đã giúp đỡ tôi.
- Xin cảm ơn cha mẹ đã quan tâm và chăm sóc tôi.

Bài 11. Báo cáo ra đề *strakf*

Li Zihao, Trường Trung học Shimen, Phạt Sơn

Tóm tắt

Bài viết này chủ yếu trình bày hai bài toán kết hợp chuỗi với cấu trúc dữ liệu, cùng một vài cách chỉnh tham số cho **KD-tree**.

1. Nguồn gốc đề bài

Kỳ thi chéo đội tuyển tập huấn quốc gia năm 2016.

2. Ý chính đề bài

Cho một chuỗi ban đầu S độ dài N , chỉ gồm các chữ cái thường. Sau đó có M thao tác, chia thành ba loại.

1. A : tăng trọng số của mọi xâu con của chuỗi gốc S bằng A thêm 1.
2. $l\ r$: tăng trọng số của mọi xâu con của chuỗi gốc S bằng $S[l, r]$ thêm 1.
3. $a\ b$: hỏi tổng trọng số của mọi xâu con $S[l, r]$ của chuỗi gốc S thỏa mãn $a \leq l \leq r \leq b$.

3. Phạm vi dữ liệu

Test	N, M	Loại thao tác	Đặc điểm dữ liệu
1	5	2, 3	Không có
2	10	2, 3	Không có
3	100	2, 3	Không có
4	$2 \cdot 10^5$	1, 2	Không có
5	$2 \cdot 10^4$	1, 3	Mọi thao tác loại 3 có $a = 1$
6	$2 \cdot 10^4$	1, 3	Mọi thao tác loại 3 có $b = N$
7–8	$2 \cdot 10^4$	1, 3	Độ dài chuỗi trong mọi thao tác loại 1 không giảm
9–10	$6 \cdot 10^3$	2, 3	Không có
11–12	$2 \cdot 10^4$	2, 3	Mọi thao tác loại 3 có $b = N$
13–15	$2 \cdot 10^4$	2, 3	Độ dài chuỗi trong mọi thao tác loại 2 không giảm
16–20	$2 \cdot 10^4$	2, 3	Không có

Kích thước tệp vào không vượt quá 0.3 MB.

4. Điểm kiểm tra

Chuỗi, cấu trúc dữ liệu, phân khối.

5. Nguồn ý tưởng

5.1. Một bài toán

Cho một chuỗi S độ dài N , chỉ gồm chữ cái thường. Ký hiệu $S[l][r]$ là xâu con của S từ l đến r .

Có q truy vấn. Mỗi truy vấn cho một đoạn $[l_i, r_i]$, hỏi giá trị lớn nhất của x thỏa mãn

$$S[l][l+x-1] = S[r-x+1][r].$$

Nói cách khác, cần tìm độ dài lớn nhất của phần đầu đoạn và phần cuối đoạn trùng nhau. Ở đây $N, q \leq 2 \cdot 10^5$.

5.2. Lời giải

Với một đoạn hỏi $[l, r]$, thực chất ta cần tìm x nhỏ nhất sao cho hậu tố bắt đầu tại l và hậu tố bắt đầu tại x có tiền tố chung dài nhất lớn hơn $r - x$.

Vì vậy ta có thể duyệt x tăng dần, rồi xử lý mọi truy vấn ngoại tuyến theo thứ tự. Sắp xếp truy vấn theo đầu trái; khi đang duyệt đến i , ta chèn các truy vấn có đầu trái nhỏ hơn i , rồi xóa tất cả truy vấn có thể nhận đáp án là i . Như vậy vấn đề còn lại là duy trì một cấu trúc có thể tìm ra các truy vấn mà đáp án hiện tại có thể là i .

Theo nhận xét trên, nếu len là độ dài tiền tố chung dài nhất của hai hậu tố liên quan, ta có điều kiện

$$i + len - 1 \geq r, \quad \text{tức là} \quad i \geq r - len + 1.$$

Do đó chỉ cần duy trì giá trị $r - len + 1$, rồi lấy ra theo thứ tự tăng dần.

Để làm việc này, ta dựng mảng hậu tố và mảng height. Từ mảng height, ta dựng một cây biểu diễn các nhóm hậu tố có cùng mức tiền tố chung. Chẳng hạn với chuỗi ababcba, các hậu tố được sắp theo thứ tự từ điển như a, ababcba, abcba, ba, babcba, bcba, cba; hình trong PDF nguồn minh họa cây nhóm những hậu tố này theo giá trị height của chúng.

Khi chèn một truy vấn $[l, r]$, ta tìm nút ứng với l , rồi đi từ nút này lên gốc. Với mỗi nút trên đường đi, chèn một cặp $(r - len, num)$, trong đó len là độ dài xâu dài nhất mà nút ấy có thể biểu diễn.

Khi xử lý các truy vấn có $x = i$, ta lại đi từ nút ứng với x lên gốc. Với mỗi nút trên đường đi, hỏi mọi cặp có khóa nhỏ hơn i , cập nhật đáp án tương ứng, rồi xóa các truy vấn ấy.

Bài toán trên có thể giải bằng phân rã nặng nhẹ và thứ tự DFS trên cây. Có thể chia các truy vấn cần hỗ trợ thành ba loại:

1. hỏi giá trị nhỏ nhất trong một cây con;
2. hỏi giá trị nhỏ nhất trên một chuỗi nặng;
3. hỏi giá trị nhỏ nhất trong các con nhẹ của các nút trên một chuỗi nặng.

Ba loại này lần lượt được xử lý bằng thứ tự DFS của cả cây, phân rã nặng nhẹ, và thứ tự DFS của các con nhẹ. Nhờ đó bài toán có thể giải trong $O((N + q) \log^2 N)$.

5.3. Nhận xét

Bài toán trên kết hợp khá tốt giữa chuỗi và cấu trúc dữ liệu, đồng thời kiểm tra đáng kể năng lực viết mã của thí sinh.

6. Giới thiệu thuật toán

6.1. Test 1–3

Dùng phương pháp vét cạn. Với mỗi thao tác sửa đổi, tăng trọng số của mọi xâu con phù hợp thêm 1. Với mỗi thao tác hỏi, trực tiếp cộng trọng số của mọi xâu con nằm trong phạm vi cần hỏi.

Độ phức tạp thời gian là $O(n^3)$, dự kiến được 15 điểm.

6.2. Test 4

Ở test này không có thao tác loại 3, nên có thể kết thúc trực tiếp. Cộng với phần trước, dự kiến được 20 điểm.

6.3. Test 5–8

Đặc điểm của nhóm test này là chỉ có thao tác sửa đổi loại 1, không có thao tác sửa đổi loại 2.

6.3.1. Đặc điểm dữ liệu. Vì dữ liệu vào được bảo đảm không vượt quá 0.3 MB, tổng độ dài các chuỗi được chèn không vượt quá $3 \cdot 10^5$.

Do đó ta có kết luận: số độ dài khác nhau của các chuỗi được chèn chỉ là $O(\sqrt{N})$.

Thật vậy, nếu mỗi độ dài từ 1 đến $\sqrt{N}$ đều xuất hiện đúng một lần, tổng độ dài đã đạt cấp $O(N)$. Điều phải chứng minh được suy ra từ đó.

6.3.2. Cách giải. Với thao tác hỏi, ta có thể xử lý riêng theo từng độ dài khác nhau. Khi đó bài toán trở thành duy trì số lượng phương án mà đầu trái của từng độ dài nằm trong một đoạn nào đó.

Với một thao tác chèn, các đầu trái mà nó ảnh hưởng có hạng hậu tố nằm trong một đoạn. Vì vậy, với mỗi đầu trái, ta ghi một cặp (a, b) , nghĩa là đầu trái nằm ở vị trí a , còn hạng hậu tố tương ứng là b . Khi

chèn, ta tăng trọng số cho mọi cặp có b nằm trong một đoạn; khi hỏi, ta cần tổng trọng số của mọi cặp có a nằm trong một đoạn.

Ta có thể dùng phân khối để giải bài toán này. Giả sử chia theo thứ tự vị trí, mỗi khối dài L . Với từng khối, ghi tổng trọng số trong khối, sắp các đầu mút theo b , và tiền xử lý để với mỗi r cho trước biết được chỉ số lớn nhất k thỏa mãn $b_k \leq r$. Đồng thời, với các khối tiền tố, ghi tổng trọng số sum_i .

Với thao tác sửa đổi, ta tính trực tiếp đóng góp cho từng khối và cho các khối tiền tố liên quan, rồi cập nhật tổng trọng số trong khối, sum_i , và đầu lười của mỗi khối. Với thao tác hỏi, dùng sum_i đã chuẩn bị để lấy tổng của các khối nguyên vẹn; phần không nguyên khối thì đẩy đầu lười và vét cạn.

Độ phức tạp của thao tác sửa đổi là $O(N/L)$, còn thao tác hỏi là $O(L\sqrt{N})$. Khi $L = N^{0.25}$, độ phức tạp nhỏ nhất là $O(N^{0.75})$. Vì vậy toàn bộ thuật toán có độ phức tạp $O(N^{1.75})$, dự kiến được thêm 20 điểm. Cộng với các phần trước, dự kiến được 40 điểm.

6.4. Test 9–10

Với nhóm dữ liệu này, ta cũng có thể dựng trước mảng hậu tố.

Với mỗi vị trí, duy trì một cấu trúc lưu mọi độ dài sửa đổi lấy vị trí ấy làm đầu trái. Với thao tác sửa đổi, dùng tìm kiếm nhị phân để tìm đoạn hạng hậu tố chứa xâu con này. Với mỗi vị trí tương ứng trong đoạn hạng ấy, chèn độ dài sửa đổi hiện tại vào cấu trúc của vị trí đó.

Với thao tác hỏi, duyệt mọi vị trí trong đoạn thao tác, rồi hỏi trong cấu trúc của vị trí ấy số độ dài sửa đổi không vượt quá một giá trị cho trước. Do đó ta cần một cấu trúc hỗ trợ chèn động và hỏi số phần tử nhỏ hơn một khóa; có thể dùng cây đoạn, cây cân bằng, hoặc cấu trúc thống kê thứ tự trong thư viện `pb_ds`.

Độ phức tạp thời gian là $O(NM \log N)$, dự kiến được thêm 10 điểm. Cộng với các phần trước, dự kiến được 50 điểm.

6.5. Test 11–12

Đặc điểm của nhóm dữ liệu này là đầu phải cố định bằng N , tức không tồn tại tình huống đầu phải vượt biên. Vì vậy ta không cần xử lý riêng theo từng độ dài.

Ta có thể dùng cách tương tự nhóm test 5–8. Với mỗi đầu trái, ghi một cặp (a, b) , nghĩa là đầu trái nằm ở vị trí a , còn hạng hậu tố tương ứng là b . Khi chèn, tăng trọng số cho mọi cặp có b nằm trong một đoạn; khi hỏi, lấy tổng trọng số của mọi cặp có a nằm trong một đoạn.

Ở trên ta dùng phân khối, nhưng tại đây cũng có thể dùng **KD-tree**. Cách này giảm được phần nào lượng mã và cũng cải thiện tốc độ chạy.

Có thể chứng minh rằng, khi mọi điểm có tọa độ đôi một khác nhau, **KD-tree** bảo đảm được độ phức tạp xấu nhất $O(\sqrt{N})$ cho truy vấn trong hình chữ nhật hai chiều. Chứng minh chia làm hai bước.

6.5.1. Một chiều là toàn miền. Trước hết, khá dễ thấy truy vấn với chiều còn lại là một đoạn bất kỳ $[l, r]$ có thể quy về truy vấn dạng $[1, r]$.

Lập luận rất đơn giản: xét nút có trung điểm mid khiến l và r nằm ở hai phía khác nhau. Khi đó truy vấn được tách thành hai truy vấn tương đương với $[l, mid]$ và $[mid + 1, r]$, mà mỗi phần có thể chuyển về dạng tiền tố trong cây con tương ứng.

Với truy vấn $[1, r]$, nếu lớp hiện tại tách theo chiều toàn miền thì hiển nhiên phải đi xuống cả hai phía. Nếu lớp hiện tại tách theo chiều $[1, r]$, thì một phía hoặc hoàn toàn không phủ, hoặc hoàn toàn được phủ; cả hai trường hợp đều xử lý được ngay, nên chỉ còn một nút cần đi tiếp.

Tổng số tầng là $\log_2 N$; chỉ khoảng một nửa số tầng cần duyệt cả hai nhánh. Vì vậy độ phức tạp là $O(2^{\log_2 N/2}) = O(\sqrt{N})$.

6.5.2. Trường hợp tổng quát. Tương tự, ta có thể quy đoạn bất kỳ trên một chiều về dạng $[1, r]$.

Khi lớp hiện tại tách theo chiều này, một phía hoặc hoàn toàn không phủ, hoặc hoàn toàn được phủ. Phần được phủ hoàn toàn trở thành bài toán mà một chiều là toàn miền, nên theo kết quả trên có độ phức tạp $O(\sqrt{N})$. Phía còn lại là bài toán tương đương trên nửa số điểm.

Do đó

$$F(N) = F(N/2) + O(\sqrt{N}),$$

và từ tổng cấp số nhân suy ra $F(N) = O(\sqrt{N})$. Chứng minh hoàn tất.

Nhờ vậy ta có thể giải nhóm này trong $O(M\sqrt{N})$, dự kiến được thêm 10 điểm. Cộng với các phần trước, dự kiến được 60 điểm.

6.6. Test 13–15

Nhóm dữ liệu này bảo đảm độ dài chuỗi được chèn không giảm, gợi ý rằng ta nên xét theo chiều độ dài.

Giả sử đoạn hỏi hiện tại là $[l, r]$. Đặt $[1, l-1]$ là đoạn A , $[l, r]$ là đoạn B , và $[r+1, N]$ là đoạn C . Ký hiệu (X, Y) là tổng trọng số của các xâu có đầu trái nằm trong đoạn X và đầu phải nằm trong đoạn Y . Chẳng hạn, (A, B) nghĩa là đầu trái nằm trong A , đầu phải nằm trong B . Giá trị cần tìm là (B, B) .

Nếu hỏi đoạn $[1, r]$, ta nhận được $(A, A) + (A, B) + (B, B)$. Vì đã có cách xử lý khi một đầu mút nằm ở biên, nếu ta tính được $(A, A) + (A, B)$ thì sẽ thu được (B, B) .

Tiếp theo, nếu hỏi toàn bộ $[1, N]$, ta nhận được

$$\begin{aligned} &(A, A) + (A, B) + (A, C) \\ &+ (B, B) + (B, C) + (C, C). \end{aligned}$$

Mặt khác, $(B, B) + (B, C) + (C, C)$ chính là kết quả hỏi đoạn $[l, N]$. Vì vậy vấn đề duy nhất còn lại là hạng tử (A, C) , nhưng hạng tử này có vẻ không có cách xử lý trực tiếp tốt.

Lúc này ta quay lại gợi ý về độ dài. Ta có điểm đột phá: nếu chỉ xét các xâu có độ dài không vượt quá $r-l+1$, thì không tồn tại hạng tử (A, C) .

Do đó ta duy trì một **KD-tree** khả persistent theo độ dài, trong đó phiên bản ứng với một độ dài chỉ chứa các đáp án có độ dài không vượt quá giá trị ấy. Với thao tác hỏi, chỉ cần tìm kiếm nhị phân phiên bản **KD-tree** ứng với độ dài lớn nhất hợp lệ, rồi hỏi trên cây đó.

Độ phức tạp thời gian là $O(M\sqrt{N})$, dự kiến được thêm 15 điểm. Cộng với các phần trước, dự kiến được 75 điểm.

6.7. Test 16–20

Nhóm dữ liệu cuối cùng thực ra chỉ cần sửa rất ít từ lời giải cho test 13–15.

Quan sát rằng bài này cho phép xử lý ngoại tuyến, và các thao tác độc lập với nhau. Vì vậy ta chỉ cần bọc thêm một tầng chia để trị CDQ ở ngoài, nhờ đó bảo đảm độ dài chèn là đơn điệu, rồi dùng trực tiếp lời giải của test 13–15.

Do có thêm CDQ, độ phức tạp thời gian là $O(M\sqrt{N}\log_2 M)$. Sau một vài điều chỉnh nhỏ, cách này có thể vượt qua toàn bộ dữ liệu, dự kiến được 100 điểm.

7. Chỉnh tham số cho KD-tree

Cuối cùng, nói thêm vài điều ngoài lề.

Phương pháp phân khối cho test 5–8 ở trên thực ra cũng có thể thay bằng **KD-tree**.

Bài toán lúc này là: với truy vấn mà một chiều là đoạn, còn chiều kia là toàn miền, ta có thể nhận được các độ phức tạp khác nhau tùy cách sắp tầng tách.

Giả sử ta muốn độ phức tạp khi chiều thứ nhất là đoạn bằng N^a , còn khi chiều thứ hai là đoạn bằng N^{1-a} . Khi đó chỉ cần chọn số tầng tách theo chiều thứ hai là $a \log_2 N$.

Tuy nhiên cần chú ý thứ tự sắp các tầng, nếu không độ phức tạp có thể bị nhân thêm một thừa số $O(\log_2 N)$. Một cách sắp khá tốt là: giả sử $a < 1 - a$, trước hết dùng phần tầng dư ra $(1 - 2a) \log_2 N$ để tách, sau đó các tầng còn lại luân phiên tách như bình thường.

Nếu dùng **KD-tree** để chỉnh tham số, so với phân khối có thể giảm được một phần không gian và hằng số.

8. Tổng kết

Nhìn chung, độ khó của bài này gần với bài thứ hai của NOI. Bài chủ yếu kiểm tra khả năng khai thác tính chất của đề và chuyển hóa chúng thành những bài toán đơn giản hơn. Mục tiêu là nối bài toán chuỗi với bài toán cấu trúc dữ liệu.

9. Lời cảm ơn

- Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Xin cảm ơn thầy Jiang Tao và thầy Shen Guanjian của Trường Trung học Shimen, Phúc Sơn, đã quan tâm và chỉ dẫn trong nhiều năm.
- Xin cảm ơn các huấn luyện viên đội tuyển tập huấn quốc gia Yu Linyun và Chen Xumin đã hướng dẫn.
- Xin cảm ơn các bạn Long Yaowei, Mai Jing, Yang Jiahong và những bạn học khác ở Trường Trung học Shimen đã giúp đỡ và gợi mở.
- Xin cảm ơn đàn anh Wang Jianhao của Trường Trung học số 1 Thiệu Hưng đã cung cấp mã mẫu.
- Xin cảm ơn những thầy cô và bạn học khác từng giúp đỡ và truyền cảm hứng cho tôi.
- Xin cảm ơn cha mẹ đã quan tâm và chăm sóc tôi.

Tài liệu tham khảo

1. Yu Jiping, *Ứng dụng thư viện pb_ds của C++ trong OI*.
2. Ren Zhizhou, *KD-tree trong các bài cấu trúc dữ liệu OI truyền thống*.
3. Luo Suiqian, *Mảng hậu tố: công cụ mạnh để xử lý chuỗi*.
4. Chen Lijie, *Ô-tô-mát hậu tố*.
5. Luo Jianqiao, *Bàn sơ lược về tư tưởng phân khối trong một lớp bài toán xử lý dữ liệu*.
6. Chen Danqi, *Từ Cash bàn về ứng dụng của một lớp thuật toán chia để trị*.
7. Xie Chao, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa.

8. Liu Rujia, *Kinh điển nhập môn thi lập trình thuật toán*, Nhà xuất bản Đại học Thanh Hoa.

Bài 12. Báo cáo ra đề *Tập hợp của quá khứ*

Wang Wenxiao, Trường Trung học Song Thập, Hạ Môn

Tóm tắt

Bài toán hợp nhất và truy vấn các tập rời nhau là một trong những bài toán mà người mới học thường gặp khá sớm. Lý do không chỉ là hình thức của nó đơn giản, mà còn vì nó có nhiều cách giải khác nhau. Trong đó, cách biểu diễn bằng tính liên thông, duy trì một rừng để cài đặt cấu trúc hợp nhất–tìm kiếm, cùng hai tối ưu nén đường đi và hợp nhất theo hạng, đều vừa gọn vừa rất hiệu quả.

Bài *Tập hợp của quá khứ* là một mở rộng đơn giản của bài toán hợp nhất và truy vấn tập rời nhau. Bài viết này tách khỏi bản thân đề thi, thảo luận nhiều cách giải cho bài toán đó, đồng thời tổng kết suy nghĩ của tác giả khi ra đề và một số đặc điểm của các lời giải khác nhau. Tác giả hy vọng những nhận xét này có thể gợi mở cho người đọc và giúp hiểu tốt hơn lớp bài toán này cũng như các bài toán liên quan.

1. Ý chính bài toán

Có n phần tử. Ban đầu mỗi phần tử thuộc một tập riêng. Có m thao tác, mỗi thao tác thuộc một trong hai loại sau:

1. hợp nhất hai tập;
2. hỏi tại một thời điểm nào đó trong quá khứ, hai phần tử có thuộc cùng một tập hay không.

Ở đây mô tả bài toán hơi khác với phát biểu đầy đủ của đề *Tập hợp của quá khứ*.

2. Giới thiệu các thuật toán chính

Sau đây gọi bài toán “hợp nhất và truy vấn các tập rời nhau” là bài toán gốc.

Điểm khác giữa bài toán mở rộng và bài toán gốc nằm ở yêu cầu **persistent** một phần: trên nền bài toán gốc, ta còn phải hỗ trợ truy vấn thông tin của các phiên bản lịch sử. Vì vậy trước hết ta xét một số cách giải cho bài toán gốc, rồi thử mở rộng chúng sang trường hợp cần **persistent** một phần. Với bài toán gốc, nhìn chung có hai kiểu phương pháp để cài đặt một cấu trúc hợp nhất–tìm kiếm: phương pháp gán nhãn và phương pháp duy trì rừng.

2.1. Phương pháp gán nhãn

Phương pháp gán nhãn cấp cho mỗi phần tử một nhãn; các phần tử có cùng nhãn được xem là nằm trong cùng một tập.

Nếu cài đặt đơn giản, mỗi truy vấn tốn $O(1)$, nhưng tổng độ phức tạp của các phép hợp nhất trong trường hợp xấu nhất có thể đạt $O(n^2)$.

2.1.1. Hợp nhất theo heuristic. Khi hợp nhất hai tập, ta có thể chọn đổi nhãn của mọi phần tử trong một tập sang nhãn của tập còn lại; thời gian khi đó phụ thuộc vào kích thước tập bị đổi nhãn. Một tối ưu hiển nhiên là mỗi lần luôn chọn tập có kích thước nhỏ hơn để đổi nhãn.

Khi đó truy vấn vẫn có độ phức tạp $O(1)$, còn tổng độ phức tạp của hợp nhất không vượt quá $O(n \log n)$. Thật vậy, mỗi lần nhân của một phần tử bị đổi, kích thước tập chứa nó ít nhất tăng gấp đôi. Vì kích thước một tập không thể vượt quá tổng số phần tử n , nhân của bất kỳ phần tử nào cũng chỉ bị đổi nhiều nhất $O(\log n)$ lần.

Persistent một phần. Toàn bộ cách làm này rất dễ cài đặt bằng mảng, nên một cách trực tiếp là dùng nó trên mảng **persistent**. Tuy cách cài đặt mảng **persistent**, độ phức tạp sẽ hơi khác nhau.

Nhưng bài toán thực ra không cần **persistent** hoàn toàn. Ta chỉ cần một mảng **persistent** một phần, có thể cài bằng cách với mỗi vị trí mở một danh sách ghi lại mọi lần sửa đổi của nó. Khi đó truy cập một phiên bản chỉ định tốn $O(\log n)$, còn truy cập phiên bản hiện tại tốn $O(1)$.

Sau khi dùng cấu trúc trên, mỗi truy vấn tốn $O(\log n)$, còn tổng độ phức tạp của các phép hợp nhất là $O(n \log n)$.

2.2. Phương pháp duy trì rừng

Phương pháp duy trì rừng dùng tính liên thông giữa hai phần tử để biểu thị chúng có thuộc cùng một tập hay không, và lưu tính liên thông ấy bằng một cấu trúc rừng. Cách trực tiếp để lưu một rừng là ghi cha của mỗi nút; đây cũng là hình thức chính của phương pháp này.

Để kiểm tra hai phần tử có nằm trong cùng một tập hay không, chỉ cần kiểm tra gốc của hai cây chứa chúng có giống nhau không.

Nếu không tối ưu, mỗi truy vấn trong trường hợp xấu nhất tốn $O(n)$, còn tổng độ phức tạp của hợp nhất trong trường hợp xấu nhất có thể đạt $O(n^2)$.

2.2.1. Nén đường đi. Tối ưu nén đường đi nghĩa là mỗi khi truy vấn gốc của cây chứa một điểm, ta đi thẳng lên tìm gốc, rồi đổi mọi điểm trên đường từ điểm đó đến gốc thành con trực tiếp của gốc. Sau tối ưu này, độ phức tạp khấu hao của mỗi thao tác là $O(\log n)$.

Vì độ phức tạp này mang tính khấu hao, cách làm rất khó mở rộng sang trường hợp **persistent** một phần.

2.2.2. Hợp nhất theo hạng. Hợp nhất theo hạng gán cho mỗi cây một hạng. Cây chỉ có một điểm có hạng 0. Khi hợp nhất hai cây, nếu hạng của chúng khác nhau, lấy gốc của cây có hạng nhỏ hơn làm con của gốc cây còn lại, và đặt hạng cây mới bằng hạng lớn hơn trong hai hạng cũ. Nếu hai hạng bằng nhau, tùy ý lấy gốc của một cây làm con của gốc cây kia, rồi đặt hạng cây mới bằng hạng cũ cộng 1.

Dễ thấy hạng tương đương với khoảng cách từ gốc đến điểm xa gốc nhất trong cây. Bằng quy nạp có thể suy ra một cây hạng i phải có ít nhất 2^i nút, nên hạng là $O(\log n)$. Do đó mỗi thao tác có độ phức tạp xấu nhất $O(\log n)$.

Persistent một phần. Cách này hiển nhiên cũng có thể mở rộng sang trường hợp **persistent** một phần.

Một cách là tiếp tục dùng mảng **persistent** hoặc mảng **persistent** một phần, nhưng ở đây ta có cách tốt hơn.

Quan sát cấu trúc của rừng và cách sửa đổi khi hợp nhất, ta thấy vài tính chất sau.

- Tập cạnh của rừng tại một thời điểm trong quá khứ chắc chắn là tập con của tập cạnh hiện tại, vì trong quá trình thao tác không có cạnh nào bị xóa.

- Nếu gán cho mỗi cạnh hiện đang tồn tại trọng số bằng thời điểm cạnh ấy xuất hiện, thì trên đường từ bất kỳ điểm nào đến gốc, trọng số cạnh tăng đơn điệu. Lý do là mỗi lần hợp nhất hai cây, cạnh mới luôn nối hai gốc của hai cây.

Vì vậy ta có thể ghi trực tiếp thời điểm xuất hiện của mỗi cạnh. Khi truy vấn, trên đường từ điểm được hỏi đến gốc, duyệt thô để tìm cạnh xuất hiện không muộn hơn thời điểm truy vấn và gần gốc nhất. Khi đó hợp nhất và truy vấn đều có độ phức tạp xấu nhất $O(\log n)$.

Có thể thấy thuật toán này vẫn khá thô bạo khi truy vấn tập chứa một điểm, nên còn không gian tối ưu.

2.2.3. Kết hợp hai tối ưu trên. Sau khi kết hợp hai tối ưu nén đường đi và hợp nhất theo hạng, độ phức tạp khâu hao là $O(\alpha(n))$, nhưng độ phức tạp xấu nhất của một thao tác vẫn là $O(\log n)$.

Persistent một phần. Khi mở rộng sang trường hợp **persistent** một phần, thuật toán này không có ưu thế so với chỉ dùng hợp nhất theo hạng; ngược lại, nén đường đi còn làm cài đặt khó hơn, nên ý nghĩa thực tế không lớn.

2.2.4. Phương pháp nhân đôi. Xét cách giải truy vấn bằng nhân đôi. Vấn đề còn lại là duy trì những thông tin cần thiết cho nhân đôi.

Với mỗi điểm x và mỗi i , ghi $f_{x,i}$ là nút nhận được sau khi đi lên 2^i bước về phía gốc. Mỗi phép hợp nhất thực chất là gán giá trị cho một $f_{x,0}$. Khi một $f_{x,i}$ đã được xác định, giá trị ấy sẽ không bị sửa nữa; đồng thời, với mọi y thỏa $f_{y,i} = x$, giá trị $f_{y,i+1}$ cũng sẽ được xác định. Với mỗi cặp (x, i) , dùng danh sách liên kết ghi lại các y thỏa $f_{y,i} = x$. Khi $f_{x,i}$ được xác định, ta dựa vào danh sách này để cập nhật mọi giá trị $f_{y,i+1}$ mới được xác định.

Nhờ các giá trị $f_{x,i}$, khi truy vấn ta có thể dễ dàng dùng nhân đôi để tìm gốc hiện tại.

Rõ ràng với mỗi x , chỉ những $i \geq 0$ và không vượt quá $O(\log n)$ mới có ý nghĩa, nên số vị trí hữu hiệu không quá $O(n \log n)$. Mỗi vị trí hữu hiệu lại được gán nhiều nhất một lần, vì vậy tổng độ phức tạp của hợp nhất cũng không vượt quá $O(n \log n)$.

Thuật toán này có độ phức tạp xấu nhất $O(\log n)$ cho một truy vấn, và tổng độ phức tạp $O(n \log n)$ cho các phép hợp nhất.

Persistent một phần. Dựa vào tính đơn điệu của trọng số cạnh trên đường từ điểm đến gốc, ta có thể dễ dàng làm thuật toán này **persistent** một phần.

Với mỗi điểm x và mỗi i , ghi thêm $t_{x,i}$ là thời điểm sớm nhất mà x liên thông với $f_{x,i}$. Giá trị $t_{x,i}$ có thể được tính tự nhiên khi duy trì $f_{x,i}$.

Khi truy vấn, dùng nhân đôi cùng tính đơn điệu của trọng số cạnh trên đường từ điểm đến gốc, ta có thể giải trong $O(\log n)$.

Sau khi làm **persistent** một phần, độ phức tạp của một truy vấn vẫn là xấu nhất $O(\log n)$, và tổng độ phức tạp của hợp nhất vẫn là $O(n \log n)$.

2.2.5. Hợp nhất theo hạng kết hợp nhân đôi. Chú ý rằng hợp nhất theo hạng dùng tính chất “khi hợp nhất có thể tự do chọn gốc của cây mới”, còn phương pháp nhân đôi chỉ dùng cấu trúc khác để tăng tốc quá trình truy vấn. Hai thuật toán này tối ưu hai phần không liên quan của cách làm đơn giản, nên hiển nhiên có thể kết hợp.

Khi dùng đồng thời hai thuật toán, độ phức tạp của một truy vấn là xấu nhất $O(\log \log n)$, còn tổng độ phức tạp của hợp nhất là $O(n \log \log n)$.

Persistent một phần. Khi mở rộng hai thuật toán này sang trường hợp **persistent** một phần, hai cách xử lý cũng thống nhất với nhau: đều ghi lại và kiểm tra thời điểm xuất hiện của các cạnh trong rừng. Vì vậy chúng vẫn có thể kết hợp.

Khi đó ta thu được một thuật toán hợp nhất–tìm kiếm **persistent** một phần với độ phức tạp xấu nhất $O(\log \log n)$ cho một truy vấn, và tổng độ phức tạp $O(n \log \log n)$ cho các phép hợp nhất.

3. Lời cảm ơn

- Xin cảm ơn cha mẹ và gia đình tôi.
- Xin cảm ơn thầy Zeng Yiqing.
- Xin cảm ơn các bạn học của tôi.
- Xin cảm ơn China Computer Federation.

Bài 13. Báo cáo ra đề *Lái tàu rời Qin Chuan*

Wu Zuofan, Trường THPT trực thuộc Đại học Sư phạm An Huy

1. Đề bài

1.1. Mô tả bài toán

Cho một cây xương rồng có n đỉnh, m cạnh, các cạnh có trọng số; đỉnh và cạnh đều được đánh số từ 1. Có q nhóm thao tác, mỗi nhóm gồm hai phần:

1. Cho k_i đường đi. Đường đi thứ j nối từ s_j đến t_j ; một số đường là đường đơn đi qua nhiều đỉnh nhất, một số đường là đường đơn đi qua ít đỉnh nhất. Hãy tính tổng trọng số của những cạnh được ít nhất một đường đi qua.
2. Sau khi hỏi xong, đổi trọng số cạnh thứ w_i thành x_i .

Một đồ thị vô hướng liên thông được gọi là **cây xương rồng** nếu mỗi cạnh thuộc nhiều nhất một chu trình đơn. Chu trình đơn ở đây là chu trình không đi qua một đỉnh lặp lại. Để đường đơn đi qua nhiều đỉnh nhất hoặc ít đỉnh nhất là duy nhất, mọi chu trình trong dữ liệu của bài đều có độ dài lẻ. Không có khuyên.

Hình minh họa trong PDF nguồn gồm ba đồ thị: một cây xương rồng hợp lệ; một đồ thị không phải cây xương rồng vì một cạnh nằm trên hai chu trình; và một đồ thị không phải cây xương rồng vì không liên thông.

1.2. Định dạng vào

Dòng đầu gồm ba số nguyên n, m, q , biểu thị số đỉnh, số cạnh và số nhóm thao tác.

Tiếp theo là m dòng, mỗi dòng gồm ba số nguyên u, v, w , biểu thị một cạnh nối u và v có trọng số w .

Tiếp theo là $2q$ dòng; cứ hai dòng mô tả một nhóm thao tác.

Dòng thứ nhất của mỗi nhóm là truy vấn. Trước hết có một số nguyên k_i , biểu thị có k_i đường đi. Giá trị k_i có thể bằng 0; khi đó bỏ qua truy vấn này và in 0. Sau đó là mô tả từng đường đi. Đường thứ j được mô tả bằng ba số nguyên $s_j, t_j, type_j$, lần lượt là đầu mút đầu, đầu mút cuối và loại đường đi. Nếu

$\text{type}_j = 0$ thì đó là đường đơn đi qua ít đỉnh nhất; nếu $\text{type}_j = 1$ thì đó là đường đơn đi qua nhiều đỉnh nhất. Hai đầu mút của một đường đi không trùng nhau.

Dòng thứ hai gồm hai số nguyên w_i, x_i , nghĩa là trọng số cạnh thứ w_i sẽ đổi thành x_i . Nếu $w_i = 0$, không có sửa đổi.

1.3. Định dạng ra

In q dòng, mỗi dòng một số nguyên, là đáp án của truy vấn tương ứng.

1.4. Giới hạn và quy ước

Bảo đảm tại mọi thời điểm, tổng độ dài cạnh nằm trong phạm vi kiểu `int`. Mọi độ dài cạnh đều là số nguyên dương.

Đặt $S = \sum_{i=1}^q k_i$. Giới hạn dữ liệu như sau.

Test	n	m	q	S	Có sửa đổi
1	$= 2$	$n - 1 \leq m \leq 2n - 2$	≤ 10	≤ 10	Có
2	$= 100$	$n - 1 \leq m \leq 2n - 2$	≤ 100	≤ 100	Có
3	$= 100000$	$m = n - 1$	≤ 100000	≤ 100000	Không
4	$= 100000$	$m = n - 1$	≤ 100000	≤ 100000	Có
5	$= 300000$	$m = n - 1$	≤ 300000	≤ 300000	Không
6	$= 300000$	$m = n - 1$	≤ 300000	≤ 300000	Có
7	$= 100000$	$n - 1 \leq m \leq 2n - 2$	≤ 100000	≤ 100000	Không
8	$= 100000$	$n - 1 \leq m \leq 2n - 2$	≤ 100000	≤ 100000	Có
9	$= 300000$	$n - 1 \leq m \leq 2n - 2$	≤ 300000	≤ 300000	Không
10	$= 300000$	$n - 1 \leq m \leq 2n - 2$	≤ 300000	≤ 300000	Có

2. Phân tích thuật toán

Trong các test 1, 3, 4, 5 và 6, cây xương rồng thực ra là một cây. Trên cây, đường đơn giữa hai đỉnh là duy nhất, nên dễ xử lý hơn nhiều. Vì vậy trước hết ta xem bài toán quen thuộc trên cây nên được giải thế nào.

2.1. Thuật toán một

Ta bắt đầu từ trường hợp dãy.

2.1.1. Thuật toán dãy thứ nhất. Trên một dãy, một đường đi chính là một đoạn. Ta sắp các đoạn theo đầu trái, quét một lần, liên tục hợp nhất hai đoạn giao nhau, rồi tính trọng số của từng đoạn rời nhau còn lại.

Vì vậy ta cần hỗ trợ sửa một điểm và hỏi một đoạn; có thể dùng cây đoạn, cây Fenwick, v.v. Mỗi thao tác cơ bản tốn $O(\log n)$, nên một nhóm thao tác có tổng độ phức tạp $O(k \log n)$.

2.1.2. Thuật toán dãy thứ hai. Ta cũng có thể dùng trực tiếp cây đoạn, đánh dấu mỗi nút xem nó đã được phủ hay chưa. Phần hơi phiền là sau mỗi truy vấn phải đặt lại cây đoạn; có thể ghi lại những nút từng bị sửa trong truy vấn rồi khôi phục thô sau khi hỏi xong, hoặc dùng cấu trúc persistent. Độ phức tạp vẫn là $O(k \log n)$.

Quay lại cây, ta cũng có thể dùng phân rã nặng nhẹ để chuyển truy vấn thành hợp các đoạn.

Phân rã nặng nhẹ đã rất phổ biến trong thi tin học, nên ở đây chỉ nhắc lại ngắn gọn. Ý tưởng cốt lõi là tách cây thành một số chuỗi sao cho mỗi đỉnh thuộc đúng một chuỗi. Khi đó một đường đi được ghép từ các đoạn trên những chuỗi ấy, và bài toán trên cây được chuyển thành bài toán trên dãy.

Ta cần một cách tách heuristic sao cho đường từ gốc đến mỗi đỉnh đi qua càng ít chuỗi càng tốt. Với mỗi đỉnh u , tìm con v có cây con lớn nhất và đánh dấu cạnh (u, v) là cạnh nặng; các cạnh còn lại là cạnh nhẹ. Các chuỗi tạo bởi cạnh nặng được gọi là chuỗi nặng.

Khi đi từ gốc xuống, mỗi lần qua cạnh nhẹ để đổi chuỗi nặng thì kích thước cây con ít nhất giảm một nửa, nên chỉ đi qua $O(\log n)$ chuỗi nặng. Do đó đường đi giữa hai đỉnh bất kỳ được chia thành $O(\log n)$ đoạn.

Như vậy bài toán trở thành hợp của $O(k \log n)$ đoạn; dùng thuật toán dãy ở trên cho độ phức tạp $O(S \log^2 n)$, dự kiến được 30 đến 50 điểm.

2.2. Thuật toán hai

Muốn có độ phức tạp tốt hơn, ta đổi cách nhìn.

Nếu chỉ có một lần hỏi, với một đường đi $\langle u, v \rangle$, ta có thể đánh dấu tại u, v và $LCA(u, v)$, rồi duyệt cả cây để quyết định mỗi cạnh có được đi qua hay không. Độ phức tạp là $O(n)$.

Vì số đỉnh quan trọng là $O(k)$, ta tự nhiên nghĩ đến cây ảo. Đây cũng là thuật toán kinh điển, nên bài viết chỉ giới thiệu ngắn gọn.

Cây ảo là cây thu được bằng cách lấy một cây con liên thông của cây gốc rồi co những chuỗi không rẽ nhánh thành một cạnh. Ta cần dựng một cây ảo chứa tất cả đỉnh quan trọng và có số đỉnh càng ít càng tốt.

Một cách dựng đơn giản là duy trì một ngăn xếp, sắp các đỉnh quan trọng theo thứ tự DFS rồi lần lượt thêm chúng vào cây ảo. Với đỉnh hiện tại, tính LCA của nó với đỉnh trên đỉnh ngăn xếp; liên tục bật ngăn xếp đến khi đỉnh trên cùng là tổ tiên của LCA, rồi đưa LCA và đỉnh hiện tại vào cây ảo, đồng thời sửa quan hệ cha của các nút liên quan.

Có thể thấy cây ảo dựng theo cách này có $O(k)$ đỉnh và mất $O(k \log n)$ thời gian.

Tiếp theo cần tính trọng số của mỗi cạnh trên cây ảo. Một cạnh trên cây ảo chính là một chuỗi $\langle u, v \rangle$ trên cây gốc, trong đó u là tổ tiên của v . Có thể dùng trực tiếp Link-Cut Tree để duy trì cây này, khi đó tính một cạnh mất $O(\log n)$.

Cách đơn giản hơn là duy trì độ dài đường từ mỗi đỉnh đến gốc. Sửa một cạnh chỉ ảnh hưởng đến các đỉnh trong một cây con; cây con lại là một đoạn liên tiếp theo thứ tự DFS, nên chỉ cần dùng cây đoạn hỗ trợ cộng đoạn. Độ phức tạp cũng là $O(\log n)$.

Sau khi biết mọi trọng số cạnh, duyệt cây ảo là nhận được đáp án.

Do dựng cây ảo mất $O(k \log n)$, có $O(k)$ cạnh trên cây ảo cần tính và mỗi cạnh tốn $O(\log n)$, mỗi nhóm thao tác mất $O(k \log n)$. Tổng thời gian là $O(S \log n)$, dự kiến được 50 điểm.

Các test còn lại đều là cây xương rồng. Để tiện mô tả thuật toán, ta đưa ra một số định nghĩa.

2.3. Một số định nghĩa

Hình trong PDF nguồn dùng một cây xương rồng gốc 1 làm ví dụ. Các đỉnh được nối sao cho có hai chu trình $10 - 11 - 12 - 10$ và $2 - 5 - 6 - 7 - 4 - 2$, cùng các nhánh $1 - 3 - 10$, $6 - 9$, $7 - 14 - 15$, $4 - 8$ và $11 - 13$. Nếu không nói khác, các ví dụ bên dưới đều nói về cây xương rồng đó.

2.3.1. Chuỗi dài nhất. Chuỗi dài nhất giữa hai đỉnh u và v là đường đơn đi qua nhiều đỉnh nhất giữa chúng. Chẳng hạn chuỗi dài nhất giữa 10 và 5 là

$$10 - 3 - 1 - 2 - 4 - 7 - 6 - 5.$$

2.3.2. Chuỗi ngắn nhất. Chuỗi ngắn nhất giữa hai đỉnh u và v là đường đơn đi qua ít đỉnh nhất giữa chúng. Chẳng hạn chuỗi ngắn nhất giữa 10 và 5 là

$$10 - 3 - 2 - 5.$$

2.3.3. Cha của một chu trình. Cha của một chu trình là đỉnh trên chu trình gần gốc nhất. Ví dụ cha của chu trình $10 - 11 - 12$ là 10. Ở đây và dưới đây, “chu trình” đều có nghĩa là chu trình đơn.

2.3.4. Cây xương rồng con. Cây xương rồng con của một đỉnh là thành phần liên thông chứa đỉnh đó sau khi xóa các cạnh nằm trên chuỗi dài nhất và chuỗi ngắn nhất từ đỉnh đó đến gốc. Ví dụ cây xương rồng con của đỉnh 6 chỉ gồm hai đỉnh 6 và 9.

2.4. Thuật toán ba

Với test 2, số đỉnh và tổng số đường trong truy vấn đều rất nhỏ, nên ta có thể tính trực tiếp mỗi chuỗi dài nhất hoặc ngắn nhất đi qua những cạnh nào, rồi cộng đáp án.

Trước hết DFS cây xương rồng để tạo một cây DFS. Khi đó mỗi chu trình gồm một chuỗi trên cây DFS và một cạnh ngược lên tổ tiên. Chẳng hạn chu trình $2 - 5 - 6 - 7 - 4$ gồm chuỗi $2 - 5 - 6 - 7 - 4$ trên cây DFS và cạnh ngược $(4, 2)$.

Với một đường đi trong truy vấn, trước hết đi theo cây DFS, rồi liệt kê từng chu trình. Nếu đường đi qua một chu trình thì có hai cách đi trong chu trình đó; nếu cách còn lại tốt hơn, thay thế đoạn tương ứng.

Ví dụ cần đi chuỗi dài nhất từ 4 đến 9. Đi trên cây DFS trước cho $4 - 7 - 6 - 9$. Chu trình $2 - 5 - 6 - 7 - 4$ bị đi qua; đoạn trong chu trình có hai cách là $4 - 7 - 6$ và $4 - 2 - 5 - 6$. Cách sau tốt hơn, nên đổi thành $4 - 2 - 5 - 6 - 9$.

Vì hai chu trình bất kỳ không có cạnh chung, liệt kê mọi chu trình mất $O(m)$. Ở đây m và n cùng bậc nên dưới đây không phân biệt thêm. Tổng độ phức tạp là $O(Sn)$; kết hợp với các thuật toán trên cây có thể đạt 60 điểm.

2.5. Chuyển hóa

Xử lý trực tiếp cây xương rồng khá khó. Ta xét việc co toàn bộ chu trình trong cây xương rồng để biến nó thành cây, rồi dùng thuật toán trên cây để giải bài toán xương rồng.

Co chu trình có một vấn đề: một đỉnh có thể nằm trong nhiều chu trình, ví dụ đỉnh 2 ở ví dụ trên. Nhưng ta nhận thấy đường từ cha của chu trình đến gốc không đi qua những đỉnh khác của chu trình đó, còn hợp của chuỗi ngắn nhất và chuỗi dài nhất từ các đỉnh khác đến gốc sẽ phủ kín chu trình. Vì vậy cha của chu trình không bị co vào chu trình ấy.

Khi đó mỗi đỉnh chỉ bị co vào chu trình chứa cạnh nối nó với cha trong cây DFS. Mỗi cạnh chỉ thuộc một chu trình, nên mỗi đỉnh của cây xương rồng tương ứng duy nhất với một đỉnh mới. Dĩ nhiên một đỉnh mới có thể đại diện cho nhiều đỉnh cũ. Nếu cạnh nối với cha không nằm trong chu trình nào thì đỉnh ấy không bị co.

Ta gọi cây thu được sau phép co này là **cây co chu trình**. Với ví dụ trên, cây co chu trình có các nút

$$\begin{aligned} a &= \{1\}, & b &= \{2, 3\}, & c &= \{10\}, & d &= \{11, 12\}, \\ e &= \{13\}, & f &= \{4, 5, 6, 7\}, & g &= \{8\}, & h &= \{9\}, \\ i &= \{14\}, & j &= \{15\}. \end{aligned}$$

Các nút b, d, f là những nút sinh ra từ phép co chu trình, mỗi nút đại diện cho một chu trình.

2.6. Ví dụ: cây xương rồng tĩnh

Xét bài toán sau. Cho một cây xương rồng có n đỉnh, m cạnh; mỗi đỉnh có một màu đen hoặc trắng, ban đầu đều là đen. Có q thao tác thuộc ba loại:

1. đảo màu chuỗi ngắn nhất từ đỉnh x đến gốc;
2. đảo màu chuỗi dài nhất từ đỉnh x đến gốc;
3. hỏi số đỉnh đen trong cây xương rồng con của đỉnh x .

Ở đây $n, q \leq 50000$.

Ta tổng quát hóa bài toán này như sau: cho một cây xương rồng, cần hỗ trợ thao tác và truy vấn trên một chuỗi dài nhất, một chuỗi ngắn nhất, hoặc một cây xương rồng con cho trước. Ở đây lấy trọng số đỉnh làm ví dụ; trường hợp trọng số cạnh tương tự.

2.6.1. Trường hợp trên cây. Nếu cây xương rồng thực ra là cây, ta có thể dùng phân rã nặng nhẹ đã nhắc ở trên. Sắp các chuỗi nặng liên nhau theo thứ tự DFS thì cây con cũng là một đoạn liên tiếp, sau đó dùng cấu trúc dữ liệu duy trì là đủ. Một chuỗi bị chia thành $O(\log n)$ đoạn, còn một cây con là $O(1)$ đoạn.

2.6.2. Phân rã nặng nhẹ trên cây co chu trình. Ta đã chuyển cây xương rồng thành cây co chu trình, nên có thể phân rã nặng nhẹ trên cây này.

Trong ví dụ, một cách chọn cạnh nặng trên cây co chu trình là

$$a - b - f - i - j, \quad b - c - d - e,$$

các cạnh nhẹ nối tới g và h . Nếu đây là cây thông thường, thứ tự sau phân rã có thể là

$$\{a, b, f, i, j, h, g, c, d, e\}.$$

Với cây xương rồng, ta cần hỏi nên trải nó thành dãy thế nào.

Xét một chuỗi nặng, chẳng hạn $a - b - f - i - j$. Một nút có thể có một cách đi nếu nó đại diện cho một đỉnh, hoặc hai cách đi nếu nó đại diện cho một chu trình. Ta xét đường từ đỉnh 15 đến đỉnh 1.

Trong hình nguồn, đỉnh màu đỏ là đỉnh vừa nằm trên chuỗi dài nhất vừa nằm trên chuỗi ngắn nhất; đỉnh màu xanh lá chỉ nằm trên chuỗi dài nhất; đỉnh màu vàng chỉ nằm trên chuỗi ngắn nhất. Ta nhận thấy: nếu một nút của cây co chu trình đại diện cho một đỉnh thì nó chắc chắn thuộc loại đỏ; nếu nó đại diện cho một chu trình, phần giao giữa chu trình này và nút kế tiếp là đỏ, ví dụ các đỉnh 2 và 7.

Với một chu trình, hai cách đi chỉ phụ thuộc vào vị trí của đầu xuất phát trên chu trình; với các nút trên cùng một chuỗi nặng, vị trí này là như nhau.

Do đó ta có thể xử lý riêng từng chuỗi nặng: tiền xử lý màu của từng đỉnh, rồi lập riêng ba dãy cho ba màu. Các dãy này thu được bằng cách mở rộng $\{a, b, f, i, j, h, g, c, d, e\}$, trong đó mỗi nút đại diện cho chu trình được mở thành một đoạn trên chu trình.

Trong ví dụ, dãy vàng là $\{4\}$, dãy đỏ là $\{1, 2, 7, 14, 15, 9, 8, 10, 11, 13\}$, và dãy xanh lá là $\{3, 5, 6, 12\}$.

Với một chuỗi ngắn nhất hoặc dài nhất, đường đi tương ứng trên cây co chu trình đi qua $O(\log n)$ chuỗi nặng và $O(\log n)$ cạnh nhẹ. Phần trên chuỗi nặng có thể biểu diễn bằng $O(\log n)$ đoạn trên ba loại dây; mỗi cạnh nhẹ có thể xét cách đi trong chu trình rồi chuyển thành $O(1)$ đoạn. Ta chỉ cần dùng cấu trúc dữ liệu để duy trì ba dãy này.

Với thao tác trên cây xương rồng con, nếu đỉnh đang xét không thuộc một chu trình trong cây co chu trình, chỉ cần tìm đoạn tương ứng trên ba dãy. Nếu đỉnh đó thuộc một chu trình, ta có thể điều chỉnh thứ tự các con của chu trình để cây xương rồng con của mỗi nút đều là một đoạn liên tiếp.

Như vậy, nhờ phân rã nặng nhẹ trên cây co chu trình, ta đạt độ phức tạp giống trường hợp cây: chuỗi bị chia thành $O(\log n)$ đoạn, cây con là $O(1)$ đoạn.

2.7. Thuật toán bốn

Ta phân rã nặng nhẹ trực tiếp trên cây co chu trình, rồi dùng cách duy trì dãy của thuật toán một. Độ phức tạp thời gian là $O(S \log^2 n)$, dự kiến được 80 đến 100 điểm.

2.8. Ví dụ: cây xương rồng của mx

Cho một cây xương rồng có n đỉnh, m cạnh. Định nghĩa khoảng cách giữa hai đỉnh là độ dài đường ngắn nhất. Có q truy vấn, mỗi truy vấn cho một tập đỉnh và hỏi khoảng cách giữa cặp đỉnh xa nhất trong tập đó.

$$n, q \leq 300000,$$

và tổng số đỉnh trong các tập truy vấn không vượt quá 300000.

2.8.1. Một truy vấn. Nếu chỉ có một truy vấn, ta làm thế nào?

Ta có thể duyệt toàn bộ cây xương rồng và làm một lần quy hoạch động trên cây xương rồng: với mỗi đỉnh, ghi đỉnh quan trọng xa nhất trong cây xương rồng con của nó; nếu gặp một chu trình thì làm thêm một lần quy hoạch động trên chu trình. Độ phức tạp là $O(n)$.

2.8.2. Cây ảo trên cây co chu trình. Chú ý số đỉnh trong tập hỏi không nhiều; giả sử là k . Tương tự trên cây, ta có thể dựng cây ảo trên cây co chu trình, rồi quy hoạch động trên cây ảo này.

Với cây xương rồng và cây co chu trình ở trên, nếu tập đỉnh là $\{8, 12, 13\}$, các đỉnh tương ứng trên cây co chu trình là $\{g, d, e\}$. Cây ảo nguồn có $b = \{2, 3\}$ làm nút chung, nối tới $d = \{11, 12\}$ và $g = \{8\}$, rồi d nối tới $e = \{13\}$.

Cây ảo này có $O(k)$ nút, nhưng một nút có thể đại diện cho cả một chu trình, ứng với $O(n)$ đỉnh trên cây xương rồng, nên không thể quy hoạch động trực tiếp. Các đỉnh thật sự cần xét chắc chắn là các đỉnh được đọc vào trong tập hỏi, hoặc là một đỉnh trên chu trình mà cây xương rồng con của nó chứa đỉnh quan trọng. Ví dụ trong nút b , đỉnh 2 có cây xương rồng con chứa đỉnh 8. Khi đó số đỉnh trên cây xương rồng cũng chỉ là $O(k)$, nên có thể quy hoạch động.

Ta cần tính độ dài cạnh trên cây ảo, tức độ dài một chuỗi ngắn nhất trong cây xương rồng. Thực tế chỉ cần xử lý đặc biệt hai chu trình chứa hai đầu nút của chuỗi ngắn nhất; phần còn lại có thể lấy trực tiếp từ giá trị đã tiền xử lý là độ dài chuỗi ngắn nhất từ một đỉnh đến gốc.

Cách làm này đòi hỏi hỗ trợ truy vấn: một đỉnh nằm trong cây xương rồng con của nút nào trên một chu trình. Có thể tìm bằng nhị phân trên thứ tự DFS. Nếu dựng thứ tự DFS theo cách tương tự phân rã nặng nhẹ ở trên, mỗi cây xương rồng con là một đoạn liên tiếp.

Vì vậy độ phức tạp của một truy vấn là $O(k \log n)$.

2.9. Thuật toán năm

Trở lại bài này, ta cũng dựng cây ảo trên cây co chu trình từ các đầu mút của những đường đi trong truy vấn; giả sử có k đường đi. Sau đó duyệt cây ảo. Ta dùng một ví dụ để xem phần nào cần thống kê và phải duy trì thế nào.

Giả sử có hai đường đi: chuỗi dài nhất từ 11 đến 14,

$$11 - 12 - 10 - 3 - 1 - 2 - 5 - 6 - 7 - 14,$$

và chuỗi ngắn nhất từ 8 đến 9,

$$8 - 4 - 7 - 6 - 9.$$

Các nút tương ứng trên cây co chu trình là d, i, h, g . Cây ảo gồm $b = \{2, 3\}$, hai nhánh tới $d = \{11, 12\}$ và $f = \{4, 5, 6, 7\}$, rồi từ f đi tới $g = \{8\}$, $h = \{9\}$, $i = \{14\}$.

2.9.1. Trong chu trình. Khác với cây, trong cây co chu trình vẫn có các cạnh nội bộ chu trình có thể đóng góp vào đáp án. Đóng góp trong chu trình xuất hiện theo hai cách.

1. Đóng góp sinh ra khi chu trình là LCA trên cây co chu trình của một đường đi. Ví dụ LCA của hai nút g và h , chứa 8 và 9, là f . Đỉnh 8 nằm trong cây xương rồng con của 4, đỉnh 9 nằm trong cây xương rồng con của 6, nên đóng góp đoạn $4 - 7 - 6$.
2. Đóng góp sinh ra khi một đường đi vượt qua chu trình này để đi lên. Ví dụ chuỗi dài nhất từ 11 đến 14 phải vượt qua chu trình f ; đỉnh 14 nằm trong cây xương rồng con của 7, nên sinh ra đóng góp là chuỗi dài nhất từ 7 đến cha của chu trình f , tức $7 - 6 - 5 - 2$.

Rõ ràng loại đóng góp thứ nhất chỉ có $O(k)$ đoạn, còn mỗi đóng góp loại thứ hai tương ứng với một cạnh của cây ảo, nên cũng chỉ có $O(k)$ đoạn.

Hai loại đóng góp đều là một số đoạn, vì vậy có thể thống kê trực tiếp bằng cách xử lý dãy của thuật toán một; sửa trọng số cạnh cũng tương tự. Phần này có độ phức tạp $O(k \log n)$.

2.9.2. Cạnh của cây ảo. Phần giống cây là các cạnh của cây ảo. Một cạnh cây ảo có thể xem là chuỗi dài nhất, chuỗi ngắn nhất, hoặc hợp của hai chuỗi ấy, từ cha của một chu trình (hoặc một đỉnh) tới một đỉnh trên chu trình nào đó (hoặc một đỉnh). Ví dụ cạnh giữa b và d có thể xem là chuỗi dài nhất từ gốc 10 của chu trình d đến đỉnh 3 trên b .

Ta duy trì, với mỗi đỉnh, ba giá trị từ đỉnh đó đến gốc: độ dài chuỗi dài nhất, độ dài chuỗi ngắn nhất, và độ dài hợp của hai chuỗi ấy. Trọng số cạnh cần hỏi chính là hiệu của hai giá trị tương ứng. Giống trường hợp cây, có thể duy trì theo thứ tự DFS.

Nếu cạnh bị sửa là cạnh từ một đỉnh không thuộc chu trình trên cây co chu trình tới cha của nó, ví dụ cạnh $10 - 3$, thì chỉ cây xương rồng con của đỉnh đó bị ảnh hưởng. Ta sửa ba loại giá trị trên đoạn tương ứng là được.

Nếu cạnh bị sửa là cạnh trên một chu trình, ta có thể chia chu trình này thành nhiều nhất ba đoạn; mỗi đoạn sẽ làm thay đổi một loại chuỗi dài nhất hoặc ngắn nhất. Vì vậy chỉ cần đặt thứ tự DFS của cùng một chu trình liên nhau là có thể duy trì thuận tiện.

Ví dụ nếu sửa cạnh $5 - 6$, thì chuỗi dài nhất của cây xương rồng con của 5 bị thay đổi, chuỗi ngắn nhất của cây xương rồng con của 6 bị thay đổi, và chuỗi dài nhất của cây xương rồng con của 4 và 7 cũng bị thay đổi.

Dùng cây đoạn để duy trì các giá trị trên. Số cạnh cần truy vấn là $O(k)$; mỗi truy vấn cạnh mất $O(\log n)$, nên tổng độ phức tạp của phần này là $O(k \log n)$.

Như vậy mỗi nhóm thao tác có độ phức tạp $O(k \log n)$, tổng thời gian là $O(S \log n)$, đủ đạt 100 điểm.

3. Ý tưởng ra đề và tổng kết

Những năm gần đây, thi tin học xuất hiện khá nhiều bài liên quan đến cây xương rồng. Bài thứ nhất ngày thứ hai của kỳ tập huấn Thanh Hoa 2015, *Cây xương rồng tĩnh*, đã gợi ý cho tác giả rất nhiều. Tuy tác giả đã nghĩ ra một phương pháp tốt hơn lời giải chuẩn, tức phương pháp phân rã nặng nhẹ trên cây co chu trình trong bài này, nhưng lại không cài đặt thành công. Điều đó khơi dậy hứng thú nghiên cứu cây xương rồng của tác giả.

Qua việc nghiên cứu cấu trúc mà tác giả gọi là “cây co chu trình”, tác giả nhận ra thuật toán cây ảo vẫn đúng trên cấu trúc này và độ phức tạp không đổi. Bản thân bài này trên cây cũng là một bài kinh điển, nên tác giả mở rộng nó và đặt thành đề này.

Bài toán này không dễ. Thí sinh cần hiểu khá sâu tính chất của cây xương rồng, nắm chắc các thuật toán kinh điển, và có năng lực cài đặt mạnh mới có thể làm được.

Tác giả đưa bài này vào kỳ kiểm tra chéo của đội tuyển tập huấn với mong muốn gợi mở thêm hướng suy nghĩ, đồng thời hy vọng mọi người quan tâm hơn đến cây xương rồng, một cấu trúc mở rộng của cây. Hiện nay các thuật toán liên quan đến cây xương rồng tuy có độ phức tạp khá tốt, nhưng mã thường rất khó viết; tác giả hy vọng sẽ có thêm những thuật toán hiệu quả hơn và thuận tiện hơn.

4. Lời cảm ơn

- Xin cảm ơn thầy Ye Guoping đã giúp đỡ và quan tâm trong học tập cũng như đời sống.
- Xin cảm ơn CCF đã cung cấp cơ hội giao lưu.
- Xin cảm ơn các bạn Luo Zhezheng, Gao Wenyuan và Xiao Ziqing đã đọc duyệt bài viết này.

Tài liệu tham khảo

1. #474, *Một số nghiên cứu về lời giải SPOJ375 QTREE*, bài tập đội tuyển tập huấn quốc gia năm 2007.
2. Xu Yinzhao, *Ứng dụng cây đoạn trong một lớp bài toán chia để trị*, luận văn đội tuyển tập huấn quốc gia năm 2014.
3. Wang Yisong, *Các thuật toán liên quan đến cây xương rồng và ứng dụng*, luận văn đội tuyển tập huấn quốc gia năm 2015.

Bài 14. Bài luyện tập thuật toán sắp xếp cơ bản

Jin Ce, Trường Học Quân Hàng Châu, Chiết Giang

Tóm tắt

Trong vòng đầu của kỳ kiểm tra chéo đội tuyển tập huấn, tác giả ra một bài toán thuật toán lấy bối cảnh là sắp xếp mảng. Bài viết này giới thiệu lời giải, ý tưởng ra đề, và một vài thảo luận mở rộng liên quan.

1. Đề bài

1.1. Mô tả bài toán

Cho một mảng $a[1], a[2], \dots, a[n]$ độ dài n . Xét một thuật toán gồm m bước; ở bước thứ i , thuật toán sắp xếp tại chỗ đoạn

$$a[l_i], a[l_i + 1], \dots, a[r_i]$$

theo thứ tự tăng không giảm.

Có q truy vấn. Mỗi truy vấn cho một mảng ban đầu a . Cần phán đoán xem thuật toán trên có thể sắp xếp mảng này thành thứ tự tăng không giảm hay không, tức cuối cùng có thỏa

$$a[i] \leq a[i + 1] \quad (i = 1, 2, \dots, n - 1)$$

hay không.

1.2. Dạng vào

Dòng đầu gồm ba số nguyên n, m, q .

m dòng tiếp theo lần lượt cho tham số l_i, r_i của từng bước trong thuật toán.

q dòng tiếp theo biểu diễn các truy vấn. Mỗi dòng gồm n số nguyên không âm cách nhau bởi dấu cách: $a[1], a[2], \dots, a[n]$.

1.3. Dạng ra

Với mỗi truy vấn, in một dòng. Nếu thuật toán sắp xếp được mảng thì in AC, ngược lại in WA.

1.4. Ví dụ vào

```
6 3 2
1 3
3 6
1 3
4 2 2 3 0 7
5 3 8 2 1 9
```

1.5. Ví dụ ra

```
AC
WA
```

1.6. Giải thích ví dụ

Truy vấn thứ nhất:

$$4\ 2\ 2\ 3\ 0\ 7 \rightarrow [2\ 2\ 4]\ 3\ 0\ 7 \rightarrow 2\ 2\ [0\ 3\ 4\ 7] \rightarrow [0\ 2\ 2]\ 3\ 4\ 7,$$

nên kết quả đúng.

Truy vấn thứ hai:

$$5\ 3\ 8\ 2\ 1\ 9 \rightarrow [3\ 5\ 8]\ 2\ 1\ 9 \rightarrow 3\ 5\ [1\ 2\ 8\ 9] \rightarrow [1\ 3\ 5]\ 2\ 8\ 9,$$

nên kết quả sai.

1.7. Quy mô dữ liệu và quy ước

Điểm	$n \leq$	$m \leq$	$q \leq$	Tính chất
1	100	100	100	
2	200	200	200	
3	1400	900000	8	
4	1500	1000000	8	
5	1400	4000	1400	
6	1500	5000	1500	
7	1200	700000	1200	
8	1300	800000	1300	$a[1..n]$ gồm $\lfloor n/2 \rfloor$ số 1 và $\lfloor n/2 \rfloor$ số 0
9	1400	900000	1400	
10	1500	1000000	1500	

Với mọi dữ liệu,

$$1 \leq n \leq 1500, \quad 1 \leq m \leq 1000000, \quad 1 \leq q \leq 1500,$$

$$1 \leq l_i \leq r_i \leq n, \quad 0 \leq a[i] \leq 1500.$$

1.8. Giới hạn thời gian

Mỗi điểm kiểm tra: 2.5 giây.

1.9. Giới hạn bộ nhớ

512 MB.

2. Tình hình điểm số

Có 12 thí sinh tham gia vòng kiểm tra chéo này.

Trước trận, tác giả dự đoán điểm của bài như sau: khoảng 1 đến 3 thí sinh đạt điểm cao 80–100, khoảng hai phần ba số thí sinh đạt ít nhất 40 điểm, và mọi thí sinh đều đạt ít nhất 20 điểm.

Kết quả thực tế là: 1 thí sinh đạt 100 điểm, 3 thí sinh đạt 50 điểm, 7 thí sinh đạt 40 điểm, và 1 thí sinh đạt 20 điểm. Nhìn chung kết quả phù hợp với dự đoán.

3. Thuật toán thô

Trước hết xét cách làm trực tiếp nhất.

Với mỗi truy vấn, ta mô phỏng m bước của thuật toán. Ở mỗi bước, sắp xếp đoạn tương ứng bằng sắp xếp nổi bọt hoặc sắp xếp chèn trong $O(n^2)$, rồi cuối cùng kiểm tra mảng có thứ tự hay không. Độ phức tạp thời gian là

$$O(qmn^2).$$

Có thể tối ưu cách làm này bằng cách thay thuật toán sắp xếp bằng quicksort hoặc `std::sort`, độ phức tạp mỗi lần sắp xếp là $O(n \log n)$. Khi đó tổng độ phức tạp trở thành

$$O(qmn \log n),$$

đủ qua khoảng 20% dữ liệu.

4. Nghịch thế

Một **nghịch thế** trong dãy $a[1..n]$ là một cặp chỉ số (i, j) thỏa

$$1 \leq i < j \leq n, \quad a[i] > a[j].$$

Đây là một khái niệm quan trọng và có liên hệ chặt với thuật toán sắp xếp.

Ví dụ 1 (bài toán kinh điển). Cho một mảng $a[1..n]$. Mỗi lần được phép đổi chỗ hai phần tử kề nhau $a[i]$ và $a[i + 1]$. Hỏi cần ít nhất bao nhiêu thao tác để sắp xếp mảng?

$$n \leq 10^5.$$

Sau mỗi phép đổi chỗ, số nghịch thế tăng hoặc giảm đúng 1. Vì vậy số nghịch thế ban đầu là một cận dưới của số thao tác. Cận này đạt được: cứ chọn một vị trí i có $a[i] > a[i + 1]$, đổi chỗ hai phần tử này, và lặp lại cho đến khi không còn nghịch thế kề nhau. Khi đó mảng chắc chắn đã được sắp xếp tăng không giảm.

Vì thế chỉ cần tính số nghịch thế ban đầu. Việc này có thể làm bằng merge sort: trong mỗi lần trộn, tính thêm số nghịch thế đi qua điểm giữa của đoạn, còn hai nửa được xử lý đệ quy.

Quay lại bài gốc. Ở điểm kiểm tra 3 và 4, q nhỏ còn n, m lớn, nên cần giảm độ phức tạp của một truy vấn.

Ta dùng ý tưởng đổi chỗ nghịch thế kề nhau để sắp xếp. Vì số nghịch thế ban đầu của mảng a chỉ là $O(n^2)$, trong toàn bộ quá trình ta thực hiện nhiều nhất $O(n^2)$ phép đổi chỗ.

Tuy nhiên như vậy vẫn chưa đủ. Mỗi lần sắp xếp một đoạn $a[l..r]$, nếu ta quét cả đoạn thì mất $r - l = O(n)$ thời gian. Khi đoạn rất dài nhưng có rất ít nghịch thế, phần quét này bị lãng phí.

Để tránh điều đó, ta đưa mọi vị trí i thỏa $a[i] > a[i + 1]$ vào một cây cân bằng, chẳng hạn `std::set`. Khi cần sắp xếp $a[l..r]$, tìm trong `set` vị trí nhỏ nhất không nhỏ hơn l . Nếu vị trí này $\geq r$ thì đoạn đã có thứ tự. Nếu không, đổi chỗ $a[i]$ và $a[i + 1]$, rồi chỉ cập nhật lại quan hệ của các cặp lân cận có thể bị ảnh hưởng:

$$(i - 1, i), \quad (i, i + 1), \quad (i + 1, i + 2).$$

Độ phức tạp tổng cộng là

$$O(q(m + n^2) \log n),$$

đủ qua khoảng 40% dữ liệu. Nếu thay `set` bằng cây van Emde Boas thì có thể đạt độ phức tạp lý thuyết tốt hơn, nhưng điều đó không cần thiết cho ý tưởng chính.

5. Dãy 0-1

Dãy 0-1 là dãy mà mọi phần tử đều bằng 0 hoặc 1. Trong các bài toán liên quan tới sắp xếp dãy số, xét riêng trường hợp 0-1 thường giúp nhìn ra cấu trúc của bài toán.

Ví dụ 2 (BestCoder Round #76 div.1, Problem 1004). Cần duy trì một mảng $a[1..n]$, ban đầu là một hoán vị của 1 đến n . Có m thao tác, mỗi thao tác thuộc một trong hai loại:

1. Sắp xếp tại chỗ đoạn $a[l..r]$ theo thứ tự tăng.
2. Sắp xếp tại chỗ đoạn $a[l..r]$ theo thứ tự giảm.

Sau khi thực hiện xong mọi thao tác, trả lời một truy vấn duy nhất: với chỉ số k đã cho, in giá trị $a[k]$.

$$n, m \leq 10^5.$$

Công cụ quen thuộc cho thao tác đoạn là cây đoạn, nhưng thao tác trong bài này khó hiện thực trực tiếp trên cây đoạn. Điểm đặc biệt là chỉ hỏi một vị trí $a[k]$, nên có thể nhị phân đáp án. Giả sử giá trị đang xét là mid . Đánh dấu các số lớn hơn mid bằng 1, các số còn lại bằng 0; hoán vị được biến thành một dãy 0-1.

Trên dãy 0-1, sắp xếp một đoạn rất đơn giản. Dùng cây đoạn có nhãn lười, hỗ trợ hỏi tổng đoạn và gán đoạn. Khi sắp xếp một đoạn, chỉ cần biết trong đoạn có bao nhiêu số 1 và bao nhiêu số 0; với sắp xếp tăng thì gán nửa đầu thành 0, nửa sau thành 1, còn sắp xếp giảm thì làm ngược lại.

Sau khi thực hiện mọi thao tác trên dãy 0-1, kiểm tra vị trí $a[k]$ là 0 hay 1, từ đó biết giá trị thật có lớn hơn mid hay không. Độ phức tạp là

$$O(n \log n + m \log^2 n).$$

Lợi ích của việc biến một hoán vị thành dãy 0-1 là ta chỉ cần quan tâm quan hệ giữa các phần tử lớn hơn mid và không lớn hơn mid . Quan hệ giữa các phần tử nằm cùng một phía của mid không còn cần xét.

Ví dụ 3 (bài toán kinh điển). Cho một mạng so sánh trên n phần tử. Hãy phán đoán nó có phải một mạng sắp xếp đúng hay không.

Nếu không phải mạng sắp xếp đúng, hãy đếm có bao nhiêu hoán vị n phần tử không được sắp xếp đúng.

Nếu kiểm tra thô, ta phải thử từng hoán vị trong $n!$ hoán vị. Thực ra chỉ cần thử 2^n dãy 0-1. Đây là **nguyên lý 0-1** của mạng sắp xếp. Chứng minh rất ngắn: nếu tồn tại một hoán vị không được sắp xếp đúng, thì kết quả cuối cùng có một nghịch thế $i < j$ với $a[i] > a[j]$. Đặt $x = a[j]$, rồi đổi mọi phần tử lớn hơn x thành 1, các phần tử còn lại thành 0. Với dãy 0-1 thu được làm đầu vào, ở đầu ra vị trí j vẫn là 0, còn vị trí i vẫn là 1, nên mạng cũng không sắp xếp đúng dãy 0-1 này.

Phần còn lại là một bài toán đếm đơn giản. Một hoán vị được sắp xếp đúng khi và chỉ khi với mọi ngưỡng k , nếu đánh dấu các phần tử lớn hơn k bằng 1 và các phần tử còn lại bằng 0, dãy 0-1 thu được đều được sắp xếp đúng. Vì vậy sau khi tiền xử lý mọi dãy 0-1 hợp lệ, một hoán vị hợp lệ tương ứng với một đường đi từ $00 \dots 0$ đến $11 \dots 1$, mỗi bước đổi một 0 thành 1, và mọi trạng thái đi qua đều hợp lệ. Số đường đi này tính được bằng quy hoạch động $O(n2^n)$.

5.1. Truy hồi ngược

Trở lại bài gốc. Ở các điểm kiểm tra còn lại, quy mô đều lớn; mô phỏng từng truy vấn xem ra rất khó. Ta đổi góc nhìn: thay vì mô phỏng, hãy nghiên cứu những dãy ban đầu nào có thể được sắp xếp.

Theo kinh nghiệm vừa nêu, trước hết xét riêng trường hợp dãy 0-1 để xử lý điểm kiểm tra 8. Giả sử dãy 0-1 có k số 1.

Với một dãy 0-1 có thể được sắp xếp đúng, hình dạng cuối cùng của nó là cố định: một đoạn 0 rồi một đoạn 1. Vì thế phân tích ngược sẽ dễ hơn. Trước thao tác thứ m , dãy chắc chắn có dạng đoạn đầu là 0, đoạn cuối là 1, còn riêng đoạn

$$a[l_m], a[l_m + 1], \dots, a[r_m]$$

có thể bị xáo trộn tùy ý. Tương tự, nếu đã biết mọi kết quả có thể có sau thao tác thứ $m - 1$, thì trước thao tác đó đoạn $a[l_{m-1}..r_{m-1}]$ chỉ cần là một hoán vị bất kỳ của đoạn đã được sắp xếp ở sau thao tác.

Mô tả hình thức như sau. Gọi A_i là tập mọi dãy 0-1 có thể được các thao tác $i + 1, i + 2, \dots, m$ sắp xếp đúng. Tập A_m chỉ chứa một dãy: một số 0 đứng trước rồi đến k số 1. Duyệt i giảm từ m đến 1. Với mỗi dãy $a \in A_i$, nếu đoạn $a[l_i..r_i]$ đã có thứ tự, thì mọi cách xáo trộn đoạn đó đều có thể là trạng thái trước thao tác i , nên được thêm vào A_{i-1} . Cuối cùng A_0 chứa đúng mọi dãy 0-1 ban đầu có thể được sắp xếp.

5.2. Phần tử đại diện

Vấn đề là mỗi lần “xáo trộn tùy ý” sinh ra rất nhiều dãy, khiến kích thước A_i có thể là cấp số mũ. Ta muốn tìm một tính chất ngắn gọn mô tả toàn bộ tập A_i . Ý tưởng tự nhiên là tìm trong A_i một dãy đặc biệt làm **phần tử đại diện**. Nó là trường hợp xấu nhất của A_i ; nếu một dãy tốt hơn phần tử đại diện thì dãy đó cũng thuộc A_i .

Cần định nghĩa thế nào là “tốt hơn”. Trong dãy 0-1, số 1 càng nằm về bên trái và số 0 càng nằm về bên phải thì dãy càng gần thứ tự giảm, tức càng xa thứ tự tăng. Vì vậy cách xáo trộn tệ nhất của một đoạn là đưa mọi số 1 trong đoạn về bên trái và mọi số 0 về bên phải, tức sắp xếp đoạn đó theo thứ tự giảm. Mọi cách xáo trộn khác đều có thể tiếp tục dịch các số 1 sang trái để đạt trạng thái xấu nhất này.

Gọi U là tập mọi dãy 0-1 độ dài n chứa đúng k số 1. Với $a \in U$, ký hiệu $\text{pos}(a, i)$ là vị trí của số 1 thứ i khi nhìn từ trái sang phải. Với hai dãy $a, b \in U$, định nghĩa a **tốt hơn** b khi và chỉ khi

$$\text{pos}(a, i) \geq \text{pos}(b, i) \quad (i = 1, 2, \dots, k).$$

Nói cách khác, có thể thu được a bằng cách dời một vài số 1 trong b sang phải. Ví dụ 100111 tốt hơn 101101.

Để tiện trình bày, đặt $f_{01}(a, l, r)$ là dãy thu được sau khi sắp xếp tăng đoạn $a[l..r]$ của dãy 0-1 a , và $f_{10}(a, l, r)$ là dãy thu được sau khi sắp xếp giảm đoạn đó. Khi đó có các tính chất sau:

1. $f_{01}(a, l, r)$ tốt hơn a , còn a tốt hơn $f_{10}(a, l, r)$.
2. Nếu $b = f_{01}(a, l, r)$ thì $f_{10}(a, l, r) = f_{10}(b, l, r)$. Nếu $c = f_{10}(a, l, r)$ thì $f_{01}(a, l, r) = f_{01}(c, l, r)$.
3. Nếu a tốt hơn b , thì $f_{01}(a, l, r)$ tốt hơn $f_{01}(b, l, r)$, và $f_{10}(a, l, r)$ tốt hơn $f_{10}(b, l, r)$.

Bây giờ chứng minh sự tồn tại của phần tử đại diện. Gọi phần tử duy nhất trong A_m là a_m . Theo quy trình truy hồi ngược xấu nhất, định nghĩa các dãy

$$a_{m-1}, a_{m-2}, \dots, a_0$$

bằng công thức

$$a_{i-1} = f_{10}(a_i, l_i, r_i).$$

Ta chứng minh bằng quy nạp rằng

$$x \in A_i \iff x \text{ tốt hơn } a_i.$$

Nếu $x \in A_i$, theo định nghĩa tồn tại

$$y = f_{01}(x, l_{i+1}, r_{i+1}) \in A_{i+1}.$$

Theo giả thiết quy nạp, y tốt hơn a_{i+1} . Dùng tính chất đơn điệu của f_{10} , ta có

$$f_{10}(y, l_{i+1}, r_{i+1}) \text{ tốt hơn } f_{10}(a_{i+1}, l_{i+1}, r_{i+1}) = a_i.$$

Mặt khác x tốt hơn $f_{10}(y, l_{i+1}, r_{i+1})$, nên x tốt hơn a_i .

Chiều ngược lại, nếu u tốt hơn a_i , xét kết quả sau thao tác $i + 1$:

$$v = f_{01}(u, l_{i+1}, r_{i+1}).$$

Từ tính chất đơn điệu, v tốt hơn

$$w = f_{01}(a_i, l_{i+1}, r_{i+1}).$$

Do $a_i = f_{10}(a_{i+1}, l_{i+1}, r_{i+1})$, suy ra w tốt hơn a_{i+1} , nên v tốt hơn a_{i+1} . Theo giả thiết quy nạp, $v \in A_{i+1}$. Vậy u được các thao tác $i + 1, \dots, m$ sắp xếp đúng, tức $u \in A_i$.

Như vậy ta không chỉ chứng minh được phần tử đại diện tồn tại, mà còn biến truy hồi giữa hai tập A_i, A_{i-1} thành truy hồi giữa hai dãy đại diện a_i, a_{i-1} .

5.3. Thuật toán cho điểm kiểm tra 8

Từ kết luận trên, với dữ liệu chỉ gồm 0 và 1, ta tiền xử lý dãy đại diện a_0 , rồi với mỗi truy vấn chỉ cần kiểm tra dãy đầu vào có tốt hơn a_0 hay không.

Bước kiểm tra một truy vấn làm trực tiếp trong $O(n)$. Bước tiền xử lý cần thực hiện các lần sắp xếp giảm trên dãy 0-1; có thể dùng kỹ thuật khử nghịch thế đã nêu ở Mục 4.

Độ phức tạp là

$$O((m + n^2) \log n + qn).$$

5.4. Thuật toán đủ điểm

Không mất tính tổng quát, giả sử mảng cần sắp xếp là một hoán vị của 1 đến n . Nếu không phải vậy, có thể rời rạc hóa; với các giá trị bằng nhau, gán thứ hạng theo thứ tự chỉ số tăng dần. Cách làm này không ảnh hưởng đến kết quả cuối cùng.

Bây giờ mở rộng thuật toán 0-1 sang hoán vị n phần tử. Với hoán vị ban đầu $a[1..n]$ và một ngưỡng t , định nghĩa dãy 0-1 b_t bởi

$$b_t[i] = 1 \iff a[i] \geq t.$$

Thuật toán sắp xếp đúng hoán vị a khi và chỉ khi với mọi $t = 2, 3, \dots, n$, thuật toán sắp xếp đúng dãy 0-1 b_t . Đây chính là nguyên lý 0-1 ở dạng phù hợp với bài toán này.

Theo thuật toán ở trên, về lý thuyết cần tiền xử lý, với từng ngưỡng t , kết quả đại diện sau khi đi ngược qua m thao tác. Làm riêng cho từng ngưỡng là quá chậm. Ta làm một việc tương đương: bắt đầu từ hoán vị

$$1, 2, \dots, n,$$

rồi duyệt các thao tác từ m về 1, mỗi lần sắp xếp giảm đoạn $[l_i, r_i]$. Sau khi kết thúc, nếu trong hoán vị thu được ta thay các giá trị nhỏ hơn t bằng 0 và các giá trị còn lại bằng 1, ta được đúng dãy đại diện c_t cần dùng cho ngưỡng t . Tính đúng đắn là trực tiếp từ định nghĩa truy hồi ngược. Đồng thời, khi t giảm đi một đơn vị, dãy c_t chỉ thay đổi ở một vị trí từ 0 thành 1.

Bây giờ cần kiểm tra với mọi $t = 2, 3, \dots, n$ xem b_t có tốt hơn c_t hay không. Đây là một bài toán kinh điển. Điều kiện tốt hơn tương đương với việc trên mọi tiền tố, số lượng số 1 của b_t không vượt quá số lượng số 1 của c_t . Vì vậy ta đặt các số 1 của c_t đóng góp +1, các số 1 của b_t đóng góp -1, rồi kiểm tra mọi tổng tiền tố có không âm hay không.

Khi quét ngưỡng t , mỗi bước chỉ thêm một cặp thay đổi +1, -1. Việc kiểm tra mọi tiền tố không âm có thể thực hiện bằng cây đoạn hỗ trợ cộng đoạn và duy trì giá trị nhỏ nhất trên toàn cục. Chi phí cho một truy vấn là

$$O(n \log n).$$

Quá trình tiền xử lý hoán vị đại diện được thực hiện bằng kỹ thuật khử nghịch thế. Do đó tổng độ phức tạp là

$$O((n^2 + m + qn) \log n),$$

đủ qua toàn bộ dữ liệu.

6. Các thuật toán lấy điểm một phần khác

Nếu thí sinh nhận ra kết luận ở trên nhưng không biết dùng kỹ thuật khử nghịch thế để tiền xử lý, vẫn có thể đạt điểm khá cao. Chẳng hạn ở điểm kiểm tra 5 và 6, m nhỏ hơn, có thể sắp xếp thô từng lần. Ở điểm kiểm tra 8, do mảng chỉ có 0 và 1, ta cũng có thể chuyển thao tác sắp xếp đoạn thành hỏi tổng đoạn và gán đoạn, rồi dùng cây đoạn.

Còn một hướng có khả năng qua điểm 5 và 6: dùng cấu trúc dữ liệu duy trì dãy, chia nó thành một số đoạn sao cho mỗi đoạn tăng không giảm, và đưa các phần tử của mỗi đoạn vào một cây cân bằng. Khi sắp xếp một đoạn, chỉ cần tách ở hai biên rồi trộn các đoạn nằm giữa; lúc trộn dùng kỹ thuật hợp nhất cây cân bằng. Tuy nhiên độ phức tạp của cách làm này không dễ phân tích.

Một hướng khác là loại bỏ các thao tác chắc chắn không bao giờ cần dùng, rồi mô phỏng thô từng truy vấn. Có thể loại bỏ thao tác vô ích như sau: lấy dãy ban đầu $a[i] = n - i + 1$, thực hiện các thao tác sắp xếp theo thứ tự ban đầu; nếu một thao tác không làm dãy thay đổi thì nó là thao tác vô ích. Hoặc lấy dãy ban đầu $a[i] = i$, đi ngược lại bằng các thao tác sắp xếp giảm, và loại bỏ những thao tác không được dùng tới.

Kết hợp các cách lấy điểm một phần này với một số tối ưu hóa, dự kiến có thể đạt 50 đến 70 điểm; điểm cụ thể phụ thuộc vào cách tối ưu và kinh nghiệm của thí sinh.

7. Xây dựng dữ liệu kiểm tra

Dữ liệu của bài này tương đối khó xây dựng. Sau đây là cách tác giả tạo từng nhóm điểm kiểm tra.

7.1. Điểm 1 và 2

Hai điểm này có $n, m, q \leq 200$. Mô phỏng thô là đủ qua, gần như không có cách sai hoặc cách quá thời gian đáng kể, nên chỉ cần sinh ngẫu nhiên. Khi sinh, cố gắng làm số lượng đáp án AC và WA cân bằng.

7.2. Điểm 3 và 4

Hai điểm này dành cho thuật toán mô phỏng mỗi truy vấn trong

$$O((n^2 + m) \log n).$$

Ở đây q nhỏ; nhưng để tránh việc thí sinh dùng cách nộp thô rồi đoán đáp án bằng OJ, không thể đặt q quá nhỏ, nên giới hạn $q \leq 8$.

Độ phức tạp của sắp xếp thô là

$$O\left(\sum (r_i - l_i) \log n\right).$$

Để chặn cách làm này, cần làm cho các đoạn sắp xếp càng dài càng tốt. Đồng thời cũng phải tránh để dãy bị sắp xếp xong quá sớm, nếu không thí sinh chỉ cần thêm lệnh dừng sớm là có thể qua. Có thể giữ vài phần tử ở biên không động, đến cuối mới cho chúng tham gia sắp xếp.

7.3. Sinh mảng truy vấn

Mảng truy vấn được sinh như sau. Sau khi xây dựng xong m bước sắp xếp, tính giống lời giải chuẩn để lấy kết quả a_0 sau khi đi ngược bằng các thao tác sắp xếp giảm, rồi thực hiện một số điều chỉnh ngẫu nhiên trên mảng này.

Nếu muốn sinh truy vấn AC, nhiều lần chọn ngẫu nhiên một cặp gần nhau (i, j) thỏa $i < j$ và $a[i] > a[j]$, rồi đổi chỗ $a[i]$ và $a[j]$. Nếu muốn sinh truy vấn WA, làm ngược lại. Cách tinh chỉnh này khiến dữ liệu AC không bị sắp xếp xong quá sớm, còn dữ liệu WA có kết quả cuối khá gần với dãy có thứ tự, nhờ đó hạn chế các cách ngẫu nhiên thiếu căn cứ. Ngoài ra, trong dữ liệu còn trộn thêm một phần mảng truy vấn gần như giảm.

7.4. Điểm 5 và 6

Ở hai điểm này, m và n cùng bậc, q lớn. Chúng dành cho thí sinh nhận ra kết luận lời giải nhưng chưa biết tiền xử lý, cũng như thí sinh có năng lực cấu trúc dữ liệu tốt.

Ở đây vẫn cần làm các đoạn sắp xếp càng dài càng tốt, nhưng cũng phải tránh sinh quá nhiều thao tác vô ích. Có thể đặt

$$mid = \lfloor n/2 \rfloor,$$

rồi lần lượt thêm các thao tác trên các đoạn

$$[1, mid], [2, mid + 1], [3, mid + 2], \dots, [n - mid + 1, n],$$

sau đó đệ quy xây dựng cho nửa trái và nửa phải. Các thao tác này đều hữu ích; số thao tác là $\Theta(n)$, độ dài trung bình của đoạn là $\Theta(n)$, và lượng nghịch thế giảm là $\Theta(n^2)$, đủ khiến phần lớn cách sai bị quá thời gian.

7.5. Điểm 7, 8, 9 và 10

Điểm 7, 9 và 10 dành cho lời giải chuẩn, với quy mô tăng dần; cách xây dựng điểm 8 gần giống chúng. Trong bốn điểm này có hai loại dữ liệu.

Loại thứ nhất giống điểm 3 và 4: các đoạn thao tác đều rất dài, nhưng số thao tác vô ích cũng tăng.

Loại thứ hai có độ dài thao tác hầu như đều bằng 2, tức mỗi lần chỉ sắp xếp hai phần tử kề nhau. Số thao tác hữu ích có thể đạt $\Theta(n^2)$; cách sinh chỉ cần mỗi lần đổi một cặp nghịch thế kề nhau. Khi đó mô phỏng thô có độ phức tạp ít nhất $\Omega(qn^2)$, về cơ bản không thể qua.

Vì loại dữ liệu thứ nhất dễ bị vượt qua bằng mẹo hơn, tác giả chỉ đặt một điểm 9 thuộc loại này; ba điểm còn lại thuộc loại thứ hai.

8. Tổng kết ra đề

Đề bài này rất ngắn gọn và dễ hiểu. Thoạt nhìn nó giống nhiều bài cấu trúc dữ liệu trước đây: cần duy trì một dãy và hỗ trợ thao tác đoạn. Nhưng khác với các bài cấu trúc dữ liệu thuần túy, nút thắt của bài không nằm ở lý thuyết cấu trúc dữ liệu cao cấp hay kỹ thuật phân tích độ phức tạp phức tạp, mà nằm ở việc nghiên cứu tính chất của thao tác trong đề, phân tích ngược để phát hiện kết luận, rồi chọn cấu trúc dữ liệu kinh điển phù hợp với kết luận đó.

Lời giải chuẩn chỉ dùng set và cây đoạn có nhãn lười cơ bản. Độ khó mã không cao; phần lớn thí sinh ở trình độ NOI đều có thể sử dụng thành thạo những công cụ này. Tuy nhiên bài có độ khó tư duy nhất định.

Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Cảm ơn hai huấn luyện viên đội tuyển tập huấn Yu Linyun và Chen Xumin đã giúp đỡ.
- Cảm ơn thầy Xu Xianyou, huấn luyện viên của tôi, đã bồi dưỡng tôi trong nhiều năm.
- Cảm ơn các bạn Du Yuhao, Xu Yinzhan, Wang Yisong và Mao Xiao đã thảo luận cùng tôi trong quá trình ra đề và đưa ra nhiều góp ý quý báu.
- Cảm ơn bạn Zhou Zichun đã kiểm tra đề.

Tài liệu tham khảo

1. Hamid Sarbazi-Azad. “Stupid Sort: A new sorting algorithm”, *Newsletter*, Computing Science Department, University of Glasgow, số 599, trang 4, 2/10/2000.
2. *BestCoder Round #76 Solution*.
3. Wang Yisong, báo cáo lời giải *Tree and Sets*.
4. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi Olympic Tin học*, Nhà xuất bản Đại học Thanh Hoa.

Bài 15. Báo cáo ra đề *move*

Yuan Yutao, Trường Yali, Trường Sa

1. Đề bài

Xét một dãy nhị phân vô hạn. Trong dãy có đúng s vị trí mang giá trị 1; các vị trí này tạo thành m đoạn liên tiếp và những đoạn ấy được cho trước. Ngoài ra cho một đa thức $f(x)$.

Với một đoạn, định nghĩa trọng số của đoạn như sau. Chọn ngẫu nhiên một vị trí bắt đầu trong đoạn. Sau đó ở mỗi bước lại chọn ngẫu nhiên vị trí tiếp theo trong đoạn, lật giá trị tại vị trí ấy, và lặp lại cho tới khi mọi số trong đoạn bằng nhau. Mỗi lần di chuyển, lấy khoảng cách vừa đi thay vào $f(x)$; trọng số của đoạn là kỳ vọng của tổng các giá trị thu được.

Có q truy vấn. Mỗi truy vấn cho một đoạn độ dài n ; ta được phép sắp xếp lại tùy ý mọi giá trị trong đoạn ấy. Hỏi có thể làm trọng số tăng thêm nhiều nhất bao nhiêu, đồng thời phải sắp xếp theo một phương án đạt cực đại. Nếu có nhiều phương án đạt cực đại, chọn phương án mà dãy vị trí các số 1 có thứ tự từ điển nhỏ nhất.

Giới hạn dữ liệu:

$$n \leq 10^9, \quad k \leq 250000, \quad m \leq 20000, \quad q \leq 15000, \quad s \leq 5 \cdot 10^7.$$

2. Giới thiệu thuật toán

2.1. Cách làm dựa trên nén trạng thái

Cách 10 điểm. Trước hết xét cách tính trọng số của một đoạn. Nếu trong một trạng thái ta ghi cả vị trí hiện tại lẫn trạng thái nhị phân hiện tại của đoạn, ta nhận được một thuật toán có số trạng thái $O(n2^n)$. Gọi $f_{i,t}$ là kỳ vọng lợi ích còn lại khi vị trí hiện tại là i , còn trạng thái nhị phân của đoạn là t . Giải trực

tiếp hệ phương trình tuyến tính trên các trạng thái này sẽ cho trọng số cần tìm. Cách này chỉ đủ cho nhóm dữ liệu nhỏ nhất.

Cách 20 điểm. Vì mỗi lần di chuyển đều là ngẫu nhiên đều, kỳ vọng lợi ích của một bước hiện tại chỉ phụ thuộc vào vị trí hiện tại. Gọi g_t là kỳ vọng khi trạng thái hiện tại là t , còn vị trí hiện tại tùy ý và không tính lợi ích của bước đang đi. Gọi h_i là kỳ vọng lợi ích thu được khi bắt đầu từ vị trí i và đi một bước; giá trị này chỉ phụ thuộc vào i , nên có thể tiền xử lý. Khi đó

$$f_{i,t} = g_t + h_i \quad (t \neq 0, t \neq 2^n - 1),$$

và

$$g_t = \frac{1}{n} \sum_{i=0}^{n-1} f_{i,t \oplus 2^i}.$$

Thay mọi f bằng g , số trạng thái giảm còn 2^n . Sau đó vẫn giải hệ phương trình để tìm mọi đáp án, đủ đạt khoảng 20 điểm.

2.2. Một thuật toán độ phức tạp đa thức

Quan sát rằng phương trình của g_t có tính đối xứng rất mạnh. Ta tách riêng đóng góp của lợi ích h_i khi di chuyển từ từng vị trí i . Gọi $c_{i,t}$ là kỳ vọng số lần đi từ vị trí i khi trạng thái của đoạn là t . Khi đó

$$g_t = \sum_{i=0}^{n-1} c_{i,t} h_i.$$

Do mỗi bước đều chọn ngẫu nhiên, $c_{i,t}$ không phụ thuộc vào thứ tự cụ thể của các vị trí khác nhau. Vì vậy, ngoài vị trí i , mọi vị trí có cùng giá trị trong trạng thái đều tương đương. Nói cách khác, $c_{i,t}$ chỉ phụ thuộc vào giá trị của bit thứ i trong t và số lượng bit 1 của t .

Gọi $a_{i,x}$ là giá trị $c_{i,t}$ tại một vị trí có giá trị x , trong một trạng thái t có đúng i bit 1; ở đây $x \in \{0, 1\}$. Xét trạng thái có i bit đầu là 1 và $n - i$ bit sau là 0, với $1 < i < n - 1$. Thay a vào phương trình chuyển của g_t , ta có

$$\begin{aligned} & \sum_{k=0}^{i-1} a_{i,1} h_k + \sum_{k=i}^{n-1} a_{i,0} h_k \\ &= \frac{1}{n} \left(\sum_{k=0}^{i-1} \left(h_k + \sum_{\substack{p < i \\ p \neq k}} a_{i-1,1} h_p + a_{i-1,0} h_k + \sum_{p=i}^{n-1} a_{i-1,0} h_p \right) \right. \\ & \quad \left. + \sum_{k=i}^{n-1} \left(h_k + \sum_{p=0}^{i-1} a_{i+1,1} h_p + a_{i+1,1} h_k + \sum_{\substack{p \geq i \\ p \neq k}} a_{i+1,0} h_p \right) \right) \\ &= \frac{1}{n} \left(\sum_{k=0}^{i-1} ((i-1)a_{i-1,1} + a_{i-1,0} + (n-i)a_{i+1,1} + 1) h_k \right. \\ & \quad \left. + \sum_{k=i}^{n-1} ((n-i-1)a_{i+1,0} + a_{i+1,1} + ia_{i-1,0} + 1) h_k \right). \end{aligned}$$

Vì các $a_{i,x}$ không phụ thuộc vào h_k , nhất định có

$$a_{i,1} = \frac{1}{n} ((i-1)a_{i-1,1} + a_{i-1,0} + (n-i)a_{i+1,1} + 1),$$

$$a_{i,0} = \frac{1}{n} ((n-i-1)a_{i+1,0} + a_{i+1,1} + ia_{i-1,0} + 1) \quad (1 < i < n-1). \quad (1)$$

Với trường hợp biên $i = 1$, xây dựng tương tự rồi thay vào g_t , ta được

$$a_{1,1} = \frac{1}{n}(n-1)a_{2,1}, \quad a_{1,0} = \frac{1}{n}((n-2)a_{2,0} + a_{2,1} + 1). \quad (2)$$

Với trường hợp biên $i = n-1$, ta được

$$a_{n-1,1} = \frac{1}{n}((n-2)a_{n-2,1} + a_{n-2,0} + 1), \quad a_{n-1,0} = \frac{1}{n}(n-1)a_{n-2,0}. \quad (3)$$

Vì $a_{i,x}$ chỉ liên hệ với các hạng tử kề nó, toàn bộ dãy $a_{i,x}$ có thể được giải trong $O(n)$ thời gian.

Để quan sát rằng khi $i < n/2$ thì $a_{i,1} < a_{i,0}$; khi $i = n/2$ thì $a_{i,1} = a_{i,0}$; còn khi $i > n/2$ thì $a_{i,1} > a_{i,0}$. Do đó nếu một đoạn truy vấn có i số 1, thì khi $i < n/2$ nên đưa các số 1 tới những vị trí có h_k nhỏ nhất, khi $i = n/2$ mọi phương án cho cùng đáp án, còn khi $i > n/2$ nên đưa các số 1 tới những vị trí có h_k lớn nhất.

Theo định nghĩa của h_x ,

$$\begin{aligned} h_x &= \frac{1}{n} \sum_{i=0}^{n-1} f(|x-i|) \\ &= \frac{1}{n} \left(\sum_{i=0}^{x-1} f(x-i) + \sum_{i=x}^{n-1} f(i-x) \right) \\ &= \frac{1}{n} \left(\sum_{i=1}^x f(i) + \sum_{i=0}^{n-x-1} f(i) \right). \end{aligned} \quad (4)$$

Do mọi hệ số của $f(x)$ đều là số nguyên không âm và hệ số bậc cao nhất khác 0, $f(x)$ tăng nghiêm ngặt. Lại có

$$h_x - h_{x-1} = \frac{1}{n}(f(x) - f(n-x)),$$

nên h_x đối xứng quanh $x = n/2$, giảm khi $x < n/2$ và tăng khi $x > n/2$. Vì vậy các vị trí có h_x nhỏ nhất luôn tạo thành một đoạn, còn các vị trí có h_x lớn nhất luôn tạo thành hai đoạn ở hai phía.

Kết luận là sau mỗi lần sắp xếp lại, các vị trí chứa 1 chắc chắn tạo thành một hoặc hai đoạn. Ta có thể dùng cây cân bằng để duy trì mọi đầu mút đoạn. Mỗi đầu mút chỉ bị xóa nhiều nhất một lần, nên tổng số đoạn là $O(m+q)$, và tổng số thao tác trên đoạn cũng là $O(m+q)$. Với mỗi truy vấn, tính đáp án trước và sau khi sắp xếp lại. Sau khi tiền xử lý tổng tiền tố của h_k , đóng góp của mỗi đoạn vào đáp án được tính trong $O(1)$.

Cách này cần $O(n)$ thời gian để tiền xử lý $a_{i,x}$, $O(nk)$ thời gian để tiền xử lý h_k , và

$$O((m+q) \log(m+q))$$

để xử lý các truy vấn, đủ đạt khoảng 30–35 điểm.

2.3. Tính nhanh h_x

Dùng số Bernoulli. Theo (4), để tính h_x chỉ cần nhanh chóng tính $\sum_{i=1}^x f(i)$. Đây là một đa thức bậc $k+1$ theo x , có thể tính bằng số Bernoulli. Nếu

$$f(x) = \sum_{i=0}^k u_i x^i,$$

thì

$$\begin{aligned}\sum_{r=1}^x f(r) &= \sum_{i=0}^k \frac{u_i}{i+1} \sum_{j=0}^i (-1)^j \binom{i+1}{j} B_j x^{i+1-j} \\ &= \sum_{p=0}^{k+1} x^p \sum_{i=p-1}^k \frac{u_i}{i+1} (-1)^{i+1-p} \binom{i+1}{i+1-p} B_{i+1-p} \\ &= \sum_{p=0}^{k+1} \frac{x^p}{p!} \sum_{i=p-1}^k \frac{u_i}{i+1} (-1)^{i+1-p} \frac{(i+1)!}{(i+1-p)!} B_{i+1-p}.\end{aligned}$$

Như vậy hệ số của x^p có thể lấy bằng một tích chập của hai dãy, chẳng hạn $F_i = i!u_i$ và $G_i = (-1)^i i!B_i$. Các số Bernoulli cũng tính được bằng một lần NTT.

Số Bernoulli có hàm sinh mũ

$$G(x) = \sum_{i=0}^{\infty} \frac{B_i}{i!} x^i = \frac{x}{e^x - 1} = \frac{1}{\sum_{i=0}^{\infty} x^i / (i+1)!}.$$

Do đó bài toán quy về tìm nghịch đảo của đa thức; cách làm chi tiết được nêu trong tài liệu tham khảo [1].

Để tìm đa thức của h_i , còn cần từ hệ số của $f(x)$ suy ra hệ số của $f(n-1-x)$, việc này cũng giải được bằng NTT. Tổng tiền tố của h_i cũng có thể xử lý bằng số Bernoulli. Nhờ đó ta tìm được đa thức biểu diễn tổng tiền tố của h_i trong $O(k \log k)$ thời gian, rồi chỉ cần thay các giá trị cần hỏi vào. Vì số đầu mút đoạn khác nhau là $O(\min(n, m+q))$, việc tính trực tiếp mọi giá trị cần dùng tốn $O(k \min(n, m+q))$, đủ đạt khoảng 45–50 điểm.

Dùng thuật toán tính giá trị đa điểm. Để tính $f(x_i)$ tại nhiều điểm x_i khác nhau, ta dùng thuật toán tính giá trị đa điểm. Ý tưởng cơ bản là

$$f(x_i) = f(x) \bmod (x - x_i).$$

Muốn tính $f(x_1), f(x_2), \dots, f(x_m)$, ta dựng cây tích các đa thức $(x - x_i)$. Ở mỗi nút, lấy phần dư của đa thức hiện tại theo tích ứng với nửa trái và nửa phải, rồi tiếp tục chia xuống dưới. Phép chia đa thức có thể làm trong $O(n \log n)$, nên toàn bộ quá trình tính giá trị đa điểm có độ phức tạp $O(n \log^2 n)$. Chi tiết về phép chia đa thức và tính giá trị đa điểm có trong tài liệu tham khảo [2].

Sau tối ưu này, lời giải đạt khoảng 60–70 điểm.

2.4. Tối ưu thêm

Trong bài này, thực ra ta chỉ cần biết đóng góp thay đổi khi một vị trí đổi từ 0 thành 1 hoặc từ 1 thành 0, tức là $a_{i,1} - a_{i,0}$. Đặt

$$d_i = a_{i,1} - a_{i,0}.$$

Từ (1) suy ra

$$\begin{aligned}n(a_{i,1} - a_{i,0}) &= ((i-1)a_{i-1,1} + a_{i-1,0} + (n-i)a_{i+1,1} + 1) \\ &\quad - ((n-i-1)a_{i+1,0} + a_{i+1,1} + ia_{i-1,0} + 1) \\ &= (i-1)(a_{i-1,1} - a_{i-1,0}) + (n-i-1)(a_{i+1,1} - a_{i+1,0}),\end{aligned}$$

tức là

$$nd_i = (n-i-1)d_{i+1} + (i-1)d_{i-1} \quad (1 < i < n-1). \quad (5)$$

Tương tự, từ (2) và (3) lần lượt có

$$nd_1 = (n-2)d_2 - 1, \quad (6)$$

$$nd_{n-1} = (n-2)d_{n-2} + 1. \quad (7)$$

Tính d_i . Theo đối xứng, $d_{n-1} = -d_1$. Cộng (5) với $i = 2, 3, \dots, m$ rồi cộng thêm (6), trong đó $1 \leq m < n - 1$, ta được

$$\begin{aligned}\sum_{i=1}^m nd_i &= \sum_{i=1}^m (n-i-1)d_{i+1} + \sum_{i=2}^m (i-1)d_{i-1} - 1 \\ &= \sum_{i=2}^{m+1} (n-i)d_i + \sum_{i=1}^{m-1} id_i - 1 \\ &= \sum_{i=2}^{m-1} nd_i + (n-m)d_m + (n-m-1)d_{m+1} + d_1 - 1.\end{aligned}$$

Rút gọn:

$$(n-m-1)d_{m+1} - md_m = (n-1)d_1 + 1.$$

Nhân hai vế với $\binom{n-1}{m}$, ta có

$$\binom{n-2}{m}d_{m+1} - \binom{n-2}{m-1}d_m = \binom{n-1}{m}\left(d_1 + \frac{1}{n-1}\right). \quad (8)$$

Cộng (8) với $m = 1, 2, \dots, k-1$, suy ra

$$\binom{n-2}{k-1}d_k - d_1 = \sum_{i=1}^{k-1} \binom{n-1}{i}\left(d_1 + \frac{1}{n-1}\right),$$

hay

$$d_k = \frac{1}{\binom{n-2}{k-1}} \left(\sum_{i=0}^{k-1} \binom{n-1}{i} \left(d_1 + \frac{1}{n-1} \right) - \frac{1}{n-1} \right). \quad (9)$$

Lấy $k = n - 1$ trong (9), rồi dùng $d_{n-1} = -d_1$, ta giải được

$$d_1 = \frac{1 - 2^{n-2}}{(n-1)2^{n-2}}. \quad (10)$$

Thay vào (9),

$$d_k = \frac{\sum_{i=0}^{k-1} \binom{n-1}{i} - 2^{n-2}}{\binom{n-2}{k-1}(n-1)2^{n-2}}. \quad (11)$$

Như vậy có thể tính mọi d_i cần dùng trong $O(k)$ thời gian. Trong dữ liệu của bài, $k \leq s \leq 5 \cdot 10^7$, nên cách này còn có thể chạy kịp nếu tối ưu đủ tốt.

Giảm bộ nhớ. Nếu lưu toàn bộ d_i , các tổ hợp và nghịch đảo trung gian, bộ nhớ $O(s)$ sẽ vượt giới hạn. Ta biến đổi (11):

$$d_k = \frac{\sum_{i=0}^{k-1} \frac{(n-1)!(k-1)!}{(n-i-1)!i!} - (k-1)!2^{n-2}}{\frac{(n-2)!}{(n-k-1)!}(n-1)2^{n-2}}.$$

Vì vậy cần tính

$$\sum_{i=0}^{k-1} \frac{(n-1)!(k-1)!}{(n-i-1)!i!}$$

và nghịch đảo của $\frac{(n-2)!}{(n-k-1)!}$.

Đặt

$$b_k = \sum_{i=0}^{k-1} \frac{(n-1)!(k-1)!}{(n-i-1)!i!}.$$

Khi đó

$$b_0 = 0, \quad b_k = (k-1)b_{k-1} + \frac{(n-1)!}{(n-k)!} \quad (k \geq 1).$$

Ta có thể quét xuôi; chỉ cần duy trì $(n-1)!/(n-k)!$, nên mỗi bước dùng $O(1)$ bộ nhớ bổ sung.

Đặt

$$e_k = \left(\frac{(n-2)!}{(n-k-1)!} \right)^{-1}.$$

Khi đó

$$e_k = (n-k-2)e_{k+1}.$$

Chỉ cần tính e_k tại giá trị k lớn nhất, rồi quét ngược để suy ra các giá trị còn lại. Trong quá trình tính toán, ta chỉ giữ lại các e_k thật sự cần cho truy vấn, nhờ đó bộ nhớ giảm xuống $O(m+q)$. Cách này đạt khoảng 90–100 điểm.

2.5. Tối ưu hằng số

Vì hằng số của NTT khá lớn, cần thêm một số tối ưu hằng số mới qua được bài.

Phân tích khấu hao. Định nghĩa thế năng $\Phi(S)$ của trạng thái S là hai lần số đoạn mang giá trị 1 trong trạng thái ấy.

Mọi điểm cần tính bằng đa điểm đến từ hai phần của mỗi truy vấn: đáp án trước khi sửa và đáp án sau khi sửa. Với phần trước khi sửa, gọi w là số đoạn 1 nằm hoàn toàn trong đoạn truy vấn, và z là số đoạn chỉ bị phủ một phần. Rõ ràng $z \leq 2$. Nếu đoạn truy vấn nằm trong một đoạn 1, số điểm cần hỏi của truy vấn này bằng 0, thế năng không đổi. Nếu không, phần này có $2w+z$ điểm cần hỏi, đồng thời thế năng giảm $2w$.

Với phần sau khi sửa, xét thay đổi trạng thái trong một truy vấn. Nếu đoạn truy vấn có i vị trí là 1, có hai trường hợp.

Khi $i \leq n/2$, sau khi sắp xếp sẽ sinh một đoạn. Ta cần hỏi hai đầu mút của đoạn này, và thế năng tăng 2.

Khi $i > n/2$, sau khi sắp xếp sẽ sinh một đoạn tiền tố và một đoạn hậu tố. Khi đó chỉ cần hỏi đầu mút phải của đoạn tiền tố và đầu mút trái của đoạn hậu tố. Nếu một đoạn đã bị phủ một phần trong phần trước, nó có thể hợp nhất với đoạn sinh ra trong phần này, nên tối đa chỉ sinh $2-z$ đoạn mới; thế năng tăng $4-2z$.

Trong trường hợp thứ nhất, thời gian khấu hao là $4+z$; trong trường hợp thứ hai, thời gian khấu hao là $6-z$. Do đó mỗi truy vấn có thời gian khấu hao không quá 6. Vì thế năng ban đầu là $2m$, tổng số điểm cần hỏi không vượt quá $2m+6q$, và trong giới hạn của bài này nhiều nhất là 130000.

Mẹo tối ưu hằng số. Khi cài NTT, có thể tiền xử lý mọi căn đơn vị cần dùng để giảm số phép nhân.

Khi cài tính giá trị đa điểm, ta thường phải lấy phần dư của một đa thức theo hai đa thức con. Lưu ý rằng để tính $f(x) \bmod g(x)$, ta cần kết quả NTT của cả $f(x)$ và $g(x)$. Với $f(x)$, chỉ cần NTT một lần rồi lưu kết quả. Với $g(x)$, do $g(x)$ được tạo bằng tích của hai đa thức, kết quả NTT của $g(x)$ đã có trong bước dựng cây tích và có thể dùng lại trực tiếp.

3. Cách sinh dữ liệu

3.1. Dữ liệu 1–10

Trong 10 điểm dữ liệu này, mọi dữ liệu đều được sinh hoàn toàn ngẫu nhiên.

3.2. Dữ liệu 11–12

Trên nền dữ liệu ngẫu nhiên, hai điểm này có thêm một số đoạn độ dài lớn. Đồng thời sinh các truy vấn có $i = n/2$.

3.3. Dữ liệu 15–20

Ở sáu điểm này, ban đầu chỉ có một đoạn. Mỗi lần chọn ngẫu nhiên hậu tố của một đoạn hiện có để truy vấn; truy vấn luôn dịch hậu tố ấy sang phải. Lặp lại việc chọn hậu tố, và khi đoạn trở nên quá ngắn thì bắt đầu lại từ đầu mút trái ban đầu, cho tới khi đủ số truy vấn. Cách này tạo được dữ liệu có nhiều điểm cần tính đa điểm, khoảng 100000 điểm.

3.4. Dữ liệu 13–14

Hai điểm này cũng nhằm tạo nhiều điểm cần tính đa điểm.

Ban đầu sinh hai đoạn p và q , có độ dài lần lượt là a và b , trong đó $b > n/2$, giữa hai đoạn có một khoảng cách nhất định. Truy vấn đầu tiên chọn đoạn q , khiến số điểm cần hỏi đạt 6. Đồng thời q bị tách thành hai đoạn q_1 và q_2 , cả hai đều có độ dài nhỏ hơn $n/2$. Truy vấn thứ hai chọn q_1 , đồng thời phủ một phần của p và q_2 , và bảo đảm q_1 nằm lệch trái trong đoạn truy vấn. Khi đó truy vấn này cũng có 6 điểm cần hỏi, đồng thời q_1 dịch sang phải. Ta có thể giữ khoảng cách giữa đầu mút trái của q_1 và đầu mút phải của q_2 không vượt quá n , nhờ đó xem q_1 và q_2 như một đoạn q mới để tiếp tục.

Do cách này cần $b > n/2$, sau khi sinh một số truy vấn thì không thể tiếp tục mãi. Có thể dùng một vài truy vấn để hợp nhất mọi đoạn về dạng gần giống ban đầu. Dữ liệu sinh theo cách này có thể đạt khoảng 125000 điểm tính đa điểm.

4. Tổng kết

Ý tưởng ra đề đến từ bài *BitToggler* của Topcoder SRM 641. Bài này mở rộng một phần bài gốc lên $n = 10^9$ và $s = 5 \cdot 10^7$. Việc rút ra kết luận chính có độ khó tư duy nhất định; đồng thời lời giải kết hợp các thuật toán đa thức, qua đó kiểm tra năng lực cài đặt của thí sinh.

5. Lời cảm ơn

Xin cảm ơn CCF đã cung cấp nền tảng học tập và giao lưu.

Xin cảm ơn thầy Wang Xingming đã dạy dỗ tôi.

Xin cảm ơn các bạn học đã giúp đỡ tôi.

Tài liệu tham khảo

1. Picks, “Inverse Element of Polynomial”,
<http://picks.logdown.com/posts/189620-inverse-element-of-polynomial>.
2. Picks, “Polynomial Division”,
<http://picks.logdown.com/posts/197262-polynomial-division>.
3. Liu Rujia, *Kinh điển nhập môn thi lập trình thuật toán*, Nhà xuất bản Đại học Thanh Hoa.